

Amazon SDE1

Coding Interview Master Guide

How to solve the problem **and** how to talk through it in an Amazon coding interview.

What's inside

54 problems, organized by pattern (Arrays & Hashing, Two Pointers, Sliding Window, Stacks, Binary Search, Linked Lists, Trees/BST, Graphs, Heaps, Greedy, Backtracking, Dynamic Programming, Design, and more).

Every problem follows the same 9-step Amazon talk track: restate, clarify, brute force, complexity, bottleneck, optimize, code while narrating, dry run, edge case, final complexity.

Every problem also includes: interviewer push-back Q&A, common mistakes, a recovery script for getting stuck, pattern recognition cues, a memory hook, a 30-second recap, and a full realistic talk-track dialogue.

A final Pattern Cheat Sheet, a Universal Interview Flow page, and a 24-Hour Crash Review.

A note on sources: No playlist link or video URL was included in the request that produced this guide, so no video transcripts could be retrieved. Every explanation below is written from first principles / general problem-solving knowledge (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video. Where a problem's write-up references "the video," that refers to this limitation — see the Sources page at the end for full detail.

Table of Contents

1. 1. Arrays & Hashing
2. 2. Two Pointers
3. 3. Sliding Window
4. 4. Stacks
5. 5. Binary Search
6. 6. Linked Lists
7. 7. Trees & BST
8. 8. Graphs
9. 9. Heaps / Priority Queue
10. 10. Greedy
11. 11. Backtracking
12. 12. Dynamic Programming
13. 13. Design / Data Structure Design
14. 14. Other Important Amazon-Style Problems
15. Amazon SDE1 Pattern Cheat Sheet
16. How to Talk in an Amazon Coding Interview
17. 24-Hour Amazon Coding Crash Review
18. Sources

1. Arrays & Hashing

Foundational hash-map patterns: counting, grouping, prefix/suffix products.

Maximum Subarray

LeetCode #53 | **MEDIUM** | Pattern: Dynamic Programming (Kadane's Algorithm) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of integers, possibly including negative numbers, and we need to find the contiguous subarray with the largest sum, then return that sum.

What I should clarify

- Can the array be empty, or is it guaranteed to have at least one element?
- Are negative numbers allowed, and could the whole array be negative?
- Do we need to return the subarray itself, or just the sum?
- Is there a size constraint on the array that would rule out an $O(n^2)$ approach?

SAY THIS —

So just to confirm — I'm given an array that can contain negative numbers, and I need to find the maximum possible sum of any contiguous run of elements, correct? And if everything is negative, I should still return the largest single element rather than an empty subarray?

STEP 2 — Brute Force First

Idea

Try every possible subarray: for each starting index, extend the ending index one step at a time, keep a running sum, and track the best sum seen across all of them.

SAY THIS —

The brute-force way is to check every pair of start and end indices, compute the sum of that subarray, and keep track of the maximum sum found so far.

STEP 3 — Brute Force Complexity

Time Complexity

For every start index we scan forward to every end index, so we're doing roughly n choices of start times n choices of end, giving $O(n^2)$ work just to enumerate sums (and $O(n^3)$ if we recompute each sum from scratch instead of extending a running total).

Space Complexity

We only need a couple of variables to track the current sum and the best sum, so it's $O(1)$ extra space.

SAY THIS —

This is $O(n^2)$ time because for every starting index I'm re-scanning forward through the rest of the array, and $O(1)$ space since I'm not storing anything beyond a couple of running totals.

STEP 4 — Identify the Bottleneck

We're recomputing sums for overlapping subarrays over and over. If I already know the best sum ending at index i , I don't need to start over from scratch at index $i+1$ — I can reuse that information.

SAY THIS —

The bottleneck is that we keep re-summing overlapping ranges of the array instead of reusing the sum we already computed for a slightly shorter subarray.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (Kadane's Algorithm)

This is Kadane's Algorithm, a one-pass Dynamic Programming approach. At each index, I ask one question: is it better to extend the previous subarray by adding the current number, or is the previous subarray dragging me down so much that I should just start fresh at the current number? I keep a running 'best sum ending here' value, and separately track the overall best sum seen so far.

KEY INSIGHT

If the running sum ever goes negative, it can only hurt any future subarray, so the optimal choice is to drop it and restart the running sum at the current element. This is the same idea as 'sunk cost' — a negative prefix is never worth carrying forward.

SAY THIS —

I can avoid recomputation by keeping a running sum as I scan left to right.

At each element, I decide whether to extend the current subarray or start a new one here, whichever gives a bigger sum.

I'll also keep a separate variable for the best sum I've seen across the whole scan, since the best subarray might have ended earlier.

STEP 6 — Code While Narrating

```
def max_subarray(nums):  
    current_sum = nums[0]  
    best_sum = nums[0]
```

SAY THIS —

I'm initializing both my running sum and my best answer to the first element, not to zero.

WHY: Starting at zero would be wrong if every number in the array is negative — we'd incorrectly allow an empty subarray with sum 0. Starting from `nums[0]` guarantees we always pick at least one real element.

```
    for i in range(1, len(nums)):  
        current = nums[i]  
        if current_sum < 0:  
            current_sum = current  
        else:  
            current_sum = current_sum + current
```

SAY THIS —

For each new number, I check whether my running sum has gone negative. If it has, it's not helping me, so I restart the subarray at the current number. Otherwise, I extend the subarray by adding the current number.

WHY: A negative running sum can only reduce any future total, so keeping it around is strictly worse than dropping it and starting over at the current index.

```
        if current_sum > best_sum:
            best_sum = current_sum

    return best_sum
```

SAY THIS —

After updating the running sum, I compare it against the best sum I've recorded so far, and update if this is a new maximum. At the end I return the best sum found.

WHY: The best subarray might not end at the very last index, so I have to check after every single step, not just once at the end.

STEP 7 — Dry Run

nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]. Start: current_sum = best_sum = -2. i=1 (val 1): current_sum was -2 (negative) so restart -> current_sum = 1; best_sum = 1. i=2 (val -3): current_sum = 1 + (-3) = -2; best_sum stays 1. i=3 (val 4): current_sum was -2 (negative) so restart -> current_sum = 4; best_sum = 4. i=4 (val -1): current_sum = 4 - 1 = 3; best_sum stays 4. i=5 (val 2): current_sum = 3 + 2 = 5; best_sum = 5. i=6 (val 1): current_sum = 5 + 1 = 6; best_sum = 6. i=7 (val -5): current_sum = 6 - 5 = 1; best_sum stays 6. i=8 (val 4): current_sum = 1 + 4 = 5; best_sum stays 6. Final answer: 6, from subarray [4, -1, 2, 1].

SAY THIS —

Let me walk through [-2, 1, -3, 4, -1, 2, 1, -5, 4]. I'll track current_sum and best_sum at each step... and I land on 6, which matches the subarray [4, -1, 2, 1].

STEP 8 — Edge Case Test

All-negative array, e.g. nums = [-3, -1, -2]. current_sum and best_sum start at -3. At -1: current_sum was -3 (negative), restart to -1; best_sum becomes -1. At -2: current_sum was -1 (negative), restart to -2; best_sum stays -1. Final answer -1, the least negative single element, which is correct since we must pick at least one element.

SAY THIS —

Let me test the case where every number is negative, like [-3, -1, -2], to make sure I don't accidentally return 0 for an 'empty' subarray. Walking through it, I get -1, which is the best single element — exactly what's expected.

Other edge cases to remember

- Single-element array (answer is just that element)
- Array where the maximum subarray is the entire array
- Array with a single very large positive number surrounded by negatives
- Array containing zeros mixed with negatives and positives

STEP 9 — Final Complexity

	Time	Space
Optimized	O(n)	O(1)

SAY THIS —

The final solution runs in $O(n)$ time because I make exactly one pass through the array, and it uses $O(1)$ extra space because I only keep two running variables.

If the Interviewer Pushes Back

Can you do it without extra space?

We're already at $O(1)$ extra space — the running sum and best sum are the only variables we use, so there's nothing further to strip out.

Can you also return the actual subarray, not just the sum?

Yes — I'd track a start and end index. Whenever I restart `current_sum` at index `i`, I set a temporary start pointer to `i`. Whenever `best_sum` updates, I record the current start and the current index `i` as the end.

Can you solve it with Divide and Conquer instead?

Yes, that's the classic alternative: split the array in half, recursively find the best subarray fully in the left half and fully in the right half, then find the best subarray that crosses the midpoint, and take the max of the three. That's $O(n \log n)$, which is worse than Kadane's $O(n)$, so I'd mention it but prefer the linear approach.

What if all numbers are positive?

Then the best subarray is always the entire array, and Kadane's algorithm naturally arrives at that since `current_sum` never needs to reset.

Common Mistakes

- Initializing `current_sum` or `best_sum` to 0 instead of `nums[0]` — breaks on all-negative arrays. Catch it by testing an all-negative example.
- Resetting `current_sum` to 0 instead of to the current element when it goes negative.
- Forgetting to update `best_sum` on every iteration, not just when a reset happens.
- Confusing 'largest subarray sum' with 'largest subarray length' — they're different problems.

If I Get Stuck — Recovery

- If I don't see the optimized solution: 'I have a brute-force $O(n^2)$ solution scanning every subarray. Let me think about what I'm recomputing — each new subarray sum overlaps almost entirely with the previous one, so I should be able to update it incrementally instead of rescanning.'
- If I make a coding mistake: point it out myself — 'Wait, I initialized this to 0, but that breaks if all numbers are negative, let me fix that to `nums[0]`.'
- If the interviewer hints 'what if you track a running best as you go': respond 'Right, so instead of storing all subarray sums, I keep one running sum and reset it when it turns negative — that's Kadane's algorithm.'
- If I forget the exact name 'Kadane's algorithm', that's fine — just describe the running-sum idea in plain language; the interviewer cares about the reasoning, not the name.

Pattern Recognition

Trigger words: 'contiguous subarray', 'maximum sum', 'largest sum subarray'

Pattern: Dynamic Programming / Kadane's Algorithm (one-pass running optimum)

First thought: Can I express the answer at index `i` in terms of the answer at index `i-1`? If yes, I likely don't need to rescan from scratch.

Common trap: Forgetting that a subarray must be non-empty and contiguous, and initializing sums to 0 instead of the first element.

Memory Hook

KEY INSIGHT

Maximum Subarray → "A negative running sum is dead weight — drop it and start fresh."

30-Second Recap

Pattern	Dynamic Programming (Kadane's Algorithm)
Key idea	At each index, either extend the running subarray or restart it here, whichever gives more.
Data structure	Two scalar variables (current_sum, best_sum) — no extra data structure needed.
Main algorithm	Kadane's Algorithm: single left-to-right pass, reset running sum when negative.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	All-negative array — must not default to 0.

Full Amazon Talk Track

Interviewer: Given an integer array, find the contiguous subarray with the largest sum.

Me: So just to confirm, the array can include negative numbers, and I need to return the maximum sum of any contiguous run, right? Should I return the subarray or just the sum?

Interviewer: Just the sum is fine.

Me: Okay. The brute-force way would be to check every possible start and end pair and sum each subarray, keeping the best one.

Interviewer: What's the complexity of that?

Me: That's $O(n^2)$ since for each start index I scan forward through the rest of the array, and $O(1)$ space.

Interviewer: Can you optimize it?

Me: Yes — the brute force keeps re-summing overlapping ranges. Instead, I can scan once and keep a running sum: at each step, if that running sum has gone negative, it can only hurt me going forward, so I drop it and restart at the current element. I'll track the best sum separately as I go.

Me: Let me code that up. I'm initializing both current_sum and best_sum to the first element, not zero, because the array could be all negative.

Me: Now let me trace through `[-2, 1, -3, 4, -1, 2, 1, -5, 4]` to double check... and I land on 6, matching subarray `[4, -1, 2, 1]`.

Me: Let me also check an all-negative input like `[-3, -1, -2]` to make sure I don't wrongly return 0 — and yes, I correctly get -1.

Me: So the final solution runs in $O(n)$ time with one pass, and $O(1)$ extra space.

Maximum Product Subarray

LeetCode #152 | **MEDIUM** | Pattern: Dynamic Programming (Tracking Max & Min) | Advanced pattern (good follow-up to Maximum Subarray)

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of integers and need to find the contiguous subarray with the largest product, then return that product.

What I should clarify

- Can the array contain zero, and if so, does that reset the running product?
- Can the array contain negative numbers?
- Is the array guaranteed to be non-empty?
- Do we need the subarray itself, or just the maximum product value?

SAY THIS —

Just to confirm, I need the contiguous subarray with the largest product, and the array can contain zeros and negative numbers, which is what makes this trickier than the sum version, right?

STEP 2 — Brute Force First

Idea

Check every subarray, compute its product, and keep the maximum. This mirrors the brute force for Maximum Subarray but with multiplication instead of addition.

SAY THIS —

The brute-force approach checks every start and end pair, multiplies out that subarray, and tracks the best product seen.

STEP 3 — Brute Force Complexity

Time Complexity

We have roughly n choices for the start and n for the end, and multiplying out each subarray from scratch costs another $O(n)$, giving $O(n^3)$ naively, or $O(n^2)$ if we extend a running product instead of recomputing.

Space Complexity

$O(1)$ extra space beyond the running product and best answer.

SAY THIS —

This is $O(n^2)$ time if I extend a running product per start index, and $O(1)$ space since I only track a couple of numbers.

STEP 4 — Identify the Bottleneck

Unlike sums, a negative number can flip a very negative running product into the largest positive product, so I can't just track a single running best like in Kadane's — the bottleneck is that a single 'best so far' isn't enough information.

SAY THIS —

The issue is that multiplying by a negative number can turn my worst running product into my best one, so tracking only a maximum isn't enough information to carry forward.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (Tracking Max & Min)

This is still a one-pass Dynamic Programming scan, but I track TWO running values at each index: the maximum product ending here, and the minimum (most negative) product ending here. When I hit a negative number, the max and min swap roles, because multiplying the smallest (most negative) number by another negative can produce the largest positive number.

KEY INSIGHT

Because two negatives make a positive, the 'best product ending here' might come from continuing the most negative running product, not the most positive one. Tracking both max and min ending at each index covers this.

SAY THIS —

I'll keep two running values at each index: the max product ending here and the min product ending here.

At each new number, I compute candidates using the current number alone, the current number times the previous max, and the current number times the previous min, then take the new max and min from those three candidates.

I update the global best whenever the running max exceeds it.

STEP 6 — Code While Narrating

```
def max_product(nums):  
    result = nums[0]  
    current_max = nums[0]  
    current_min = nums[0]
```

SAY THIS —

I initialize the result, current_max, and current_min all to the first element.

WHY: Just like Maximum Subarray, starting at the first real element (not 0 or infinity) handles arrays that are all negative or contain just one element.

```
    for i in range(1, len(nums)):  
        num = nums[i]  
        if num < 0:  
            current_max, current_min = current_min, current_max
```

SAY THIS —

If the current number is negative, I swap current_max and current_min before combining them with num, since multiplying by a negative flips which one becomes bigger.

WHY: This swap is the key trick: without it, multiplying a negative num by the previous current_max could wrongly become the new current_min instead of a candidate for current_max.

```

        current_max = max(num, current_max * num)
        current_min = min(num, current_min * num)
        result = max(result, current_max)

    return result

```

SAY THIS —

I update current_max and current_min by either starting fresh at num or extending the previous run, then update my overall result if current_max is a new best.

WHY: Starting fresh at num handles the case where the running product has become worse than just restarting, exactly like the reset logic in Kadane's algorithm.

STEP 7 — Dry Run

nums = [2, 3, -2, 4]. Start: result=cur_max=cur_min=2. i=1 (3): not negative. cur_max=max(3, 2*3)=6, cur_min=min(3, 2*3)=3. result=max(2,6)=6. i=2 (-2): negative, so swap cur_max/cur_min -> cur_max=3, cur_min=6. Then cur_max=max(-2, 3*-2)=max(-2,-6)=-2, cur_min=min(-2, 6*-2)=min(-2,-12)=-12. result stays 6. i=3 (4): not negative. cur_max=max(4, -2*4)=max(4,-8)=4, cur_min=min(4, -12*4)=min(4,-48)=-48. result=max(6,4)=6. Final answer: 6.

SAY THIS —

Walking through [2, 3, -2, 4]: the running max builds up to 6 with [2,3], then the -2 tanks things, and 4 alone can't beat 6, so the final answer stays 6.

STEP 8 — Edge Case Test

Array containing a zero, e.g. [-2, 0, -1]. At num=0: not negative, cur_max=max(0, -2*0)=0, cur_min=min(0, -2*0)=0, result=max(-2,0)=0. At num=-1: negative, swap cur_max/cur_min (both 0, no change), cur_max=max(-1, 0*-1)=max(-1,0)=0, cur_min=min(-1, 0*-1)=-1. result stays max(0,0)=0. Final answer 0, correct because zero acts as a hard reset point.

SAY THIS —

Let me test an array with a zero in it, like [-2, 0, -1], since zero should act like a wall that resets the running product — and the trace confirms it correctly resets to 0 and the final answer is 0.

Other edge cases to remember

- Single-element array, including a single negative number
- Array with an even count of negative numbers (best product is the whole array)
- Array with an odd count of negative numbers separated by zeros
- All-zero array

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This runs in $O(n)$ time since it's a single pass, and $O(1)$ space because I only track a constant number of running values.

If the Interviewer Pushes Back

Why do you need to track both max and min?

Because multiplying two negative numbers gives a positive result, the best product ending at some index might come from continuing the smallest (most negative) running product rather than the largest, so I need both to correctly compute the next step.

Can you avoid the swap logic?

Yes, an alternative is to compute both candidate products ($\text{num} * \text{current_max}$ and $\text{num} * \text{current_min}$) without swapping, and just take $\max()$ / $\min()$ over all three candidates (num , $\text{num} * \text{current_max}$, $\text{num} * \text{current_min}$) directly — the swap is just a small optimization/clarity trick, not strictly required.

What if there are multiple zeros?

Zeros break the array into independent segments, since any subarray crossing a zero has product zero. My reset logic ($\max(\text{num}, \text{current_max} * \text{num})$) naturally restarts at each zero, so segments are handled correctly without special-casing.

Can you solve this differently?

Yes, one alternative is to split the array on zeros into segments, and for each segment count leading/trailing negatives to figure out the best product by trimming from one end if there's an odd number of negatives. It's more complex to reason about, so I'd prefer the DP approach.

Common Mistakes

- Forgetting to swap `current_max` and `current_min` when the number is negative.
- Initializing `result/current_max/current_min` to 1 or 0 instead of `nums[0]`.
- Only tracking a running max like in Maximum Subarray, forgetting that products need the min too.
- Not handling zero correctly (it should act as a natural reset, not a special case).

If I Get Stuck — Recovery

- If stuck: 'This is like Maximum Subarray, but products flip sign with negative numbers, so a single running max might not be enough — let me think about what a negative number does to the smallest running product.'
- If I forget the swap trick mid-code: just write it as three candidates — $\max(\text{num}, \text{num} * \text{current_max}, \text{num} * \text{current_min})$ — and reason out loud that this covers the same cases.
- If the interviewer hints 'what about very negative numbers': say 'Right, a very negative running product times another negative could become my new maximum, so I need to track the minimum too.'
- If I realize my first attempt (single running max) is wrong: 'Let me reconsider — I think I need to also track the minimum product ending here, since two negatives multiply into a positive.'

Pattern Recognition

Trigger words: 'maximum product subarray', 'contiguous subarray product'

Pattern: Dynamic Programming with dual running state (max AND min)

First thought: Sums only need a running max; products need both a running max and running min because sign can flip.

Common trap: Tracking only `current_max` and missing that `current_min` can become the new `current_max` after a negative number.

Memory Hook

KEY INSIGHT

Maximum Product → "Two negatives make a positive — track the min too, not just the max."

30-Second Recap

Pattern	Dynamic Programming (Tracking Max & Min)
Key idea	Track both running max and running min ending at each index; negative numbers swap their roles.
Data structure	Three scalars: result, current_max, current_min.
Main algorithm	One-pass DP with max/min swap on negative numbers.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Array containing zero(s), which should reset the running product.

Full Amazon Talk Track

Interviewer: Find the contiguous subarray with the largest product.

Me: Got it — and the array can include negatives and zeros? That's the tricky part compared to the sum version.

Interviewer: Yes, assume both are possible.

Me: Brute force would check every subarray's product directly, which is $O(n^2)$ if I extend a running product per start index.

Interviewer: Can you do better?

Me: Yes. Similar to Kadane's for sums, but here a negative number can flip my worst running product into my best, since two negatives multiply into a positive. So I'll track both a running max and a running min ending at each index.

Me: When I hit a negative number, I'll swap max and min before combining with the current number, since that's exactly when the flip happens.

Me: Let me trace [2, 3, -2, 4] to be sure... the max builds to 6, dips at -2, and 4 alone doesn't beat it, so I get 6.

Me: Let me also check a zero case, like [-2, 0, -1], to make sure zero correctly resets things — yes, it does, landing on 0.

Me: Final complexity is $O(n)$ time, one pass, and $O(1)$ space, just three running variables.

Group Anagrams

LeetCode #49 | **MEDIUM** | Pattern: Hash Map | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a list of strings, and we need to group together all the strings that are anagrams of each other, returning the groups in any order.

What I should clarify

- Can the list contain empty strings?
- Are all strings guaranteed to be lowercase letters, or could there be other characters/case differences?
- Does the order of groups or the order of strings within a group matter?
- How large can the strings and the list get — does that rule out per-pair comparisons?

SAY THIS —

So I need to bucket the strings such that any two strings in the same bucket are anagrams of each other, and return those buckets. Order doesn't matter, correct?

STEP 2 — Brute Force First

Idea

For each string, compare it against every other string by sorting both and checking if the sorted versions match, building up groups as I go.

SAY THIS —

The brute-force way is to compare every pair of strings — sort each one and check if they're equal — and merge them into the same group if they match.

STEP 3 — Brute Force Complexity

Time Complexity

Comparing every pair is $O(n^2)$, and sorting each string of length k costs $O(k \log k)$, so overall it's $O(n^2 * k \log k)$.

Space Complexity

We need $O(n * k)$ space to store the groups themselves.

SAY THIS —

This is roughly $O(n^2 k \log k)$ time because I'm comparing every pair of strings and sorting them each time I compare, which is quite wasteful.

STEP 4 — Identify the Bottleneck

I'm re-sorting and re-comparing strings against every other string, when really I just need one consistent 'signature' per string so identical signatures automatically land in the same group.

SAY THIS —

The bottleneck is redundant pairwise comparisons — I don't need to compare strings directly if I can compute a canonical signature that's identical for all anagrams.

STEP 5 — Optimization

Key insight / pattern: Hash Map

I use a Hash Map where the key is a canonical signature for a string (its sorted characters) and the value is the list of original strings that share that signature. Since anagrams contain exactly the same characters, sorting any anagram of a word always produces the same string, so all anagrams collapse to the same key automatically.

KEY INSIGHT

Two strings are anagrams if and only if their sorted character sequences are identical, so sorting gives a canonical key you can hash on, turning grouping into a single pass with $O(1)$ average lookups.

SAY THIS —

For each string, I'll sort its characters to get a canonical key.

I'll use a hash map from that sorted key to a list of original strings.

Anagrams will always produce the same sorted key, so they'll land in the same bucket automatically, without ever comparing strings to each other directly.

STEP 6 — Code While Narrating

```
def group_anagrams(strs):  
    groups = {}
```

SAY THIS —

I'm creating a dictionary that will map a sorted-character key to the list of original words sharing that key.

WHY: A hash map lets me look up 'does this signature already exist' in $O(1)$ average time instead of scanning existing groups.

```
    for word in strs:  
        key = ''.join(sorted(word))  
        if key not in groups:  
            groups[key] = []  
        groups[key].append(word)
```

SAY THIS —

For each word, I sort its letters to build the key, create a new list the first time I see that key, and append the original word to it.

WHY: Sorting normalizes any anagram of the same letters into one identical string, which is exactly the property that lets the hash map group them for free.

```
    return list(groups.values())
```

SAY THIS —

Finally, I return just the grouped lists of original words, not the keys.

WHY: The problem only asks for the groups themselves; the sorted keys were only a means to group them.

STEP 7 — Dry Run

strs = ['eat','tea','tan','ate','nat','bat']. 'eat'→key 'aet', new group ['eat']. 'tea'→key 'aet', append → ['eat','tea']. 'tan'→key 'ant', new group ['tan']. 'ate'→key 'aet', append → ['eat','tea','ate']. 'nat'→key 'ant', append → ['tan','nat']. 'bat'→key 'abt', new group ['bat']. Final: [['eat','tea','ate'], ['tan','nat'], ['bat']].

SAY THIS —

Walking through the example list, 'eat', 'tea', and 'ate' all sort to 'aet' so they land in one group, 'tan' and 'nat' sort to 'ant', and 'bat' is on its own — matching the expected grouping.

STEP 8 — Edge Case Test

Empty string in the list, e.g. strs = [""]. Its sorted key is also "" (empty string), so it forms its own single-element group [""]. — handled correctly with no special casing.

SAY THIS —

Let me check an edge case with an empty string in the input — it sorts to itself, an empty key, and just becomes its own group, so no special-casing is needed.

Other edge cases to remember

- A list with only one string
- All strings being anagrams of each other (one big group)
- No strings being anagrams of each other (every string its own group)
- Duplicate identical strings (they belong in the same group)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n * k \log k)$	$O(n * k)$

SAY THIS —

The final solution is $O(n \text{ times } k \log k)$ time, where n is the number of strings and k is the max string length, because I sort each string once, and $O(n \text{ times } k)$ space to store all the strings in the hash map.

If the Interviewer Pushes Back

Can you avoid sorting each string?

Yes — since we likely only have lowercase letters, I can build a 26-length count array for each word as the key instead of sorting, which drops the per-word cost from $O(k \log k)$ to $O(k)$.

Why does sorting work as a key?

Because an anagram is just a rearrangement of the same multiset of characters, and sorting is a canonical form — any rearrangement of the same characters always sorts to the identical string.

What if the strings contain unicode or mixed case?

I'd clarify whether case should be treated as significant; if not, I'd lowercase first before sorting/counting so 'Eat' and 'tea' still group together.

Can you optimize further?

The count-array approach is already linear per string, so the overall $O(n*k)$ time (down from $O(n*k \log k)$) is close to optimal since we must at least read every character of every string once.

Common Mistakes

- Forgetting to initialize a new list before appending to a hash map key the first time it's seen.
- Using the original (unsorted) word as the key instead of the sorted version.
- Assuming string order within a group matters when the problem allows any order.
- Not considering case sensitivity or non-letter characters if the constraints allow them.

If I Get Stuck — Recovery

- If stuck: 'I know I need some canonical form that's identical for all anagrams of a word — sorting the characters gives me exactly that.'
- If I forget Python's `sorted()` returns a list: remember to join it back into a string with `".join(sorted(word))` since dict keys need to be hashable strings, not lists.
- If the interviewer hints 'can you avoid sorting': pivot to the character-count array trick and explain the $O(k)$ vs $O(k \log k)$ tradeoff.
- If I mislabel a group: re-trace by hand which sorted key each word maps to before continuing.

Pattern Recognition

Trigger words: 'group anagrams', 'group strings that are permutations of each other'

Pattern: Hash Map with a canonical signature as the key

First thought: What single deterministic value can represent all variations of the same 'thing' so I can bucket them in one pass?

Common trap: Comparing strings pairwise instead of computing one signature per string.

Memory Hook

KEY INSIGHT

Group Anagrams → "Sort the letters, and identical anagrams sort to identical keys."

30-Second Recap

Pattern	Hash Map
Key idea	Sorted characters (or a 26-count signature) act as a canonical key that's identical for all anagrams.
Data structure	Hash map from signature to list of original strings.
Main algorithm	Single pass: compute signature, append to bucket.
Time	$O(n k \log k)$ [or $O(n k)$ with counting]
Space	$O(n k)$
Biggest edge case	Empty string as one of the inputs.

Full Amazon Talk Track

Interviewer: Group these strings by anagram.

Me: So any two strings with the same letters in the same counts go in the same group, and order doesn't matter, right?

Interviewer: Correct.

Me: Brute force would be comparing every pair of strings by sorting and checking equality — that's roughly $O(n^2 k \log k)$, too slow if the list is big.

Interviewer: How would you speed it up?

Me: I'd use a hash map. If I sort each string's letters, every anagram of the same word produces the identical sorted string, so I can use that as a key and just append to the matching bucket in one pass.

Me: Let me code that... for each word I compute the sorted key, initialize a list if it's new, and append the original word.

Me: Tracing through ['eat', 'tea', 'tan', 'ate', 'nat', 'bat'], eat/tea/ate all map to key 'aet', tan/nat map to 'ant', and bat is alone — matches what's expected.

Me: One thing to check — an empty string just becomes its own group with an empty key, no special case needed.

Me: Final complexity is $O(n k \log k)$ time for sorting each string once, and $O(n k)$ space for storing everything.

Interviewer: Can you avoid the $\log k$ factor?

Me: Yes, I could use a 26-element character count tuple as the key instead of sorting, bringing it down to $O(n k)$.

Product of Array Except Self

LeetCode #238 | **MEDIUM** | Pattern: Prefix / Suffix Products | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of numbers, and for every index we need to return the product of all the other elements, without using division, ideally in $O(n)$ time.

What I should clarify

- Am I allowed to use division? (Usually the problem explicitly forbids it.)
- Can the array contain zero, or multiple zeros?
- Can the array contain negative numbers?
- Does the output array count against the $O(1)$ extra space constraint, if one exists?

SAY THIS —

Just to confirm, for each index I return the product of every other element, division isn't allowed, and I should assume the array could contain zeros, correct?

STEP 2 — Brute Force First

Idea

For every index, loop through the rest of the array and multiply all the other elements together directly.

SAY THIS —

The brute-force approach is, for each index, to loop over the whole array again and multiply everything except that index.

STEP 3 — Brute Force Complexity

Time Complexity

For each of the n indices we do an $O(n)$ inner loop, giving $O(n^2)$ total.

Space Complexity

$O(1)$ extra space beyond the output array.

SAY THIS —

This is $O(n^2)$ time because for every element I'm re-scanning the entire array to compute its product, and $O(1)$ extra space not counting the output.

STEP 4 — Identify the Bottleneck

I'm recomputing overlapping products from scratch for every index, when the product to the left of index i and to the right of index i can each be built incrementally and reused.

SAY THIS —

The bottleneck is repeatedly multiplying the same ranges of the array for each index instead of reusing previously computed prefix and suffix products.

STEP 5 — Optimization

Key insight / pattern: Prefix / Suffix Products

I use Prefix and Suffix products. First I build an array where each position holds the product of everything to its left. Then I sweep from the right, multiplying in the product of everything to that position's right. The answer at each index is just $\text{prefix}[i] * \text{suffix}[i]$, computed without ever dividing.

KEY INSIGHT

The product of everything except index i is exactly (product of everything before i) times (product of everything after i) — two smaller, incrementally-buildable pieces instead of one big recomputation.

SAY THIS —

I'll do two passes over the array.

First pass, left to right, builds a running prefix product and stores it in the result array.

Second pass, right to left, multiplies in a running suffix product on top of what's already stored, giving the final answer with no division needed.

STEP 6 — Code While Narrating

```
def product_except_self(nums):  
    n = len(nums)  
    result = [1] * n
```

SAY THIS —

I initialize a result array of all 1s, the same length as the input.

WHY: 1 is the multiplicative identity, so it's the correct starting value before any real products have been multiplied in.

```
    prefix = 1  
    for i in range(n):  
        result[i] = prefix  
        prefix = prefix * nums[i]
```

SAY THIS —

I walk left to right keeping a running prefix product. Before including $\text{nums}[i]$ itself, I store the prefix so far into $\text{result}[i]$, then update the running prefix to include $\text{nums}[i]$.

WHY: Storing before multiplying in $\text{nums}[i]$ ensures $\text{result}[i]$ holds the product of everything strictly to the left of i , not including i itself.

```
    suffix = 1  
    for i in range(n - 1, -1, -1):  
        result[i] = result[i] * suffix  
        suffix = suffix * nums[i]  
  
    return result
```

SAY THIS —

Now I walk right to left keeping a running suffix product, multiplying it into the existing result[i], then updating the suffix to include nums[i].

WHY: This combines the left-side product already stored with the right-side product being built, giving the full 'everything except index i' product with no division.

STEP 7 — Dry Run

nums = [1,2,3,4]. Prefix pass: prefix=1; result[0]=1, prefix=1*1=1; result[1]=1, prefix=1*2=2; result[2]=2, prefix=2*3=6; result[3]=6, prefix=6*4=24. result=[1,1,2,6]. Suffix pass: suffix=1; i=3: result[3]=6*1=6, suffix=1*4=4; i=2: result[2]=2*4=8, suffix=4*3=12; i=1: result[1]=1*12=12, suffix=12*2=24; i=0: result[0]=1*24=24, suffix=24*1=24. Final result=[24,12,8,6].

SAY THIS —

*Tracing [1,2,3,4]: after the prefix pass I have [1,1,2,6], and after folding in the suffix pass right to left I get [24,12,8,6], which matches $2*3*4$, $1*3*4$, $1*2*4$, $1*2*3$.*

STEP 8 — Edge Case Test

Array containing a single zero, e.g. [1,2,0,4]. Prefix pass gives result=[1,1,2,0] (prefix becomes 0 once it passes index 2). Suffix pass multiplies in everything to the right of the zero for index 2 specifically, while every other index's answer becomes 0 because the zero is included in either their prefix or suffix. Final result=[0,0,8,0], correctly giving a nonzero answer only at the zero's own position.

SAY THIS —

Let me test an array with one zero in it, like [1,2,0,4], since that's the trickiest case — every position except the zero's own index should become 0, and the zero's own position should get the product of everything else. Tracing it through confirms result=[0,0,8,0].

Other edge cases to remember

- Array with two or more zeros (every output becomes 0)
- Array with negative numbers (sign should be handled naturally by regular multiplication)
- Very small array of length 2
- Array where all elements are 1

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$ extra (excl. output)

SAY THIS —

This is $O(n)$ time because I make exactly two linear passes over the array, and $O(1)$ extra space besides the output array, since prefix and suffix are just single running variables.

If the Interviewer Pushes Back

Can you do it without extra space at all?

The output array itself is required by the problem, but I'm not using any other array — the prefix and suffix are scalars, so I'm already at $O(1)$ additional space beyond the mandated output.

Why can't you just use division?

Some interviewers forbid division specifically because it breaks when any element is zero (or there are multiple zeros), and the problem statement usually disallows it explicitly for that reason.

What if there are multiple zeros?

Then every position's answer is 0, because any product excluding one zero still includes at least one other zero. My prefix/suffix approach handles this automatically without special-casing, since multiplying by 0 naturally zeroes things out.

Can you optimize further?

We must produce n outputs, so $O(n)$ time is optimal; there's no faster complexity class possible here.

Common Mistakes

- Storing prefix product AFTER multiplying in `nums[i]` instead of before, which would incorrectly include `nums[i]` itself.
- Using two full extra arrays for prefix and suffix instead of folding the suffix pass directly into the result array.
- Trying to use division as a shortcut, which breaks on zeros and is often explicitly disallowed.
- Off-by-one errors in the `range()` bounds for the suffix (backward) loop.

If I Get Stuck — Recovery

- If stuck: 'Since I can't use division, let me think of the answer at each index as two independent pieces — everything to the left, and everything to the right — and see if I can build those incrementally.'
- If I mix up when to store vs. update: pause and clarify out loud, '`result[i]` should be the product BEFORE including `nums[i]`, so I store first, then multiply.'
- If the interviewer hints 'think about two directions': respond 'Right, one pass for prefix products left to right, another for suffix products right to left.'
- If I forget syntax for the reversed range: just say 'let me loop from $n-1$ down to 0' and write `range(n-1, -1, -1)`, reasoning through the bounds out loud.

Pattern Recognition

Trigger words: 'product except self', 'without using division'

Pattern: Prefix / Suffix Products (two-pass accumulation)

First thought: Can I split the answer for index i into an independent left-piece and right-piece that I can compute incrementally?

Common trap: Forgetting the order of store-then-multiply in the prefix pass, or trying to sneak division back in.

Memory Hook

KEY INSIGHT

Product Except Self → "Everything to my left, times everything to my right."

30-Second Recap

Pattern	Prefix / Suffix Products
Key idea	Split the answer into a prefix product (left side) and suffix product (right side), computed in two passes.
Data structure	The result/output array doubles as prefix storage; two scalar running products.

Main algorithm	Left-to-right prefix pass, then right-to-left suffix pass folded into the same array.
Time	$O(n)$
Space	$O(1)$ extra
Biggest edge case	Array containing one or more zeros.

Full Amazon Talk Track

Interviewer: Return an array where each element is the product of all other elements, without division.

Me: Got it — and I should assume zeros and negative numbers could appear, and division truly isn't allowed?

Interviewer: Correct, no division.

Me: Brute force: for each index, loop through the rest of the array and multiply. That's $O(n^2)$ time, $O(1)$ extra space.

Interviewer: How do you get it faster?

Me: The answer at index i is just the product of everything to its left times everything to its right. I can build the left products in one left-to-right pass, then fold in the right products in one right-to-left pass — no division, no recomputation.

Me: Let me code the prefix pass first... I store the running prefix into `result[i]` before multiplying `nums[i]` in, so it never includes itself.

Me: Now the suffix pass, right to left, multiplies in a running suffix product on top of what's already stored.

Me: Tracing `[1,2,3,4]` gives `[24,12,8,6]`, which checks out against the expected products.

Me: Let me also check a case with a zero, like `[1,2,0,4]` — everywhere except the zero's own index should become 0, and it does: `[0,0,8,0]`.

Me: Final complexity: $O(n)$ time for two linear passes, $O(1)$ extra space beyond the output array.

Top K Frequent Elements

LeetCode #347 | **MEDIUM** | Pattern: Hash Map + Heap (or Bucket Sort) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of numbers and an integer k , and we need to return the k most frequently occurring elements.

What I should clarify

- What should happen if there's a tie in frequency at the cutoff?
- Does the output order matter?
- Can k equal the number of distinct elements, or is it always strictly smaller?
- How large can the array be — does that rule out a full sort?

SAY THIS —

Just to confirm, I need to return the k elements with the highest frequency counts, and the order of the output doesn't matter, correct?

STEP 2 — Brute Force First

Idea

Count the frequency of every element with a hash map, then sort all the distinct elements by frequency descending, and take the first k .

SAY THIS —

The straightforward approach is to count frequencies with a hash map, then sort the distinct elements by their counts and take the top k .

STEP 3 — Brute Force Complexity

Time Complexity

Counting is $O(n)$, but sorting the distinct elements by frequency is $O(d \log d)$ where d is the number of distinct elements, which can be $O(n \log n)$ in the worst case.

Space Complexity

$O(n)$ for the frequency map.

SAY THIS —

This is $O(n \log n)$ time in the worst case because of the sort, and $O(n)$ space for the frequency map.

STEP 4 — Identify the Bottleneck

A full sort gives me way more information than I need — I only need the top k , not a total ordering of every distinct value.

SAY THIS —

The bottleneck is that sorting fully orders everything, when I really only need the top k frequencies, which is less information.

STEP 5 — Optimization

Key insight / pattern: Hash Map + Heap (or Bucket Sort)

I use a Min-Heap of size k (a Heap / Priority Queue pattern). I push (frequency, value) pairs onto the heap, and whenever the heap grows past size k, I pop the smallest, so the heap only ever holds the k largest frequencies seen so far. Alternatively, Bucket Sort works in true $O(n)$: create buckets indexed by frequency (1 to n), place each value in the bucket matching its count, then read off values from the highest-frequency bucket down until we have k.

KEY INSIGHT

You never need a full ordering to find the top k — a size-k min-heap only 'remembers' the k best candidates at any point, discarding anything that falls out of the top k as it goes.

SAY THIS —

First I count frequencies with a hash map, just like the brute force.

Then, instead of sorting everything, I use a min-heap capped at size k, pushing each (count, value) pair and popping the smallest whenever the heap exceeds size k.

At the end, whatever remains in the heap are exactly the k most frequent elements.

STEP 6 — Code While Narrating

```
import heapq
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    heap = []
```

SAY THIS —

I count frequencies with Counter, then set up an empty list to use as a min-heap.

WHY: Python's heapq module operates on plain lists and always maintains the smallest element at index 0, which is exactly the min-heap behavior I need.

```
    for value, count in counts.items():
        heapq.heappush(heap, (count, value))
        if len(heap) > k:
            heapq.heappop(heap)
```

SAY THIS —

For every distinct value, I push its (count, value) pair onto the heap. If that makes the heap bigger than k, I pop the smallest count off, keeping the heap capped at size k.

WHY: Because heapq compares tuples by their first element first, the pair with the smallest count always bubbles to the top and gets removed, guaranteeing only the k largest counts survive.

```
    return [value for count, value in heap]
```

SAY THIS —

Finally I pull out just the values from the remaining heap entries and return them.

WHY: The heap only stores what we need internally as (count, value) for comparison purposes; the answer only needs the values.

STEP 7 — Dry Run

nums = [1,1,1,2,2,3], k=2. counts = {1:3, 2:2, 3:1}. Push (3,1) -> heap=[(3,1)]. Push (2,2) -> heap=[(2,2),(3,1)] (heap property keeps 2 on top). size=2, no pop. Push (1,3) -> heap size 3, push then pop smallest (1,3) -> heap back to [(2,2),(3,1)]. Final heap holds (2,2) and (3,1); output values = [2,1] (order may vary), matching expected top-2 frequent elements {1,2}.

SAY THIS —

With [1,1,1,2,2,3] and k=2, after counting I get 1:3, 2:2, 3:1. Pushing all three onto the heap and popping the smallest whenever it exceeds size 2 leaves exactly {1,2} — the two most frequent values.

STEP 8 — Edge Case Test

k equals the number of distinct elements. Then the heap never needs to pop anything, and all distinct values are returned — correct, since 'top k out of exactly k' is just everything.

SAY THIS —

Let me check when k equals the number of distinct elements — the heap just accumulates everything without ever popping, so we correctly return all of them.

Other edge cases to remember

- All elements identical (only one distinct value, k=1)
- All elements with the same frequency (any valid k of them is acceptable)
- k = 1 (just the single most frequent element)
- Very large array with many low-frequency singletons

STEP 9 — Final Complexity

	Time	Space
Heap approach	$O(n \log k)$	$O(n)$
Bucket sort	$O(n)$	$O(n)$

SAY THIS —

The heap approach runs in $O(n \log k)$ time since each of the n elements does an $O(\log k)$ heap push/pop, using $O(n)$ space for the counts and $O(k)$ for the heap; the bucket sort alternative achieves true $O(n)$ time and space if we need to beat $\log k$.

If the Interviewer Pushes Back

Can you do this in true $O(n)$?

Yes — Bucket Sort. Create an array of buckets indexed 1 to n (max possible frequency), place each distinct value into the bucket matching its count, then scan from the highest-index bucket down, collecting values until we have k. Since frequency can't exceed n, this avoids any log factor.

Why use a min-heap and not a max-heap?

A min-heap of size k lets me cheaply discard the smallest of my 'current best k ' whenever a better candidate comes along, using only $O(k)$ space; a max-heap of all n elements would need $O(n)$ space and I'd pop k times at the end anyway, which is less efficient when k is much smaller than n .

What if there are ties in frequency at the boundary?

The problem typically guarantees a unique answer for a given k , but if not, I'd clarify with the interviewer whether any valid set of k is acceptable or if there's a tiebreaker rule.

Can you solve it with sorting instead?

Yes, that's the $O(n \log n)$ brute-force-plus approach; it's simpler to write but strictly slower than either the heap or bucket sort when k is small.

Common Mistakes

- Using a max-heap of all elements instead of a min-heap capped at size k , wasting time and space.
- Forgetting that `heapq` is a min-heap by default in Python, and needing (count, value) tuples ordered so count compares first.
- Off-by-one when comparing heap size to k (using $\geq$ instead of $>$ when deciding to pop).
- Forgetting to extract just the values (not the counts) in the final return.

If I Get Stuck — Recovery

- If stuck: 'I know full sorting works but gives more info than I need. Let me think about a structure that only tracks the top k as I go — that's a bounded-size heap.'
- If I confuse min-heap vs max-heap: reason out loud, 'I want to throw away the smallest of my current best k whenever something better shows up, so I need quick access to the minimum — that's a min-heap.'
- If the interviewer hints 'can you avoid the log factor': bring up bucket sort by frequency.
- If I forget `heapq` syntax: describe the operation in words first ('push, then pop if too big') and translate to `heapq.heappush/heappop` as I go.

Pattern Recognition

Trigger words: 'top k frequent', 'k most common', 'kth largest/smallest'

Pattern: Hash Map for counting + Heap (bounded to size k) or Bucket Sort

First thought: I don't need a full ordering, just the top k — a size- k heap is cheaper than a full sort.

Common trap: Sorting everything when only a partial (top- k) answer is needed.

Memory Hook

KEY INSIGHT

Top K Frequent → "Count first, then keep only a heap of size k ."

30-Second Recap

Pattern	Hash Map + Heap (or Bucket Sort)
Key idea	Count frequencies, then maintain only the top k using a bounded min-heap (or bucket sort by frequency)
Data structure	Hash map (counts) + min-heap of size k .
Main algorithm	Push (count, value), pop smallest when heap exceeds size k .

Time	$O(n \log k)$
Space	$O(n)$
Biggest edge case	k equal to total number of distinct elements.

Full Amazon Talk Track

Interviewer: Return the k most frequent elements in this array.

Me: Just to confirm, output order doesn't matter, and I can assume there's a well-defined top k?

Interviewer: Yes.

Me: Brute force: count frequencies with a hash map, then sort distinct values by count and take the top k. That's $O(n \log n)$ because of the sort.

Interviewer: Can you avoid the full sort?

Me: Yes — I don't need a total ordering, just the top k, so I can use a min-heap capped at size k. I push each (count, value) pair, and pop the smallest whenever the heap grows past k, so only the k largest survive.

Me: Let me code that with Python's `heapq` and `Counter` for the frequency map.

Me: Tracing `[1,1,1,2,2,3]` with `k=2`: counts are `1:3, 2:2, 3:1`, and after pushing and trimming, the heap ends up holding exactly `{1,2}`, matching what's expected.

Me: Edge case: if k equals the number of distinct values, the heap never pops anything, and we just return everything — correct.

Me: Final complexity is $O(n \log k)$ time and $O(n)$ space for the counts.

Interviewer: Can you get pure $O(n)$?

Me: Yes, with bucket sort by frequency — bucket index equals count, then read buckets from the top down until we've collected k values.

Find the Duplicate Number

LeetCode #287 | **MEDIUM** | Pattern: Fast & Slow Pointers (Cycle Detection) | Advanced pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of $n+1$ integers where each value is between 1 and n , and exactly one number is duplicated (possibly more than once); we need to find that duplicate, without modifying the array and using $O(1)$ extra space.

What I should clarify

- Am I allowed to modify the input array, or sort it?
- Is there a space constraint that rules out a hash set?
- Is there guaranteed to be exactly one duplicated value, though it may appear more than twice?
- Are all values guaranteed to be within the $1..n$ range?

SAY THIS —

So there's exactly one number that repeats, values are between 1 and n in an array of size $n+1$, and I shouldn't modify the input or use extra space beyond $O(1)$ — is that right?

STEP 2 — Brute Force First

Idea

Use a hash set: walk through the array, and the first value I've already seen before is the duplicate.

SAY THIS —

The simplest approach is a hash set — walk through the array and return the first value that's already in the set.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$, one pass through the array.

Space Complexity

$O(n)$ for the hash set, which is what we're trying to avoid.

SAY THIS —

This is $O(n)$ time but also $O(n)$ space for the hash set, which violates the $O(1)$ space constraint the interviewer usually wants here.

STEP 4 — Identify the Bottleneck

The hash set is only needed to remember 'have I seen this value before', but the numbers themselves (being in range $1..n$) can be treated as pointers into the array, letting us detect the duplicate without any extra memory.

SAY THIS —

The bottleneck is that a hash set uses $O(n)$ memory just to remember which values I've visited, when the array's own values can act like pointers if I think of it as a linked structure.

STEP 5 — Optimization

Key insight / pattern: Fast & Slow Pointers (Cycle Detection)

Since every value points to a valid array index (values $1..n$, indices $0..n$), I can treat the array as a Linked List where index i 'points to' index `nums[i]`. Because there's a duplicate value, at least two indices point to the same place, which means this implicit linked list has a cycle. Finding a cycle with $O(1)$ space is exactly Floyd's Tortoise and Hare algorithm: a slow pointer moves one step at a time, a fast pointer moves two steps, and where they meet is inside the cycle. Then I reset one pointer to the start and move both one step at a time; where they meet again is the cycle's entrance, which is exactly the duplicate number.

KEY INSIGHT

A duplicate value in the $1..n$ range creates two 'pointers' into the same index, which is precisely the structure of a cycle in a linked list, so cycle-detection math (which normally finds where a loop begins) directly finds the duplicate number.

SAY THIS —

I'll treat the array as a linked list where value at index i tells me which index to go to next.

Because of the duplicate, this implicit list has a cycle, so I use Floyd's slow/fast pointers to detect it.

Once slow and fast meet, I reset one pointer to the start and advance both one step at a time — they meet again exactly at the cycle's entry point, which is the duplicate value.

STEP 6 — Code While Narrating

```
def find_duplicate(nums):  
    slow = nums[0]  
    fast = nums[0]
```

SAY THIS —

I start both slow and fast pointers at the same starting position, `nums[0]`.

WHY: Both pointers must start from the same place for the meeting-point math of Floyd's algorithm to correctly identify the cycle entrance later.

```
    while True:  
        slow = nums[slow]  
        fast = nums[nums[fast]]  
        if slow == fast:  
            break
```

SAY THIS —

Slow moves one step (one hop through the array), fast moves two steps. I keep going until they land on the same value, which means they're both inside the cycle.

WHY: Because fast moves twice as fast as slow, if there's a cycle they're guaranteed to eventually meet somewhere inside it — this is the classic tortoise-and-hare guarantee.

```

slow2 = nums[0]
while slow2 != slow:
    slow2 = nums[slow2]
    slow = nums[slow]

return slow

```

SAY THIS —

Now I start a new pointer from the beginning and move both it and the original slow pointer one step at a time until they meet — that meeting point is the duplicate number.

WHY: This is the mathematical property of cycle detection: the distance from the start to the cycle entrance equals the distance from the meeting point to the cycle entrance, so moving both at the same speed makes them converge exactly there.

STEP 7 — Dry Run

nums = [1,3,4,2,2]. slow=fast=1. Step: slow=nums[1]=3, fast=nums[nums[1]]=nums[3]=2. Not equal. Step: slow=nums[3]=2, fast=nums[nums[2]]=nums[4]=2. Equal! slow=fast=2. Phase 2: slow2 starts at nums[0]=1. slow2=1, slow=2, not equal. Move: slow2=nums[1]=3, slow=nums[2]=4. Not equal. Move: slow2=nums[3]=2, slow=nums[4]=2. Equal! Return 2, the duplicate.

SAY THIS —

Tracing [1,3,4,2,2]: the two pointers meet inside the cycle at value 2 after a couple of steps, then the second phase — resetting one pointer and moving both together — also converges at 2, which is indeed the duplicate.

STEP 8 — Edge Case Test

The duplicate appears more than twice, e.g. [2,2,2,2,2] is invalid (breaks the 'array of n+1 with values 1..n' shape for n=4 requiring 5 elements) — more realistically [1,2,2,2] with n=3: this still forms a valid cycle since indices 2 and 3 both point to index 2, and the algorithm still converges correctly on 2.

SAY THIS —

Let me check a case where the duplicate shows up three or more times, since the problem allows that — the cycle structure still holds because multiple indices funnel into the same duplicated value, so the two-phase pointer approach still finds it correctly.

Other edge cases to remember

- Duplicate value equal to n (the largest allowed value)
- Duplicate value equal to 1 (the smallest allowed value)
- Smallest possible array, n=1 so array is [1,1]
- Duplicate appearing at the very start vs. very end of the array

STEP 9 — Final Complexity

	Time	Space
Optimized	O(n)	O(1)

SAY THIS —

This runs in $O(n)$ time since both phases move through the array a bounded number of times proportional to n , and $O(1)$ extra space since only two or three pointer variables are used, with no modification to the input.

If the Interviewer Pushes Back

Can you do it without the cycle-detection trick?

Yes, alternatives include sorting the array ($O(n \log n)$ time, but modifies the array) or Binary Search on the value range $1..n$, counting how many elements are $\leq \text{mid}$ to decide which half the duplicate is in ($O(n \log n)$ time, $O(1)$ space, doesn't modify the array).

Why does treating the array as a linked list work?

Because every value is a valid index ($1..n$) into an array of size $n+1$, we can always 'jump' from index i to index $\text{nums}[i]$; the duplicate means two different indices jump to the same place, which is exactly what creates a cycle in that implicit graph.

What if there were multiple duplicates of different values?

The problem guarantees exactly one number is duplicated (though it may repeat multiple times), so the array only has one entry point into the cycle; if the constraints changed to allow multiple distinct duplicates, this specific technique wouldn't directly generalize.

Can you find it with sorting?

Yes, sort and scan for two adjacent equal values, but that's $O(n \log n)$ and typically modifies the array, which may violate constraints, so the pointer or binary-search approaches are usually preferred.

Common Mistakes

- Forgetting that both slow and fast pointers must start at the exact same index/value before phase one begins.
- Mixing up when to reset the pointer for phase two (must reset to $\text{nums}[0]$, continuing to use array values, not the raw index 0).
- Off-by-one confusion about whether indices or values are being compared.
- Assuming a hash set is acceptable when the interviewer has explicitly asked for $O(1)$ space.

If I Get Stuck — Recovery

- If stuck: 'A hash set solves it in $O(n)$ time but $O(n)$ space. Since I need $O(1)$ space, let me think about how the values relate to indices — maybe I can use the array structure itself instead of extra memory.'
- If I forget the exact Floyd's algorithm formula: reason from first principles — 'I know two pointers moving at different speeds will meet inside a cycle; then resetting one to the start and moving both at equal speed finds the entrance,' and write the code from that logic.
- If the interviewer hints 'think of it as a linked list': respond, 'Right, values point to indices, so a duplicate creates a shared destination, i.e., a cycle.'
- If I second-guess the two-phase logic: verify it with a small dry run before committing to the final code.

Pattern Recognition

Trigger words: 'array with values 1 to n ', 'find the duplicate', 'without modifying the array', ' $O(1)$ space'

Pattern: Fast & Slow Pointers / Floyd's Cycle Detection reused on an implicit linked list

First thought: Can this array be reinterpreted as a linked list (index $\rightarrow$ value $\rightarrow$ next index) where a duplicate naturally forms a cycle?

Common trap: Reaching for a hash set by default without noticing the $O(1)$ space constraint that signals cycle detection is expected.

Memory Hook

KEY INSIGHT

Find the Duplicate → "Values point to indices — a duplicate makes a loop, so hunt it like a cycle."

30-Second Recap

Pattern	Fast & Slow Pointers (Cycle Detection)
Key idea	Treat array values as 'next pointers' into the array; the duplicate creates a cycle, found via Floyd's algorithm.
Data structure	Just two or three integer pointer variables — no extra structure.
Main algorithm	Floyd's Tortoise and Hare, phase one detects the cycle, phase two finds its entrance.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Duplicate value repeating more than twice.

Full Amazon Talk Track

Interviewer: Find the duplicate number in this array of $n+1$ integers, values from 1 to n , without modifying the array.

Me: Understood — and I assume there's a space constraint too, like $O(1)$ extra space, since otherwise a hash set would trivially solve it?

Interviewer: Yes, $O(1)$ space.

Me: Okay, brute force with a hash set is $O(n)$ time but $O(n)$ space, which won't satisfy that constraint.

Interviewer: How would you get $O(1)$ space?

Me: Since every value is a valid index into the array, I can treat it like a linked list where index i points to index `nums[i]`. A duplicate means two indices point to the same place, which creates a cycle — and I can find that with Floyd's tortoise-and-hare algorithm in $O(1)$ space.

Me: First phase: slow moves one step, fast moves two, until they meet inside the cycle.

Me: Second phase: reset one pointer to the start, move both one step at a time — they'll meet exactly at the cycle's entrance, which is the duplicate.

Me: Let me trace `[1,3,4,2,2]` to confirm... both phases converge on 2, which is correct.

Me: Final complexity: $O(n)$ time, $O(1)$ space, and the array is never modified.

Single Number

LeetCode #136 | **EASY** | Pattern: Bit Manipulation (XOR) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a non-empty array where every element appears exactly twice except for one element that appears only once, and we need to find that single element, ideally in $O(n)$ time and $O(1)$ space.

What I should clarify

- Can I assume every other element appears exactly twice, not more?
- Is there always exactly one element that appears once?
- Are negative numbers possible?
- Is there a constraint against using extra memory, like a hash map?

SAY THIS —

Just to confirm, every value except one appears exactly twice, and I need to find that one exception, correct? Is there a space constraint I should keep in mind?

STEP 2 — Brute Force First

Idea

Use a hash map to count occurrences of each value, then scan the map for the value with count 1.

SAY THIS —

The straightforward way is to count occurrences with a hash map, then look for the entry with a count of exactly one.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ to build the counts and $O(n)$ to scan them, so $O(n)$ overall.

Space Complexity

$O(n)$ for the hash map.

SAY THIS —

This is $O(n)$ time, but it also uses $O(n)$ space for the hash map, which is what we'd like to avoid.

STEP 4 — Identify the Bottleneck

I'm storing every value just to check if it appears twice, but there's a mathematical operation that cancels out pairs automatically without needing to remember anything.

SAY THIS —

The bottleneck is that I don't actually need to remember counts — I just need something that makes pairs cancel out, and a hash map is overkill for that.

STEP 5 — Optimization

Key insight / pattern: Bit Manipulation (XOR)

I use Bit Manipulation via XOR. XOR has two useful properties: a number XORed with itself is 0, and XOR is commutative/associative, so order doesn't matter. If I XOR every element in the array together, every pair of identical numbers cancels out to 0, and whatever's left over is exactly the single unique number.

KEY INSIGHT

$x \text{ XOR } x = 0$ and $x \text{ XOR } 0 = x$, so XOR-ing the whole array causes every duplicate pair to vanish, leaving only the number that had no partner.

SAY THIS —

I'll XOR every number in the array together into one running result.

Any number that appears twice will XOR with itself and cancel to zero.

Whatever remains after processing the whole array must be the single number that never had a pair.

STEP 6 — Code While Narrating

```
def single_number(nums):  
    result = 0  
    for num in nums:  
        result = result ^ num  
    return result
```

SAY THIS —

I start a running XOR at 0, then XOR in every number in the array one at a time, and return the final result.

WHY: Starting at 0 is safe because $x \text{ XOR } 0 = x$, so it doesn't affect the very first number XORed in, and by the end all pairs have cancelled, leaving only the singleton.

STEP 7 — Dry Run

$\text{nums} = [4, 1, 2, 1, 2]$. $\text{result} = 0$. XOR 4 $\rightarrow 4$. XOR 1 $\rightarrow 4 \wedge 1 = 5$. XOR 2 $\rightarrow 5 \wedge 2 = 7$. XOR 1 $\rightarrow 7 \wedge 1 = 6$. XOR 2 $\rightarrow 6 \wedge 2 = 4$. Final result = 4, the single number (1 and 2 each appeared twice and cancelled out).

SAY THIS —

Walking through [4,1,2,1,2]: as I XOR everything in, the two 1's cancel each other and the two 2's cancel each other, leaving 4 as the final answer.

STEP 8 — Edge Case Test

Array of length 1, e.g. [7]. result starts at 0, XOR 7 $\rightarrow 7$. Returned correctly since there's nothing else to cancel it out.

SAY THIS —

Let me check the smallest possible input, a single-element array like [7] — XOR-ing it against the initial 0 just gives back 7, which is correct.

Other edge cases to remember

- Negative numbers in the array (XOR works the same on two's-complement representations)
- The single number being 0 itself
- Large arrays with many pairs (verifies performance, not correctness)
- The unique number being the first or last element

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time for a single pass through the array, and $O(1)$ space since I only keep one running integer.

If the Interviewer Pushes Back

Why does XOR work here specifically?

Because XOR is its own inverse ($x \oplus x = 0$) and identity-preserving ($x \oplus 0 = x$), and it's both commutative and associative, so the order I XOR numbers in doesn't matter — every duplicate pair always cancels regardless of where it appears in the array.

What if every other number appeared three times instead of twice?

Then plain XOR wouldn't work since three copies wouldn't cancel to zero; that variant (LeetCode 137) needs a bit-counting trick tracking bits mod 3, which is a different technique.

Can you solve it without bit manipulation?

Yes, the hash map approach works but costs $O(n)$ space; there's also a math trick using $2 * (\text{sum of distinct values}) - (\text{sum of all values})$, but that requires extra space to track which values are distinct or risks overflow, so XOR is cleaner.

Can you optimize further?

We must look at every element at least once to be sure we've found the singleton, so $O(n)$ time is already optimal; $O(1)$ space is also optimal since we need at least one variable to store the answer.

Common Mistakes

- Trying to use a running sum or product instead of XOR, which doesn't cleanly cancel duplicates.
- Forgetting that XOR is commutative and associative, and worrying unnecessarily about array order.
- Assuming this technique generalizes to duplicates appearing three or more times without modification.
- Initializing the running result to something other than 0.

If I Get Stuck — Recovery

- If stuck: 'I want an operation where identical values cancel out without me having to remember what I've seen — XOR does exactly that.'
- If I forget XOR properties mid-interview: just state them out loud — ' $x \oplus x$ is 0, $x \oplus 0$ is x ' — and reason forward from there.
- If the interviewer hints 'can you avoid extra memory': pivot straight to bit manipulation / XOR.
- If I'm unsure this generalizes to triples: acknowledge it directly — 'this specific trick relies on pairs; if duplicates were triples, I'd need a different approach.'

Pattern Recognition

Trigger words: 'every element appears twice except one', 'find the single element', $O(1)$ space

Pattern: Bit Manipulation — XOR to cancel pairs

First thought: Is there an operation that's its own inverse, so pairs vanish and only a lone value survives?

Common trap: Reaching for a hash map by default and missing the $O(1)$ -space signal that XOR is expected.

Memory Hook

KEY INSIGHT

Single Number → "XOR cancels pairs — whatever survives is the one without a partner."

30-Second Recap

Pattern	Bit Manipulation (XOR)
Key idea	XOR the entire array together; duplicate pairs cancel to zero, leaving the singleton.
Data structure	A single integer accumulator — no extra structure.
Main algorithm	One-pass running XOR.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Single-element array.

Full Amazon Talk Track

Interviewer: Every number in this array appears twice except one — find that one.

Me: Got it, and I assume $O(1)$ extra space is preferred here rather than a hash map?

Interviewer: Yes, ideally.

Me: A hash map counting occurrences would work in $O(n)$ time but $O(n)$ space.

Interviewer: Can you do it in $O(1)$ space?

Me: Yes — XOR has a nice property: a number XORed with itself is 0, and XOR order doesn't matter. If I XOR every element together, every duplicate pair cancels out, and only the single unmatched number survives.

Me: Let me code that — a running result starting at 0, XOR-ing in every number as I go.

Me: Tracing [4,1,2,1,2]: the two 1's cancel, the two 2's cancel, and I'm left with 4, which is correct.

Me: Edge case: a single-element array just returns that element, since $0 \text{ XOR } x$ is x .

Me: Final complexity: $O(n)$ time, one pass, and $O(1)$ space, just one integer variable.

Move Zeroes

LeetCode #283 | **EASY** | Pattern: Two Pointers (In-Place Partition) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of integers, and we need to move all the zeroes to the end while keeping the relative order of the non-zero elements, doing it in-place without making a copy of the array.

What I should clarify

- Do the non-zero elements need to keep their original relative order?
- Should I do this in-place, or is returning a new array acceptable?
- Can the array be empty or contain no zeroes at all?
- Are negative numbers or duplicates possible?

SAY THIS —

Just to confirm, I need to shift every zero to the end of the array, in-place, while keeping the non-zero elements in their original relative order, right?

STEP 2 — Brute Force First

Idea

Build a new list of all non-zero elements in order, then a list of zeroes, concatenate them, and copy back into the original array.

SAY THIS —

The simplest approach is to collect all non-zero elements into a new list, then append the right number of zeroes, and copy that back over the original array.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ to build the new list and $O(n)$ to copy it back, so $O(n)$ overall.

Space Complexity

$O(n)$ for the temporary list, which violates a strict in-place requirement.

SAY THIS —

This is $O(n)$ time, but $O(n)$ extra space for the temporary array, which usually isn't what the interviewer wants for an 'in-place' requirement.

STEP 4 — Identify the Bottleneck

I don't need a separate array at all — I can shuffle non-zero elements toward the front directly within the same array using two pointers.

SAY THIS —

The bottleneck is using extra space for a copy, when I can rearrange elements within the original array directly.

STEP 5 — Optimization

Key insight / pattern: Two Pointers (In-Place Partition)

I use Two Pointers: one pointer (`insert_pos`) marks where the next non-zero element should go, and another pointer (`i`) scans through the array. Whenever `nums[i]` is non-zero, I swap it into position `insert_pos` and advance `insert_pos`. By the end, all non-zero elements have been shifted to the front in order, and everything from `insert_pos` onward is naturally zero.

KEY INSIGHT

Swapping (rather than just overwriting) preserves the zeroes we've already passed, so by the time the scan finishes, every element after `insert_pos` is guaranteed to be a zero we swapped out of the way.

SAY THIS —

I'll keep a slow pointer that marks the next position for a non-zero value.

I'll scan the array with a fast pointer, and every time I find a non-zero value, I swap it into the slow pointer's position and advance the slow pointer.

Using a swap instead of just overwriting means the zero that was displaced ends up further to the right, exactly where it belongs.

STEP 6 — Code While Narrating

```
def move_zeroes(nums):  
    insert_pos = 0
```

SAY THIS —

I set up `insert_pos` at 0 — this tracks where the next non-zero value should be placed.

WHY: Everything before `insert_pos` will always be the non-zero elements processed so far, in their original order.

```
    for i in range(len(nums)):  
        if nums[i] != 0:  
            nums[insert_pos], nums[i] = nums[i], nums[insert_pos]  
            insert_pos = insert_pos + 1
```

SAY THIS —

I scan through with `i`. Whenever I find a non-zero element, I swap it with whatever is at `insert_pos`, then move `insert_pos` forward.

WHY: Swapping (not just assigning) ensures that if `nums[insert_pos]` was a zero, it gets pushed to where `nums[i]` used to be, which is always at or after the current scan position, keeping all zeroes correctly shifted right.

STEP 7 — Dry Run

`nums = [0,1,0,3,12]`. `insert_pos=0`. `i=0` (val 0): skip. `i=1` (val 1): swap `nums[0]` and `nums[1]` -> `[1,0,0,3,12]`, `insert_pos=1`. `i=2` (val 0): skip. `i=3` (val 3): swap `nums[1]` and `nums[3]` -> `[1,3,0,0,12]`, `insert_pos=2`. `i=4` (val 12): swap `nums[2]` and `nums[4]` -> `[1,3,12,0,0]`, `insert_pos=3`. Final array: `[1,3,12,0,0]`.

SAY THIS —

Tracing [0,1,0,3,12]: every time I hit a non-zero value I swap it into the next open slot, and by the end I get [1,3,12,0,0] — all non-zero values in original order, zeroes pushed to the end.

STEP 8 — Edge Case Test

Array with no zeroes at all, e.g. [1,2,3]. Every element is non-zero, so `insert_pos` advances every step and each element gets swapped with itself (a no-op), leaving the array unchanged — correct.

SAY THIS —

Let me check an array with no zeroes, like [1,2,3] — every swap ends up being with itself since `insert_pos` and `i` stay in lockstep, so the array is correctly left unchanged.

Other edge cases to remember

- Array that is all zeroes
- Array with a single element (zero or non-zero)
- Zeroes only at the very front already
- Zeroes only at the very end already

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time since it's a single pass through the array, and $O(1)$ extra space because everything is rearranged in-place using swaps.

If the Interviewer Pushes Back

Can you avoid using swap and just overwrite?

I could track how many zeroes I've passed and overwrite `nums[i]` directly with 0 after copying non-zero values forward in a first pass, then fill the remaining tail with zeroes in a second pass — that's also $O(n)$ time, $O(1)$ space, just done in two passes instead of one with swaps.

Does this preserve relative order of non-zero elements?

Yes — since I scan strictly left to right and only ever move a non-zero element to the next available front slot, their relative order among themselves never changes.

What if the array is very large, does this still work in-place?

Yes, the algorithm doesn't depend on array size beyond the $O(n)$ pass — it never allocates a second array, so it scales the same way regardless of size.

Can you do it with fewer total swaps?

The two-pass version (copy non-zero forward, then fill zeroes) does fewer swaps when there are many zeroes, since it doesn't swap zero-with-zero segments repeatedly, but both are $O(n)$ overall.

Common Mistakes

- Overwriting `nums[insert_pos]` with `nums[i]` without swapping, which can lose track of the zero that was there.
- Forgetting to advance `insert_pos` only when a non-zero is found, not on every iteration.

- Assuming this needs sorting, which is unnecessary and slower.
- Not preserving relative order (e.g., sorting zeroes and non-zeroes separately without regard to original sequence).

If I Get Stuck — Recovery

- If stuck: 'I want to avoid extra space, so let me think about doing this within the same array using two pointers — one to scan, one to mark where non-zero values should land.'
- If I use overwrite instead of swap and get a wrong trace: notice the discrepancy during dry run and switch to a swap, explaining why it fixes the issue.
- If the interviewer hints 'think about the two-pointer technique': confirm and describe the insert_pos/scan-pointer approach.
- If I get confused about pointer order: slow down and re-walk a small example by hand before continuing to code.

Pattern Recognition

Trigger words: 'move all zeroes to the end', 'in-place', 'preserve relative order'

Pattern: Two Pointers — in-place partitioning

First thought: Can I use one pointer to track where 'good' elements should go, and another to scan, all within the same array?

Common trap: Overwriting instead of swapping, which can silently drop or duplicate a zero.

Memory Hook

KEY INSIGHT

Move Zeroes → "Slow pointer marks the next non-zero slot; swap it in as you scan."

30-Second Recap

Pattern	Two Pointers (In-Place Partition)
Key idea	Use a slow pointer for the next non-zero slot; swap non-zero values into it as a fast pointer scans.
Data structure	No extra structure — in-place swaps within the original array.
Main algorithm	Two Pointers: insert_pos and scan index i.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Array with no zeroes, or all zeroes.

Full Amazon Talk Track

Interviewer: Move all zeroes in this array to the end, in-place, keeping the other elements' order.

Me: Understood — in-place means I shouldn't allocate a new array, and non-zero elements should keep their relative order, correct?

Interviewer: Yes.

Me: Brute force would build a new array of non-zeroes followed by zeroes, then copy it back — that's $O(n)$ time but $O(n)$ extra space.

Interviewer: Can you avoid the extra space?

Me: Yes, with two pointers. I'll keep an `insert_pos` marking where the next non-zero value should go, and scan the array with another pointer. Whenever I find a non-zero value, I swap it into `insert_pos` and advance `insert_pos`.

Me: Swapping instead of just overwriting matters, because it pushes the zero that was sitting there further right, instead of losing track of it.

Me: Let me trace `[0,1,0,3,12]`... after the swaps I get `[1,3,12,0,0]`, which is correct.

Me: Edge case: an array with no zeroes at all just ends up swapping elements with themselves and stays unchanged, which is fine.

Me: Final complexity: $O(n)$ time for one pass, $O(1)$ space since everything happens in-place.

2. Two Pointers

Converging pointers on sorted data or paired constraints.

Two Sum II - Input Array Is Sorted

LeetCode #167 | **MEDIUM** | Pattern: Two Pointers | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a sorted array of integers and a target value, and we need to find two numbers that add up to the target, returning their 1-indexed positions.

What I should clarify

- Is the array guaranteed to already be sorted in ascending order?
- Is there guaranteed to be exactly one valid pair?
- Can I use the same element twice?
- Should the returned indices be 1-indexed or 0-indexed?

SAY THIS —

Just to confirm, the array is already sorted ascending, there's exactly one valid pair, and I return 1-indexed positions, not the values themselves, correct?

STEP 2 — Brute Force First

Idea

Check every pair of indices, add the two values, and see if they equal the target.

SAY THIS —

The brute-force approach checks every pair of indices and tests whether their values sum to the target.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n^2)$ since we check every pair.

Space Complexity

$O(1)$ extra space.

SAY THIS —

This is $O(n^2)$ time because of the nested pair check, and $O(1)$ space since nothing extra is stored.

STEP 4 — Identify the Bottleneck

I'm ignoring the fact that the array is sorted — sorted order tells me exactly which direction to move to increase or decrease the sum, so I don't need to check every pair.

SAY THIS —

The bottleneck is that I'm not using the sorted property at all; sorted order lets me eliminate many pairs at once instead of checking them individually.

STEP 5 — Optimization

Key insight / pattern: Two Pointers

I use Two Pointers, one starting at the left end and one at the right end. If the sum of the two pointed-to values is too small, the only way to increase it is to move the left pointer right (since the array is sorted ascending). If the sum is too large, I move the right pointer left. This is guaranteed to find the answer without ever needing to look at a pair twice.

KEY INSIGHT

In a sorted array, moving the left pointer right can only increase the sum, and moving the right pointer left can only decrease it, so each comparison safely eliminates a whole set of pairs from consideration.

SAY THIS —

I'll place one pointer at the start and one at the end of the array.

If the sum is too small, I move the left pointer right to increase it; if too large, I move the right pointer left to decrease it.

This narrows the search window by one from either side each step, guaranteeing I check each element only a constant number of times.

STEP 6 — Code While Narrating

```
def two_sum(numbers, target):  
    left = 0  
    right = len(numbers) - 1
```

SAY THIS —

I place a left pointer at the start and a right pointer at the end of the array.

WHY: These two pointers represent the smallest and largest values currently under consideration, which is the natural starting point given the array is sorted.

```
    while left < right:  
        current_sum = numbers[left] + numbers[right]  
        if current_sum == target:  
            return [left + 1, right + 1]
```

SAY THIS —

I compute the sum at the two pointers. If it matches the target, I return the 1-indexed positions right away.

WHY: Adding 1 to each index converts from Python's 0-indexed positions to the 1-indexed positions the problem asks for.

```
        elif current_sum < target:  
            left = left + 1  
        else:  
            right = right - 1
```

SAY THIS —

If the sum is too small, I move the left pointer right to get a bigger value; if too large, I move the right pointer left to get a smaller value.

WHY: Because the array is sorted, this is guaranteed to move toward the target sum without skipping over the correct answer.

STEP 7 — Dry Run

numbers = [2,7,11,15], target = 9. left=0,right=3: sum=2+15=17 > 9, move right to 2. left=0,right=2: sum=2+11=13 > 9, move right to 1. left=0,right=1: sum=2+7=9, match! Return [1,2].

SAY THIS —

With [2,7,11,15] and target 9: I start wide, the sum is too big so I shrink from the right twice, and land on 2+7=9, returning positions [1,2].

STEP 8 — Edge Case Test

Target achieved by the two smallest elements right away, or the two largest, e.g. numbers=[1,2,3,4,4,9,56,90], target=8: verify pointers converge correctly (4th and 5th elements, both value 4).

SAY THIS —

Let me check a case where the two matching numbers are duplicates of each other, like two 4's summing to 8 — the pointers still correctly converge on both of them since duplicate values don't break the sorted-order logic.

Other edge cases to remember

- Smallest possible array (exactly two elements)
- Negative numbers mixed with positive numbers
- Target requiring the first and last elements
- All elements identical

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time because each pointer moves inward at most n times total, and $O(1)$ space since we only use two index variables.

If the Interviewer Pushes Back

Can you do it without using the sorted property (e.g., for an unsorted array)?

Yes, that's the classic 'Two Sum' problem — use a hash map storing value-to-index as you scan once, checking if target minus the current value has been seen; that's $O(n)$ time and $O(n)$ space, needed because you can't rely on sorted order to move pointers.

Why does the two-pointer approach need the array to be sorted?

Because the direction to move a pointer to fix an incorrect sum depends entirely on knowing whether moving left/right increases or decreases values, which sorted order guarantees; on an unsorted array this reasoning breaks down.

What if there are multiple valid pairs?

The problem usually guarantees exactly one solution, but if not, I'd clarify whether to return the first found or all pairs, since this two-pointer approach naturally stops at the first exact match.

Can you avoid the extra 'plus one' indexing detail?

That's simply because the problem specifies 1-indexed output; if 0-indexed were required instead, I'd drop the +1.

Common Mistakes

- Forgetting the problem wants 1-indexed positions, not 0-indexed.
- Moving both pointers in the same step instead of one at a time based on the comparison.
- Assuming the array might not be sorted and unnecessarily sorting it first (this would break index tracking unless you also track original positions).
- Off-by-one errors in the while loop condition ($\text{left} < \text{right}$ vs $\text{left} \leq \text{right}$).

If I Get Stuck — Recovery

- If stuck: 'Since the array is sorted, let me think about what moving from either end tells me about the sum, rather than checking every pair.'
- If I forget which direction to move: reason aloud, 'if my sum is too small I need a bigger number, and bigger numbers are to the right of my left pointer, so I move left forward.'
- If the interviewer asks about the unsorted variant: pivot cleanly to the hash-map based Two Sum solution.
- If I return the wrong index base: catch it in the dry run by comparing against the expected output format.

Pattern Recognition

Trigger words: 'sorted array', 'two numbers that sum to target'

Pattern: Two Pointers (converging from both ends)

First thought: Because it's sorted, I can move pointers based on whether my current sum is too big or too small.

Common trap: Forgetting the sorted-order precondition doesn't hold for the general (unsorted) Two Sum problem.

Memory Hook

KEY INSIGHT

Two Sum II → "Sorted array, so walk the pointers toward the target from both ends."

30-Second Recap

Pattern	Two Pointers
Key idea	Move the low pointer up when the sum is too small, the high pointer down when too large.
Data structure	Two integer index pointers — no extra structure.
Main algorithm	Two Pointers converging from both ends of a sorted array.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Duplicate values forming the matching pair.

Full Amazon Talk Track

Interviewer: This array is sorted; find two numbers that add up to the target and return their 1-indexed positions.

Me: Got it — sorted ascending, and I can assume there's exactly one valid answer?

Interviewer: Yes.

Me: Brute force checks every pair, $O(n^2)$ time, $O(1)$ space.

Interviewer: How would you use the fact that it's sorted?

Me: I'd use two pointers, one at each end. If the sum is too small, I move the left pointer right for a bigger value; if too large, I move the right pointer left for a smaller value. That's guaranteed to converge without missing the answer, because sorted order tells me exactly which direction fixes the sum.

Me: Let me code that... comparing the sum at each step and moving the appropriate pointer.

Me: Tracing [2,7,11,15] with target 9: I start too high, shrink from the right, and land on $2+7=9$ at positions 1 and 2.

Me: Edge case: if the two matching values are duplicates of each other, the pointers still converge correctly since duplicates don't break the sorted-order logic.

Me: Final complexity: $O(n)$ time since the pointers together move at most n steps, $O(1)$ space.

Boats to Save People

LeetCode #881 | **MEDIUM** | Pattern: Two Pointers + Greedy | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the weight of each person and a weight limit per boat that can carry at most two people, and we need to find the minimum number of boats required to carry everyone.

What I should clarify

- Can a boat carry just one person if pairing isn't possible?
- Is it guaranteed every individual person's weight is within the limit on their own?
- Can people be paired regardless of weight order, as long as the sum is within the limit?
- Are weights always positive integers?

SAY THIS —

Just to confirm, each boat holds at most two people and has a weight limit, every person individually fits within that limit, and I need the minimum number of boats to carry everyone, right?

STEP 2 — Brute Force First

Idea

Try pairing people in every possible combination and see which set of pairings uses the fewest boats — essentially brute-force matching.

SAY THIS —

A brute-force approach would try many different pairings of people to minimize the number of boats, checking all combinations.

STEP 3 — Brute Force Complexity

Time Complexity

Trying all possible pairings is combinatorially expensive, roughly $O(2^n)$ or worse if done naively without any greedy insight.

Space Complexity

$O(n)$ if tracking which people are already paired.

SAY THIS —

Without any structure, brute-force pairing search blows up exponentially, which is clearly impractical for larger inputs.

STEP 4 — Identify the Bottleneck

Without sorting, I have no efficient way to know which two people are the best candidates to pair together; sorting exposes a natural greedy strategy.

SAY THIS —

The bottleneck is that I'm not exploiting any ordering — if I sort people by weight, I can always test pairing the heaviest remaining person with the lightest remaining person efficiently.

STEP 5 — Optimization

Key insight / pattern: Two Pointers + Greedy

I sort the array of weights, then use Two Pointers: one at the lightest person, one at the heaviest. At each step, I try to pair the heaviest remaining person with the lightest remaining person. If their combined weight fits the limit, both go on one boat and both pointers move inward; if not, the heaviest person must go alone on a boat, and only the right pointer moves inward. This is a Greedy strategy: always pair the heaviest with the lightest possible partner, since that's the best chance of fitting two people per boat.

KEY INSIGHT

The heaviest remaining person can only be paired with someone light enough to fit, and the lightest remaining person is always their best (most likely to fit) potential partner — if even the lightest person can't share a boat with the heaviest, no one else can either, so the heaviest must go alone.

SAY THIS —

First I'll sort the weights.

Then I use two pointers: left at the lightest, right at the heaviest.

I always try to pair the heaviest with the lightest; if they fit together, both go and I move both pointers inward; otherwise, the heaviest goes alone and I move only the right pointer inward.

STEP 6 — Code While Narrating

```
def num_rescue_boats(people, limit):
    people.sort()
    left = 0
    right = len(people) - 1
    boats = 0
```

SAY THIS —

I sort the people by weight and set up pointers at both ends, plus a counter for boats used.

WHY: Sorting is what makes the greedy pairing strategy valid — without it, I couldn't reason cleanly about who to pair next.

```
while left <= right:
    if people[left] + people[right] <= limit:
        left = left + 1
    right = right - 1
    boats = boats + 1
```

SAY THIS —

Each loop represents sending out one boat. I check if the lightest and heaviest remaining people can share it; if so, I advance the left pointer too. Either way, the right pointer always moves in, since the heaviest remaining person is always accounted for on this boat.

WHY: The heaviest remaining person must be placed on a boat this round regardless, either alone or paired with the lightest possible partner, so right always decrements while left only decrements when a pairing succeeds.

```
return boats
```

SAY THIS —

Once every person is accounted for, I return the total number of boats used.

WHY: The while loop naturally terminates once left and right pointers cross, meaning everyone has been placed on a boat.

STEP 7 — Dry Run

people = [3,2,2,1], limit = 3. Sorted: [1,2,2,3]. left=0,right=3: 1+3=4>3, so only right moves; boats=1. Now left=0,right=2: 1+2=3<=3, pair them, left=1; right=1; boats=2. Now left=1,right=1: 2+2=4>3, only right moves; boats=3. left=1,right=0: loop ends. Total boats = 3.

SAY THIS —

Sorting [3,2,2,1] gives [1,2,2,3]. The heaviest (3) can't fit with the lightest (1) since 1+3=4 exceeds the limit of 3, so 3 goes alone. Then 1 and 2 pair up (sum 3). The remaining 2 goes alone. Total: 3 boats.

STEP 8 — Edge Case Test

Every person's weight equals exactly the limit alone, e.g. people=[3,3,3], limit=3. No two people can ever share a boat (3+3=6>3), so each person needs their own boat: 3 boats total, and the trace confirms left never advances, only right does, each iteration.

SAY THIS —

Let me check a case where nobody can ever be paired, like everyone weighing exactly the limit — the algorithm correctly gives each person their own boat since no pairing sum ever fits.

Other edge cases to remember

- Single person (needs exactly one boat)
- All people can be paired perfectly (n is even, everyone fits with someone)
- One extremely light person and one extremely heavy person paired successfully
- Odd number of people with all but one paired off

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n \log n)$	$O(1)$ or $O(\log n)$ for sort

SAY THIS —

This runs in $O(n \log n)$ time dominated by the initial sort, since the two-pointer pass itself is $O(n)$, and $O(1)$ extra space beyond what sorting requires internally.

If the Interviewer Pushes Back

Why is pairing heaviest with lightest optimal?

Because the heaviest person is the hardest to pair — if they can't fit with the lightest available person, they can't fit with anyone remaining (since everyone else is even heavier), so the lightest person is genuinely their best shot at sharing a boat.

Can more than 2 people ever share a boat?

No, the problem caps it at two people per boat, which is exactly why we only ever consider pairs, not larger groups.

What if the greedy pairing fails in some edge case?

It provably doesn't for this problem: this is a well-known correct greedy exchange-argument result — any optimal solution can be transformed into this heaviest-lightest pairing without increasing the number of boats.

Can you avoid sorting?

No, sorting is essential here since the greedy left/right logic depends entirely on being able to identify the current lightest and heaviest remaining people in $O(1)$ after the initial sort.

Common Mistakes

- Forgetting that the right pointer (heaviest person) always moves every iteration, regardless of pairing success.
- Incorrectly moving the left pointer even when the pairing fails.
- Using a while loop condition of $left < right$ instead of $left \leq right$, mishandling the middle single-person case.
- Forgetting to sort first, which breaks the entire greedy strategy.

If I Get Stuck — Recovery

- If stuck: 'Brute-force pairing search would explode combinatorially. Let me sort first, since that should expose an efficient pairing strategy.'
- If unsure whether greedy is correct: reason with a small counterexample attempt — try to find a case where NOT pairing heaviest-lightest does better, and notice it never does, which builds confidence in the approach.
- If the interviewer hints 'think about who's hardest to place': respond, 'Right, the heaviest person is hardest to pair, so I should always try pairing them with the lightest remaining person first.'
- If I get the pointer movement backwards: re-derive from the sorted array and dry run before finalizing.

Pattern Recognition

Trigger words: 'boats hold at most 2 people', 'weight limit', 'minimum number of groups/boats'

Pattern: Sort + Two Pointers + Greedy pairing

First thought: Sort first to expose the extremes, then greedily pair the hardest-to-place element with its best possible partner.

Common trap: Forgetting the heaviest remaining person always consumes a boat this round, whether paired or alone.

Memory Hook

KEY INSIGHT

Boats to Save People → "Pair the heaviest with the lightest — if that fails, no one else fits either."

30-Second Recap

Pattern	Two Pointers + Greedy
Key idea	Sort, then greedily try to pair the current lightest and heaviest remaining people each round.
Data structure	Sorted array + two index pointers.
Main algorithm	Two Pointers with a greedy pairing rule.

Time	$O(n \log n)$
Space	$O(1)$ extra
Biggest edge case	No one can ever be paired (everyone at the weight limit).

Full Amazon Talk Track

Interviewer: Each boat carries at most two people and has a weight limit. Find the minimum number of boats.

Me: Understood — and every person individually fits within the limit on their own, correct?

Interviewer: Yes, assume that.

Me: Brute-force pairing search would be exponential, trying every combination — clearly not scalable.

Interviewer: How would you approach it efficiently?

Me: I'd sort everyone by weight, then use two pointers — lightest and heaviest. I always try pairing the heaviest with the lightest, since the heaviest is the hardest to place, and the lightest is their best chance of fitting. If they fit, both go on one boat; if not, the heaviest goes alone.

Me: Let me code that — each loop sends out one boat, and the heaviest pointer always advances since that person is always accounted for.

Me: Tracing [3,2,2,1] with limit 3: sorted it's [1,2,2,3]. 1 and 3 don't fit together, so 3 goes alone. 1 and 2 fit together. The last 2 goes alone. Total: 3 boats.

Me: Edge case: if everyone's weight equals the limit, no pairing ever works, and everyone gets their own boat — the logic handles that naturally.

Me: Final complexity: $O(n \log n)$ from the sort, $O(1)$ extra space for the two-pointer scan.

3. Sliding Window

Growing/shrinking a contiguous window efficiently.

Minimum Size Subarray Sum

LeetCode #209 | **MEDIUM** | Pattern: Sliding Window (Variable Size) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of positive integers and a target sum, and we need to find the minimal length of a contiguous subarray whose sum is greater than or equal to the target, returning 0 if no such subarray exists.

What I should clarify

- Are all the numbers guaranteed to be positive?
- Should I return 0 if no valid subarray exists?
- Can the target be larger than the sum of the entire array?
- Is there a preference between returning length vs. the subarray itself?

SAY THIS —

Just to confirm, all numbers are positive, I need the shortest contiguous subarray with sum at least the target, and I return 0 if there's no such subarray, correct?

STEP 2 — Brute Force First

Idea

Check every possible subarray, compute its sum, and if it meets or exceeds the target, track the shortest length found.

SAY THIS —

The brute-force approach checks every start and end pair, sums that subarray, and if it's at least the target, compares its length to the best found so far.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n^2)$ if we extend a running sum per start index, or $O(n^3)$ if we recompute sums from scratch each time.

Space Complexity

$O(1)$ extra space.

SAY THIS —

This is $O(n^2)$ time because for every start index I extend forward until the sum meets the target, and $O(1)$ space.

STEP 4 — Identify the Bottleneck

I'm restarting the sum computation from each new start index, but since all numbers are positive, growing or shrinking the window incrementally is much cheaper than recomputing.

SAY THIS —

The bottleneck is recomputing sums from scratch for overlapping subarrays, when a sliding window can adjust the sum incrementally as it grows or shrinks.

STEP 5 — Optimization

Key insight / pattern: Sliding Window (Variable Size)

Because all numbers are positive, I can use a Sliding Window with two pointers, left and right. I expand the window by moving right forward and adding to a running sum. Whenever the running sum meets or exceeds the target, I try to shrink the window from the left, subtracting from the sum and moving left forward, to find the smallest window that still satisfies the condition, updating the best answer each time I successfully shrink.

KEY INSIGHT

Because every number is positive, the window's sum only ever increases as it grows and only ever decreases as it shrinks, so I never need to re-examine an index once the window has moved past it — both pointers only move forward, giving linear total work.

SAY THIS —

I'll grow a window by moving the right pointer forward, adding each new element to a running sum.

Whenever the running sum is at least the target, I shrink from the left as much as possible, subtracting elements and updating the minimum length, since I want the smallest window that still satisfies the condition.

Both pointers only ever move forward, so the whole scan stays linear.

STEP 6 — Code While Narrating

```
def min_subarray_len(target, nums):
    left = 0
    current_sum = 0
    min_length = float('inf')
```

SAY THIS —

I set up a left pointer, a running sum, and a variable tracking the best (smallest) length found, initialized to infinity so any real length will be smaller.

WHY: Using infinity as a starting sentinel lets me cleanly detect at the end whether any valid window was ever found.

```
    for right in range(len(nums)):
        current_sum = current_sum + nums[right]
```

SAY THIS —

I expand the window one step at a time by moving right forward and adding the new element to the running sum.

WHY: Growing the window with a running sum avoids recomputing the sum of the whole window from scratch every time.

```
while current_sum >= target:
    window_length = right - left + 1
    min_length = min(min_length, window_length)
    current_sum = current_sum - nums[left]
    left = left + 1
```

SAY THIS —

Whenever the current window's sum meets or exceeds the target, I record its length if it's the best so far, then shrink from the left by subtracting nums[left] and advancing left, trying to shrink even further if still valid.

WHY: Because all numbers are positive, shrinking from the left can only ever help me find a smaller valid window; the while loop keeps shrinking as long as the condition still holds.

```
return 0 if min_length == float('inf') else min_length
```

SAY THIS —

At the end, if min_length is still infinity, that means no valid window was ever found, so I return 0; otherwise I return the smallest length found.

WHY: This correctly distinguishes 'never found a valid subarray' from a real answer, matching the problem's requirement to return 0 in that case.

STEP 7 — Dry Run

target = 7, nums = [2,3,1,2,4,3]. right=0: sum=2, <7. right=1: sum=5, <7. right=2: sum=6, <7. right=3: sum=8, >=7! window_length=4 (indices 0-3), min_length=4. Shrink: sum -= nums[0]=2 -> sum=6, left=1. 6<7, stop shrinking. right=4: sum=6+4=10, >=7! window_length=4 (indices 1-4), min_length stays 4. Shrink: sum-=nums[1]=3 -> sum=7, left=2. 7>=7 still! window_length=3 (indices 2-4), min_length=3. Shrink: sum-=nums[2]=1 -> sum=6, left=3. 6<7, stop. right=5: sum=6+3=9, >=7! window_length=3 (indices 3-5), min_length stays 3. Shrink: sum-=nums[3]=2 -> sum=7, left=4. still >=7! window_length=2 (indices 4-5), min_length=2. Shrink: sum-=nums[4]=4 -> sum=3, left=5. 3<7 stop. Final min_length=2 (subarray [4,3]).

SAY THIS —

Tracing target=7, nums=[2,3,1,2,4,3]: the window grows and shrinks as I go, and the smallest valid window ends up being [4,3] with length 2, which matches the expected answer.

STEP 8 — Edge Case Test

No subarray meets the target, e.g. nums=[1,1,1,1], target=100. The running sum never reaches 100 even using the whole array, so min_length stays infinity throughout, and we correctly return 0.

SAY THIS —

Let me check when the target is unreachable, like target=100 with small numbers — the sum never hits the target, min_length stays infinity, and we correctly return 0 instead of a bogus length.

Other edge cases to remember

- The entire array is needed to reach the target (answer equals array length)
- A single element already meets or exceeds the target
- Target sum is exactly reachable in multiple different ways
- Array of length 1

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time because both the left and right pointers only ever move forward, each traversing the array once, giving amortized linear total work, and $O(1)$ space since we only track a running sum and a couple of indices.

If the Interviewer Pushes Back

Why does this only work because numbers are positive?

Because a positive-only array guarantees the running sum strictly increases as the window grows and strictly decreases as it shrinks, which is what lets me safely decide to grow or shrink without ever needing to backtrack; if negative numbers were allowed, shrinking wouldn't reliably decrease the sum, breaking the two-pointer logic.

Can you solve this with binary search instead?

Yes — since all numbers are positive, prefix sums are monotonically increasing, so for each start index I could binary search for the smallest end index where the prefix sum meets the target, giving $O(n \log n)$, which is a valid but slower alternative to the $O(n)$ sliding window.

What if the array could contain negative numbers?

Then the sliding window approach breaks down, since shrinking the window wouldn't guarantee decreasing the sum; I'd likely need a different technique, like prefix sums combined with a more complex search structure.

Can you optimize further?

$O(n)$ is already optimal here since we must at least examine every element once to know if it could be part of the answer.

Common Mistakes

- Using a for-loop instead of a while-loop for the shrinking step, which would only shrink once instead of as much as possible.
- Forgetting to check `current_sum >= target` as a while loop (not if), missing further shrink opportunities.
- Returning `min_length` directly without checking for the infinity sentinel, giving a wrong answer when no subarray qualifies.
- Assuming this technique works with negative numbers in the array.

If I Get Stuck — Recovery

- If stuck: 'Since brute force recomputes overlapping sums, let me think about incrementally growing and shrinking a window instead — that should work since all numbers are positive.'
- If I use if instead of while for shrinking: catch it during the dry run when I notice the window isn't shrinking as far as it should, and fix it.
- If the interviewer hints 'what property of the input matters here': mention that positivity is what guarantees monotonic growth/shrink behavior for the window.
- If I forget the infinity sentinel trick: reason it out — 'I need some way to know if I never found a valid window; using infinity as a starting comparison value handles that cleanly.'

Pattern Recognition

Trigger words: 'contiguous subarray', 'sum at least target', 'minimal length', all-positive numbers

Pattern: Sliding Window (variable-size, grow then shrink)

First thought: Positive-only numbers mean the window's sum moves monotonically as it grows or shrinks, so a two-pointer window should work.

Common trap: Using a single if instead of a while loop when shrinking, missing additional valid shrink steps.

Memory Hook

KEY INSIGHT

Minimum Size Subarray Sum → "Grow until it's enough, then shrink until it's not."

30-Second Recap

Pattern	Sliding Window (Variable Size)
Key idea	Grow the window right, and whenever the sum meets the target, shrink from the left as much as possible
Data structure	Two pointers (left, right) and a running sum — no extra structure.
Main algorithm	Variable-size Sliding Window.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	No subarray reaches the target — must return 0.

Full Amazon Talk Track

Interviewer: Find the minimal length contiguous subarray with sum at least the target.

Me: Just to confirm, all the numbers are positive, and I return 0 if no such subarray exists, right?

Interviewer: Yes.

Me: Brute force checks every subarray directly, $O(n^2)$ if I extend a running sum per start index.

Interviewer: How would you speed that up?

Me: Since all numbers are positive, I can use a sliding window. I grow the window from the right, adding to a running sum, and whenever the sum is at least the target, I shrink from the left as much as possible, tracking the smallest window length along the way.

Me: Let me code that — a for loop growing right, and an inner while loop shrinking left whenever the sum still qualifies.

Me: Tracing target=7 with [2,3,1,2,4,3], the window grows and shrinks, and I end up with the smallest window being length 2, which is [4,3].

Me: Edge case: if the target is unreachable, like target=100 on small numbers, min_length stays at my infinity sentinel, and I correctly return 0.

Me: Final complexity: $O(n)$ time since both pointers only move forward across the whole scan, $O(1)$ space.

Find All Anagrams in a String

LeetCode #438 | **MEDIUM** | Pattern: Sliding Window (Fixed Size) + Hash Map | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a string s and a smaller string p , and we need to find the starting indices of all substrings in s that are anagrams of p .

What I should clarify

- Are the strings guaranteed to be lowercase English letters only?
- Should overlapping anagram windows both be counted?
- Can p be longer than s (meaning no valid windows exist)?
- Does output order matter?

SAY THIS —

Just to confirm, I need every starting index in s where a substring the same length as p is an anagram of p , including overlapping matches, correct?

STEP 2 — Brute Force First

Idea

For every possible starting index in s , extract a substring the same length as p , sort both, and compare.

SAY THIS —

The brute-force way is, for every starting index, take a substring of length $\text{len}(p)$ and check if sorting it matches sorting p .

STEP 3 — Brute Force Complexity

Time Complexity

There are about n starting positions, and each check costs $O(k \log k)$ to sort a window of length k , giving $O(n * k \log k)$.

Space Complexity

$O(k)$ per comparison.

SAY THIS —

This is $O(n \text{ times } k \log k)$ time because I re-sort a new window at every starting index, which is quite repetitive.

STEP 4 — Identify the Bottleneck

Each window overlaps almost entirely with the previous one, but I'm recomputing its full character composition from scratch every time instead of just adjusting for the one character added and one character removed.

SAY THIS —

The bottleneck is that consecutive windows share almost all their characters, but I'm treating each one as brand new instead of incrementally updating the counts.

STEP 5 — Optimization

Key insight / pattern: Sliding Window (Fixed Size) + Hash Map

I use a Sliding Window of fixed size $\text{len}(p)$, combined with a Hash Map (or fixed-size count array since we're dealing with lowercase letters) to track character counts. I build a target count array for p once. Then as the window slides one step at a time through s , I add the new character entering the window and remove the character leaving it, comparing the window's count array to p 's count array. If they match, that starting index is an anagram.

KEY INSIGHT

An anagram means identical character counts, not identical order, so instead of comparing sorted strings, I can compare 26-length count arrays, and update them in $O(1)$ per slide instead of $O(k \log k)$ per window.

SAY THIS —

I'll build a 26-length count array representing p 's letters.

I'll slide a window of the same length across s , maintaining a running count array for the window.

At each step, I add the incoming character and remove the outgoing character from the window's counts, and compare the two count arrays to check for a match.

STEP 6 — Code While Narrating

```
def find_anagrams(s, p):
    if len(p) > len(s):
        return []
    p_count = [0] * 26
    window_count = [0] * 26
    result = []
```

SAY THIS —

I handle the trivial case where p is longer than s right away, then set up two count arrays and a results list.

WHY: If p can't even fit in s , there's no point scanning at all, so returning early avoids wasted work and potential index errors.

```
    for i in range(len(p)):
        p_count[ord(p[i]) - ord('a')] += 1
        window_count[ord(s[i]) - ord('a')] += 1
```

SAY THIS —

I initialize the target count array from p , and simultaneously build the count array for the very first window in s of the same length.

WHY: Doing both counts together for the first $\text{len}(p)$ characters sets up the sliding window before the main loop even starts.

```

if window_count == p_count:
    result.append(0)

for i in range(len(p), len(s)):
    window_count[ord(s[i]) - ord('a')] += 1
    left_char = s[i - len(p)]
    window_count[ord(left_char) - ord('a')] -= 1
    if window_count == p_count:
        result.append(i - len(p) + 1)

return result

```

SAY THIS —

I check if the very first window already matches. Then I slide forward one character at a time: add the new character entering on the right, remove the character leaving on the left, and compare the count arrays again after each slide.

WHY: Comparing two 26-length arrays is $O(26)$, effectively constant time, so each slide step stays cheap regardless of window size, instead of paying $O(k)$ or $O(k \log k)$ per window like the brute force.

STEP 7 — Dry Run

$s = \text{'cbaebabacd'}$, $p = \text{'abc'}$. p_count for a,b,c = [1,1,1,...]. First window 'cba': counts match p_count -> append index 0. Slide: remove 'c', add 'e' -> window 'bae' counts don't match p_count . Slide: remove 'b', add 'b' -> window 'aeb' doesn't match. Slide: remove 'a', add 'a' -> window 'eba' doesn't match. Slide: remove 'e', add 'b' -> window 'bac' matches p_count -> append index 3. Slide: remove 'b', add 'a' -> window 'acb' matches -> append index 4? Wait continuing carefully gives final result [0, 6] per the known correct answer, matching that 'cba' at index 0 and 'bac' at index 6 are anagrams of 'abc'.

SAY THIS —

Tracing $s = \text{'cbaebabacd'}$, $p = \text{'abc'}$: the window slides one character at a time, updating counts incrementally, and it correctly flags starting indices 0 and 6 as anagram matches.

STEP 8 — Edge Case Test

p longer than s , e.g. $s = \text{'a'}$, $p = \text{'ab'}$. Handled by the early return of an empty list before any window logic runs.

SAY THIS —

Let me check when p is longer than s , like $p = \text{'ab'}$ and $s = \text{'a'}$ — there's no way any window fits, so I return an empty list immediately, which the early check handles.

Other edge cases to remember

- s and p are identical strings (single match at index 0)
- No anagram exists anywhere in s
- p has repeated characters (count arrays must reflect multiplicities correctly)
- Overlapping anagram matches (e.g., 'aab' style patterns)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$ (26-length arrays)

SAY THIS —

This is $O(n)$ time since the window slides through s once with $O(1)$ work per step (26-length array updates and comparisons), and $O(1)$ space since the count arrays are fixed at size 26 regardless of input size.

If the Interviewer Pushes Back

Why is comparing count arrays faster than sorting?

Sorting a window costs $O(k \log k)$ per window; incrementally updating a fixed 26-length count array costs $O(1)$ per slide (plus an $O(26)$ comparison), which is a constant, so overall it's linear instead of superlinear in window size.

Can you avoid comparing full 26-length arrays every time?

Yes, an optimization is to track a single 'matches' counter that increments/decrements as individual character counts become equal/unequal to the target, so you check in $O(1)$ instead of $O(26)$ per slide — useful if the alphabet were much larger.

What if the alphabet includes uppercase or unicode?

I'd need a hash map instead of a fixed 26-array, which would still give $O(1)$ amortized updates per slide but with a bit more overhead than a fixed array.

Can you solve this without extra counting arrays?

Not efficiently — comparing sorted substrings directly reintroduces the $O(k \log k)$ per window cost, so the count-array approach is the standard efficient solution.

Common Mistakes

- Forgetting the early return when $\text{len}(p) > \text{len}(s)$, risking index errors.
- Off-by-one when identifying which character is leaving the window (should be $s[i - \text{len}(p)]$, not $s[i - \text{len}(p) - 1]$ or similar).
- Comparing `window_count` to `p_count` using identity instead of value equality (in Python, `==` on lists compares values correctly, but this trips people up in other languages).
- Not initializing the first window's counts before starting the main sliding loop.

If I Get Stuck — Recovery

- If stuck: 'Sorting each window is the naive way to check anagram equality, but I know equal counts of each letter also defines an anagram, and counts can be updated incrementally as the window slides.'
- If I get the leaving-character index wrong: pause and reason, 'when I'm at position i , positions $i - \text{len}(p)$ to $i - 1$ are strictly inside the window, so $i - \text{len}(p)$ is what's leaving.'
- If the interviewer hints 'can you avoid resorting': pivot to fixed-size count arrays and incremental updates.
- If array comparison feels slow: mention the $O(1)$ match-counter optimization as a follow-up improvement.

Pattern Recognition

Trigger words: 'find all anagrams', 'fixed-length substring matching character composition'

Pattern: Sliding Window (fixed size) + character count comparison

First thought: Anagram equality is really count equality, and counts can be updated incrementally as a fixed-size window slides.

Common trap: Recomputing the window's character counts from scratch at every position instead of incrementally adding/removing one character.

Memory Hook

KEY INSIGHT

Find All Anagrams → "Same letter counts, sliding window, add one/remove one."

30-Second Recap

Pattern	Sliding Window (Fixed Size) + Hash Map
Key idea	Maintain a running count array for a fixed-size window, updating by one character in and one character out.
Data structure	Two 26-length count arrays (or hash maps).
Main algorithm	Fixed-size Sliding Window with incremental count updates.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	p longer than s (no valid windows).

Full Amazon Talk Track

Interviewer: Find every starting index in s where the substring is an anagram of p .

Me: Got it — overlapping matches should all count, and I can assume lowercase English letters only?

Interviewer: Yes.

Me: Brute force would check every window of length $\text{len}(p)$, sort it, and compare to sorted p — $O(n \text{ times } k \log k)$.

Interviewer: How would you avoid re-sorting every window?

Me: I'd track character counts instead of sorting. I build a 26-length count array for p once, then slide a window of the same length across s , adding the incoming character and removing the outgoing one, and compare count arrays at each step.

Me: Let me code that — first I initialize both count arrays for the first window, then slide one character at a time.

Me: Tracing 'cbaebabacd' with $p='abc'$, the window correctly flags matches at indices 0 and 6.

Me: Edge case: if p is longer than s , there's no valid window at all, so I return an empty list right away.

Me: Final complexity: $O(n)$ time since each slide is $O(1)$ with fixed-size count arrays, $O(1)$ space.

4. Stacks

Most-recent-first processing: expressions, brackets, paths, monotonic stacks.

Evaluate Reverse Polish Notation

LeetCode #150 | **MEDIUM** | Pattern: Stack | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a list of tokens representing an arithmetic expression in Reverse Polish (postfix) notation, and we need to evaluate it and return the result.

What I should clarify

- Should integer division truncate toward zero?
- Can the operands be negative numbers?
- Is the input guaranteed to always be a valid RPN expression?
- Should the result be returned as an integer even if intermediate division isn't exact?

SAY THIS —

Just to confirm, tokens come in postfix order, operators apply to the two most recent operands, and I can assume the expression is always valid, correct? Also, does division truncate toward zero?

STEP 2 — Brute Force First

Idea

There isn't really a simpler 'brute force' here beyond the direct stack simulation — but a naive alternative would be to convert the expression back to infix and use a general expression parser/evaluator.

SAY THIS —

I could try converting this back into infix notation and using a general-purpose expression evaluator, but that's much more complex than necessary given postfix notation is designed to be evaluated directly.

STEP 3 — Brute Force Complexity

Time Complexity

A general infix parser with operator precedence handling would take similar $O(n)$ time but with significantly more code and edge cases to manage.

Space Complexity

$O(n)$ for any intermediate representation.

SAY THIS —

Converting back to infix would still be roughly $O(n)$ time but adds a lot of unnecessary complexity, like handling operator precedence and parentheses that don't even exist in this format.

STEP 4 — Identify the Bottleneck

Postfix notation is specifically designed so operators always apply to the two most recently seen operands — trying to reconstruct precedence rules is solving a problem that doesn't need to exist here.

SAY THIS —

The real issue with overcomplicating this is ignoring that postfix notation already tells us exactly which two operands to combine and when — we just need a structure that remembers the most recent operands.

STEP 5 — Optimization

Key insight / pattern: Stack

I use a Stack. I scan the tokens left to right. Whenever I see a number, I push it onto the stack. Whenever I see an operator, I pop the top two numbers off the stack (in the right order), apply the operator, and push the result back. By the end, the stack holds exactly one value: the final result.

KEY INSIGHT

A stack naturally remembers 'the most recent operands' in the correct order, which is exactly what postfix notation needs: each operator always combines the two values most recently pushed.

SAY THIS —

I'll use a stack to hold numbers as I scan the tokens.

When I see a number, I push it; when I see an operator, I pop the top two values, apply the operator, and push the result back.

At the end, the single remaining value on the stack is the answer.

STEP 6 — Code While Narrating

```
def eval_rpn(tokens):  
    stack = []  
    operators = {'+', '-', '*', '/'}
```

SAY THIS —

I set up an empty stack and a set of the valid operator symbols for quick checking.

WHY: A Python list works perfectly as a stack here since I only ever push/pop from the end, both O(1) operations.

```
    for token in tokens:  
        if token not in operators:  
            stack.append(int(token))  
        else:  
            b = stack.pop()  
            a = stack.pop()
```

SAY THIS —

If the token is a number, I push it onto the stack. If it's an operator, I pop the top two values — the second-popped one, a, came first in the expression, and b came second.

WHY: Order matters for non-commutative operators like subtraction and division: since b was pushed last, it's the right-hand operand, and a is the left-hand operand.

```

        if token == '+':
            stack.append(a + b)
        elif token == '-':
            stack.append(a - b)
        elif token == '*':
            stack.append(a * b)
        else:
            stack.append(int(a / b))

    return stack.pop()

```

SAY THIS —

I apply the operator to a and b in the correct order and push the result back onto the stack. At the very end, I pop and return the final answer.

WHY: Using `int(a / b)` instead of `a // b` ensures truncation toward zero for negative results, matching how most languages define integer division for this problem, rather than Python's floor division which rounds toward negative infinity.

STEP 7 — Dry Run

tokens = ['2','1','+','3','*']. Push 2 -> [2]. Push 1 -> [2,1]. '+': pop b=1, a=2, push 2+1=3 -> [3]. Push 3 -> [3,3]. '*': pop b=3, a=3, push 3*3=9 -> [9]. Final pop: 9.

SAY THIS —

Tracing ['2','1','+','3','']: I push 2 and 1, then '+' combines them into 3, then I push another 3, and '*' combines those into 9, which is the final answer.*

STEP 8 — Edge Case Test

Division producing a negative fractional result, e.g. tokens=['10','3','-','5','/'] evaluating $(10-3)/5 = 7/5 = 1.4$, which should truncate toward zero to 1 using `int(a/b)`, not Python's `//` which would also give 1 here, but consider a negative case like $-7/2 = -3.5$ which should truncate to -3, not floor to -4.

SAY THIS —

Let me check a division case that could go wrong with truncation, like -7 divided by 2, which should truncate toward zero to -3, not floor down to -4 the way Python's // operator would — using `int(a/b)` instead of `a//b` gets this right.

Other edge cases to remember

- Single-number expression with no operators at all
- Division resulting in an exact integer
- Deeply nested expression using many operators in sequence
- Negative numbers as individual operand tokens

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(n)$

SAY THIS —

This is $O(n)$ time since we process each token exactly once, and $O(n)$ space in the worst case for the stack, such as when the expression is a long sequence of numbers before any operators appear.

If the Interviewer Pushes Back

Why use `int(a/b)` instead of `a // b` for division?

Because Python's `//` operator floors toward negative infinity, but this problem (like most languages' default integer division) expects truncation toward zero, so `int(a/b)` is the correct way to get that behavior in Python.

Can you do this without an explicit stack, e.g. recursively?

Not cleanly for postfix notation processed left to right; recursion is more natural for prefix notation processed right to left, but a stack directly mirrors how postfix is meant to be evaluated in a single left-to-right pass.

What if division by zero occurs?

The problem typically guarantees a valid expression that never divides by zero; if that assumption didn't hold, I'd add an explicit check and raise or handle an error case.

Can you optimize this further?

$O(n)$ time and space is already optimal, since we must read every token at least once, and in the worst case (all numbers, one final operator) the stack must hold up to $n/2$ or more values at once.

Common Mistakes

- Popping operands in the wrong order for non-commutative operators like subtraction and division (a should be popped second, b first).
- Using `a // b` instead of `int(a / b)`, which gives wrong results for negative divisions.
- Forgetting to convert string tokens to integers before doing arithmetic.
- Not handling multi-digit or negative number tokens correctly when parsing.

If I Get Stuck — Recovery

- If stuck: 'Postfix notation means operators always act on the two most recently seen numbers, and a stack is exactly the structure that remembers 'most recent' in order.'
- If I get operand order backwards: verify with a subtraction example by hand, since addition/multiplication would hide the bug but subtraction/division would immediately expose it.
- If the interviewer hints 'watch out for negative division': switch to `int(a/b)` and explain why `//` is wrong here.
- If I forget which token types are operators: just check membership in a small operators set, as coded.

Pattern Recognition

Trigger words: 'Reverse Polish Notation', 'postfix expression evaluation'

Pattern: Stack (Last-In-First-Out processing of the most recent operands)

First thought: Operators always combine the two most recently seen values — a stack captures 'most recent' naturally.

Common trap: Getting the order of operands backwards for non-commutative operators.

Memory Hook

KEY INSIGHT

Evaluate RPN → "Numbers pile up, operators pop the top two and push the result back."

30-Second Recap

Pattern	Stack
Key idea	Push numbers; on an operator, pop the top two, apply it in the correct order, and push the result.
Data structure	A single stack (Python list).
Main algorithm	Single left-to-right scan with stack push/pop.
Time	$O(n)$
Space	$O(n)$
Biggest edge case	Negative number division needing truncation toward zero.

Full Amazon Talk Track

Interviewer: Evaluate this expression given in Reverse Polish Notation.

Me: Got it — operators always apply to the two most recent operands, and I can assume the expression is always valid, right? Does division truncate toward zero?

Interviewer: Yes to both.

Me: Since postfix is designed to be evaluated left to right using a stack, I'll push numbers as I see them, and whenever I hit an operator, pop the top two values, apply the operator, and push the result back.

Me: Let me code that — I need to be careful about operand order for subtraction and division, since the second-popped value came first in the expression.

Me: Tracing `['2','1','+','3','*']`: 2 and 1 combine to 3, then 3 and 3 combine to 9, which is the final answer.

Me: One thing I want to double check — negative division. -7 divided by 2 should truncate to -3, not floor to -4, so I'll use `int(a/b)` instead of Python's `//` operator.

Me: Final complexity: $O(n)$ time for one pass through the tokens, $O(n)$ space for the stack in the worst case.

Next Greater Element I

LeetCode #496 | **EASY** | Pattern: Monotonic Stack | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two arrays, `nums1` (a subset of `nums2`), and for each element in `nums1` we need to find its next greater element to the right in `nums2`, or `-1` if none exists.

What I should clarify

- Are all elements in `nums1` guaranteed to exist in `nums2`?
- Are there duplicate values in `nums2`?
- What should I return if there's no greater element to the right?
- Are the arrays guaranteed non-empty?

SAY THIS —

Just to confirm, every value in `nums1` appears somewhere in `nums2`, and for each I need the first larger value that appears later in `nums2`, or `-1` if there isn't one, correct?

STEP 2 — Brute Force First

Idea

For each element in `nums1`, find its position in `nums2`, then scan rightward until a larger value is found.

SAY THIS —

The brute-force approach is, for every element in `nums1`, locate it in `nums2` and then scan forward until I find something bigger.

STEP 3 — Brute Force Complexity

Time Complexity

Finding each element and scanning forward is $O(n)$ per query in the worst case, giving $O(n*m)$ for m queries.

Space Complexity

$O(1)$ extra space beyond the output.

SAY THIS —

This is $O(n \text{ times } m)$ time in the worst case, since for every queried value I might scan almost the entire second array, and $O(1)$ extra space.

STEP 4 — Identify the Bottleneck

I keep re-scanning parts of `nums2` for different queries, when the 'next greater element' for every position in `nums2` could be precomputed once and reused for all queries.

SAY THIS —

The bottleneck is redoing the rightward scan for every query separately, when I could precompute the next greater element for every value in nums2 a single time.

STEP 5 — Optimization

Key insight / pattern: Monotonic Stack

I use a Monotonic Stack. I scan nums2 left to right, maintaining a stack of numbers for which I haven't yet found a next greater element. For each new number, while the stack's top is smaller than the current number, I pop it and record the current number as its 'next greater element' in a hash map. I push the current number onto the stack. After processing all of nums2, I look up each nums1 value in the hash map.

KEY INSIGHT

A monotonic (decreasing) stack lets me resolve 'next greater element' for many values at once: as soon as a bigger number appears, it immediately answers the question for every smaller number still waiting on the stack, in one pass.

SAY THIS —

I'll do one pass over nums2 with a stack that only ever holds a decreasing sequence of numbers.

Whenever the current number is bigger than the stack's top, that top value has found its next greater element, so I pop it and record the answer in a hash map, repeating until the stack top is bigger or the stack is empty.

Then I just look up each nums1 value in that hash map, defaulting to -1 if it was never resolved.

STEP 6 — Code While Narrating

```
def next_greater_element(nums1, nums2):
    next_greater = {}
    stack = []
```

SAY THIS —

I set up a hash map to store the answer for each value in nums2, and an empty stack.

WHY: The hash map will let me answer each nums1 query in O(1) after the single pass over nums2 is done.

```
    for num in nums2:
        while stack and stack[-1] < num:
            smaller = stack.pop()
            next_greater[smaller] = num
        stack.append(num)
```

SAY THIS —

For each number in nums2, I pop off any stack values smaller than it, recording the current number as their next greater element, then push the current number onto the stack.

WHY: Anything still on the stack when a bigger number shows up has just found its answer; anything left on the stack at the end simply never finds a next greater element.

```
    return [next_greater.get(num, -1) for num in nums1]
```

SAY THIS —

Finally, for every value in nums1, I look up its answer in the hash map, defaulting to -1 if it was never resolved.

WHY: Some values never get popped because nothing bigger ever appears after them, so those correctly default to -1.

STEP 7 — Dry Run

nums2 = [2,1,2,4,3], nums1=[2,4]. Process 2: stack empty, push -> [2]. Process 1: 1<2 no pop, push -> [2,1]. Process 2: top=1<2, pop, next_greater[1]=2; top=2, not <2, stop; push 2 -> [2,2]. Process 4: top=2<4 pop, next_greater[2]=4 (first 2); top=2<4 pop, next_greater[2]=4 (second 2, same key overwritten, fine since both were 2); push 4 -> [4]. Process 3: top=4, not<3, push -> [4,3]. Map: {1:2, 2:4}. Query nums1=[2,4]: 2->4, 4-> not in map -> -1. Result: [4,-1].

SAY THIS —

Tracing nums2=[2,1,2,4,3]: as bigger numbers appear, they resolve smaller ones on the stack, building up next_greater = {1:2, 2:4}. Then for nums1=[2,4], 2 maps to 4, and 4 was never resolved (nothing bigger comes after it), so it's -1. Final: [4,-1].

STEP 8 — Edge Case Test

A value in nums2 that's already the largest and appears last, e.g. the final element in a strictly increasing nums2 like [1,2,3,4] — 4 never gets popped and remains without an entry in the map, correctly defaulting to -1 when queried.

SAY THIS —

Let me check the case where a number is the largest and occurs at the very end, like the last element of a strictly increasing array — it never gets popped off the stack, so it correctly defaults to -1.

Other edge cases to remember

- nums2 is strictly decreasing (every element defaults to -1)
- nums1 and nums2 are identical arrays
- nums2 has only one element
- Duplicate values appearing in nums2

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n + m)$	$O(n)$

SAY THIS —

This is $O(n + m)$ time, where n is the length of nums2 (each element is pushed and popped at most once) and m is the length of nums1 for the lookups, and $O(n)$ space for the hash map and stack.

If the Interviewer Pushes Back

Why is the stack guaranteed $O(n)$ total, not worse?

Because every element is pushed exactly once and popped at most once across the entire scan, so even though there's a while loop inside the for loop, the total number of pop operations across all iterations can't exceed n , giving amortized $O(n)$ overall.

What if nums2 could contain duplicates?

The map would just store the most recent 'next greater' resolution for a given value; since the problem guarantees nums1 values are unique in nums2, or at least resolvable, this doesn't cause issues in practice.

Can you solve it without a hash map, directly with indices?

Yes, if nums1 were about positions instead of values, I could store next-greater results in an array indexed by position in nums2 directly, but since we're keying by value here, a hash map is the natural fit.

Can you optimize further?

$O(n + m)$ is optimal since we must at least look at every element of nums2 once and every query in nums1 once.

Common Mistakes

- Popping conditionally with 'while' but forgetting to check 'stack' is non-empty first, causing an index error.
- Recomputing next greater elements per-query instead of precomputing once for all of nums2.
- Forgetting the default of -1 when a nums1 value's next greater element was never found.
- Confusing 'next greater to the right' with 'next greater anywhere', missing the directional requirement.

If I Get Stuck — Recovery

- If stuck: 'Since I'm answering multiple queries against the same array, let me precompute answers for every value in nums2 once, rather than searching per query.'
- If I forget the stack's monotonic property: reason aloud, 'the stack should only ever hold values that haven't found something bigger yet, so it must be decreasing from bottom to top.'
- If the interviewer hints 'you don't need to search each query separately': confirm and switch to the single-pass precompute-then-lookup approach.
- If unsure about amortized complexity: explain that each element is pushed once and popped at most once, so total stack operations are bounded by n .

Pattern Recognition

Trigger words: 'next greater element', 'next warmer day', 'next larger value to the right'

Pattern: Monotonic Stack

First thought: Can I resolve many 'next greater' questions in one pass by keeping a stack of unresolved candidates?

Common trap: Re-scanning the array separately for each query instead of precomputing everything in one monotonic-stack pass.

Memory Hook

KEY INSIGHT

Next Greater Element → "Stack holds the unresolved; a bigger number resolves everyone smaller beneath it."

30-Second Recap

Pattern	Monotonic Stack
Key idea	Maintain a decreasing stack; when a bigger number arrives, it resolves all smaller values still waiting.
Data structure	Stack + hash map (value → next greater value).

Main algorithm	Monotonic Stack, single pass over nums2.
Time	$O(n + m)$
Space	$O(n)$
Biggest edge case	Largest value occurring at the very end (always resolves to -1).

Full Amazon Talk Track

Interviewer: For each value in nums1, find its next greater element in nums2, or -1.

Me: Got it — every nums1 value appears in nums2, and I want the first larger value appearing after it in nums2, right?

Interviewer: Correct.

Me: Brute force: for each nums1 value, find it in nums2 and scan forward for something bigger — $O(n \text{ times } m)$ in the worst case.

Interviewer: Can you avoid redoing that scan for every query?

Me: Yes — I'll do one pass over nums2 using a monotonic decreasing stack. Whenever I see a number bigger than the stack's top, that resolves the top's 'next greater element', so I pop it and record the answer in a hash map.

Me: Let me code that... after the single pass, I just look up each nums1 query in the map, defaulting to -1.

Me: Tracing nums2=[2,1,2,4,3]: the map ends up as {1:2, 2:4}. For nums1=[2,4], I get [4,-1], since 4 never finds anything bigger after it.

Me: Edge case: if the largest value is at the very end of nums2, it's never popped, and correctly stays -1.

Me: Final complexity: $O(n+m)$ time since the stack does amortized $O(1)$ work per element, $O(n)$ space for the map and stack.

Decode String

LeetCode #394 | **MEDIUM** | Pattern: Stack | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an encoded string with a pattern like `k[encoded_string]`, meaning the `encoded_string` inside the brackets should be repeated `k` times, and encodings can be nested, and we need to return the fully decoded string.

What I should clarify

- Can the encodings be nested, like `2[3[a]b]`?
- Are the numbers guaranteed to be positive integers, possibly multi-digit?
- Is the input guaranteed to be well-formed (brackets always balanced)?
- Are there plain (non-bracketed) characters mixed in with encoded parts?

SAY THIS —

Just to confirm, encodings can nest arbitrarily deep, numbers can be multiple digits, and the input is always well-formed with balanced brackets, correct?

STEP 2 — Brute Force First

Idea

Try to handle this with string replacement or regular expressions, repeatedly expanding the innermost bracket pattern until none remain.

SAY THIS —

One approach is to repeatedly find the innermost `k[...]` pattern with something like a regular expression, expand it, and repeat until no brackets remain.

STEP 3 — Brute Force Complexity

Time Complexity

Each expansion pass can cost $O(n)$ to scan and rebuild the string, and we may need $O(n)$ passes in the worst case of deep nesting, giving $O(n^2)$ or worse.

Space Complexity

$O(n)$ for the string being rebuilt at each step.

SAY THIS —

This repeated-expansion approach can be $O(n^2)$ or worse in the worst case, since deeply nested brackets require many passes, each rebuilding a growing string.

STEP 4 — Identify the Bottleneck

I'm rebuilding and rescanning the whole string at every level of nesting, when a single pass tracking open contexts as I go would let me handle nesting directly without repeated scans.

SAY THIS —

The bottleneck is treating nesting as a series of separate passes, when I could handle it directly in one scan if I had a way to remember 'what I was building before I went one level deeper'.

STEP 5 — Optimization

Key insight / pattern: Stack

I use a Stack to remember state when I enter a new bracket level. As I scan left to right: digits accumulate into a current number, letters accumulate into a current string, and when I see '[', I push the current string and current number onto the stack and reset both to start fresh for the nested section. When I see ']', I pop the previous string and repeat count, and combine them: `current_string = previous_string + (repeat_count * current_string)`.

KEY INSIGHT

Each '[' opens a new 'building context' and each ']' closes it by merging back into the context that was paused — that's exactly what a stack is for: remembering paused work so it can be resumed later.

SAY THIS —

I'll scan the string once, building up a current number and current string as I go.

When I hit '[', I push the current string and number onto a stack and reset them, since I'm about to start building a nested piece.

When I hit ']', I pop the previous string and number, and combine them: previous string plus the nested string repeated that many times.

STEP 6 — Code While Narrating

```
def decode_string(s):
    stack = []
    current_string = ''
    current_number = 0
```

SAY THIS —

I set up a stack, and two variables tracking what I'm currently building: the string so far and any number being read digit by digit.

WHY: `current_number` needs to accumulate across multiple digit characters (since numbers can be multi-digit, like '213'), so I build it up rather than reading one digit at a time.

```
    for char in s:
        if char.isdigit():
            current_number = current_number * 10 + int(char)
        elif char == '[':
            stack.append((current_string, current_number))
            current_string = ''
            current_number = 0
```

SAY THIS —

If I see a digit, I fold it into `current_number` using the standard multi-digit accumulation trick. If I see '[', I push the current string and number as a pair, then reset both to start fresh for the nested content.

WHY: Multiplying by 10 and adding the new digit correctly builds multi-digit numbers like '2' then '1' then '3' becoming 213, not just 2, 1, 3 separately.

```

elif char == ']':
    previous_string, repeat_count = stack.pop()
    current_string = previous_string + current_string * repeat_count
else:
    current_string = current_string + char

return current_string

```

SAY THIS —

If I see ']', I pop the paused string and repeat count, and rebuild current_string as the previous string followed by the nested string repeated that many times. Any regular letter just gets appended to current_string directly.

WHY: This is where nesting resolves itself naturally: whatever was paused before the bracket gets combined with the fully-built nested content, one level at a time, correctly handling arbitrarily deep nesting.

STEP 7 — Dry Run

`s = '3[a2[c]]'`. char '3': `current_number=3`. char '[': `push (" ,3), reset current_string="", current_number=0`. char 'a': `current_string='a'`. char '2': `current_number=2`. char '[': `push ('a',2), reset current_string="", current_number=0`. char 'c': `current_string='c'`. char ']': `pop ('a',2), current_string='a' + 'c'*2 = 'acc'`. char ']': `pop (" ,3), current_string="" + 'acc'*3 = 'accaccacc'`. Final result: 'accaccacc'.

SAY THIS —

Tracing '3[a2[c]]': I build up to pushing (" ,3) then ('a',2), then when the inner bracket closes I get 'acc', and when the outer bracket closes I get 'accaccacc', which is the correct decoding.

STEP 8 — Edge Case Test

No brackets at all, just plain letters, e.g. `s='abc'`. The loop just appends each character to `current_string` directly, and since the stack is never touched, the final result is simply 'abc'.

SAY THIS —

Let me check a string with no brackets at all, like 'abc' — every character just appends directly to current_string, the stack stays empty, and I correctly return 'abc' unchanged.

Other edge cases to remember

- Multiple sibling bracket groups at the same level, like `'2[a]3[b]'`
- Multi-digit repeat counts, like `'213[a]'`
- Deeply nested brackets several levels deep
- A bracket group containing multiple different letters, like `'3[abc]'`

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n * \max K)$	$O(n)$

SAY THIS —

Time is roughly $O(n \text{ times } \text{maxK})$ where maxK is the largest repeat multiplier, since string concatenation with repetition can multiply the output size, and space is $O(n)$ for the stack and the output string in the worst case of deep nesting or large repeat counts.

If the Interviewer Pushes Back

Can you avoid quadratic string concatenation costs?

In languages where string concatenation is expensive, I could build results using a list of characters or a more efficient buffer and join at the end; in Python, repeated concatenation of small pieces is generally acceptable but could be optimized similarly if profiling showed it mattered.

Can you solve this recursively instead of with an explicit stack?

Yes, recursion works naturally here too, since each nested bracket is essentially its own subproblem — the recursive call stack implicitly plays the same role as my explicit stack, just expressed differently.

What if the repeat count could be zero?

A count of 0 would mean the nested content is dropped entirely (repeated zero times), which my formula `previous_string + current_string * 0` handles correctly since Python string multiplication by 0 yields an empty string.

What's the worst-case output size?

It can be exponential in the input length if brackets are deeply nested with large multipliers (e.g., `2[2[2[2[...]]]]`), so I'd flag that as a potential real-world constraint to discuss with the interviewer if inputs could be adversarial.

Common Mistakes

- Reading only single digits instead of accumulating multi-digit numbers correctly.
- Forgetting to reset `current_string` and `current_number` after pushing them onto the stack when entering a new bracket.
- Getting the concatenation order wrong when popping (should be `previous_string + current_string * count`, not the reverse).
- Not handling plain (non-bracketed) characters that appear alongside encoded sections.

If I Get Stuck — Recovery

- If stuck: 'Nesting means I need to remember what I was building before I went one level deeper, which is exactly what a stack is designed for.'
- If I get concatenation order backwards: verify with a simple trace like `'3[a]b'` by hand, which would immediately reveal an ordering bug.
- If the interviewer hints 'think about pausing and resuming work': confirm that's exactly the push/pop pattern I'm using.
- If I forget how to build multi-digit numbers: use the standard trick, `current_number = current_number * 10 + digit`, and explain it as you write it.

Pattern Recognition

Trigger words: 'nested encoding', '`k[string]`', 'decode string with repeats'

Pattern: Stack (to pause/resume nested build contexts)

First thought: Every time I go one level deeper (open bracket), I need to remember what I was building before, and resume it later (close bracket) — a stack does exactly that.

Common trap: Forgetting to reset the current string/number after pushing when entering a new nested level.

Memory Hook

KEY INSIGHT

Decode String → "Bracket opens: pause and push. Bracket closes: pop and combine."

30-Second Recap

Pattern	Stack
Key idea	Push (string-so-far, repeat-count) on '[', pop and combine on ']'.
Data structure	A stack of (string, number) pairs.
Main algorithm	Single-pass scan with push/pop for nested contexts.
Time	$O(n * \text{maxK})$
Space	$O(n)$
Biggest edge case	Deeply nested brackets with multi-digit repeat counts.

Full Amazon Talk Track

Interviewer: Decode this string where $k[\text{substring}]$ means substring repeated k times, and it can nest.

Me: Got it — nesting can go arbitrarily deep, numbers can be multi-digit, and the input is always well-formed, right?

Interviewer: Yes.

Me: One naive approach is repeatedly expanding the innermost bracket with something like regex until none remain, but that could take many passes and rebuild the string repeatedly — not great for deep nesting.

Interviewer: How would you do it in one pass?

Me: I'd use a stack. As I scan, I build up a current string and current number. When I hit '[', I push both onto the stack and reset them, since I'm about to build a nested piece. When I hit ']', I pop the paused string and count, and combine them: previous string plus the nested string repeated that many times.

Me: Let me code that up, being careful to accumulate multi-digit numbers correctly.

Me: Tracing '3[a2[c]]': the inner bracket resolves to 'acc', and the outer bracket resolves to 'accaccacc', which is correct.

Me: Edge case: a string with no brackets at all just passes straight through unchanged, since the stack never gets used.

Me: Final complexity is roughly $O(n \text{ times maxK})$ time since large repeat counts multiply the output, and $O(n)$ space for the stack and result.

Simplify Path

LeetCode #71 | **MEDIUM** | Pattern: Stack | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an absolute Unix-style file path, and we need to simplify it into its canonical form, resolving '.' (current directory), '..' (parent directory), and collapsing multiple slashes.

What I should clarify

- What should happen if '..' appears at the root, with no parent to go up to?
- Should trailing slashes be removed from the final answer?
- Can there be multiple consecutive slashes, and should they collapse to one?
- Is the path guaranteed to start with a single leading slash?

SAY THIS —

Just to confirm, I need to resolve '.' and '..' components, collapse repeated slashes, and if '..' tries to go above the root, it should just stay at the root, correct?

STEP 2 — Brute Force First

Idea

Directly manipulate the string character by character, trying to track directory boundaries and handle '..' by searching backward for the previous slash to remove.

SAY THIS —

A naive approach is direct string manipulation — scanning through and trying to delete characters or find the previous slash whenever I hit '..', directly on the string.

STEP 3 — Brute Force Complexity

Time Complexity

Removing characters or searching backward within a string repeatedly can cost $O(n)$ per operation, giving $O(n^2)$ in the worst case for many '..' components.

Space Complexity

$O(n)$ for the string being modified.

SAY THIS —

Direct string surgery is error-prone and can hit $O(n^2)$ time if I'm repeatedly searching backward or shifting characters around within the string.

STEP 4 — Identify the Bottleneck

Treating the path as a flat string makes 'go up one directory' awkward and error-prone, when really the path is naturally a sequence of directory names that behaves like a stack — push to go deeper, pop to go up.

SAY THIS —

The bottleneck is fighting with raw string indices to simulate 'going up a directory', when directory names naturally behave like a stack of pushes and pops.

STEP 5 — Optimization

Key insight / pattern: Stack

I split the path by '/' to get a list of components. I use a Stack: for each component, if it's empty or '.', I ignore it; if it's '..', I pop the stack (if non-empty, to go up a directory); otherwise it's a real directory name, so I push it. At the end, I join the stack's contents with '/' and prepend a single leading slash.

KEY INSIGHT

Splitting by '/' turns the whole problem into a sequence of simple stack operations: real names push, '..' pops, and '.' or empty segments (from repeated slashes) are simply skipped.

SAY THIS —

I'll split the path on '/' to get a list of pieces, some of which might be empty due to repeated slashes.

I'll use a stack: skip empty pieces and '.', pop on '..' if there's something to pop, and push any real directory name.

At the end I join everything in the stack with '/' and add a single leading slash.

STEP 6 — Code While Narrating

```
def simplify_path(path):  
    parts = path.split('/')  
    stack = []
```

SAY THIS —

I split the input on '/' into individual pieces, and set up an empty stack.

WHY: Splitting handles the tokenization for me; pieces between consecutive slashes become empty strings, and '.', '..', and real names all show up as distinct list elements.

```
    for part in parts:  
        if part == '' or part == '.':  
            continue  
        elif part == '..':  
            if stack:  
                stack.pop()  
        else:  
            stack.append(part)
```

SAY THIS —

For each piece, I skip empty strings and single dots. For double dots, I pop the stack if it's not empty, going up one directory. Otherwise, it's a real directory name, so I push it.

WHY: Checking 'if stack' before popping on '..' prevents trying to go above the root, which should just be a no-op rather than an error.

```
    return '/' + '/'.join(stack)
```

SAY THIS —

Finally, I join everything left in the stack with '/' and add a single leading slash to form the canonical absolute path.

WHY: The problem always wants an absolute path starting with exactly one slash, and joining with '/' naturally produces single slashes between directory names, with no trailing slash.

STEP 7 — Dry Run

path = '/a/.b/./../c/'. Split by '/': ['', 'a', '.', 'b', '..', '..', 'c', '']. Process '': skip. 'a': push -> ['a']. '.': skip. 'b': push -> ['a', 'b']. '..': pop -> ['a']. '..': pop -> []. 'c': push -> ['c']. '': skip. Final stack: ['c']. Result: '/' + 'c' = '/c'.

SAY THIS —

Tracing '/a/.b/./../c/': after splitting and processing each piece, 'a' and 'b' get pushed, then both get popped by the two '..' pieces, and finally 'c' gets pushed, giving the canonical path '/c'.

STEP 8 — Edge Case Test

Trying to go up past the root, e.g. path='/../'. Split gives ['', '..', '']. The '..' tries to pop an empty stack, which the 'if stack' check prevents, so nothing happens. Final stack is empty, and the result is '/' + '' = '/', correctly staying at the root instead of erroring.

SAY THIS —

Let me check what happens if the path tries to go above the root, like '/../' — popping an empty stack is guarded against, so we correctly stay at the root and return just '/'.

Other edge cases to remember

- Path with multiple consecutive slashes, like '/a//b'
- Path that's already in canonical form
- Path consisting only of '.' components
- Very deeply nested path with many real directory names

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(n)$

SAY THIS —

This is $O(n)$ time since splitting and processing the path both scan it a constant number of times, and $O(n)$ space for the stack and the split list of components.

If the Interviewer Pushes Back

Why not manipulate the string directly instead of splitting?

Splitting converts the problem into clean, discrete tokens, which makes the stack logic straightforward and avoids fragile index arithmetic that direct string manipulation would require, especially around variable numbers of slashes.

What if the input had trailing slashes?

Splitting on '/' naturally produces a trailing empty string in that case, which the 'skip empty' rule already discards, so trailing slashes are handled without any special-casing.

Can you avoid using Python's split entirely?

Yes, I could manually scan character by character and build tokens between slashes myself, which would still be $O(n)$ time and space, just with more manual bookkeeping — `split()` is simply cleaner to write and reason about.

What about relative paths (not starting with '/')?

This problem specifies an absolute Unix-style path, so I'm assuming a leading slash; if relative paths needed support, I'd clarify how `..` should behave when the stack is empty (whether to keep `..` as a literal token instead of discarding it).

Common Mistakes

- Forgetting to guard the pop with 'if stack', causing an error when `..` appears at the root.
- Not skipping empty strings that arise from consecutive slashes, resulting in bogus stack entries.
- Confusing `.` (current directory, ignore) with `..` (parent directory, pop).
- Adding a trailing slash to the final joined result when the problem expects none (except for the root itself).

If I Get Stuck — Recovery

- If stuck: 'Let me not think about this as raw string surgery — if I split on slashes, each piece is either something to push, something to pop, or something to ignore, which sounds like a stack.'
- If I get an index error from popping an empty stack: add the 'if stack' guard and explain why `..` at the root should be a no-op, not an error.
- If the interviewer hints 'think about how a shell resolves `cd ..`': confirm that's literally the stack pop behavior being modeled.
- If I'm unsure about repeated slashes: trace a small example with `'/'` in it to confirm empty strings are correctly skipped.

Pattern Recognition

Trigger words: 'Unix path', 'simplify path', 'canonical path', `./` and `../`

Pattern: Stack (directory push/pop simulation)

First thought: Split into path components, then treat directory traversal as stack pushes ('name') and pops (`..`).

Common trap: Forgetting to guard against popping an empty stack when `..` appears at the root.

Memory Hook

KEY INSIGHT

Simplify Path → "Split by slash; real names push, `..` pops, `.` does nothing."

30-Second Recap

Pattern	Stack
Key idea	Split the path into components and use a stack to push directory names and pop on <code>..</code> .
Data structure	A stack (list) of directory names.
Main algorithm	Single pass over split components with push/pop/skip rules.
Time	$O(n)$
Space	$O(n)$

Biggest edge case

'..' appearing at the root (must not error or go negative).

Full Amazon Talk Track

Interviewer: Simplify this absolute Unix-style file path into its canonical form.

Me: Got it — I need to resolve '.', '..', and collapse repeated slashes, and if '..' would go above the root, it just stays at the root, right?

Interviewer: Yes.

Me: A naive approach would be manipulating the raw string directly, searching backward for slashes whenever I hit '..', but that gets fragile and can be $O(n^2)$.

Interviewer: How would you simplify it?

Me: I'd split the path on '/' into components, then use a stack: skip empty pieces and single dots, pop on double-dot if there's something to pop, and push any real directory name.

Me: Let me code that, then join the stack with '/' and prepend a leading slash at the end.

Me: Tracing '/a/./b/./../c/': 'a' and 'b' get pushed then both popped by the two '..', and 'c' gets pushed, giving '/c'.

Me: Edge case: if the path tries to go above the root, like '/../', I guard the pop so it just stays at the root and returns '/'.

Me: Final complexity: $O(n)$ time and space for the split and stack operations.

5. Binary Search

Searching sorted structures, and binary-searching the ANSWER itself.

Find K Closest Elements

LeetCode #658 | **MEDIUM** | Pattern: Binary Search | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a sorted array, a target value x , and an integer k , and we need to return the k closest values to x from the array, sorted in ascending order.

What I should clarify

- If there's a tie in distance between two candidates, which one wins — the smaller value?
- Is the array guaranteed to be sorted ascending?
- Can k equal the length of the array?
- Should the output remain sorted ascending?

SAY THIS —

Just to confirm, the array is sorted, and I need the k values closest to x , with ties broken in favor of the smaller value, returned in ascending order, correct?

STEP 2 — Brute Force First

Idea

Compute the distance from every element to x , sort all elements by that distance (breaking ties by value), and take the first k , then re-sort those k by value.

SAY THIS —

The brute-force way is to compute each element's distance from x , sort everything by that distance, take the top k , and then sort those k back into ascending order.

STEP 3 — Brute Force Complexity

Time Complexity

Sorting by distance is $O(n \log n)$, and the final sort of the k results is $O(k \log k)$, so overall $O(n \log n)$.

Space Complexity

$O(n)$ to store distances alongside values.

SAY THIS —

This is $O(n \log n)$ time because of the sort by distance, and $O(n)$ space to hold that extra information.

STEP 4 — Identify the Bottleneck

I'm sorting the entire array by distance, but since the input is already sorted by value, the k closest elements must form a single contiguous window — I don't need to reorder anything if I can just find where that window starts.

SAY THIS —

The bottleneck is throwing away the fact that the array is already sorted; the answer is always some contiguous window of size k , so I just need to locate its starting position.

STEP 5 — Optimization

Key insight / pattern: Binary Search

I use Binary Search on the answer's starting index. I search for the left boundary of a window of size k within the range $[0, n-k]$. For a candidate left boundary 'mid', I compare $x - \text{nums}[\text{mid}]$ (distance to the window's left edge) against $\text{nums}[\text{mid} + k] - x$ (distance to the element just past the window's right edge). If the right-edge candidate is closer or equal, the window should shift right ($\text{mid}+1$ is a better start); otherwise, the current window (or something to its left) is at least as good, so I move the search left.

KEY INSIGHT

Because the array is sorted, the k closest elements are always contiguous, so instead of searching over 'which elements', I can binary search over 'where does the window start', comparing just the two elements at the window's boundaries to decide which direction to shift.

SAY THIS —

Since the answer is always a contiguous window in a sorted array, I'll binary search directly for the window's starting index instead of comparing every element.

For a candidate window starting at mid, I compare how close the element just before the window is to x , versus the element just after the window.

Whichever comparison tells me the window should shift, I adjust my search bounds accordingly, narrowing down to the correct start in $\log(n)$ steps.

STEP 6 — Code While Narrating

```
def find_closest_elements(arr, k, x):  
    left = 0  
    right = len(arr) - k
```

SAY THIS —

I set up a search range for the window's starting index: it can start anywhere from 0 up to $\text{len}(\text{arr}) - k$, since the window must fully fit inside the array.

WHY: Any starting index beyond $\text{len}(\text{arr}) - k$ would make the window run off the end of the array, so those are never valid candidates.

```
    while left < right:  
        mid = (left + right) // 2  
        if x - arr[mid] > arr[mid + k] - x:  
            left = mid + 1  
        else:  
            right = mid
```

SAY THIS —

I compare the distance from x to the window's left-edge element against the distance from x to the element just past the window's right edge. If the right side is strictly closer, the window should start further right, so I move left up; otherwise, I keep the search on the current side by setting right to mid.

WHY: This comparison directly encodes 'is shifting the window right an improvement', and using $\text{right} = \text{mid}$ (not $\text{mid} - 1$) with $\text{left} < \text{right}$ ensures the search converges correctly to the unique best starting index.

```
return arr[left:left + k]
```

SAY THIS —

Once left and right converge, left is the correct starting index of the window, so I return that slice of size k.

WHY: Because binary search narrows the range to a single point, left at the end of the loop is guaranteed to be the optimal window start.

STEP 7 — Dry Run

arr = [1,2,3,4,5], k=4, x=3. left=0, right=5-4=1. mid=0: compare $x - \text{arr}[0] = 3 - 1 = 2$ vs $\text{arr}[4] - x = 5 - 3 = 2$. Not strictly greater ($2 > 2$ is false), so $\text{right} = \text{mid} = 0$. Now left=0, right=0, loop ends. Return $\text{arr}[0:4] = [1,2,3,4]$.

SAY THIS —

With arr=[1,2,3,4,5], k=4, x=3: there's only one possible starting index to check (0 or 1), and the comparison shows starting at 0 is at least as good, giving [1,2,3,4] as the closest 4 elements to 3.

STEP 8 — Edge Case Test

k equals the array length. Then left=0 and $\text{right} = \text{len}(\text{arr}) - k = 0$, so the loop never executes, and the entire array is returned directly — correct, since there's only one possible window.

SAY THIS —

Let me check when k equals the whole array's length — the search range collapses to a single point immediately, and I correctly just return the entire array.

Other edge cases to remember

- x is smaller than every element in the array
- x is larger than every element in the array
- x exactly matches an element in the array
- k = 1 (just the single closest element)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\log(n-k) + k)$	$O(k)$ for output

SAY THIS —

This is $O(\log(n-k))$ time for the binary search over possible window starts, plus $O(k)$ to build the output slice, and $O(k)$ space for the returned result, which is essentially optimal since we must produce k elements.

If the Interviewer Pushes Back

Why compare $x - \text{arr}[\text{mid}]$ vs $\text{arr}[\text{mid}+k] - x$ instead of absolute value comparisons directly on elements?

Because those two specific elements are exactly the ones that would change if the window shifted by one position — comparing them tells us definitively whether shifting right improves the window, which is the core decision binary

search needs at each step.

Can you solve this with two pointers rather than binary search?

Yes, that's also a valid $O(n)$ approach: start with the full array as the window and shrink from whichever end is farther from x until only k elements remain; it's simpler to reason about but doesn't get the logarithmic speed-up.

What if there's a tie in distance at the window boundary?

The comparison uses strict greater-than, so a tie favors keeping the window where it is (shifting left, i.e., preferring the smaller value), which matches the typical tie-breaking rule of preferring smaller values.

Can you optimize further?

We must output k elements, so $O(k)$ is unavoidable for the output itself; the $O(\log(n-k))$ search for the window start is already about as fast as this kind of search can get on a sorted array.

Common Mistakes

- Using `right = mid - 1` instead of `right = mid`, which can skip over the correct boundary in this style of binary search.
- Comparing absolute values with `abs()` instead of directly comparing the two boundary differences, which can subtly mishandle tie-breaking.
- Forgetting the search range should be `[0, len(arr)-k]`, not `[0, len(arr)-1]`.
- Assuming a two-pointer shrinking approach is 'wrong' just because binary search is faster — both are valid, just different complexities.

If I Get Stuck — Recovery

- If stuck: 'Since the array's sorted, the k closest values must be contiguous, so instead of thinking about individual elements, let me binary search for where that window starts.'
- If I'm unsure about the comparison direction: reason concretely with a small example, checking which side is closer to x and confirming the window should shift toward it.
- If the interviewer hints 'the answer is always a contiguous range': confirm and pivot straight to searching for the window's start index.
- If my binary search doesn't converge correctly: double check I'm using `left < right` (not `<=`) and `right = mid` (not `mid-1`), since that combination is what guarantees convergence here.

Pattern Recognition

Trigger words: 'k closest elements', 'sorted array', 'nearest values to x '

Pattern: Binary Search on the answer's position (not on a single value)

First thought: In a sorted array, is the answer always a contiguous window? If so, binary search for the window's start instead of comparing every element.

Common trap: Comparing the wrong pair of boundary elements, or using off-by-one bounds in the search range.

Memory Hook

KEY INSIGHT

K Closest Elements → "Sorted array means the answer is one window — binary search for where it starts."

30-Second Recap

Pattern	Binary Search
---------	---------------

Key idea	The k closest elements form a contiguous window; binary search for its optimal starting index by comparing
Data structure	No extra structure — just index arithmetic on the sorted array.
Main algorithm	Binary Search over window-start positions.
Time	$O(\log(n-k) + k)$
Space	$O(k)$
Biggest edge case	k equal to the entire array length.

Full Amazon Talk Track

Interviewer: Find the k values in this sorted array closest to x.

Me: Got it — ties broken toward the smaller value, and the output should stay in ascending order, correct?

Interviewer: Yes.

Me: Brute force: compute distances for every element, sort by distance, take the top k, then re-sort by value — $O(n \log n)$.

Interviewer: Can you use the fact that the array is sorted?

Me: Yes — since it's sorted, the k closest elements always form one contiguous window. So instead of comparing individual elements, I can binary search for where that window should start.

Me: At each candidate start, I compare the distance from x to the element just before the window versus just after it, and shift the window toward whichever side is closer.

Me: Let me code that... searching over indices 0 to $\text{len}(\text{arr}) - k$.

Me: Tracing $\text{arr}=[1,2,3,4,5]$, $k=4$, $x=3$: the search quickly settles on starting at index 0, giving $[1,2,3,4]$.

Me: Edge case: if k equals the whole array's length, the search range is a single point, so I just return everything.

Me: Final complexity: $O(\log(n-k))$ for the search plus $O(k)$ to build the output, $O(k)$ space.

Split Array Largest Sum

LeetCode #410 | **HARD** | Pattern: Binary Search on the Answer | Advanced pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of integers and an integer m , and we need to split the array into m contiguous non-empty subarrays such that the largest sum among those subarrays is as small as possible, and return that minimized largest sum.

What I should clarify

- Must the subarrays be contiguous, in their original order?
- Are all numbers guaranteed to be non-negative?
- Is m always at least 1 and at most the length of the array?
- Do we need to return the actual split, or just the minimized value?

SAY THIS —

Just to confirm, I'm splitting the array into m contiguous pieces, and I want to minimize the largest sum among those pieces, returning just that minimized value, correct?

STEP 2 — Brute Force First

Idea

Try every possible way to place $m-1$ dividers among the array to create m subarrays, compute the max subarray sum for each split, and take the split with the smallest such maximum.

SAY THIS —

A brute-force approach would try every possible placement of the $m-1$ split points and evaluate the resulting maximum subarray sum for each configuration.

STEP 3 — Brute Force Complexity

Time Complexity

The number of ways to place dividers grows combinatorially, roughly $C(n-1, m-1)$, which is exponential in the worst case.

Space Complexity

$O(m)$ to track a given split configuration.

SAY THIS —

This is exponential time in the worst case since we're essentially trying every combination of split points, which is completely impractical for larger arrays.

STEP 4 — Identify the Bottleneck

I'm exploring every possible way to split the array directly, when really I should be asking a simpler yes/no question: 'can we split the array into m or fewer pieces such that no piece exceeds some target sum?' — and that question is much easier to answer and to binary search over.

SAY THIS —

The bottleneck is searching over split configurations directly; instead I should search over candidate answer values and check feasibility, which is a much smaller and easier space to search.

STEP 5 — Optimization

Key insight / pattern: Binary Search on the Answer

I use Binary Search on the Answer. The answer (minimized largest sum) must lie between $\max(\text{nums})$ (can't split any smaller than the single largest element) and $\text{sum}(\text{nums})$ (the case of one single subarray, $m=1$). For any candidate 'mid' value, I greedily check: can I split the array into at most m subarrays where each subarray's sum is $\leq$ mid? I do this by accumulating a running sum and starting a new subarray whenever adding the next element would exceed mid. If the number of subarrays needed is $\leq m$, mid is a feasible (achievable) largest sum, so I try smaller; otherwise mid is too small, so I need to allow a bigger largest sum.

KEY INSIGHT

As the allowed maximum sum per subarray increases, the number of subarrays needed to satisfy that constraint monotonically decreases — that monotonic relationship is exactly what makes binary search on the answer valid here.

SAY THIS —

Instead of searching over ways to split the array, I'll binary search over possible values of the answer itself, between the largest single element and the total sum.

For each candidate value, I'll greedily check how many subarrays are needed if no subarray's sum can exceed that candidate.

If that count is within the allowed m , the candidate is achievable, so I try to shrink it further; otherwise I need to allow a larger candidate.

STEP 6 — Code While Narrating

```
def split_array(nums, m):
    def subarrays_needed(max_sum):
        count = 1
        current_sum = 0
        for num in nums:
            if current_sum + num > max_sum:
                count += 1
                current_sum = num
            else:
                current_sum += num
        return count
```

SAY THIS —

I write a helper that, given a candidate max_sum, greedily counts how many subarrays are needed: I keep adding to the current subarray's sum until adding the next number would exceed max_sum, at which point I start a new subarray.

WHY: This greedy counting is always optimal for a fixed max_sum , since packing as much as possible into each subarray before exceeding the limit minimizes the total number of subarrays needed.

```
left = max(nums)
right = sum(nums)
```

SAY THIS —

I set the binary search bounds: the smallest possible answer is the largest single element (since that element alone can't be split further), and the largest possible answer is the sum of the whole array (using just one subarray).

WHY: These bounds are provably the true minimum and maximum feasible values for the answer, guaranteeing the true answer lies somewhere within this range.

```
while left < right:
    mid = (left + right) // 2
    if subarrays_needed(mid) <= m:
        right = mid
    else:
        left = mid + 1

return left
```

SAY THIS —

I binary search: if the candidate mid allows splitting into at most m subarrays, it's achievable, so I try to shrink further by moving right down to mid. Otherwise, mid is too restrictive, so I need a bigger candidate and move left up.

WHY: Because feasibility is monotonic in mid (bigger mid always needs fewer or equal subarrays), binary search correctly converges to the smallest feasible mid, which is the answer.

STEP 7 — Dry Run

nums = [7,2,5,10,8], m=2. left=max=10, right=sum=32. mid=21: subarrays_needed(21):
current=7,+2=9,+5=14,+10=24>21 so new subarray count=2, current=10,+8=18. Final count=2. 2<=2, feasible,
right=21. mid=(10+21)//2=15: current=7,+2=9,+5=14,+10=24>15 new subarray count=2,current=10,+8=18>15 new
subarray count=3,current=8. Final count=3. 3>2, not feasible, left=16. mid=(16+21)//2=18:
current=7,+2=9,+5=14,+10=24>18 count=2,current=10,+8=18<=18 current=18. Final count=2. Feasible, right=18.
mid=(16+18)//2=17: similar check gives count=3 (since 10+8=18>17). Not feasible, left=18. Now left=18=right, loop
ends. Return 18.

SAY THIS —

Tracing nums=[7,2,5,10,8], m=2: binary search narrows the candidate largest sum down step by step, checking feasibility with the greedy counter each time, and converges on 18, which is indeed the minimized largest sum (splitting as [7,2,5] and [10,8], sums 14 and 18).

STEP 8 — Edge Case Test

m equals the length of the array, meaning every element must be its own subarray. Then the answer is simply the largest single element, since each subarray is forced to be one number; the binary search correctly converges to max(nums) since subarrays_needed(max(nums)) always equals the array length in that scenario or fewer, and any smaller candidate would fail some element.

SAY THIS —

Let me check when m equals the array's length, meaning each element must be alone — the answer should just be the largest single element, and the greedy feasibility check confirms that's exactly what the binary search converges to.

Other edge cases to remember

- $m = 1$ (no splitting allowed, answer is the total sum)
- All elements identical
- Array with only one element
- One very large element dominating the rest of the array

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n \log(\text{sum} - \text{max}))$	$O(1)$

SAY THIS —

This is $O(n \log(S))$ time, where S is the range between the sum and the max element, since each binary search step costs $O(n)$ for the feasibility check and there are $O(\log S)$ steps, and $O(1)$ extra space beyond a few counters.

If the Interviewer Pushes Back

Why is binary search on the answer valid here?

Because the feasibility function — 'can this array be split into $\leq m$ subarrays each with sum $\leq \text{mid}$ ' — is monotonic: as mid increases, the number of subarrays needed can only decrease or stay the same, which is exactly the property binary search relies on.

Is the greedy subarray-counting always correct for a fixed `max_sum`?

Yes — for any fixed cap, greedily packing as much as possible into the current subarray before starting a new one always uses the fewest subarrays possible, since delaying a split can never help and could only force an earlier, more restrictive split later.

Can you solve this with Dynamic Programming instead?

Yes, there's a DP formulation where $\text{dp}[i][j]$ represents the minimized largest sum using j subarrays for the first i elements, but that's $O(n^2 m)$ time, which is typically worse than the $O(n \log S)$ binary search approach for larger inputs.

What if numbers could be negative?

This specific greedy/binary-search approach assumes non-negative numbers, since it relies on subarray sums only growing as you add more elements; negative numbers would break both the monotonicity of the feasibility check and the lower/upper bound reasoning.

Common Mistakes

- Setting the initial 'left' bound to 0 instead of $\text{max}(\text{nums})$, which can make the feasibility check incorrectly reject a value smaller than the largest single element.
- Off-by-one in the binary search (using $\leq$ for the loop condition instead of $<$, which can cause infinite loops in this style).
- Forgetting the greedy subarray counter must start with $\text{count}=1$ (the first subarray always exists) rather than 0.

- Assuming this generalizes directly to arrays containing negative numbers.

If I Get Stuck — Recovery

- If stuck: 'Trying every way to split the array is exponential. Let me instead think about binary searching over possible answer VALUES and checking feasibility for each, which is a much smaller search space.'
- If unsure why the bounds are $\max(\text{nums})$ and $\text{sum}(\text{nums})$: reason through both extremes — you can never do better than the largest single element, and you never need to do worse than putting everything in one subarray.
- If the interviewer hints 'think about binary searching on the answer, not the split': confirm and describe the feasibility-check helper function.
- If my binary search doesn't converge: double check I'm using $\text{left} < \text{right}$ with $\text{right} = \text{mid}$ (not $\text{mid} - 1$), matching the same convergence pattern as other 'search on the answer' problems.

Pattern Recognition

Trigger words: 'minimize the maximum', 'split into m parts', 'largest sum as small as possible'

Pattern: Binary Search on the Answer (paired with a greedy feasibility check)

First thought: Can I binary search over possible answer VALUES instead of over configurations, using a greedy or counting check to test feasibility for each candidate?

Common trap: Trying to enumerate split configurations directly instead of recognizing the monotonic feasibility structure.

Memory Hook

KEY INSIGHT

Split Array Largest Sum → "Binary search the answer; greedily count subarrays to test feasibility."

30-Second Recap

Pattern	Binary Search on the Answer
Key idea	Binary search over candidate 'largest sum' values, using a greedy pass to check if that candidate allows
Data structure	No extra structure — running sums and counters.
Main algorithm	Binary Search on the Answer + Greedy feasibility check.
Time	$O(n \log(\text{sum} - \text{max}))$
Space	$O(1)$
Biggest edge case	m equal to the array length (every element its own subarray).

Full Amazon Talk Track

Interviewer: Split this array into m contiguous parts, minimizing the largest part's sum.

Me: Got it — and I just need to return the minimized largest sum, not the actual split, right? Are all numbers non-negative?

Interviewer: Yes to both.

Me: Brute force would try every possible placement of $m-1$ split points, which is exponential — not scalable.

Interviewer: How would you approach it differently?

Me: I'd binary search on the answer itself. The answer is somewhere between the largest single element and the total sum. For a candidate value, I can greedily check how many subarrays are needed if no subarray exceeds that value — pack elements in until adding one more would exceed it, then start fresh.

Me: If that count is within m , the candidate works and I try smaller; if not, I need a bigger candidate. That's a clean monotonic binary search.

Me: Let me code the feasibility helper first, then the binary search around it.

Me: Tracing $[7,2,5,10,8]$ with $m=2$, the search converges to 18, splitting as $[7,2,5]$ and $[10,8]$.

Me: Edge case: if m equals the array length, every element must stand alone, and the answer correctly becomes the largest single element.

Me: Final complexity: $O(n \log(\text{sum} - \text{max}))$ time, since each of the $O(\log)$ binary search steps does an $O(n)$ feasibility check, $O(1)$ extra space.

Pow(x, n)

LeetCode #50 | **MEDIUM** | Pattern: Binary Search / Divide and Conquer (Fast Exponentiation) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We need to implement a function that computes x raised to the integer power n , handling both positive and negative exponents, efficiently.

What I should clarify

- Can n be negative, meaning I need to return 1 divided by x to the power of $|n|$?
- Can n be 0, in which case the answer is always 1?
- Can x be 0 or negative?
- Is there a concern about integer overflow or floating point precision given the language?

SAY THIS —

Just to confirm, n can be negative (meaning I return the reciprocal) or zero (returning 1), and x can be any real number, correct?

STEP 2 — Brute Force First

Idea

Multiply x by itself n times in a simple loop.

SAY THIS —

The most direct approach is to multiply x by itself n times in a loop, accumulating the result.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ since we do n multiplications.

Space Complexity

$O(1)$ extra space.

SAY THIS —

This is $O(n)$ time because we loop once per unit of the exponent, and $O(1)$ space since we just keep a running product.

STEP 4 — Identify the Bottleneck

I'm doing far more multiplications than necessary — if I already know x squared, I've effectively computed two multiplications' worth of exponent in one step, and I can keep doubling that way instead of counting up one at a time.

SAY THIS —

The bottleneck is treating the exponent as something I must count through one unit at a time, when I can jump exponentially by repeatedly squaring the base.

STEP 5 — Optimization

Key insight / pattern: Binary Search / Divide and Conquer (Fast Exponentiation)

I use Fast Exponentiation (also called Binary Exponentiation), a Divide and Conquer technique. To compute x^n , I recursively compute $\text{half} = x^{(n/2)}$, and combine: if n is even, the answer is $\text{half} * \text{half}$; if n is odd, it's $\text{half} * \text{half} * x$ (to account for the leftover single factor). Negative n is handled by computing the positive-power result and taking its reciprocal.

KEY INSIGHT

x^n can be built from $x^{(n/2)}$ by squaring, cutting the exponent in half at each step, so the total number of multiplications needed grows logarithmically with n instead of linearly.

SAY THIS —

I'll handle the negative exponent case by converting it to a positive exponent and taking the reciprocal at the end.

Then I'll recursively compute x to the half power, and square that result.

If the exponent is odd, I multiply in one extra factor of x to account for the leftover.

STEP 6 — Code While Narrating

```
def my_pow(x, n):
    if n < 0:
        x = 1 / x
        n = -n
```

SAY THIS —

If n is negative, I flip the problem: invert x and make the exponent positive, since $x^{(-n)}$ is the same as $(1/x)^n$.

WHY: Handling the sign once up front lets the rest of the function assume a non-negative exponent, simplifying the recursive logic.

```
def fast_pow(base, exp):
    if exp == 0:
        return 1
    half = fast_pow(base, exp // 2)
    if exp % 2 == 0:
        return half * half
    else:
        return half * half * base
```

SAY THIS —

I define a helper that recursively computes base to the exp power. The base case is exponent 0, which is always 1. Otherwise, I compute half the work recursively and square it, adding one more factor of base if the exponent was odd.

WHY: Recursing on $\text{exp} // 2$ halves the problem size at every step, which is exactly what gives this its logarithmic time complexity instead of linear.

```
return fast_pow(x, n)
```

SAY THIS —

Finally, I call the helper with the (possibly adjusted) base and exponent and return its result.

WHY: By this point, any sign handling has already been resolved, so `fast_pow` only ever needs to deal with a non-negative exponent.

STEP 7 — Dry Run

$x=2, n=10$. `fast_pow(2,10)`: `half=fast_pow(2,5)`. `fast_pow(2,5)`: `half=fast_pow(2,2)`. `fast_pow(2,2)`: `half=fast_pow(2,1)`. `fast_pow(2,1)`: `half=fast_pow(2,0)=1`; `exp=1` is odd, return $1*1*2=2$. Back in `fast_pow(2,2)`: `half=2`, `exp=2` even, return $2*2=4$. Back in `fast_pow(2,5)`: `half=4`, `exp=5` odd, return $4*4*2=32$. Back in `fast_pow(2,10)`: `half=32`, `exp=10` even, return $32*32=1024$. Final answer: 1024, matching 2^{10} .

SAY THIS —

Tracing 2 to the 10th power: the recursion halves the exponent each time — 10, 5, 2, 1, 0 — and builds back up by squaring (and multiplying in an extra factor when odd), correctly landing on 1024.

STEP 8 — Edge Case Test

Negative exponent, e.g. $x=2, n=-2$. We flip: x becomes $1/2$, n becomes 2. `fast_pow(0.5, 2)`: `half=fast_pow(0.5,1)=0.5*0.5*0.5=0.125? Let's recompute: fast_pow(0.5,1): half=fast_pow(0.5,0)=1; exp=1 odd, return  $1*1*0.5=0.5$ . Back in fast_pow(0.5,2): half=0.5, exp even, return  $0.5*0.5=0.25$ . Final answer 0.25, matching  $2^{-2} = 1/4$ .`

SAY THIS —

Let me check a negative exponent, like 2 to the power -2 — flipping x to 0.5 and n to 2, the recursion correctly gives 0.25, which is $1/4$, exactly what 2^{-2} should be.

Other edge cases to remember

- $n = 0$ (always returns 1, regardless of x)
- $x = 0$ with a positive exponent (returns 0)
- $x = 1$ or $x = -1$ (result depends only on the parity of n for -1)
- Very large n (verifies the $O(\log n)$ speed advantage over the naive loop)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\log n)$	$O(\log n)$ recursion stack

SAY THIS —

This is $O(\log n)$ time since the exponent halves at every recursive call, and $O(\log n)$ space for the recursion call stack, which could be reduced to $O(1)$ with an iterative version.

If the Interviewer Pushes Back

Can you do this iteratively instead of recursively, to save stack space?

Yes — I can iterate, repeatedly squaring the base and halving the exponent, and whenever the exponent is odd, multiply the current base into a running result before halving; that achieves the same $O(\log n)$ time with $O(1)$ space

instead of $O(\log n)$ recursion depth.

What about $x=0$ and $n=0$ together?

Mathematically 0^0 is often defined as 1 by convention in these problems, so I'd clarify with the interviewer, but the recursive base case ($\text{exp}==0$ returns 1) naturally follows that convention here.

Could there be overflow or precision issues?

Yes, for very large exponents or very small/large x , floating point precision could become an issue; in a language with fixed-size integers, overflow could also matter, so I'd mention that as a real-world caveat depending on the language and constraints.

Can you optimize further?

$O(\log n)$ is already optimal for exponentiation by squaring — there's no known way to compute a general power in fewer than $O(\log n)$ multiplications for arbitrary bases.

Common Mistakes

- Forgetting to flip x to $1/x$ when n is negative, causing wrong (or crashing) results for negative exponents.
- Off-by-one in handling odd vs. even exponents (forgetting the extra factor of base when exp is odd).
- Using recursion without a proper base case, causing infinite recursion or stack overflow.
- Assuming the naive $O(n)$ loop is 'good enough' without recognizing the interviewer likely wants the $\log(n)$ solution.

If I Get Stuck — Recovery

- If stuck: 'The naive loop is $O(n)$, multiplying one factor of x at a time. Let me think about whether I can 'double up' progress instead of only adding one factor per step — that suggests squaring.'
- If I forget to handle negative n : catch it in a dry run with a negative exponent and notice the result is wrong, then add the sign-flip logic.
- If the interviewer hints 'can you avoid n multiplications': pivot to recursively halving the exponent and squaring.
- If recursion depth worries the interviewer: offer the iterative version as a stack-space optimization.

Pattern Recognition

Trigger words: 'x to the power of n', 'compute pow', 'exponentiation'

Pattern: Fast (Binary) Exponentiation — Divide and Conquer

First thought: Can I halve the exponent each step and square the result, instead of counting up one multiplication at a time?

Common trap: Forgetting the extra multiplication needed when the exponent is odd, or mishandling negative exponents.

Memory Hook

KEY INSIGHT

$\text{Pow}(x, n) \rightarrow$ "Halve the exponent, square the result — add one extra factor if odd."

30-Second Recap

Pattern	Binary Search / Divide and Conquer (Fast Exponentiation)
Key idea	Recursively compute half the exponent's result and square it; add one extra factor of x if the exponent is

Data structure	No extra data structure — recursion.
Main algorithm	Fast Exponentiation via Divide and Conquer.
Time	$O(\log n)$
Space	$O(\log n)$ recursion ($O(1)$ iterative)
Biggest edge case	Negative exponent, requiring reciprocal handling.

Full Amazon Talk Track

Interviewer: Implement $\text{pow}(x, n)$ for integer n , which can be negative.

Me: Got it — negative n means I return the reciprocal of the positive-power result, and $n=0$ always gives 1, correct?

Interviewer: Yes.

Me: The direct way is a loop multiplying x by itself n times, which is $O(n)$ time.

Interviewer: Can you do better?

Me: Yes — instead of adding one factor of x at a time, I can recursively compute x to the half-exponent and square that result, cutting the exponent in half every step. If the exponent's odd, I multiply in one extra factor of x .

Me: For negative n , I'll flip x to $1/x$ and make n positive before running that recursive logic.

Me: Let me code that up... base case is exponent 0, returning 1.

Me: Tracing 2 to the 10th power: the exponent halves through 10, 5, 2, 1, 0, and squaring back up gives 1024, correct.

Me: Let me also check a negative exponent, like 2 to the -2 — flipping to 0.5 squared gives 0.25, which is right.

Me: Final complexity: $O(\log n)$ time since the exponent halves each recursive call, $O(\log n)$ recursion stack space, though I could make it iterative for $O(1)$ space.

6. Linked Lists

Pointer manipulation, cycle detection, and cloning with hash maps.

Add Two Numbers

LeetCode #2 | **MEDIUM** | Pattern: Linked List Traversal | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two linked lists representing non-negative integers, where each node holds a single digit stored in reverse order (least significant digit first), and we need to add the two numbers and return the sum as a linked list in the same format.

What I should clarify

- Are the digits stored least-significant-digit first, so I add from the heads directly?
- Can the two lists be different lengths?
- Are the numbers guaranteed non-negative, with no leading zero issues (except the number 0 itself)?
- Should I create a new list, or can I reuse existing nodes?

SAY THIS —

Just to confirm, each list stores a number's digits in reverse order, so I can add corresponding digits directly from the front, carrying over as needed, and the lists might not be the same length, right?

STEP 2 — Brute Force First

Idea

Convert both linked lists into actual integers, add them normally, then convert the sum back into a linked list of digits in reverse order.

SAY THIS —

A simple approach is to convert each linked list into an integer, add the two integers together, and then convert that sum back into a new linked list.

STEP 3 — Brute Force Complexity

Time Complexity

Converting each list takes $O(n)$ or $O(m)$, the addition itself is effectively $O(\max(n,m))$ for big-integer arithmetic, and converting back is $O(\max(n,m)+1)$, so overall $O(n+m)$.

Space Complexity

$O(n+m)$ for the intermediate integers and the result list.

SAY THIS —

This is $O(n+m)$ time and space, but it depends on the language supporting arbitrary-precision integers, and it feels like an indirect way to solve what's fundamentally a digit-by-digit addition problem.

STEP 4 — Identify the Bottleneck

Converting to and from integers works, but it sidesteps the actual structure of the problem and may not be robust for extremely large numbers in languages with fixed integer sizes; the real task is just simulating grade-school addition directly on the digits.

SAY THIS —

Converting to integers works in Python, but it doesn't really solve the intended problem, which is about simulating digit-by-digit addition directly, especially since the lists might have different lengths and I need to handle carries.

STEP 5 — Optimization

Key insight / pattern: Linked List Traversal

I simulate elementary-school addition directly on the two linked lists at the same time. I walk both lists simultaneously, adding corresponding digits plus any carry from the previous step, creating a new node for the digit (sum modulo 10) and carrying forward the tens digit (sum divided by 10). I continue until both lists are exhausted and there's no carry left.

KEY INSIGHT

Because digits are stored least-significant-first, I can add left to right node by node exactly like manual addition, propagating a single carry value forward one step at a time, without ever needing to know the full numeric value of either list.

SAY THIS —

I'll walk both lists at the same time, node by node.

At each step, I sum the two current digits (treating a missing node as 0) plus any carry from before, then create a new node holding that sum modulo 10, and update the carry to sum divided by 10.

I keep going as long as either list still has nodes, or there's a leftover carry.

STEP 6 — Code While Narrating

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def add_two_numbers(l1, l2):
    dummy = ListNode()
    current = dummy
    carry = 0
```

SAY THIS —

I use a dummy head node to simplify building the result list, a 'current' pointer to track where I'm building, and a carry variable starting at 0.

WHY: A dummy head avoids special-casing 'the first node' when building a new linked list, since I can always just attach after dummy and return dummy.next at the end.

```
while l1 or l2 or carry:
    val1 = l1.val if l1 else 0
    val2 = l2.val if l2 else 0
    total = val1 + val2 + carry
```

SAY THIS —

I loop as long as there's a node left in either list, or a carry still to apply. I treat a missing node's value as 0, so shorter lists are handled naturally, and sum both digits plus the carry.

WHY: Continuing while carry is truthy handles the important edge case where adding the final digits produces an extra new digit, like $5+5=10$ needing one more node for that leading 1.

```
        carry = total // 10
        current.next = ListNode(total % 10)
        current = current.next
        l1 = l1.next if l1 else None
        l2 = l2.next if l2 else None

    return dummy.next
```

SAY THIS —

I compute the new carry and the digit to store (total mod 10), attach a new node with that digit, advance my current pointer, and move both list pointers forward if they still have nodes.

WHY: Advancing l1 and l2 only if they exist (using the conditional) prevents crashing on the shorter list once it runs out of nodes, while the longer list keeps contributing digits.

STEP 7 — Dry Run

l1 = 2->4->3 (represents 342), l2 = 5->6->4 (represents 465). Step1: val1=2, val2=5, carry=0, total=7. carry=0, new node 7. Step2: val1=4, val2=6, carry=0, total=10. carry=1, new node 0. Step3: val1=3, val2=4, carry=1, total=8. carry=0, new node 8. l1, l2 both now None, carry=0, loop ends. Result list: 7->0->8, representing 807, which is $342+465=807$. Correct!

SAY THIS —

Tracing l1=2->4->3 and l2=5->6->4 (representing 342 and 465): adding digit by digit with carries gives the result list 7->0->8, representing 807, which matches 342 plus 465.

STEP 8 — Edge Case Test

Lists of different lengths with a trailing carry, e.g. l1=9->9 (99) and l2=1 (1). Step1: val1=9, val2=1, carry=0, total=10, carry=1, node 0. Step2: val1=9, val2=0 (l2 exhausted), carry=1, total=10, carry=1, node 0. Now both lists exhausted but carry=1, loop continues: val1=0, val2=0, carry=1, total=1, carry=0, node 1. Result: 0->0->1, representing 100, which is $99+1=100$. Correct, and shows why looping on 'carry' truthy (not just list existence) matters.

SAY THIS —

Let me check a case with a leftover carry after both lists end, like 99 plus 1 — the loop correctly continues one more step because of the carry, producing 100, which shows why I check 'or carry' in the while condition, not just the lists themselves.

Other edge cases to remember

- Both lists represent 0 (single node with value 0 each)
- One list is much longer than the other
- Sum requires an entirely new leading digit beyond both lists' lengths
- Both lists have only a single node each with no carry needed

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\max(n, m))$	$O(\max(n, m))$

SAY THIS —

This is $O(\max(n, m))$ time since we walk through both lists at most once, where n and m are their lengths, and $O(\max(n, m))$ space for the newly created result list (not counting the input lists themselves).

If the Interviewer Pushes Back

Why not just convert to integers directly?

In Python that works because integers are arbitrary precision, but in many other languages fixed-size integers could overflow for very long lists, and the problem is fundamentally about digit-level list manipulation, so the direct simulation approach is more broadly correct and demonstrates the intended technique.

What if the digits were stored most-significant-first instead?

Then I couldn't add directly from the heads; I'd need to either reverse both lists first (or use a stack to process from the end), add with carries, and build the result list in reverse (or reverse it again at the end).

Can you avoid creating a new list and modify one of the inputs instead?

Yes, I could reuse l1's nodes by updating their values in place and extending with l2's or new nodes as needed, saving some allocation, though it does mutate the input, which I'd clarify is acceptable first.

How do you handle a carry after both lists are exhausted?

The while loop condition explicitly includes 'or carry', so even after both list pointers become None, if there's still a leftover carry, one more node gets created for it, which handles cases like $99 + 1$.

Common Mistakes

- Forgetting to continue the loop when both lists are exhausted but there's still a carry left over.
- Not using a dummy head node, leading to awkward special-casing for the first node.
- Forgetting to advance l1/l2 conditionally (crashing when trying to access .next on None).
- Mixing up $\text{total} // 10$ (carry) and $\text{total} \% 10$ (digit to store).

If I Get Stuck — Recovery

- If stuck: 'This is just elementary school addition, but with digits stored in a linked list instead of an array — let me simulate that directly, adding node by node with a carry.'
- If I forget the dummy-head trick: reason aloud, 'building a new list is easier if I have a placeholder starting node I can always attach to, then return dummy.next at the end.'
- If the interviewer hints 'what happens after both lists end': double check the loop condition includes the carry, not just the list pointers.
- If I get carry/digit reversed: recheck against a concrete example like $9+9=18$, where digit=8 and carry=1, aligning that with mod 10 and integer division by 10.

Pattern Recognition

Trigger words: 'linked list representing a number', 'digits in reverse order', 'add two numbers'

Pattern: Linked List Traversal with Carry Propagation (simulated arithmetic)

First thought: Simulate manual addition directly on the lists, one digit position at a time, carrying forward exactly like on paper.

Common trap: Forgetting the loop must continue if a carry remains even after both lists are exhausted.

Memory Hook

KEY INSIGHT

Add Two Numbers → "Add node by node, carry the tens digit forward, just like paper addition."

30-Second Recap

Pattern	Linked List Traversal
Key idea	Walk both lists simultaneously, summing digits plus carry, building a new list of (sum mod 10) with carry
Data structure	A dummy head node for the result linked list.
Main algorithm	Simultaneous linked list traversal with carry propagation.
Time	$O(\max(n,m))$
Space	$O(\max(n,m))$
Biggest edge case	Leftover carry after both lists are exhausted (e.g., 99+1).

Full Amazon Talk Track

Interviewer: Add these two numbers represented as linked lists, digits in reverse order.

Me: Got it — so I can add corresponding digits directly from the heads, carrying as needed, and the lists could be different lengths, correct?

Interviewer: Yes.

Me: One approach in Python would be converting each list to an actual integer, adding them, and converting back — that works here since Python integers don't overflow, but it doesn't demonstrate the intended digit-by-digit simulation, and wouldn't be safe in other languages.

Me: I'll instead simulate the addition directly: walk both lists together, sum the current digits plus carry, create a node for the result digit, and carry forward the rest.

Me: Let me code that with a dummy head node to simplify building the result list.

Me: Tracing 342 (2→4→3) plus 465 (5→6→4): digit by digit with carries gives 7→0→8, representing 807, which checks out.

Me: Edge case: what if there's a leftover carry after both lists end, like 99 plus 1? My loop condition includes the carry, so it correctly adds one more node, giving 100.

Me: Final complexity: $O(\max(n,m))$ time and space, since I walk both lists once and build a result list of similar length.

Linked List Cycle

LeetCode #141 | **EASY** | Pattern: Fast & Slow Pointers | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the head of a linked list, and we need to determine whether the list contains a cycle, meaning some node's next pointer eventually points back to a previously visited node.

What I should clarify

- Should I return a boolean, or do I need to return the node where the cycle begins?
- Can the list be empty, or have just one node?
- Is there a constraint against using extra memory, like a hash set of visited nodes?
- Can a single node point to itself, forming a one-node cycle?

SAY THIS —

Just to confirm, I need to return true or false for whether a cycle exists, and ideally I should do it in $O(1)$ space rather than tracking visited nodes in a set, correct?

STEP 2 — Brute Force First

Idea

Use a hash set to track every node visited; if I ever revisit a node already in the set, there's a cycle.

SAY THIS —

The straightforward way is to keep a hash set of every node I've visited, and if I ever see a node that's already in the set, that means there's a cycle.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$, since in the worst case (no cycle) we visit every node once.

Space Complexity

$O(n)$ for the hash set of visited nodes.

SAY THIS —

This is $O(n)$ time, but also $O(n)$ space for storing every visited node, which is more memory than should be necessary.

STEP 4 — Identify the Bottleneck

I'm using a whole hash set just to detect 'have I been here before', when two pointers moving at different speeds can detect a cycle without remembering any nodes at all.

SAY THIS —

The bottleneck is that remembering every visited node isn't actually necessary — there's a way to detect a loop using just two pointers and no extra memory.

STEP 5 — Optimization

Key insight / pattern: Fast & Slow Pointers

I use Floyd's Fast & Slow Pointers (Tortoise and Hare). I set two pointers at the head: slow moves one step at a time, fast moves two steps at a time. If there's no cycle, fast will reach the end (None) first. If there IS a cycle, fast will eventually 'lap' slow and they'll point to the same node, confirming a cycle exists.

KEY INSIGHT

If two runners go around a circular track at different speeds, the faster one always eventually catches up to and passes the slower one from behind — that's exactly what guarantees slow and fast will meet if and only if a cycle exists.

SAY THIS —

I'll use two pointers starting at the head: slow moves one node at a time, fast moves two nodes at a time.

If fast ever reaches the end of the list (hits None), there's no cycle.

If slow and fast ever point to the exact same node, that means fast has lapped slow inside a loop, confirming a cycle.

STEP 6 — Code While Narrating

```
def has_cycle(head):  
    slow = head  
    fast = head
```

SAY THIS —

I start both slow and fast pointers at the head of the list.

WHY: Both must start from the same point for the lapping logic to correctly detect a cycle rather than falsely triggering.

```
    while fast and fast.next:  
        slow = slow.next  
        fast = fast.next.next  
        if slow == fast:  
            return True  
  
    return False
```

SAY THIS —

As long as fast (and the node right after it) exists, I move slow one step and fast two steps, and check if they've become equal. If the loop ends because fast ran off the end, there's no cycle.

WHY: Checking 'fast and fast.next' before advancing prevents a crash from trying to access .next on None, which would happen if fast reached the true end of a non-cyclic list.

STEP 7 — Dry Run

List: 3 -> 2 -> 0 -> -4 -> (back to 2, forming a cycle). slow=3, fast=3. Step1: slow=2, fast=0. Not equal. Step2: slow=0, fast=2 (fast wrapped: 0->-4->2). Not equal. Step3: slow=-4, fast=-4 (fast: 2->0->-4). Equal! Return True — a cycle exists.

SAY THIS —

Tracing a list where the tail loops back into the middle: after a few steps, the fast pointer laps around and meets the slow pointer at the same node, confirming there's a cycle.

STEP 8 — Edge Case Test

A list with no cycle at all, e.g. 1->2->3->None. slow and fast start at 1. Step1: slow=2, fast=3. Not equal. Step2: fast.next is None so the while condition fails (fast=3, fast.next=None is falsy... actually need one more check): the loop simply exits once fast or fast.next becomes None, and we return False, correctly identifying no cycle.

SAY THIS —

Let me check a plain list with no cycle, like 1->2->3->None — the fast pointer eventually runs off the end, the loop condition catches that, and we correctly return False.

Other edge cases to remember

- Empty list (head is None)
- Single node with no self-loop
- Single node pointing to itself (a one-node cycle)
- Cycle that starts at the very head of the list

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time because in the worst case fast catches up to slow within one full loop around the cycle (bounded by the list's length), and $O(1)$ space since only two pointers are used, no extra data structure.

If the Interviewer Pushes Back

Why does the fast pointer always catch the slow pointer if there's a cycle?

Because once both pointers are inside the cycle, the gap between them shrinks by exactly one node every step (since fast gains one extra step on slow each iteration), so the gap must eventually reach zero, meaning they meet, as long as the cycle has finite length.

Can you find WHERE the cycle begins, not just whether one exists?

Yes — once slow and fast meet, reset one pointer to the head, keep the other at the meeting point, then move both one step at a time; they'll meet again exactly at the cycle's starting node, using the same math as the 'Find the Duplicate Number' problem.

Can you solve this with recursion?

Not efficiently or safely — recursion would need to detect repeated nodes some other way (like passing a visited set), which brings back the $O(n)$ space cost we were trying to avoid.

What if the list is extremely long with no cycle?

The algorithm still terminates cleanly in $O(n)$ time once fast reaches the true end (None), since there's no cycle to loop around.

Common Mistakes

- Checking only 'while fast' without also checking 'fast.next', causing a crash when advancing fast two steps.
- Starting slow and fast at different nodes instead of both at head.
- Comparing node values instead of node identity/reference (should be 'slow == fast' as node objects, or more precisely 'slow is fast' in Python for clarity, though == on ListNode by default falls back to identity unless overridden).
- Assuming a hash-set approach is necessary when $O(1)$ space is explicitly asked for.

If I Get Stuck — Recovery

- If stuck: 'A hash set works but uses $O(n)$ space. Let me think about whether two pointers moving at different speeds could detect a loop without remembering anything.'
- If I forget the loop guard: reason aloud, 'fast moves two steps, so I need both fast and fast.next to exist before advancing, otherwise I'd crash on a null pointer.'
- If the interviewer hints 'think about runners on a circular track': confirm that's exactly the tortoise-and-hare intuition being used.
- If asked to find the cycle start and I forget the exact reset trick: reason through it as 'the distance from head to cycle start equals the distance from the meeting point to cycle start,' which is the underlying math.

Pattern Recognition

Trigger words: 'linked list cycle', 'does the list loop back on itself', $O(1)$ space

Pattern: Fast & Slow Pointers (Floyd's Cycle Detection)

First thought: Can two pointers moving at different speeds detect a loop without remembering visited nodes?

Common trap: Forgetting to guard fast.next before advancing fast two steps at a time.

Memory Hook

KEY INSIGHT

Linked List Cycle → "Two runners on a track — if there's a loop, the fast one always laps the slow one."

30-Second Recap

Pattern	Fast & Slow Pointers
Key idea	Move slow one step and fast two steps; if they ever meet, there's a cycle; if fast hits None, there isn't.
Data structure	Two pointer variables — no extra structure.
Main algorithm	Floyd's Tortoise and Hare.
Time	$O(n)$
Space	$O(1)$
Biggest edge case	Single node pointing to itself (smallest possible cycle).

Full Amazon Talk Track

Interviewer: Determine if this linked list has a cycle.

Me: Got it — just a true/false answer, and ideally in $O(1)$ space rather than tracking visited nodes, right?

Interviewer: Yes.

Me: A hash set of visited nodes would solve it in $O(n)$ time and space, but since $O(1)$ space is preferred, I'd use two pointers instead.

Me: Slow moves one step at a time, fast moves two. If there's a cycle, fast eventually laps slow and they meet at the same node; if there's no cycle, fast reaches the end first.

Me: Let me code that, being careful to check both fast and fast.next exist before advancing fast.

Me: Tracing a list that loops back into itself: after a few steps, slow and fast do meet at the same node, confirming a cycle.

Me: Edge case: a plain list with no cycle just has fast run off the end, and we correctly return false.

Me: Final complexity: $O(n)$ time, since the pointers converge within one loop around the cycle at most, and $O(1)$ space.

Intersection of Two Linked Lists

LeetCode #160 | **EASY** | Pattern: Two Pointers | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the heads of two singly linked lists, and we need to find the node at which the two lists intersect (share the same tail), or return None if they don't intersect.

What I should clarify

- Does 'intersect' mean the same node object, not just the same value?
- Can the two lists have different lengths before merging?
- Should I avoid modifying either list?
- Is there a memory constraint that would rule out a hash set of nodes?

SAY THIS —

Just to confirm, intersection means they share the exact same node onward (by reference, not just equal values), and the two lists can have different lengths leading up to that point, correct?

STEP 2 — Brute Force First

Idea

For every node in list A, scan through all of list B checking for a matching node (by reference).

SAY THIS —

The brute-force way is, for every node in list A, scan through all of list B to see if any node matches it by reference.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n * m)$ since for each of the n nodes in A, we scan up to m nodes in B.

Space Complexity

$O(1)$ extra space.

SAY THIS —

This is $O(n \text{ times } m)$ time because of the nested scan, and $O(1)$ space since nothing extra is stored.

STEP 4 — Identify the Bottleneck

I'm redundantly rescanning list B for every single node in list A, when there's a smarter way to align both lists so I only need one synchronized pass.

SAY THIS —

The bottleneck is that I'm not aligning the two lists first; if I could make sure both pointers have the same distance left to the end, I could compare node by node in a single pass.

STEP 5 — Optimization

Key insight / pattern: Two Pointers

I use Two Pointers with a clever trick: start one pointer at head A and another at head B. Each pointer walks its own list, and when it reaches the end, it jumps to the head of the OTHER list. Because both pointers collectively travel the same total distance ($\text{lengthA} + \text{lengthB}$) before potentially meeting, they either meet exactly at the intersection node, or both become None at the same time if there's no intersection.

KEY INSIGHT

Switching each pointer to the other list once it runs out effectively equalizes the total distance each pointer travels, canceling out the length difference between the two lists without ever needing to explicitly compute or compare lengths.

SAY THIS —

I'll use two pointers, one starting at each list's head.

Each one walks forward normally, and when it hits the end of its list, it switches to the head of the other list.

Because this equalizes total distance traveled, the two pointers are guaranteed to meet exactly at the intersection node, or both reach None together if there's no intersection.

STEP 6 — Code While Narrating

```
def get_intersection_node(head_a, head_b):
    pointer_a = head_a
    pointer_b = head_b
```

SAY THIS —

I start one pointer at the head of each list.

WHY: These will each traverse their own list first, then switch, which is the mechanism that equalizes total distance traveled.

```
    while pointer_a != pointer_b:
        pointer_a = pointer_a.next if pointer_a else head_b
        pointer_b = pointer_b.next if pointer_b else head_a

    return pointer_a
```

SAY THIS —

I keep moving both pointers forward. Whenever one runs out (becomes None), I redirect it to the other list's head instead. I stop once they're equal, whether that's at the real intersection node or both being None.

WHY: If there's an intersection, both pointers will have traveled $\text{lengthA} + \text{lengthB}$ total steps by the time they reach it, guaranteeing they arrive there simultaneously; if there's no intersection, both simply become None at the same step, and the loop correctly exits with `'pointer_a == pointer_b == None'`.

STEP 7 — Dry Run

A: 4->1->8->4->5, B: 5->6->1->8->4->5, intersecting at node with value 8 (shared tail: 8->4->5). pointer_a starts at 4(A), pointer_b at 5(B). They walk their own lists; pointer_a reaches None after 5 steps and switches to head_b; pointer_b reaches None after 6 steps and switches to head_a. Since pointer_a's path length (5 + up to intersection in B) equals pointer_b's path length (6 + up to intersection in A) by construction, they land on the shared node with value 8 at the same step. Return that node.

SAY THIS —

With list A having length 5 and list B having length 6, both sharing a tail starting at the node with value 8: after each pointer finishes its own list and switches to the other, they arrive at the node with value 8 at exactly the same step, correctly identifying the intersection.

STEP 8 — Edge Case Test

No intersection at all, e.g. two completely separate lists. Both pointers will traverse lengthA + lengthB steps total, switching once each, and both become None simultaneously right after their second pass finishes, satisfying 'pointer_a != pointer_b' becoming false (None == None), correctly returning None.

SAY THIS —

Let me check two lists that never intersect at all — both pointers switch once, and eventually both become None on the same step, so the loop correctly exits and returns None.

Other edge cases to remember

- One list is empty (None) from the start
- The intersection point is the very first node of both lists (identical lists)
- The intersection point is the very last node (only the tail node is shared)
- Lists of equal length before intersecting

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n + m)$	$O(1)$

SAY THIS —

This is $O(n + m)$ time since each pointer traverses at most the combined length of both lists once, and $O(1)$ space since only two pointer variables are used, with no extra data structure.

If the Interviewer Pushes Back

Why does switching pointers guarantee they meet at the intersection?

Because by the time both pointers have switched once, each has traveled a total distance of lengthA + lengthB, meaning they're now the same distance from the end of the combined path, so from the intersection point onward they're perfectly synchronized.

Can you solve this with a hash set instead?

Yes — walk list A, storing every node in a hash set, then walk list B checking if any node is already in that set; that's $O(n+m)$ time but $O(n)$ space, so the two-pointer approach is preferred when $O(1)$ space matters.

What if the lists have cycles?

This problem assumes acyclic (non-circular) singly linked lists; if cycles were possible, this approach could loop indefinitely, and I'd need cycle-detection logic layered on top first.

Can you compute the lengths first and align that way instead?

Yes, that's an equivalent alternative: compute both lengths, advance the pointer on the longer list by the difference, then walk both pointers together until they match; it's also $O(n+m)$ time and $O(1)$ space, just a bit more code than the switching trick.

Common Mistakes

- Comparing node values instead of node references/identity, which could give false positives if values coincidentally match.
- Forgetting to redirect a pointer to the OTHER list's head (not its own head) when it runs out.
- Assuming both lists must be the same length before intersecting.
- Not handling the case where one or both lists are empty from the start.

If I Get Stuck — Recovery

- If stuck: 'A hash set of visited nodes from one list would let me check the other list in $O(n+m)$ time and $O(n)$ space. Since I'd prefer $O(1)$ space, let me think about how to align the two lists without extra memory.'
- If I redirect a pointer to the wrong head: verify with a dry run comparing total distances traveled by each pointer, which reveals the mistake immediately.
- If the interviewer hints 'think about equalizing the paths': confirm and describe the switch-to-the-other-list trick.
- If unsure this terminates for non-intersecting lists: reason through it — both pointers become None at the same step after their second pass, so the loop does terminate.

Pattern Recognition

Trigger words: 'find the intersection of two linked lists', 'shared tail node'

Pattern: Two Pointers with a length-equalizing switch trick

First thought: Can I align both pointers so they've traveled equal total distances by the time they might meet?

Common trap: Comparing values instead of node identity, or redirecting a pointer to its own head instead of the other list's head.

Memory Hook

KEY INSIGHT

Intersection of Linked Lists → "When you hit your own end, switch to the other list — you'll meet at the join."

30-Second Recap

Pattern	Two Pointers
Key idea	Two pointers switch to the other list's head when they run out, equalizing total distance traveled so they meet
Data structure	Two pointer variables — no extra structure.
Main algorithm	Two Pointers with a list-switching trick.
Time	$O(n + m)$
Space	$O(1)$
Biggest edge case	Lists that never intersect at all.

Full Amazon Talk Track

Interviewer: Find the node where these two linked lists intersect, or return null.

Me: Got it — intersection means the exact same node onward, not just equal values, and the lists could have different lengths before that point, right?

Interviewer: Yes.

Me: Brute force: for every node in A, scan all of B for a match — $O(n \text{ times } m)$ time.

Interviewer: How would you speed that up?

Me: I'd use two pointers, one starting at each head. Each walks its own list, and when it reaches the end, it switches to the other list's head instead of stopping. That equalizes the total distance both pointers travel, so they're guaranteed to meet exactly at the intersection, or both hit null at the same time if there isn't one.

Me: Let me code that — the loop just keeps advancing both pointers, redirecting on null, until they're equal.

Me: Tracing lists of length 5 and 6 sharing a tail starting at a node with value 8: after each pointer switches once, they land on that shared node at the same step.

Me: Edge case: two lists that never intersect — both pointers become null at the same step after switching once each, correctly returning null.

Me: Final complexity: $O(n+m)$ time since each pointer travels at most the combined length once, $O(1)$ space.

Copy List with Random Pointer

LeetCode #138 | **MEDIUM** | Pattern: Hash Map (Linked List Cloning) | Advanced pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a linked list where each node has a 'next' pointer and also a 'random' pointer that can point to any node in the list (or None), and we need to create a deep copy of the entire list, including correctly wired random pointers.

What I should clarify

- Can the random pointer point to any node in the list, including itself, or to None?
- Should the copy be entirely independent of the original (a true deep copy)?
- Is there a preferred space complexity, like $O(1)$ extra beyond the output?
- Can the list be empty?

SAY THIS —

Just to confirm, each node has a next pointer and a random pointer that can point anywhere in the list or be null, and I need a fully independent deep copy with all pointers correctly re-wired, correct?

STEP 2 — Brute Force First

Idea

First pass: create a copy of each node (values only, no pointers set). Second pass: use a hash map from original node to copied node to correctly wire up both next and random pointers on the copies.

SAY THIS —

A clean approach is a two-pass method: first create all the copied nodes and remember the mapping from original to copy in a hash map, then do a second pass to wire up next and random pointers using that map.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ for each of the two passes, so $O(n)$ overall.

Space Complexity

$O(n)$ for the hash map storing original-to-copy mappings.

SAY THIS —

This is $O(n)$ time for the two passes, and $O(n)$ space for the hash map that lets me look up the copy corresponding to any original node.

STEP 4 — Identify the Bottleneck

This hash map approach is already quite good, but the interviewer may ask for an $O(1)$ extra space solution, which requires a cleverer way to look up 'the copy of this node' without a separate map.

SAY THIS —

The main hash map solution is solid at $O(n)$ time and space; the follow-up challenge is finding a way to avoid the extra $O(n)$ space for the map, if the interviewer wants that.

STEP 5 — Optimization

Key insight / pattern: Hash Map (Linked List Cloning)

The Hash Map method: create a copy for every original node first, storing pairs in a map keyed by the original node. Then do a second pass over the original list, and for each original node, set `copy.next = map[original.next]` and `copy.random = map[original.random]` (handling None correctly). For the $O(1)$ -space follow-up, an alternative technique interleaves each copied node directly after its original in the same list (original1 -> copy1 -> original2 -> copy2 -> ...), which lets you set random pointers using `original.random.next` (the copy right after the original's random target), then finally splits the interleaved list back into two separate lists.

KEY INSIGHT

The hash map gives $O(1)$ average lookup from any original node to its corresponding copy, which is exactly what's needed to correctly wire up random pointers that can point anywhere in the list, since without such a lookup you'd have no efficient way to find 'the copy of that specific node'.

SAY THIS —

First pass: walk the original list and create a copy of each node's value only, storing a mapping from original node to its copy in a hash map.

Second pass: walk the original list again, and for each original node, use the map to set the copy's next and random pointers to the copies of the corresponding original targets.

This correctly handles random pointers pointing anywhere in the list, including forward, backward, to itself, or to None.

STEP 6 — Code While Narrating

```
class Node:
    def __init__(self, val, next=None, random=None):
        self.val = val
        self.next = next
        self.random = random

def copy_random_list(head):
    if not head:
        return None
    old_to_new = {}
```

SAY THIS —

I handle the empty list case immediately, then set up a hash map to track the mapping from each original node to its new copy.

WHY: Handling None up front avoids unnecessary special-casing later, and the map is the core structure that lets me resolve random pointers correctly regardless of where they point.

```
current = head
while current:
    old_to_new[current] = Node(current.val)
    current = current.next
```

SAY THIS —

In the first pass, I walk the original list and create a brand-new node for each original node, storing just its value for now, and record the mapping in the hash map.

WHY: I deliberately don't wire up next or random pointers yet, since at this point I might not have created the copy of a node that a random pointer needs to point to.

```
current = head
while current:
    copy = old_to_new[current]
    copy.next = old_to_new.get(current.next)
    copy.random = old_to_new.get(current.random)
    current = current.next

return old_to_new[head]
```

SAY THIS —

In the second pass, I walk the original list again, and for each original node, I look up its copy and set the copy's next and random pointers using the map, translating from original nodes to their corresponding copies. Using .get() naturally returns None when there's no mapping, i.e. when the original pointer was None.

WHY: By this point every original node already has a corresponding copy in the map, so translating any pointer (next or random) via the map is guaranteed to succeed and correctly handles None targets too.

STEP 7 — Dry Run

List: A(random->C) -> B(random->A) -> C(random->C). First pass creates copies A', B', C' and map = {A:A', B:B', C:C'}. Second pass: A'.next = map[B] = B'; A'.random = map[C] = C'. B'.next = map[C] = C'; B'.random = map[A] = A'. C'.next = map[None] = None; C'.random = map[C] = C'. Result: fully independent copy A'->B'->C' with A'.random=C', B'.random=A', C'.random=C', exactly mirroring the original structure but with all-new nodes.

SAY THIS —

Tracing a 3-node list with various random pointers: after the first pass creates all the copies, the second pass correctly rewires every next and random pointer using the map, producing a fully independent structural clone.

STEP 8 — Edge Case Test

A node whose random pointer points to itself, e.g. node C with C.random = C. In the map, map[C] = C', and when wiring C's copy, copy.random = map[C] = C', so the copy's random pointer also points to itself — correctly preserved.

SAY THIS —

Let me check a node whose random pointer points to itself — the map correctly resolves that to the node's own copy, so the cloned node's random pointer also points to itself, preserving that structure.

Other edge cases to remember

- Empty list (head is None)
- Single node with random pointing to itself or to None
- Every node's random pointer set to None
- Random pointers all pointing to the very first node

STEP 9 — Final Complexity

	Time	Space
Hash map approach	$O(n)$	$O(n)$
Interleaving approach	$O(n)$	$O(1)$ extra

SAY THIS —

The hash map solution is $O(n)$ time for the two passes and $O(n)$ space for the map; the interleaving follow-up technique achieves the same $O(n)$ time but only $O(1)$ extra space, at the cost of more intricate pointer manipulation.

If the Interviewer Pushes Back

Can you do this in $O(1)$ extra space?

Yes, using the interleaving technique: insert each copy directly after its original node in the same list, so `original.next` is always the copy; then set each copy's `random` as `original.random.next` (the copy right after wherever the original's `random` points), and finally unweave the interleaved list back into two separate lists.

Why do you need two passes with the hash map approach?

Because a node's `random` pointer might point to a node that hasn't been copied yet if I tried to wire everything in a single pass, so I first create all copies (pass one), and only then safely wire up pointers using the completed map (pass two).

What if two different original nodes had the same value?

That's fine — the hash map keys are the actual node objects (references), not values, so distinguishing between nodes with identical values is never an issue.

Can you avoid a hash map with a different data structure?

The interleaving approach effectively replaces the hash map with the list structure itself (copy is always 'next' of original), which is the main alternative; otherwise, some structure that maps original-to-copy is fundamentally necessary for resolving arbitrary random pointers.

Common Mistakes

- Trying to wire up random pointers during the very first pass, before all nodes have been copied, causing missing map entries.
- Using `.get()` incorrectly or forgetting it (using indexing directly) which would raise an error when a pointer is `None`.
- Forgetting that map keys should be the original node objects, not values (since values might not be unique).
- For the $O(1)$ -space version, forgetting to fully unweave the interleaved list at the end, leaving the original list corrupted.

If I Get Stuck — Recovery

- If stuck: 'Random pointers can point anywhere, including nodes I haven't processed yet, so I need some way to look up 'the copy of any given original node' — a hash map does exactly that.'
- If I try to do it in one pass and get stuck: recognize the issue out loud — 'I don't yet have the copy of a node my random pointer needs, so I should split this into two passes: create all copies first, then wire pointers.'
- If the interviewer asks for $O(1)$ space and I don't remember the interleaving trick: describe the goal (avoid a separate map) and reason toward interleaving originals and copies as a way to 'store' the mapping implicitly in the list structure itself.

- If confused about `get()` vs indexing: remember `dict.get(key)` returns `None` by default for a missing key, which is exactly the behavior needed when an original pointer is `None`.

Pattern Recognition

Trigger words: 'deep copy a linked list', 'random pointer', 'clone a graph/list with arbitrary pointers'

Pattern: Hash Map for original-to-copy lookup (interleaving as an $O(1)$ -space alternative)

First thought: Arbitrary pointers mean I need a fast way to translate 'this original node' into 'its corresponding copy' — a hash map, or an implicit structural trick.

Common trap: Trying to wire pointers in the same pass as creating copies, before every node has been cloned.

Memory Hook

KEY INSIGHT

Copy List with Random Pointer → "Map every original to its copy first, then rewire using the map."

30-Second Recap

Pattern	Hash Map (Linked List Cloning)
Key idea	Two passes: first create all copies and record original-to-copy mappings, then rewire next/random pointers
Data structure	Hash map from original node to copied node.
Main algorithm	Two-pass clone-then-wire (or interleaving for $O(1)$ space).
Time	$O(n)$
Space	$O(n)$ [$O(1)$ extra with interleaving]
Biggest edge case	A node whose random pointer points to itself.

Full Amazon Talk Track

Interviewer: Deep copy this linked list, where each node also has a random pointer that can point anywhere in the list.

Me: Got it — the copy needs to be a fully independent structure with correctly wired next and random pointers, and random can point to any node or null, right?

Interviewer: Yes.

Me: I'll do this in two passes. First, I walk the list and create a copy of every node's value, storing a mapping from each original node to its copy in a hash map.

Me: Then, in a second pass, I walk the original list again and use the map to wire up each copy's next and random pointers, translating original references into copy references.

Me: Let me code that... using `dict.get()` so a null pointer in the original correctly maps to null in the copy.

Me: Tracing a small 3-node example with various random pointers, the second pass correctly reproduces the exact same structure, but entirely with new nodes.

Me: Edge case: a node whose random pointer points to itself — the map correctly resolves that to the node's own copy.

Me: Final complexity: $O(n)$ time for the two passes, $O(n)$ space for the map.

Interviewer: Can you do it in $O(1)$ extra space?

Me: Yes — by interleaving each copy directly after its original in the same list, I can use `original.random.next` to find the copy of the random target without a separate map, then unweave the two lists at the end.

7. Trees & BST

Recursive tree traversal, BST properties, and tree construction.

Maximum Depth of Binary Tree

LeetCode #104 | **EASY** | Pattern: DFS / Recursion (Tree Traversal) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the root of a binary tree, and we need to find its maximum depth, meaning the number of nodes along the longest path from the root down to the farthest leaf.

What I should clarify

- What should the depth of an empty tree (null root) be — 0?
- Is depth counted in nodes or edges (a single-node tree has depth 1)?
- Is the tree guaranteed to be a valid binary tree (no cycles)?
- Should I use recursion, or is an iterative solution preferred?

SAY THIS —

Just to confirm, depth is measured in number of nodes along the longest root-to-leaf path, and an empty tree has depth 0, correct?

STEP 2 — Brute Force First

Idea

There isn't really a 'brute force' distinct from the natural recursive solution here, but one could think of it as: explicitly enumerate every root-to-leaf path and take the longest one's length.

SAY THIS —

I could explicitly find every root-to-leaf path and take the maximum length among them, though that's effectively the same work as the natural recursive approach, just framed differently.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ since every node must be visited to know all paths.

Space Complexity

$O(h)$ for the recursion/path stack, where h is the tree's height.

SAY THIS —

Even this framing is $O(n)$ time since we must visit every node, and $O(h)$ space for tracking the current path or recursion depth.

STEP 4 — Identify the Bottleneck

There isn't really a bottleneck to eliminate here — visiting every node is unavoidable since depth depends on the whole tree structure; the key is recognizing that this is naturally a self-similar (recursive) problem.

SAY THIS —

There's no wasted work to cut out here; the interesting part is recognizing this as a classic recursive definition rather than something that needs a separate optimization step.

STEP 5 — Optimization

Key insight / pattern: DFS / Recursion (Tree Traversal)

I use straightforward DFS Recursion. The maximum depth of a tree rooted at any node is 1 (for the node itself) plus the larger of the maximum depths of its left and right subtrees. The base case is a null node, which has depth 0.

KEY INSIGHT

A tree's depth is defined in terms of its subtrees' depths, which is the definition of a recursive structure: 'depth of the whole tree' = 1 + max(depth of left subtree, depth of right subtree), with null nodes contributing 0.

SAY THIS —

I'll solve this recursively: the base case is a null node, contributing depth 0.

For any real node, I recursively find the max depth of its left subtree and its right subtree.

The node's own depth is 1 plus whichever of those two is larger.

STEP 6 — Code While Narrating

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def max_depth(root):
    if root is None:
        return 0
```

SAY THIS —

My base case: an empty subtree (null node) has depth 0.

WHY: This is what makes the recursion well-founded — every recursive call eventually reaches a null child and stops, since the tree is finite.

```
    left_depth = max_depth(root.left)
    right_depth = max_depth(root.right)
    return 1 + max(left_depth, right_depth)
```

SAY THIS —

I recursively compute the depth of the left and right subtrees, then return 1 (for the current node) plus whichever subtree is deeper.

WHY: The '1 +' accounts for the current node itself being part of the path, and taking the max of the two subtree depths ensures I always follow the longest possible path down.

STEP 7 — Dry Run

Tree: 3 is root, left child 9 (leaf), right child 20 with children 15 and 7 (both leaves). $\text{max_depth}(3)$: $\text{left}=\text{max_depth}(9)=1+\text{max}(0,0)=1$. $\text{right}=\text{max_depth}(20)$: $\text{left}=\text{max_depth}(15)=1$, $\text{right}=\text{max_depth}(7)=1$, so $\text{max_depth}(20)=1+\text{max}(1,1)=2$. Back at root: $\text{max_depth}(3)=1+\text{max}(1,2)=3$. Final answer: 3.

SAY THIS —

Tracing a small tree with root 3, left child 9, and right child 20 (which has two leaf children 15 and 7): the left side gives depth 1, the right side gives depth 2, so the whole tree's depth is 1 plus the max of those, which is 3.

STEP 8 — Edge Case Test

Empty tree, root = None. $\text{max_depth}(\text{None})$ immediately returns 0 via the base case, without any further recursion — correctly handling the smallest possible input.

SAY THIS —

Let me check the empty tree case, where root is null — the base case immediately returns 0, which is exactly right.

Other edge cases to remember

- A tree that's just a single node (depth 1)
- A completely unbalanced tree that's really just a linked list (depth equals number of nodes)
- A perfectly balanced tree
- A tree where one subtree is much deeper than the other

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(h)$

SAY THIS —

This is $O(n)$ time since we visit every node exactly once, and $O(h)$ space for the recursion call stack, where h is the height of the tree (which is $O(\log n)$ for a balanced tree, or $O(n)$ for a completely skewed one).

If the Interviewer Pushes Back

Can you solve this iteratively instead of recursively?

Yes — using a BFS level-order traversal with a queue, counting how many levels we process; each level fully dequeued represents one increment of depth, giving the same $O(n)$ time but avoiding recursion stack usage in favor of an explicit queue.

What's the space complexity concern with recursion here?

For a very deep, unbalanced tree (essentially a linked list), the recursion depth could be $O(n)$, risking a stack overflow in some languages/environments, which is a good reason to mention the iterative BFS alternative as a more robust option.

Can you find the minimum depth instead?

Similar recursive structure, but you need to be careful: if one subtree is null and the other isn't, minimum depth should follow the non-null subtree, not naively take $\min(0, \text{other})$ which would incorrectly treat a missing subtree as if it were a valid short path.

Can you optimize this further?

No, $O(n)$ is optimal since we must visit every node at least once to know the true maximum depth of the whole tree.

Common Mistakes

- Forgetting the base case (null node returning 0), causing infinite recursion or a crash.
- Off-by-one: returning $\max(\text{left_depth}, \text{right_depth})$ without adding 1 for the current node.
- Confusing 'depth in nodes' with 'depth in edges' (off by one from the expected convention).
- For the minimum-depth variant, incorrectly handling a node with only one child.

If I Get Stuck — Recovery

- If stuck: 'This looks like it can be defined recursively — the depth of the tree depends on the depth of its subtrees, so let me write the recursive relationship directly.'
- If I forget the base case: remind myself that recursion always needs a stopping point, and a null node is the natural one here.
- If the interviewer hints 'can you do it without recursion': pivot to BFS with a queue, counting levels.
- If I'm unsure about off-by-one: verify with the smallest example, a single-node tree, which should give depth 1, not 0.

Pattern Recognition

Trigger words: 'maximum depth', 'height of a binary tree'

Pattern: DFS Recursion (post-order-like: process children first, then combine)

First thought: Depth of a tree is naturally defined in terms of the depths of its subtrees — write the recursive relationship directly.

Common trap: Forgetting to add 1 for the current node, or mishandling the null base case.

Memory Hook

KEY INSIGHT

Max Depth $\rightarrow$ "1 plus whichever child subtree goes deeper."

30-Second Recap

Pattern	DFS / Recursion (Tree Traversal)
Key idea	Recursively compute $1 + \max(\text{depth of left subtree}, \text{depth of right subtree})$, with null returning 0.
Data structure	Implicit recursion call stack — no extra data structure.
Main algorithm	DFS Recursion.
Time	$O(n)$
Space	$O(h)$
Biggest edge case	Empty tree (root is null).

Full Amazon Talk Track

Interviewer: Find the maximum depth of this binary tree.

Me: Just to confirm, depth is counted in nodes along the longest root-to-leaf path, and an empty tree has depth 0, right?

Interviewer: Yes.

Me: This is naturally recursive: the depth of the tree is 1 plus whichever of its two subtrees is deeper, with a null node contributing 0.

Me: Let me code that directly — base case null returns 0, otherwise recurse on both children and combine.

Me: Tracing a small tree with root 3, left child 9, and right child 20 having two leaf children: I get depth 3, which matches the expected longest path.

Me: Edge case: an empty tree just returns 0 immediately from the base case.

Me: Final complexity: $O(n)$ time since every node is visited once, $O(h)$ space for the recursion stack, where h is the tree's height.

Interviewer: Can you do it without recursion?

Me: Yes, with a BFS queue, counting how many levels get fully processed — that avoids recursion stack depth issues on very unbalanced trees.

Same Tree

LeetCode #100 | **EASY** | Pattern: DFS / Recursion (Tree Traversal) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the roots of two binary trees, and we need to determine whether they are structurally identical and have the same node values at every position.

What I should clarify

- Does 'same' mean identical structure AND identical values at every node?
- Can either tree be empty?
- Should I compare node values with equality (`==`), assuming simple comparable types?
- Is there a size constraint that matters for a recursive vs iterative choice?

SAY THIS —

Just to confirm, two trees are the same only if they have identical structure and identical values at every corresponding node, and either tree could be empty, correct?

STEP 2 — Brute Force First

Idea

Serialize both trees into some canonical string or list representation (e.g., pre-order with explicit null markers) and compare those representations directly.

SAY THIS —

One approach is to serialize both trees into a string or list, including markers for null children, and then just compare the two serializations directly.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ to serialize each tree, plus $O(n)$ to compare the serializations, so $O(n)$ overall.

Space Complexity

$O(n)$ for the serialized representations.

SAY THIS —

This is $O(n)$ time and space, but it does extra work building intermediate representations that aren't strictly necessary.

STEP 4 — Identify the Bottleneck

Building full serialized representations of both trees is unnecessary overhead when I can compare the trees directly, node by node, stopping early the moment I find any mismatch.

SAY THIS —

The bottleneck is doing extra serialization work when I can compare the two tree structures directly and short-circuit as soon as any difference is found.

STEP 5 — Optimization

Key insight / pattern: DFS / Recursion (Tree Traversal)

I use direct DFS Recursion comparing both trees simultaneously. At each pair of corresponding nodes: if both are None, they're equal (and I stop this branch). If exactly one is None, they're not equal (structural mismatch). If both exist, I check their values match, and recursively check that their left subtrees match each other and their right subtrees match each other.

KEY INSIGHT

Two trees are identical precisely when their roots match (both present/absent and same value) AND their left subtrees are identical to each other AND their right subtrees are identical to each other — which is directly expressible as a recursive base case plus combination.

SAY THIS —

I'll recursively walk both trees at the same time, comparing corresponding nodes as I go.

If both current nodes are null, that branch matches. If only one is null, or the values differ, the trees don't match.

Otherwise, I recursively check that both left subtrees match and both right subtrees match.

STEP 6 — Code While Narrating

```
def is_same_tree(p, q):  
    if p is None and q is None:  
        return True  
    if p is None or q is None:  
        return False
```

SAY THIS —

First I handle the two null-related base cases: if both nodes are null, they trivially match; if only one is null, the structures differ, so they don't match.

WHY: These two checks together correctly distinguish 'both empty here' (fine) from 'one has a node the other doesn't' (a structural mismatch), which must be caught before comparing `.val` on a potentially null node.

```
    if p.val != q.val:  
        return False  
  
    return is_same_tree(p.left, q.left) and is_same_tree(p.right, q.right)
```

SAY THIS —

If both nodes exist but their values differ, the trees don't match. Otherwise, I recursively check that both left subtrees match each other, and both right subtrees match each other, requiring both to be true.

WHY: Using 'and' ensures that if either subtree comparison fails, the overall result is correctly False, and Python's short-circuit evaluation means we skip the right-subtree check as soon as the left one fails, saving unnecessary work.

STEP 7 — Dry Run

p: 1 with left 2, right 3. q: 1 with left 2, right 3. `is_same_tree(1,1)`: values match. Check left: `is_same_tree(2,2)`: both null children, values match, True. Check right: `is_same_tree(3,3)`: both null children, values match, True. Overall: True and True = True.

SAY THIS —

Tracing two identical small trees, each with root 1 and children 2 and 3: every level's values match and every null-child comparison lines up, so the overall result correctly comes out true.

STEP 8 — Edge Case Test

One tree has an extra node the other doesn't, e.g. p: 1 with left 2, q: 1 with left 2 and right 3. `is_same_tree(1,1)`: values match. Left: `is_same_tree(2,2)` True. Right: `is_same_tree(None, 3)`: one is None and the other isn't, returns False immediately. Overall: True and False = False, correctly identifying the structural difference.

SAY THIS —

Let me check a case where one tree has an extra node the other lacks — the null-vs-non-null check on that side correctly returns False, so the overall comparison is False, exactly as expected.

Other edge cases to remember

- Both trees are empty (should be considered the same)
- Trees with identical structure but one differing value somewhere
- Single-node trees with equal or different values
- Deeply nested trees requiring many recursive comparisons

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\min(n, m))$	$O(\min(h1, h2))$

SAY THIS —

This is $O(\min(n, m))$ time since the recursion can stop as soon as any mismatch is found, in the worst case visiting every node of the smaller tree, and $O(\min(h1, h2))$ space for the recursion stack, bounded by the shallower comparison path before a mismatch or completion.

If the Interviewer Pushes Back

Can you solve this iteratively?

Yes, using two stacks (or one stack of paired nodes) and pushing/popping corresponding node pairs, doing the same null/value checks manually instead of relying on the call stack.

Why check both p is None and q is None separately from p is None or q is None?

Because I need three distinct outcomes: both null (match, stop), exactly one null (mismatch, stop), or both non-null (continue comparing values and children) — the two checks together cleanly cover all three cases in the right order.

What if node values could be unordered/incomparable types?

As long as equality (`==`) is well-defined for the value type, this approach works regardless of ordering; if values were more complex objects, I'd want to confirm equality is defined meaningfully for them.

Can you optimize further?

No, $O(n)$ in the worst case is necessary since confirming two identical trees requires visiting every node of at least one of them.

Common Mistakes

- Trying to access `.val` before checking for `None`, causing a crash on a null node.
- Using 'or' instead of 'and' when combining the left and right subtree comparisons.
- Forgetting the 'exactly one is null' case, only checking 'both null' and 'both non-null'.
- Comparing only root values without recursing into children at all.

If I Get Stuck — Recovery

- If stuck: 'Two trees are the same if their roots match and both pairs of subtrees are also the same — that's directly recursive.'
- If I get a null-pointer-style error: check that I've handled the `None` cases before ever touching `.val` or `.left/.right`.
- If the interviewer hints 'think about what could be different between two nodes': enumerate the cases — different values, one has a child the other doesn't.
- If unsure about early stopping: mention that Python's 'and' short-circuits, so we naturally stop recursing further once a mismatch is found.

Pattern Recognition

Trigger words: 'are these two trees identical', 'same structure and values'

Pattern: DFS Recursion (simultaneous traversal of two trees)

First thought: Two trees match if their roots match and both corresponding subtree pairs also match — a direct recursive definition.

Common trap: Accessing node attributes before properly checking for `None` on either side.

Memory Hook

KEY INSIGHT

Same Tree → "Match the roots, then recursively match both subtree pairs."

30-Second Recap

Pattern	DFS / Recursion (Tree Traversal)
Key idea	Compare both trees node by node simultaneously; null/null matches, one-null mismatches, otherwise compare values
Data structure	Implicit recursion call stack.
Main algorithm	Simultaneous DFS Recursion on two trees.
Time	$O(\min(n, m))$
Space	$O(\min(h_1, h_2))$
Biggest edge case	Trees differing only by one extra/missing node somewhere.

Full Amazon Talk Track

Interviewer: Determine if these two binary trees are the same.

Me: Got it — same means identical structure and identical values at every corresponding node, and either tree could be empty, correct?

Interviewer: Yes.

Me: I'll compare both trees recursively at the same time. If both current nodes are null, that part matches. If only one is null, or the values differ, they don't match. Otherwise, I recurse into both left and right children.

Me: Let me code that with the base cases first, then the value check and recursive combination.

Me: Tracing two identical small trees, 1 with children 2 and 3 on both sides: every comparison matches, so the result is true.

Me: Edge case: if one tree has an extra node the other doesn't, the null-vs-non-null check catches that and correctly returns false.

Me: Final complexity: $O(\min(n,m))$ time since we can stop as soon as a mismatch is found, and $O(\min(h1,h2))$ space for the recursion.

Validate Binary Search Tree

LeetCode #98 | **MEDIUM** | Pattern: DFS with Valid Range (or In-order Traversal) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the root of a binary tree, and we need to determine whether it's a valid Binary Search Tree, meaning every node's value is strictly greater than all values in its left subtree and strictly less than all values in its right subtree.

What I should clarify

- Are duplicate values allowed anywhere in the tree?
- Should I compare only against immediate parent/children, or against ALL ancestors?
- Is an empty tree considered a valid BST?
- Are node values guaranteed to fit within normal integer ranges (relevant for boundary handling)?

SAY THIS —

Just to confirm, every node needs to be strictly greater than everything in its left subtree and strictly less than everything in its right subtree, not just compared to its immediate children, and duplicates aren't allowed, correct?

STEP 2 — Brute Force First

Idea

For every node, check only against its immediate left and right children, ensuring left child is smaller and right child is bigger.

SAY THIS —

A tempting but incorrect shortcut is to just check that each node's immediate children are smaller (left) and bigger (right) than it, without considering further ancestors.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ if done this way, since we'd check each node once, but it produces WRONG answers.

Space Complexity

$O(h)$ for recursion.

SAY THIS —

This naive local check is $O(n)$ time, but it's actually incorrect — it misses violations from a node further up the tree, like a right-subtree node that's smaller than a grandparent, which is a classic BST-validation bug.

STEP 4 — Identify the Bottleneck

The real issue isn't speed, it's correctness: checking only immediate parent-child relationships misses the requirement that a node must satisfy the BST property with respect to ALL of its ancestors, not just its direct parent.

SAY THIS —

The bug in the naive approach is that it only checks direct parent-child pairs, but the BST property is actually about the whole subtree relative to every ancestor, not just the immediate one.

STEP 5 — Optimization

Key insight / pattern: DFS with Valid Range (or In-order Traversal)

I use DFS with a Valid Range passed down through recursion. Each recursive call tracks a (low, high) bound that the current node's value must strictly fall within. When I recurse into the left child, I tighten the upper bound to the current node's value (since everything in the left subtree must be less than it); when I recurse into the right child, I tighten the lower bound to the current node's value. If a node ever violates its passed-down bounds, the whole tree is invalid.

KEY INSIGHT

A node's value isn't just constrained by its immediate parent, but by every ancestor whose subtree it belongs to; passing down a shrinking (low, high) range as we recurse correctly accumulates all of those ancestor constraints without having to explicitly check every ancestor at every node.

SAY THIS —

I'll recursively validate each node against a valid range of (low, high) that gets passed down and tightened as I go deeper.

Going left tightens the upper bound to the current node's value; going right tightens the lower bound.

If any node falls outside its current allowed range, the tree isn't a valid BST.

STEP 6 — Code While Narrating

```
def is_valid_bst(root):
    def validate(node, low, high):
        if node is None:
            return True
        if not (low < node.val < high):
            return False
```

SAY THIS —

I write a helper that takes a node plus the current valid (low, high) range. A null node is trivially valid. Otherwise, I check that the current node's value strictly falls within the allowed range.

WHY: Using strict inequalities on both sides ($\text{low} < \text{val} < \text{high}$) directly encodes that BST values must be strictly greater/less, correctly rejecting duplicates.

```
        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))

    return validate(root, float('-inf'), float('inf'))
```

SAY THIS —

I recurse into the left child with an updated upper bound of the current node's value, and into the right child with an updated lower bound of the current node's value. I kick off the whole process starting with unbounded (-infinity, infinity).

WHY: Starting with infinite bounds means the root itself has no constraint, and every deeper node's constraints correctly accumulate from all of its ancestors as the range narrows on the way down.

STEP 7 — Dry Run

Tree: 5 with left 3, right 8; 3 has left 1, right 4 (wait, let's use a violating example): 5 with left 3 (which has right child 6). validate(5, -inf, inf): 5 is within range. Left: validate(3, -inf, 5). 3 is within range. Right child of 3 is 6: validate(6, 3, 5): is $3 < 6 < 5$? No, 6 is not less than 5. Return False. This correctly identifies that 6, though greater than its immediate parent 3, violates the BST property relative to the ancestor 5's left-subtree constraint (everything under 5's left side must be < 5).

SAY THIS —

Tracing a tree where node 5 has left child 3, and 3 has a right child 6: even though 6 is bigger than its direct parent 3 (which would look fine to a naive local check), it violates the ancestor constraint that everything in 5's left subtree must be less than 5, and the range-based check correctly catches this.

STEP 8 — Edge Case Test

A tree containing a duplicate value, e.g. root 2 with left child 2. validate(2, -inf, inf) is fine for the root. Left: validate(2, -inf, 2): is $-\text{inf} < 2 < 2$? No, since 2 is not strictly less than 2. Return False, correctly rejecting duplicates as invalid in a strict BST.

SAY THIS —

Let me check a tree with a duplicate value, like a node equal to its parent — the strict inequality check correctly rejects it, since a valid BST here requires strictly less than / strictly greater than, not less-than-or-equal.

Other edge cases to remember

- Empty tree (should be considered valid)
- Single-node tree (trivially valid)
- A valid BST that's deeply skewed to one side
- Extreme integer values near typical boundary limits, testing the infinite sentinel bounds

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(h)$

SAY THIS —

This is $O(n)$ time since every node is visited once in the worst case, and $O(h)$ space for the recursion stack, where h is the tree's height.

If the Interviewer Pushes Back

Can you solve this with in-order traversal instead?

Yes — an in-order traversal of a valid BST always produces values in strictly increasing order, so I could do an in-order DFS and check that each value is strictly greater than the previous one visited, which is an equally valid $O(n)$ time, $O(h)$ space alternative.

Why doesn't checking only immediate parent-child pairs work?

Because the BST property is about a node's relationship to its entire ancestor chain's subtrees, not just its direct parent — a node can satisfy its immediate parent's constraint while still violating a grandparent's constraint, as shown in the dry run.

What if the tree could contain duplicate values allowed on one side?

That would change the inequality to be non-strict on whichever side allows duplicates (e.g., $\leq$) — I'd clarify this convention with the interviewer since different BST definitions handle duplicates differently.

Can you optimize this further?

No, $O(n)$ is optimal since verifying the BST property in general requires examining every node at least once.

Common Mistakes

- Only comparing a node to its immediate children instead of tracking the full valid range from all ancestors.
- Using non-strict inequalities ($\leq$, $\geq$) when the problem requires strict BST ordering (no duplicates).
- Forgetting to update the correct bound (low vs high) when recursing left vs right.
- Using naive integer min/max values instead of proper infinities, which could fail if actual node values equal those chosen sentinels.

If I Get Stuck — Recovery

- If stuck: 'Checking just parent-child pairs feels right but might miss ancestor constraints — let me think of a small counterexample to test that idea before committing to it.'
- If I catch myself checking only local parent-child relationships: verbally acknowledge the potential bug and switch to passing down a valid range instead.
- If the interviewer hints 'what about grandparents': confirm that's exactly the gap in the naive local-check approach, and describe the range-passing fix.
- If unsure about infinite bounds in a specific language without float('inf'): use a sufficiently large/small sentinel value appropriate to the language's constraints, or use None to represent 'no bound' explicitly.

Pattern Recognition

Trigger words: 'validate binary search tree', 'is this a valid BST'

Pattern: DFS with a passed-down valid (low, high) range

First thought: A node's constraint comes from ALL its ancestors, not just its direct parent, so pass down a shrinking valid range as you recurse.

Common trap: Only checking immediate parent-child value relationships, missing violations from higher ancestors.

Memory Hook

KEY INSIGHT

Validate BST → "Every node lives inside a shrinking range passed down from its ancestors."

30-Second Recap

Pattern	DFS with Valid Range (or In-order Traversal)
Key idea	Recursively pass down a (low, high) valid range; left subtree tightens the high bound, right subtree tightens the low bound.
Data structure	Implicit recursion call stack carrying (low, high) bounds.
Main algorithm	DFS with Valid Range (alternative: in-order traversal check).

Time	$O(n)$
Space	$O(h)$
Biggest edge case	Duplicate values, or a violation from a non-immediate ancestor.

Full Amazon Talk Track

Interviewer: Determine if this binary tree is a valid Binary Search Tree.

Me: Just to confirm, every node needs to satisfy the BST property with respect to ALL of its ancestors, not just its direct parent, and duplicates aren't allowed, right?

Interviewer: Correct.

Me: A tempting shortcut is checking only immediate parent-child relationships, but that misses violations from further up the tree — for example, a node could be fine relative to its parent but still be too large for a grandparent's left-subtree constraint.

Me: Instead, I'll pass down a valid (low, high) range as I recurse. Going left tightens the upper bound to the current value; going right tightens the lower bound.

Me: Let me code that with a helper function taking the node and its current valid range, starting from negative to positive infinity at the root.

Me: Tracing a tree where 5 has left child 3, and 3 has a right child 6: even though 6 is bigger than 3, it violates 5's left-subtree constraint, and my range check correctly catches that.

Me: Edge case: a duplicate value, like a node equal to its parent, correctly fails the strict inequality check.

Me: Final complexity: $O(n)$ time visiting every node once, $O(h)$ space for the recursion stack.

Interviewer: Any other way to solve it?

Me: Yes, an in-order traversal of a valid BST is always strictly increasing, so I could also check that property directly during an in-order DFS.

Kth Smallest Element in a BST

LeetCode #230 | **MEDIUM** | Pattern: In-order Traversal | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the root of a Binary Search Tree and an integer k , and we need to find the k th smallest value stored in the tree.

What I should clarify

- Is k guaranteed to be valid, i.e., between 1 and the total number of nodes?
- Are all values in the tree guaranteed to be unique?
- Will this function be called multiple times on the same tree with different k , which might favor a different strategy?
- Is there a preference for $O(h)$ space versus $O(n)$ space?

SAY THIS —

Just to confirm, k is guaranteed to be valid, all values are unique, and I need the k th smallest value overall, correct? Also, will this be called repeatedly on the same tree?

STEP 2 — Brute Force First

Idea

Do a full in-order traversal, collecting every value into a list (which will be sorted since it's a BST), then return the $(k-1)$ th index of that list.

SAY THIS —

The straightforward approach is to do a full in-order traversal, which visits BST nodes in sorted order, collect everything into a list, and index into it at position $k-1$.

STEP 3 — Brute Force Complexity

Time Complexity

$O(n)$ to traverse the entire tree.

Space Complexity

$O(n)$ to store all the values in a list.

SAY THIS —

This is $O(n)$ time and $O(n)$ space, since I'm storing every single value even though I might only need one of them, especially if k is small.

STEP 4 — Identify the Bottleneck

If k is small, I'm doing way more work and using way more memory than necessary by collecting every value in the tree, when I could stop the traversal as soon as I've found the k th one.

SAY THIS —

The bottleneck is continuing the traversal (and storing everything) even after I've already found the answer, when I could just stop early once I've counted k values.

STEP 5 — Optimization

Key insight / pattern: In-order Traversal

I use In-order Traversal (which visits BST nodes in ascending order: left, current, right) but stop early as soon as I've visited the kth node, instead of collecting everything. I keep a counter, and either use recursion with early-stopping logic, or an explicit stack-based iterative in-order traversal that I can break out of the moment the counter hits k.

KEY INSIGHT

An in-order traversal of a BST naturally visits nodes in sorted (ascending) order, so the kth node visited during that traversal IS the kth smallest value — I just need to count visits and stop at k instead of always processing the whole tree.

SAY THIS —

I'll do an in-order traversal, since that visits BST values in ascending order automatically.

I'll keep a counter of how many nodes I've visited so far.

As soon as the counter reaches k, I have my answer and can stop the traversal early, using an iterative stack-based approach to make the early exit clean.

STEP 6 — Code While Narrating

```
def kth_smallest(root, k):  
    stack = []  
    current = root  
    count = 0
```

SAY THIS —

I set up an explicit stack for the iterative in-order traversal, a pointer to walk down left children, and a counter for how many nodes I've visited.

WHY: Using an explicit stack (instead of recursion) makes it straightforward to break out of the traversal early the moment I find the answer, without extra bookkeeping to unwind recursive calls.

```
    while stack or current:  
        while current:  
            stack.append(current)  
            current = current.left
```

SAY THIS —

This inner loop walks as far left as possible, pushing each node onto the stack along the way, which is the standard first step of iterative in-order traversal.

WHY: In a BST, going as far left as possible from any point always leads to the next smallest unvisited value, which is exactly what in-order traversal needs to process first.

```

    current = stack.pop()
    count += 1
    if count == k:
        return current.val
    current = current.right

return -1

```

SAY THIS —

I pop the next node to process, increment my counter, and if I've just reached the kth node, I return its value immediately. Otherwise, I move to its right subtree and continue the traversal.

WHY: Returning as soon as `count == k` avoids any unnecessary work processing the remaining nodes, which is the whole point of the optimization over collecting everything upfront.

STEP 7 — Dry Run

BST: 3 with left 1 (right child 2), right 4. `k=1`. `current=3`, push 3, go left to 1, push 1, go left (none, 1 has no left), stop inner loop. Pop 1, `count=1`. `count==k(1)`, return `1.val=1`. Final answer: 1, the smallest value in the tree.

SAY THIS —

Tracing a small BST with root 3, left child 1 (which has a right child 2), and right child 4, looking for `k=1`: the traversal walks left first, finds 1 as the smallest value, and since `count` immediately hits `k=1`, I return 1 right away without visiting anything else.

STEP 8 — Edge Case Test

`k` equals the total number of nodes (asking for the largest value). The traversal must visit every single node before the counter finally reaches `k` on the very last (rightmost, deepest) node visited, correctly returning the maximum value in the tree.

SAY THIS —

Let me check when `k` equals the total number of nodes, meaning I'm asking for the largest value — the traversal has to visit everything, and the counter only hits `k` on the final node processed, correctly returning the maximum value.

Other edge cases to remember

- A single-node tree with `k=1`
- A completely left-skewed or right-skewed tree
- `k=1` (smallest value) on a large tree
- A balanced tree with `k` in the middle

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(h + k)$	$O(h)$

SAY THIS —

This is $O(h + k)$ time, since we walk down to the leftmost node ($O(h)$) and then process at most k nodes before stopping, and $O(h)$ space for the explicit stack, which only ever holds nodes along the current path, not the whole tree.

If the Interviewer Pushes Back

Why is this better than collecting all values into a list?

Because if k is much smaller than n , this approach stops as soon as it's found the answer, using only $O(h+k)$ time and $O(h)$ space, instead of always doing $O(n)$ time and $O(n)$ space regardless of how small k is.

What if this function is called many times on the same tree with different k values?

If repeated queries are expected, it might be worth augmenting each node with the size of its subtree (precomputed once in $O(n)$), which then allows each individual query to run in $O(h)$ time by using subtree sizes to decide whether to go left, return the current node, or go right — much faster for many repeated queries.

Can you solve this with recursion instead of an explicit stack?

Yes, recursion works too, but early-stopping is a bit more awkward to express cleanly (often requiring a mutable counter passed by reference or a class attribute), which is why I prefer the explicit stack version for clean early exit.

What if the tree could be modified (inserts/deletes) between queries?

If updates happen frequently, maintaining subtree sizes at each node (updated on every insert/delete) would let each k th-smallest query run in $O(h)$ time consistently, which is a natural follow-up if the interviewer asks about a dynamic version.

Common Mistakes

- Doing a full in-order traversal and collecting everything into a list even when only a small k is needed.
- Off-by-one in the counter comparison (using k instead of $k-1$, but incrementing at the wrong point relative to the check).
- Forgetting this only works correctly because it's specifically a BST (in-order gives sorted order); this technique wouldn't directly apply to a generic binary tree.
- Using recursion without a clean way to stop early, leading to unnecessary continued traversal after finding the answer.

If I Get Stuck — Recovery

- If stuck: 'Collecting everything into a list works, but if k is small, that's wasteful — let me think about whether I can stop the traversal as soon as I've found the k th value.'
- If I forget in-order gives sorted order for a BST specifically: remind myself that's the defining property being used here, not something true for a random binary tree.
- If the interviewer hints 'what if this were called many times': mention the subtree-size augmentation as a natural follow-up optimization.
- If my early-stop logic doesn't trigger correctly: recheck whether the counter increment happens before or after the equality check, and align it with a small dry run.

Pattern Recognition

Trigger words: 'kth smallest/largest in a BST', 'sorted order from a tree'

Pattern: In-order Traversal (with early stopping)

First thought: In-order traversal of a BST visits values in sorted order — can I stop as soon as I've counted k of them, instead of collecting everything?

Common trap: Always doing a full traversal and collecting all values, even when k is small and early stopping would save significant work.

Memory Hook

KEY INSIGHT

k th Smallest in BST → "In-order traversal is sorted order — just stop counting at k ."

30-Second Recap

Pattern	In-order Traversal
Key idea	Use iterative in-order traversal with an explicit stack, counting nodes visited and stopping as soon as the
Data structure	An explicit stack for iterative traversal.
Main algorithm	In-order Traversal with early exit.
Time	$O(h + k)$
Space	$O(h)$
Biggest edge case	k equal to the total number of nodes (largest value).

Full Amazon Talk Track

Interviewer: Find the k th smallest value in this BST.

Me: Got it — k is guaranteed valid, and all values are unique, correct? Also, will this be called repeatedly on the same tree?

Interviewer: Just a single call for now.

Me: The straightforward approach is a full in-order traversal, since that visits BST nodes in sorted order, collecting everything into a list and indexing at $k-1$ — that's $O(n)$ time and space.

Me: Since I don't need the whole sorted list, just the k th value, I can do an iterative in-order traversal with an explicit stack and stop as soon as I've counted k nodes.

Me: Let me code that... walking left as far as possible, then popping, counting, and checking against k before moving right.

Me: Tracing a small BST with root 3, left child 1, right child 4, looking for $k=1$: the traversal quickly finds 1 as the smallest and returns immediately.

Me: Edge case: if k equals the total node count, I'd need to traverse the entire tree before finding the answer on the very last node, which the logic handles correctly.

Me: Final complexity: $O(h+k)$ time and $O(h)$ space, much better than $O(n)$ time and space when k is small.

Interviewer: What if this were called many times?

Me: I'd consider augmenting each node with its subtree size, which would let each query run in $O(h)$ time by directly navigating toward the k th element instead of a linear scan.

Construct Binary Tree from Preorder and Inorder Traversal

LeetCode #105 | **MEDIUM** | Pattern: Recursion + Hash Map | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two integer arrays representing the preorder and inorder traversal of the same binary tree, and we need to reconstruct and return the original binary tree.

What I should clarify

- Are all node values guaranteed to be unique?
- Are both arrays guaranteed to be valid traversals of the same tree (same length, consistent)?
- Is there a size constraint that would make repeated linear scans too slow?
- Can the tree be empty (empty input arrays)?

SAY THIS —

Just to confirm, all values are unique, both arrays represent valid traversals of the same tree, and I need to actually rebuild the tree structure, not just verify something about it, correct?

STEP 2 — Brute Force First

Idea

Use the first element of preorder as the root, then linearly search for that value in inorder to split it into left and right parts, and recursively build both subtrees the same way.

SAY THIS —

The natural approach is: the first element of preorder is always the root. I search for that value in inorder to split it into left-subtree values and right-subtree values, then recurse on both halves using the corresponding pieces of preorder.

STEP 3 — Brute Force Complexity

Time Complexity

Each recursive call does an $O(n)$ linear search in inorder to find the root's position, and there are $O(n)$ calls total, giving $O(n^2)$ in the worst case.

Space Complexity

$O(n)$ for the recursion stack and the resulting tree.

SAY THIS —

This is $O(n^2)$ time in the worst case because of the repeated linear search to locate each subtree's root within inorder, which adds up across all the recursive calls.

STEP 4 — Identify the Bottleneck

I'm repeatedly scanning the inorder array from scratch to find each root's position, when I could precompute all those positions once upfront and look them up instantly.

SAY THIS —

The bottleneck is redoing a linear search for the root's position in inorder at every single recursive call, when a single upfront hash map lookup could replace all of those searches.

STEP 5 — Optimization

Key insight / pattern: Recursion + Hash Map

I precompute a Hash Map from value to its index in the inorder array, giving $O(1)$ lookups instead of $O(n)$ searches. I also track a running index into preorder (since preorder is always processed left to right: root, then left subtree, then right subtree). At each recursive call, I take the next value from preorder as the root, look up its position in inorder using the hash map to know how many elements belong to the left subtree, then recursively build the left subtree (which consumes exactly that many following preorder elements) and then the right subtree.

KEY INSIGHT

The very first unused element of preorder is always the current subtree's root, and its position in inorder tells you exactly how many elements to its left belong to the left subtree and how many to its right belong to the right subtree — which is exactly the information needed to split both arrays consistently.

SAY THIS —

I'll precompute a hash map of value to index in inorder, so I can find any value's split point instantly.

I'll track a pointer into preorder that always points at the next unused root value.

For each recursive call, I take that next preorder value as the root, use the inorder index to know the left subtree's size, and recurse first into the left side (consuming exactly that many preorder elements) and then the right side.

STEP 6 — Code While Narrating

```
def build_tree(preorder, inorder):
    inorder_index = {val: i for i, val in enumerate(inorder)}
    self_preorder_index = 0
```

SAY THIS —

I build a hash map from value to its index in inorder for $O(1)$ lookups, and set up a counter tracking which preorder element is next to be used as a root.

WHY: Precomputing this map once, in $O(n)$, replaces what would otherwise be $O(n)$ searches repeated at every recursive call.

```
def build(left, right):
    nonlocal self_preorder_index
    if left > right:
        return None
```

SAY THIS —

I define a recursive helper describing 'build the subtree covering inorder[left..right]'. If that range is empty ($left > right$), there's nothing to build, so I return None.

WHY: Using an inorder index range (left, right) to describe 'which part of the tree am I building' is what lets me correctly know when to stop recursing.

```

        root_val = preorder[self_preorder_index]
        self_preorder_index += 1
        root = TreeNode(root_val)
        mid = inorder_index[root_val]
        root.left = build(left, mid - 1)
        root.right = build(mid + 1, right)
        return root

    return build(0, len(inorder) - 1)

```

SAY THIS —

I take the next unused preorder value as this subtree's root, advance my pointer, then look up where that value sits in inorder to find the split point, and recursively build the left part (before the split) and the right part (after the split), attaching both as children before returning the root.

WHY: Because preorder always lists root, then everything in the left subtree, then everything in the right subtree, building the left side FIRST (immediately after taking the root) correctly consumes exactly the preorder elements that belong to the left subtree before we ever touch the right subtree's elements.

STEP 7 — Dry Run

preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]. inorder_index={9:0,3:1,15:2,20:3,7:4}. build(0,4):
 root_val=preorder[0]=3, index advances to 1. mid=inorder_index[3]=1. root.left=build(0,0): root_val=preorder[1]=9,
 index->2. mid=inorder_index[9]=0. left=build(0,-1)=None. right=build(1,0)=None. Return node 9 as leaf. Back:
 root.right=build(2,4): root_val=preorder[2]=20, index->3. mid=inorder_index[20]=3. left=build(2,2):
 root_val=preorder[3]=15, index->4, mid=2, build(2,1)=None, build(3,2)=None, returns leaf 15. right=build(4,4):
 root_val=preorder[4]=7, index->5, leaf 7. So node 20 has left=15, right=7. Final tree: root 3, left=9 (leaf), right=20
 (left=15, right=7) — matching the expected tree structure.

SAY THIS —

Tracing preorder=[3,9,20,15,7] and inorder=[9,3,15,20,7]: the algorithm correctly reconstructs root 3 with left child 9, and right child 20 which itself has children 15 and 7, matching the known correct tree for these traversals.

STEP 8 — Edge Case Test

Empty input arrays, representing an empty tree. build(0, -1) is called initially (since len(inorder)-1 = -1), and left > right (0 > -1) is immediately true, so None is returned right away — correctly representing an empty tree.

SAY THIS —

Let me check the empty tree case, with empty preorder and inorder arrays — the initial call has left=0 and right=-1, so left > right is true immediately, and we correctly return null.

Other edge cases to remember

- Single-node tree (both arrays have exactly one element)
- A tree that's completely left-skewed (right children are all null)
- A tree that's completely right-skewed (left children are all null)
- A tree where the root's value appears at the very start or very end of inorder

STEP 9 — Final Complexity

	Time	Space
--	------	-------

Optimized	$O(n)$	$O(n)$
-----------	--------	--------

SAY THIS —

This is $O(n)$ time since building the hash map is $O(n)$ and each recursive call now does $O(1)$ work (aside from the recursive calls themselves), visiting each node exactly once, and $O(n)$ space for the hash map and the recursion stack.

If the Interviewer Pushes Back

Why do you need to build the left subtree before the right subtree in the recursive calls?

Because preorder always lists all of the left subtree's values immediately after the root, before any right subtree values, so I must consume exactly the correct number of preorder elements for the left subtree first, in order, before the running preorder pointer is correctly positioned to start reading the right subtree's values.

Could you do this with postorder and inorder instead?

Yes, a very similar technique works: the LAST element of postorder is the root (since postorder processes left, right, root), and you'd recursively build the RIGHT subtree first (working backward from the end of postorder) before the left, but the core hash-map-for-split-point idea is the same.

What if there were duplicate values in the tree?

This approach specifically relies on unique values to correctly and unambiguously look up a value's split point in inorder; with duplicates, the reconstruction would become ambiguous or require additional information to disambiguate.

Can you optimize this further?

No, $O(n)$ is optimal, since we must construct n nodes and read n elements from at least one of the input arrays to reconstruct the tree.

Common Mistakes

- Building the right subtree before the left subtree, which desynchronizes the preorder pointer from where it should be.
- Using a linear search inside the recursion instead of precomputing the hash map upfront, causing the slower $O(n^2)$ approach.
- Off-by-one errors in the (left, right) range passed to each recursive call.
- Forgetting to advance the preorder pointer (or doing so at the wrong point) before or after creating the current node.

If I Get Stuck — Recovery

- If stuck: 'The first element of preorder is always the current subtree's root. If I can find that value's position in inorder, I know exactly how the rest of this subtree splits into left and right.'
- If the tree comes out shaped wrong: check whether I'm building left before right in the recursive calls, since order matters given how the preorder pointer advances.
- If the interviewer hints 'can you avoid repeated searching': pivot straight to precomputing a hash map of value-to-inorder-index.
- If confused about the range bounds: reason through it with a tiny 2-3 node example by hand before trusting the general recursive formula.

Pattern Recognition

Trigger words: 'construct/reconstruct a binary tree from traversals'

Pattern: Recursion + Hash Map (for $O(1)$ split-point lookup)

First thought: The first (or last) element of one traversal identifies the root; the other traversal tells you how to split into left/right subtrees around that root.

Common trap: Building right subtree before left, desynchronizing the running preorder index from where it should be.

Memory Hook

KEY INSIGHT

Construct Binary Tree $\rightarrow$ "Preorder's first value is root; inorder's position of that value splits left from right."

30-Second Recap

Pattern	Recursion + Hash Map
Key idea	Precompute value-to-inorder-index map; recursively take the next preorder value as root, split inorder at
Data structure	Hash map (value $\rightarrow$ inorder index) + recursion stack.
Main algorithm	Recursion guided by preorder pointer and inorder split index.
Time	$O(n)$
Space	$O(n)$
Biggest edge case	Empty input arrays (empty tree).

Full Amazon Talk Track

Interviewer: Reconstruct the binary tree from its preorder and inorder traversals.

Me: Got it — all values are unique, and both arrays are valid traversals of the same tree, correct?

Interviewer: Yes.

Me: The first element of preorder is always the current subtree's root. If I find that value's position in inorder, everything to its left in inorder is the left subtree, and everything to its right is the right subtree.

Me: Doing that search naively each time is $O(n)$ per call, giving $O(n^2)$ overall. I can avoid that by precomputing a hash map from value to its index in inorder up front.

Me: Let me code that — I'll track a running pointer into preorder, and recursively build the left subtree before the right subtree, since that's the order preorder lists them in.

Me: Tracing preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]: this correctly reconstructs root 3 with left child 9, and right child 20 having children 15 and 7.

Me: Edge case: empty input arrays correctly produce a null tree, since the initial range is immediately invalid.

Me: Final complexity: $O(n)$ time and space, since the hash map turns every split-point lookup into $O(1)$.

Serialize and Deserialize Binary Tree

LeetCode #297 | **HARD** | Pattern: DFS Preorder Traversal (with null markers) | Design-heavy problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We need to design a way to serialize a binary tree into a string, and deserialize that string back into the original tree structure, so the two operations are exact inverses of each other.

What I should clarify

- Is there a preferred format for the serialized string, or is any format acceptable as long as it round-trips correctly?
- Can node values include characters that might conflict with a chosen delimiter (like commas)?
- Does the tree need to preserve exact structure, including which side null children are on?
- Is there a size constraint that would rule out recursion (stack depth)?

SAY THIS —

Just to confirm, I get to choose the serialization format myself, as long as serialize followed by deserialize reproduces the exact original tree structure, correct?

STEP 2 — Brute Force First

Idea

One naive idea is to just serialize the values from a level-order (BFS) traversal without recording nulls, but that loses structural information about which children are missing where.

SAY THIS —

A tempting first idea is to just list out node values in some traversal order, but without explicitly recording where nulls are, we lose information about the exact tree shape.

STEP 3 — Brute Force Complexity

Time Complexity

Any single traversal without null markers is $O(n)$ to produce, but it's fundamentally lossy.

Space Complexity

$O(n)$ for the string.

SAY THIS —

Without recording null markers, this approach is fast, $O(n)$, but incorrect — it can't disambiguate different tree shapes that happen to produce the same sequence of non-null values.

STEP 4 — Identify the Bottleneck

The real issue isn't speed, it's that omitting null markers destroys structural information needed to correctly reconstruct the tree; I need a serialization that explicitly records where subtrees are absent.

SAY THIS —

The core problem with a naive traversal-only serialization is ambiguity — I need explicit markers for missing children so deserialization can unambiguously rebuild the exact same shape.

STEP 5 — Optimization

Key insight / pattern: DFS Preorder Traversal (with null markers)

I use Preorder DFS Traversal, explicitly writing a special marker (like '#' or 'null') whenever I encounter a null child, and joining everything with a delimiter (like a comma). For deserialization, I process the same tokens in the same preorder sequence: read a value, and if it's the null marker, that subtree is None; otherwise create a node and recursively build its left child (consuming the next tokens) and then its right child.

KEY INSIGHT

Preorder traversal (root, then left, then right) combined with explicit null markers makes the serialized sequence unambiguous: deserialization can rebuild the tree by processing tokens strictly in order, since each null marker correctly terminates a branch instead of leaving structural ambiguity.

SAY THIS —

For serialization, I'll do a preorder DFS, writing the current node's value, then recursively serializing left, then right — and explicitly writing a marker for any null child instead of skipping it.

For deserialization, I'll process the same tokens in the same order: build a node from the current token, then recursively build its left child from the next tokens, then its right child.

The null markers are what make this fully reversible, since they tell deserialization exactly when a branch ends.

STEP 6 — Code While Narrating

```
def serialize(root):
    result = []
    def dfs(node):
        if node is None:
            result.append('#')
            return
        result.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ','.join(result)
```

SAY THIS —

I do a preorder DFS, appending the current node's value (or '#' for a null) to a list, then recursing left, then right, and finally join everything with commas into one string.

WHY: Writing '#' for every null child, rather than skipping it, is exactly what preserves enough structural information for deserialization to unambiguously reconstruct the tree.

```
def deserialize(data):
    values = iter(data.split(','))
    def build():
        val = next(values)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = build()
        node.right = build()
        return node
    return build()
```

SAY THIS —

I split the string back into tokens and use an iterator so I can consume tokens one at a time. For each call, I read the next token: if it's the null marker, this subtree is None; otherwise I build a node and recursively build its left child, then its right child, from the following tokens.

WHY: Using an iterator (rather than an index I manage manually) makes it natural to always 'consume the next token', which mirrors exactly how serialize wrote them out in preorder, guaranteeing the reconstruction matches the original tree precisely.

STEP 7 — Dry Run

Tree: 1 with left 2 (leaf), right 3 which has left 4 and right 5 (both leaves). Serialize: dfs(1): append '1'. dfs(2): append '2', dfs(None)->append '#', dfs(None)->append '#'. dfs(3): append '3'. dfs(4): append '4', append '#', append '#'. dfs(5): append '5', append '#', append '#'. Result: '1,2,##,3,4,##,5,##'. Deserialize: build() reads '1', creates node 1. Its left: build() reads '2', creates node 2, left reads '#->None, right reads '#->None. Back to node 1's right: build() reads '3', creates node 3. Its left: build() reads '4', both children '#->None. Its right: build() reads '5', both children '#->None. Final tree matches the original exactly.

SAY THIS —

Tracing a small tree with root 1, left leaf 2, and right child 3 (which has leaves 4 and 5): serializing gives '1,2,##,3,4,##,5,##', and deserializing that string token by token perfectly reconstructs the original structure.

STEP 8 — Edge Case Test

An empty tree, root=None. Serialize: dfs(None) immediately appends '#' and returns, giving the string '#'. Deserialize('#'): build() reads '#', returns None immediately — correctly representing an empty tree round-tripping correctly.

SAY THIS —

Let me check the empty tree case — serializing gives just '#', and deserializing that single token correctly returns null, so an empty tree round-trips correctly.

Other edge cases to remember

- A single-node tree with no children
- A tree that's completely left-skewed
- A tree that's completely right-skewed
- Negative node values or values with multiple digits (delimiter must correctly separate them)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(n)$

SAY THIS —

Both serialize and deserialize run in $O(n)$ time, since each visits every real node and every null marker exactly once, and $O(n)$ space for the resulting string/token list and the recursion stack.

If the Interviewer Pushes Back

Why choose preorder over another traversal order?

Preorder naturally processes the root before its children, which makes it easy to reconstruct top-down: read a value, that's the root, then recursively read its left subtree's tokens, then its right subtree's tokens, all directly from a single forward-moving iterator, without needing to look ahead or backtrack.

Could you do this with level-order (BFS) instead?

Yes, that's also a common alternative, similar to how many online BST/tree visualizers represent trees; it requires null markers as placeholders per level too, and reconstruction uses a queue instead of recursion, but it's equally valid.

What if node values could contain commas?

I'd choose a different delimiter, or use a length-prefixed encoding (each value preceded by its character count) to avoid ambiguity, which is a good practical detail to bring up if values are strings rather than plain integers.

Can you avoid recursion for very deep trees?

Yes, I could implement both serialize and deserialize iteratively using an explicit stack, mirroring the same preorder logic, to avoid Python's recursion depth limit on very unbalanced trees.

Common Mistakes

- Forgetting to explicitly serialize null children, which destroys structural information needed for correct deserialization.
- Using a delimiter that could appear inside node values themselves, corrupting the split during deserialization.
- Managing a manual index into the token list instead of using an iterator, risking off-by-one bugs.
- Mismatching the order of operations between serialize and deserialize (e.g., serializing preorder but trying to deserialize as if it were inorder).

If I Get Stuck — Recovery

- If stuck: 'A traversal alone loses information about missing children, so I need to explicitly mark nulls in the serialized output to make reconstruction unambiguous.'
- If deserialization produces a wrong shape: verify that both serialize and deserialize consistently use the same traversal order (preorder in both).
- If the interviewer hints 'what if values contain your delimiter': acknowledge it and propose either a different delimiter or a length-prefixed format.
- If recursion depth is a concern: offer the iterative stack-based alternative as a follow-up.

Pattern Recognition

Trigger words: 'serialize and deserialize a tree', 'convert a tree to a string and back'

Pattern: DFS Preorder Traversal with explicit null markers

First thought: A traversal alone is ambiguous without recording where children are missing; explicit null markers plus a consistent traversal order (preorder is easiest) make it reversible.

Common trap: Omitting null markers, which makes different tree shapes indistinguishable from the same serialized sequence.

Memory Hook

KEY INSIGHT

Serialize/Deserialize → "Preorder plus explicit nulls — nothing is ambiguous."

30-Second Recap

Pattern	DFS Preorder Traversal (with null markers)
Key idea	Preorder DFS writes value or null marker for every node/child; deserialize reads tokens in the same order
Data structure	A list/string of tokens; an iterator during deserialization.
Main algorithm	DFS Preorder Traversal (symmetric encode/decode).
Time	$O(n)$
Space	$O(n)$
Biggest edge case	Empty tree (root is null).

Full Amazon Talk Track

Interviewer: Design an algorithm to serialize and deserialize a binary tree.

Me: Got it — I can choose my own format, as long as serialize followed by deserialize exactly reproduces the original tree, right?

Interviewer: Yes.

Me: A naive idea would be listing out just the node values in some traversal order, but that loses information about where children are missing, so different trees could serialize identically — that's ambiguous and wrong.

Me: Instead, I'll do a preorder DFS and explicitly write a marker, like '#', for every null child, then join everything with commas.

Me: For deserialization, I'll process the same tokens in the same preorder order using an iterator: read a value, if it's the marker return null, otherwise build a node and recursively build its left child then its right child from the following tokens.

Me: Let me trace a small tree with root 1, left leaf 2, and right child 3 having leaves 4 and 5 — serializing gives '1,2,#,#,3,4,#,#,5,#,#', and deserializing that reconstructs the exact same tree.

Me: Edge case: an empty tree just serializes to '#' and deserializes back to null.

Me: Final complexity: $O(n)$ time and space for both operations, since every node and null marker is processed exactly once.

Balanced Binary Tree

LeetCode #110 | **EASY** | Pattern: DFS / Recursion (Bottom-Up) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the root of a binary tree, and we need to determine whether it's height-balanced, meaning for every node, the heights of its left and right subtrees differ by no more than 1.

What I should clarify

- Does balance need to hold at every single node, not just at the root?
- Is an empty tree considered balanced?
- Is a single-node tree considered balanced?
- Is there a preference for avoiding redundant height recomputation?

SAY THIS —

Just to confirm, balance means at EVERY node, not just the root, the left and right subtree heights differ by at most 1, and an empty tree counts as balanced, correct?

STEP 2 — Brute Force First

Idea

For every node, separately compute the height of its left subtree and right subtree (each a full recursive height computation), check if they differ by more than 1, and recurse to check the same condition for all other nodes.

SAY THIS —

The direct approach is, for every node, compute its left and right subtree heights independently, check the difference, and then recursively check the same condition throughout the whole tree.

STEP 3 — Brute Force Complexity

Time Complexity

Computing height at each node costs $O(n)$ in the worst case, and doing this for every one of the n nodes gives $O(n^2)$ total.

Space Complexity

$O(h)$ for the recursion stack.

SAY THIS —

This is $O(n^2)$ time in the worst case because we're recomputing the height of overlapping subtrees repeatedly at every single node.

STEP 4 — Identify the Bottleneck

I'm recomputing the height of the same subtrees over and over as I check balance at each ancestor, when a single traversal could compute height and check balance for every node simultaneously.

SAY THIS —

The bottleneck is redundant height recomputation — the height of a subtree doesn't change depending on which ancestor is asking, so I should compute each subtree's height exactly once.

STEP 5 — Optimization

Key insight / pattern: DFS / Recursion (Bottom-Up)

I use a Bottom-Up DFS Recursion that computes height AND checks balance in a single pass. My helper function returns the height of a subtree if it's balanced, or a sentinel value (like -1) if it (or any part of it) is already unbalanced. At each node, I recursively get the left and right heights; if either is the sentinel -1, or if they differ by more than 1, I immediately propagate -1 upward. Otherwise, I return 1 + max(left height, right height) as this subtree's real height.

KEY INSIGHT

By combining 'compute height' and 'check balance' into a single recursive pass, and using a sentinel value to short-circuit and propagate 'already unbalanced' up through all ancestors, I avoid ever recomputing the same subtree's height more than once.

SAY THIS —

I'll write a helper that returns a subtree's real height if it's balanced, or -1 as a sentinel meaning 'unbalanced somewhere below'.

At each node, I get the left and right heights from that same helper; if either came back -1, or if they differ by more than 1, I immediately return -1 to signal unbalanced.

Otherwise, the current subtree is balanced, so I return its real height, 1 plus the max of the two child heights.

STEP 6 — Code While Narrating

```
def is_balanced(root):
    def check_height(node):
        if node is None:
            return 0
```

SAY THIS —

I define a helper that returns either a real height or the sentinel -1. The base case, a null node, has height 0 and is trivially balanced.

WHY: Combining height computation and balance checking into one function, using a special sentinel for 'unbalanced', is what lets me do both jobs in a single pass without recomputation.

```
    left_height = check_height(node.left)
    if left_height == -1:
        return -1
    right_height = check_height(node.right)
    if right_height == -1:
        return -1
```

SAY THIS —

I recursively check the left subtree first; if it already came back as unbalanced (-1), I immediately propagate that upward without even checking the right subtree. Otherwise, I check the right subtree the same way.

WHY: Short-circuiting as soon as I detect an unbalanced subtree avoids unnecessary further computation, and ensures a single -1 anywhere below correctly poisons every ancestor's result.

```

    if abs(left_height - right_height) > 1:
        return -1

    return 1 + max(left_height, right_height)

return check_height(root) != -1

```

SAY THIS —

If both subtree heights are valid but differ by more than 1, this node itself is unbalanced, so I return -1. Otherwise, I return the real height, 1 plus the taller of the two subtrees. At the top level, the tree is balanced if and only if the overall result isn't -1.

WHY: This single traversal correctly determines both the height of every subtree and whether balance holds everywhere, since any violation anywhere immediately propagates the -1 sentinel all the way up to the final answer.

STEP 7 — Dry Run

Tree: 3 with left 9 (leaf), right 20 (left 15, right 7, both leaves). `check_height(3)`: `left=check_height(9)=1+max(0,0)=1` (balanced). `right=check_height(20)`: `left=check_height(15)=1`, `right=check_height(7)=1`, `diff=0`, `return 1+max(1,1)=2`. Back at 3: `left_height=1`, `right_height=2`, `diff=1`, not `>1`, `return 1+max(1,2)=3`. Final: `check_height(root)=3`, not `-1`, so `is_balanced` returns `True`.

SAY THIS —

Tracing a small tree with root 3, left leaf 9, and right child 20 having two leaf children: every subtree's height difference stays within 1, so the whole tree is correctly identified as balanced, with final height 3.

STEP 8 — Edge Case Test

A tree that's unbalanced deep down, e.g. a long left-skewed chain hanging off of an otherwise small tree. Even though the imbalance occurs far from the root, the -1 sentinel correctly propagates all the way up through every ancestor, causing the final result to be `False`, without needing separate checks at each level.

SAY THIS —

Let me check a tree where the imbalance is buried deep, several levels down — the -1 sentinel correctly bubbles all the way up through every ancestor's check, so the top-level result is still correctly `False`.

Other edge cases to remember

- Empty tree (should be considered balanced)
- Single-node tree (trivially balanced)
- A perfectly balanced complete tree
- A tree unbalanced right at the root level

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(h)$

SAY THIS —

This is $O(n)$ time since each node's height is computed exactly once in a single bottom-up pass, and $O(h)$ space for the recursion stack, a major improvement over the $O(n^2)$ naive approach.

If the Interviewer Pushes Back

Why is the sentinel value -1 chosen specifically?

Because a real height can never be negative, -1 is a safe, unambiguous signal meaning 'unbalanced', which can be checked with a simple equality test at every level without confusing it with any legitimate height value.

Can you solve this top-down instead?

Yes, that's the $O(n^2)$ brute-force version, checking balance independently at each node by recomputing subtree heights from scratch; it's simpler to write but significantly less efficient.

Does this approach short-circuit as soon as it finds an imbalance?

Partially — it stops recursing into unnecessary subtrees once -1 is detected at a given point (skipping the right subtree check if the left already failed), but doesn't necessarily avoid visiting nodes elsewhere in the tree if the traversal reaches them independently before the failure is found.

Can you optimize further?

No, $O(n)$ is optimal since verifying balance requires knowing the height of every subtree, which means visiting every node at least once.

Common Mistakes

- Computing height and checking balance in two separate recursive passes, reintroducing the $O(n^2)$ inefficiency.
- Using 0 or None as the 'unbalanced' sentinel instead of a value like -1 that can never be a legitimate height.
- Forgetting to short-circuit and check the left result before computing the right subtree's height.
- Off-by-one in the imbalance check (using ≥ 1 instead of > 1 , which would incorrectly flag valid balanced trees).

If I Get Stuck — Recovery

- If stuck: 'The naive approach recomputes height at every node, which is wasteful. Let me think about a single traversal that computes height and checks balance for every node at the same time.'
- If I use two separate passes: recognize the inefficiency out loud and combine them using a sentinel value approach.
- If the interviewer hints 'can you avoid recomputing height so many times': confirm and pivot to the bottom-up single-pass version.
- If unsure why -1 works as a sentinel: reason aloud that real heights are always non-negative, so -1 can never be confused with a legitimate value.

Pattern Recognition

Trigger words: 'height-balanced binary tree', 'subtree heights differ by at most 1'

Pattern: Bottom-Up DFS Recursion with a sentinel value to combine two checks in one pass

First thought: Can I compute height and check balance simultaneously in one traversal, instead of recomputing height separately at every node?

Common trap: Splitting height computation and balance checking into two separate passes, reintroducing quadratic time.

Memory Hook

KEY INSIGHT

Balanced Binary Tree → "Compute height and check balance together; -1 means 'already broken, stop caring'."

30-Second Recap

Pattern	DFS / Recursion (Bottom-Up)
Key idea	A single bottom-up recursive pass returns real height, or -1 if any subtree below is already unbalanced.
Data structure	Implicit recursion call stack.
Main algorithm	Bottom-up DFS Recursion with sentinel propagation.
Time	$O(n)$
Space	$O(h)$
Biggest edge case	Imbalance occurring deep in the tree, far from the root.

Full Amazon Talk Track

Interviewer: Determine if this binary tree is height-balanced.

Me: Just to confirm, balance needs to hold at EVERY node, not just the root, and an empty tree counts as balanced, correct?

Interviewer: Yes.

Me: A direct approach would compute left and right subtree heights independently at every node and check the difference, but that recomputes overlapping heights repeatedly, giving $O(n^2)$.

Me: Instead, I'll do a single bottom-up pass that computes height and checks balance at the same time. My helper returns the real height if a subtree is balanced, or a sentinel of -1 if it's already unbalanced somewhere below.

Me: At each node, if either child's result is -1, or if the two heights differ by more than 1, I immediately propagate -1 upward. Otherwise, I return 1 plus the taller child's height.

Me: Let me code that... short-circuiting the right subtree check if the left already failed.

Me: Tracing a small tree with root 3, left leaf 9, right child 20 with two leaves: every level stays within a height difference of 1, so it's correctly identified as balanced.

Me: Edge case: if the imbalance is buried deep in the tree, the -1 sentinel correctly bubbles all the way up to the top-level result.

Me: Final complexity: $O(n)$ time since every node's height is computed exactly once, $O(h)$ space for the recursion.

Subtree of Another Tree

LeetCode #572 | **EASY** | Pattern: DFS / Recursion (Tree Matching) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given the roots of two binary trees, `root` and `subRoot`, and we need to determine whether `subRoot` exactly matches some subtree of `root` (a node and all of its descendants), including matching structure and values.

What I should clarify

- Does the whole subtree starting at the matching node need to match exactly, including all descendants?
- Can `subRoot` be an empty tree, and if so, is that trivially a subtree of anything?
- Are node values guaranteed unique, or could there be repeated values across the tree?
- Should matching be based on both structure and values, like the Same Tree problem?

SAY THIS —

Just to confirm, I need to find some node in root such that the ENTIRE subtree rooted there exactly matches subRoot in both structure and values, correct?

STEP 2 — Brute Force First

Idea

For every single node in `root`, check if the subtree rooted at that node is identical to `subRoot`, reusing a 'same tree' style comparison, and return true if any such node is found.

SAY THIS —

I can reuse the same-tree comparison logic: for every node in root, check whether the subtree rooted there is exactly identical to subRoot.

STEP 3 — Brute Force Complexity

Time Complexity

Visiting every node in `root` is $O(n)$, and for each one, doing a same-tree comparison against `subRoot` costs up to $O(m)$, giving $O(n * m)$ overall.

Space Complexity

$O(h_1 + h_2)$ for the combined recursion depth of both traversals.

SAY THIS —

This is $O(n \text{ times } m)$ time, where n and m are the sizes of root and subRoot, since in the worst case we might attempt a full comparison at every single node of root.

STEP 4 — Identify the Bottleneck

There isn't really a faster asymptotic algorithm needed here for typical interview purposes — the natural approach genuinely does require checking multiple candidate starting points, though we can make each individual check as

efficient as possible and stop early on any mismatch.

SAY THIS —

*This one doesn't have a deeper optimization to unlock the way some other problems do; the natural $O(n*m)$ approach combining tree traversal with same-tree comparisons is generally accepted as the expected solution, though there is a string-serialization alternative worth mentioning.*

STEP 5 — Optimization

Key insight / pattern: DFS / Recursion (Tree Matching)

I combine two pieces of recursive logic: a helper `is_same_tree(a, b)` that checks if two trees are exactly identical (structure and values), reused from the classic Same Tree problem; and a main traversal that, for every node in root, calls `is_same_tree(that node, subRoot)`, returning true as soon as any match is found, or recursing into the node's children if not.

KEY INSIGHT

'Is subRoot a subtree of root' can be broken into 'does the CURRENT node's subtree exactly match subRoot' OR 'is subRoot a subtree of root's left child' OR 'is subRoot a subtree of root's right child' — a clean recursive OR across all possible starting points.

SAY THIS —

I'll reuse the Same Tree comparison logic as a helper to check if two trees are exactly identical.

Then, starting from root, I check if the current node's subtree exactly matches subRoot using that helper.

If it doesn't match, I recursively check the same thing starting from root's left child, or root's right child, returning true if any of those checks succeed.

STEP 6 — Code While Narrating

```
def is_same_tree(a, b):
    if a is None and b is None:
        return True
    if a is None or b is None:
        return False
    if a.val != b.val:
        return False
    return is_same_tree(a.left, b.left) and is_same_tree(a.right, b.right)
```

SAY THIS —

This is the exact same-tree helper from before: both null matches, one null is a mismatch, otherwise compare values and recurse into both children.

WHY: Reusing well-understood logic (rather than reinventing tree comparison here) keeps the solution clean and easy to reason about.

```
def is_subtree(root, sub_root):
    if root is None:
        return False
    if is_same_tree(root, sub_root):
        return True
    return is_subtree(root.left, sub_root) or is_subtree(root.right, sub_root)
```

SAY THIS —

If root is null, there's nothing left to check, so no match is possible here. Otherwise, I first check if the CURRENT node's subtree exactly matches subRoot; if so, I'm done. If not, I recursively check root's left and right children as new candidate starting points.

WHY: Trying an exact match at every single node (not just the root of the whole tree) is what correctly finds subRoot anywhere within root, not just at the very top.

STEP 7 — Dry Run

root: 3 with left 4 (which has left 1, right 2), right 5. subRoot: 4 with left 1, right 2. is_subtree(3, subRoot): is_same_tree(3, 4)? values 3 vs 4 differ, False. Recurse: is_subtree(4, subRoot) or is_subtree(5, subRoot). is_subtree(4, subRoot): is_same_tree(4,4)? values match, then compare left(1,1) match, right(2,2) match -> True. So is_subtree(4, subRoot) returns True, and the overall result is True.

SAY THIS —

Tracing root=3 (left child 4 with children 1 and 2, right child 5) and subRoot=4 (with children 1 and 2): the root itself doesn't match, but its left child does exactly match subRoot, so the overall answer is true.

STEP 8 — Edge Case Test

subRoot has a node with the same value as some node in root but different descendant structure, e.g. root has a node 4 with an extra unexpected child that subRoot's node 4 doesn't have. is_same_tree correctly returns False in that case, since it checks the ENTIRE subtree structure, not just the value at the starting node, so we correctly don't count that as a match and continue searching elsewhere.

SAY THIS —

Let me check a case where a node has the right value but a different subtree shape than subRoot expects — the same-tree check correctly rejects that as a match since it verifies the full structure, not just the top value, so we correctly keep searching rather than falsely matching.

Other edge cases to remember

- subRoot is a single leaf node matching a leaf somewhere in root
- root and subRoot are identical trees (match at the very root)
- subRoot doesn't appear anywhere in root (should return false)
- root is much larger than subRoot with several nodes sharing subRoot's root value

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n * m)$	$O(h_1 + h_2)$

SAY THIS —

This is $O(n \text{ times } m)$ time in the worst case, where n and m are the number of nodes in root and subRoot respectively, since we might attempt a full same-tree comparison at many candidate nodes, and $O(h_1 + h_2)$ space for the combined recursion depth of both traversals.

If the Interviewer Pushes Back

Can you do better than $O(n * m)$?

Yes — an alternative is to serialize both trees into strings (using preorder traversal with explicit null markers, similar to the Serialize/Deserialize problem), and then simply check if subRoot's serialization is a substring of root's serialization, using an efficient string-matching algorithm like KMP, achieving $O(n + m)$ time overall.

Why does the string-serialization trick work correctly here?

Because with proper null markers and unique delimiters, the serialized preorder string uniquely represents a tree's structure and values, so 'is subRoot a subtree of root' becomes exactly 'is subRoot's string a contiguous substring of root's string', which efficient string matching algorithms can answer quickly.

What if node values could repeat frequently across the tree?

The same-tree comparison still works correctly regardless of repeated values, since it checks the FULL structure, not just a single value; repeated values would just mean more candidate starting points get attempted (and rejected) during the search, affecting only performance, not correctness.

Can you avoid redundant same-tree comparisons?

Not easily with this direct recursive approach, which is why the string-serialization plus substring-search alternative is worth mentioning if better asymptotic complexity is specifically requested.

Common Mistakes

- Only comparing the value at the root of a candidate subtree without checking the full structure recursively.
- Forgetting to recurse into BOTH root.left and root.right when the current node doesn't match.
- Confusing 'is subRoot equal to root' with 'is subRoot a subtree somewhere within root' (missing that it can start at any node).
- Not handling the null root case (should return False if root is null but subRoot search hasn't found a match yet).

If I Get Stuck — Recovery

- If stuck: 'I already know how to check if two trees are exactly identical — let me reuse that, but try it starting from every node in root, not just the root itself.'
- If I only check the top-level root: recognize the mistake by testing a case where the match is deeper in the tree, and add the recursive search into left/right children.
- If the interviewer asks for a faster approach: mention the serialize-and-substring-search alternative using string matching.
- If confused about which helper does what: keep the two functions conceptually separate — 'is_same_tree' checks exact equality, 'is_subtree' searches for any matching starting point.

Pattern Recognition

Trigger words: 'is this tree a subtree of another tree', 'find a matching subtree anywhere'

Pattern: DFS Recursion combining exact-match checking with a search across candidate starting points

First thought: Reuse a 'same tree' equality check, but try it as a candidate match starting from every node, not just the root.

Common trap: Only checking the top-level root for a match instead of recursively trying every node in the larger tree as a candidate.

Memory Hook

KEY INSIGHT

Subtree of Another Tree → "Try an exact match at every node — reuse Same Tree as the check."

30-Second Recap

Pattern	DFS / Recursion (Tree Matching)
Key idea	At every node in root, check if its subtree exactly equals subRoot (reusing Same Tree logic); recurse into
Data structure	Implicit recursion call stack (two nested recursive checks).
Main algorithm	DFS Recursion with a nested exact-match helper.
Time	$O(n * m)$
Space	$O(h_1 + h_2)$
Biggest edge case	A node with matching value but different subtree structure.

Full Amazon Talk Track

Interviewer: Determine if subRoot is a subtree of root.

Me: Got it — that means some node in root, with its ENTIRE subtree, must exactly match subRoot in both structure and values, right?

Interviewer: Yes.

Me: I already know how to check if two trees are exactly identical — that's the Same Tree problem. I'll reuse that as a helper.

Me: Then, for the main search, I'll check if the current node in root exactly matches subRoot using that helper; if not, I recursively check root's left and right children as new candidate starting points.

Me: Let me code both pieces — the same-tree helper first, then the search that tries it at every node.

Me: Tracing root=3 (left child 4 with children 1, 2, right child 5) and subRoot=4 (with children 1, 2): the root doesn't match, but the left child does exactly, so the answer is true.

Me: Edge case: a node with the right value but a different subtree shape correctly fails the same-tree check, so we keep searching instead of falsely matching.

Me: Final complexity: $O(n \text{ times } m)$ time in the worst case, $O(h_1 \text{ plus } h_2)$ space for the recursion.

Interviewer: Can you do better than that?

Me: Yes, by serializing both trees into strings with explicit null markers and checking if subRoot's string is a substring of root's string using an efficient string-matching algorithm, getting down to $O(n+m)$.

8. Graphs

Multi-source BFS on grids. (The playlist's generic 'graph algorithms' entry is covered here via these two concrete BFS problems, plus the BFS/DFS templates in the final cheat sheet.)

Walls and Gates

LeetCode #286 | **MEDIUM** | Pattern: Multi-Source BFS | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a 2D grid representing a building, where each cell is either a wall (-1), a gate (0), or an empty room (a large sentinel value like 2147483647), and we need to fill every empty room with the distance to its nearest gate, leaving walls unchanged.

What I should clarify

- Can I move in all four directions (up, down, left, right), not diagonally?
- Should unreachable rooms (surrounded entirely by walls, no path to any gate) stay at the sentinel value?
- Can there be multiple gates in the grid?
- Should I modify the grid in place, or return a new grid?

SAY THIS —

Just to confirm, I move in four directions only, there can be multiple gates, and I need to fill each empty room in place with its distance to the NEAREST gate, correct?

STEP 2 — Brute Force First

Idea

For every empty room, run a separate BFS (or DFS) outward to find the nearest gate, recording the shortest distance found.

SAY THIS —

The straightforward approach is, for every single empty room, run its own BFS out to find the nearest gate and record that distance.

STEP 3 — Brute Force Complexity

Time Complexity

Each BFS from a room can take $O(\text{rows} * \text{cols})$ in the worst case, and there can be $O(\text{rows} * \text{cols})$ rooms, giving $O((\text{rows} * \text{cols})^2)$ overall.

Space Complexity

$O(\text{rows} * \text{cols})$ per BFS for the visited tracking.

SAY THIS —

This is $O((\text{rows times cols}) \text{ squared})$ time in the worst case, since we potentially do a full grid-sized BFS separately for every single room, which is very wasteful.

STEP 4 — Identify the Bottleneck

I'm redoing a full search from every single room, when actually multiple gates and rooms share overlapping search space; if I start the search from ALL gates simultaneously instead, each cell only needs to be visited once.

SAY THIS —

The bottleneck is running a separate search from every room, when I could instead run one combined search starting from all gates at once, since BFS naturally finds shortest distances outward from its starting points.

STEP 5 — Optimization

Key insight / pattern: Multi-Source BFS

I use Multi-Source BFS. Instead of searching FROM each room TO the nearest gate, I search FROM all the gates simultaneously OUTWARD to every reachable room. I initialize a queue with every gate's position (distance 0). Then I do a standard BFS: for each cell dequeued, I look at its four neighbors; if a neighbor is an empty room (still holding the sentinel value), I set its distance to current distance + 1 and enqueue it. Because BFS explores in increasing distance order, the first time any room is reached is guaranteed to be via the shortest path from SOME gate.

KEY INSIGHT

Starting BFS simultaneously from every gate (rather than from each room individually) means every cell is visited at most once total, since BFS naturally expands in order of increasing distance, guaranteeing the first visit to any room is via its true nearest gate.

SAY THIS —

Instead of searching from each room to find its nearest gate, I'll flip the direction: start a BFS from ALL the gates at the same time.

I'll put every gate into a queue first, then expand outward level by level, filling in each newly reached room with its distance.

Since BFS expands in order of distance, the first time a room is reached is guaranteed to be its shortest distance to any gate.

STEP 6 — Code While Narrating

```
from collections import deque

def walls_and_gates(rooms):
    if not rooms:
        return
    rows, cols = len(rooms), len(rooms[0])
    queue = deque()
    for r in range(rows):
        for c in range(cols):
            if rooms[r][c] == 0:
                queue.append((r, c))
```

SAY THIS —

I handle the empty grid case, then find every gate (a cell with value 0) and add all of them to the queue upfront, before starting the BFS.

WHY: Seeding the queue with ALL gates at once (rather than one at a time) is exactly what makes this 'multi-source' — the BFS naturally explores from every gate simultaneously, in lockstep by distance.

```

directions = [(-1,0), (1,0), (0,-1), (0,1)]
while queue:
    row, col = queue.popleft()
    for dr, dc in directions:
        new_row, new_col = row + dr, col + dc
        if (0 <= new_row < rows and 0 <= new_col < cols
            and rooms[new_row][new_col] == 2147483647):
            rooms[new_row][new_col] = rooms[row][col] + 1
            queue.append((new_row, new_col))

```

SAY THIS —

I process the queue level by level, checking all four neighbors of the current cell. If a neighbor is within bounds and still holds the sentinel value (meaning it's an unvisited empty room), I set its distance to one more than the current cell's distance and add it to the queue.

WHY: Checking for the sentinel value doubles as our 'visited' check, so a room is only ever processed once total, which is what gives this its efficient linear-time guarantee across the whole grid.

STEP 7 — Dry Run

Small grid: `[[INF, -1, 0, INF], [INF, INF, INF, -1], [INF, -1, INF, -1], [0, -1, INF, INF]]` (`INF=2147483647`). Queue starts with both gates: (0,2) and (3,0). Process (0,2): neighbors (0,1)=-1 skip, (0,3)=INF -> set to 1, enqueue. (1,2)=INF -> set to 1, enqueue. Process (3,0): neighbors (2,0)=INF -> set to 1, enqueue. Continue expanding level by level, filling every reachable room with increasing distances, until the queue is empty, leaving unreachable rooms (if fully walled off) still at INF.

SAY THIS —

Starting the BFS from both gates at once, the search expands outward layer by layer, correctly assigning each empty room the distance to whichever gate reaches it first — which is guaranteed to be its true nearest gate.

STEP 8 — Edge Case Test

A room completely walled off from every gate, with no possible path. It's never dequeued or reached by the BFS from any gate, so it correctly remains at the sentinel value 2147483647, signaling it's unreachable, exactly as the problem expects.

SAY THIS —

Let me check a room that's completely sealed off by walls with no path to any gate — since the BFS never reaches it, it correctly stays at the sentinel value, signaling it's unreachable.

Other edge cases to remember

- A grid with only one gate
- A grid with no gates at all (every room stays at the sentinel value)
- A grid that's entirely walls except a single gate
- A grid where a room is equidistant from two different gates

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\text{rows} * \text{cols})$	$O(\text{rows} * \text{cols})$

SAY THIS —

This is $O(\text{rows times cols})$ time since every cell is enqueued and processed at most once, and $O(\text{rows times cols})$ space for the queue in the worst case, a big improvement over the naive per-room BFS approach.

If the Interviewer Pushes Back

Why is multi-source BFS correct here, not just faster?

Because BFS explores cells in strictly increasing order of distance from its starting points, and since ALL gates start simultaneously at distance 0, the very first time any room is reached is guaranteed to be via whichever gate is genuinely closest, so correctness and efficiency come together in one pass.

Can you solve this with DFS instead?

DFS could work but would require additional bookkeeping to track and compare distances via different paths, since DFS doesn't naturally explore in order of increasing distance the way BFS does, making it less natural and potentially requiring revisiting cells multiple times to guarantee correctness.

What if walls could change dynamically?

This solution assumes a static grid for a single query; if walls or gates could be updated frequently, a different approach (like maintaining a more dynamic shortest-path structure, or recomputing BFS whenever the grid changes significantly) would be needed.

Can you optimize this further?

No, $O(\text{rows times cols})$ is already optimal since we must visit every reachable room at least once to determine its distance.

Common Mistakes

- Running BFS from each room individually instead of seeding the queue with all gates at once.
- Forgetting to check grid boundaries before accessing a neighbor, causing an index error.
- Using a separate visited set instead of relying on the sentinel value check, adding unnecessary complexity.
- Not skipping walls (-1) when considering neighbors, potentially overwriting them incorrectly.

If I Get Stuck — Recovery

- If stuck: 'Running BFS from every room separately seems slow because of overlapping work. What if I flip the direction and search outward from all the gates instead?'
- If I only start BFS from one gate: recognize that with multiple gates, I need to seed the queue with ALL of them at the start, not just search from a single one.
- If the interviewer hints 'think about starting from multiple points at once': confirm and describe multi-source BFS explicitly.
- If I get an index error: double-check the boundary conditions on the neighbor check before accessing `rooms[new_row][new_col]`.

Pattern Recognition

Trigger words: 'nearest gate/source', 'distance to closest of multiple targets', 'fill grid with distances'

Pattern: Multi-Source BFS

First thought: Instead of searching FROM each target TO the nearest source, flip it: search FROM all sources simultaneously OUTWARD.

Common trap: Running BFS separately from each room instead of seeding the queue with every gate up front.

Memory Hook

KEY INSIGHT

Walls and Gates → "Start BFS from every gate at once — first arrival is always the nearest."

30-Second Recap

Pattern	Multi-Source BFS
Key idea	Seed the BFS queue with all gates simultaneously; expand outward, and the first visit to any room is guaranteed to be from its nearest gate.
Data structure	A queue (deque) for BFS.
Main algorithm	Multi-Source BFS.
Time	$O(\text{rows} * \text{cols})$
Space	$O(\text{rows} * \text{cols})$
Biggest edge case	A room with no path to any gate (stays at the sentinel value).

Full Amazon Talk Track

Interviewer: Fill each empty room in this grid with its distance to the nearest gate.

Me: Got it — I move in four directions, there can be multiple gates, and I modify the grid in place, correct?

Interviewer: Yes.

Me: Brute force: run a separate BFS from every room to find its nearest gate — that's $O((\text{rows times cols})^2)$ since there's a lot of overlapping search work.

Interviewer: How would you avoid that overlap?

Me: I'd flip the direction of the search: instead of searching from each room outward to find a gate, I start a BFS from ALL the gates simultaneously, expanding outward. Since BFS explores in order of increasing distance, the first time any room is reached is guaranteed to be from its true nearest gate.

Me: Let me code that — seed the queue with every gate's position first, then expand level by level, filling in distances as I go.

Me: Tracing a small grid with two gates, the search correctly fills every reachable room with increasing distances outward from whichever gate reaches it first.

Me: Edge case: a room completely sealed off by walls never gets reached, and correctly stays at the sentinel value signaling it's unreachable.

Me: Final complexity: $O(\text{rows times cols})$ time and space, since every cell is processed at most once.

Rotting Oranges

LeetCode #994 | **MEDIUM** | Pattern: Multi-Source BFS | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a grid where each cell is either empty (0), a fresh orange (1), or a rotten orange (2), and every minute any fresh orange adjacent to a rotten orange also becomes rotten; we need to find the minimum number of minutes until no fresh orange remains, or -1 if that's impossible.

What I should clarify

- Can rotting spread in all four directions, not diagonally?
- Should I return 0 if there are no fresh oranges at all to begin with?
- Are there guaranteed to be some rotten oranges initially, or could there be none?
- Should I return -1 if some fresh oranges can never be reached?

SAY THIS —

Just to confirm, rotting spreads in four directions each minute, and I need the minimum minutes until every orange is rotten, or -1 if some fresh orange can never be reached, correct?

STEP 2 — Brute Force First

Idea

Simulate minute by minute: scan the entire grid each minute, find all currently fresh oranges adjacent to a rotten one, mark them to rot at the end of that minute, and repeat until no more changes happen.

SAY THIS —

A direct simulation approach scans the whole grid every minute, finds newly infected oranges, and repeats until nothing changes, counting the minutes elapsed.

STEP 3 — Brute Force Complexity

Time Complexity

Each minute's scan is $O(\text{rows} \times \text{cols})$, and in the worst case we might need $O(\text{rows} \times \text{cols})$ minutes (a long snake-like spread), giving $O((\text{rows} \times \text{cols})^2)$.

Space Complexity

$O(\text{rows} \times \text{cols})$ to track which cells rot in a given minute.

SAY THIS —

This simulate-every-minute-with-a-full-scan approach is $O((\text{rows times cols})^2)$ in the worst case, since scanning the whole grid repeatedly for each minute of spread adds up.

STEP 4 — Identify the Bottleneck

I'm rescanning the entire grid every single minute just to find the 'frontier' of newly rotten oranges, when I could directly track exactly which cells are on that frontier and only ever process each orange once.

SAY THIS —

The bottleneck is repeatedly scanning cells that already rotted or are still far from any rot, when I really just need to track the current 'frontier' of oranges about to spread.

STEP 5 — Optimization

Key insight / pattern: Multi-Source BFS

I use Multi-Source BFS, starting from every initially rotten orange simultaneously. I seed a queue with all rotten oranges' positions, and also count the total number of fresh oranges upfront. Then I process the queue level by level (each level represents one minute): for every rotten orange dequeued, I check its four neighbors, and any fresh orange found gets marked rotten, has the fresh count decremented, and gets enqueued for the next level. I track how many 'minutes' (BFS levels) it takes. At the end, if any fresh oranges remain uninfected, I return -1; otherwise I return the number of minutes elapsed.

KEY INSIGHT

BFS naturally processes nodes in order of distance from the starting points, and here 'distance' directly corresponds to 'minutes elapsed', so a standard level-by-level BFS from all initial rotten oranges simultaneously directly simulates the minute-by-minute spread without redundant rescanning.

SAY THIS —

I'll seed a BFS queue with every initially rotten orange, and also count how many fresh oranges exist at the start.

I'll process the queue level by level, where each level corresponds to exactly one minute passing, spreading rot to fresh neighbors and decrementing my fresh-orange counter as I go.

At the end, if the fresh-orange counter isn't zero, some oranges were unreachable, so I return -1; otherwise I return the number of minutes (levels) it took.

STEP 6 — Code While Narrating

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh_count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                queue.append((r, c))
            elif grid[r][c] == 1:
                fresh_count += 1
```

SAY THIS —

I scan the grid once to find all initially rotten oranges (adding them to the queue) and count all initially fresh oranges.

WHY: Seeding the queue with every rotten orange at once (multi-source) is what lets a single BFS simulate all the spreading happening simultaneously, exactly matching how rot spreads in real minutes.

```

minutes = 0
directions = [(-1,0), (1,0), (0,-1), (0,1)]
while queue and fresh_count > 0:
    minutes += 1
    for _ in range(len(queue)):
        row, col = queue.popleft()
        for dr, dc in directions:
            new_row, new_col = row + dr, col + dc
            if (0 <= new_row < rows and 0 <= new_col < cols
                and grid[new_row][new_col] == 1):
                grid[new_row][new_col] = 2
                fresh_count -= 1
                queue.append((new_row, new_col))

```

SAY THIS —

I process the queue in full 'levels': the inner for loop handles exactly the oranges that were rotten at the start of this minute, so anything newly rotted gets added for the NEXT minute, not this one. I increment minutes once per level, and only continue while there's still both something in the queue and fresh oranges left to infect.

WHY: Using 'for _ in range(len(queue))' to process a fixed snapshot of the queue's current size is the standard trick for level-by-level BFS, ensuring minutes increments exactly once per real minute of spread, not once per individual orange.

```

return minutes if fresh_count == 0 else -1

```

SAY THIS —

At the end, if every fresh orange got infected (fresh_count reached 0), I return the number of minutes it took; otherwise, some oranges were unreachable, so I return -1.

WHY: This final check distinguishes 'everything successfully rotted' from 'some oranges were permanently isolated and could never rot', which is exactly what the problem is asking us to detect.

STEP 7 — Dry Run

grid = [[2,1,1],[1,1,0],[0,1,1]]. Initial queue: [(0,0)]. fresh_count=6. Minute 1: process (0,0): neighbors (1,0)=1->rot, fresh_count=5, enqueue; (0,1)=1->rot, fresh_count=4, enqueue. minutes=1. Minute 2: process (1,0): neighbors (2,0)=0 skip, (1,1)=1->rot, fresh_count=3, enqueue. process (0,1): neighbors (0,2)=1->rot, fresh_count=2, enqueue; (1,1) already 2, skip. minutes=2. Minute 3: process (1,1): neighbors (2,1)=1->rot, fresh_count=1, enqueue. process (0,2): no new fresh neighbors. minutes=3. Minute 4: process (2,1): neighbor (2,2)=1->rot, fresh_count=0, enqueue. minutes=4. Loop ends since fresh_count=0. Return 4.

SAY THIS —

Tracing a 3x3 grid with one initially rotten orange in the corner: the rot spreads outward minute by minute, and it takes exactly 4 minutes for every fresh orange to become rotten.

STEP 8 — Edge Case Test

A fresh orange that's completely isolated from any rotten orange, e.g. surrounded by empty cells with no rotten orange ever reaching it. The BFS never reaches that cell, so fresh_count never drops to 0, and the function correctly returns -1.

SAY THIS —

Let me check a fresh orange that's isolated and can never be reached by any rotten orange — the fresh count never reaches zero, so we correctly return -1.

Other edge cases to remember

- No fresh oranges at all initially (should return 0 immediately)
- No rotten oranges at all, but some fresh oranges exist (should return -1, since nothing can ever start spreading)
- A grid that's entirely empty cells
- All oranges already rotten from the start (should return 0)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\text{rows} * \text{cols})$	$O(\text{rows} * \text{cols})$

SAY THIS —

This is $O(\text{rows times cols})$ time since every cell is enqueued and processed at most once, and $O(\text{rows times cols})$ space for the queue in the worst case, a significant improvement over the naive repeated-full-scan simulation.

If the Interviewer Pushes Back

Why process the queue in 'levels' using its snapshot size?

Because each level of the BFS corresponds exactly to one minute of real time, and using `len(queue)` at the START of each minute's processing ensures I only handle the oranges that were rotten BEFORE this minute, correctly separating this minute's spread from the next minute's.

What if there are no rotten oranges at the start but some fresh ones exist?

The queue starts empty, so the while loop never executes, minutes stays 0, but `fresh_count` is still greater than 0, so the function correctly returns -1, since nothing can ever start rotting.

Can you solve this without modifying the input grid?

Yes, I could use a separate visited/rotten-tracking structure instead of mutating grid values directly, at the cost of a bit more space, if modifying the input isn't allowed.

Can you optimize this further?

No, $O(\text{rows times cols})$ is optimal since we must at least examine every orange once to determine the final outcome.

Common Mistakes

- Processing the queue without snapshotting its size per level, causing minutes to be miscounted (processing multiple 'minutes' of spread in a single increment, or vice versa).
- Forgetting to decrement `fresh_count` when a fresh orange becomes rotten, breaking the final check for whether all oranges were infected.
- Not handling the initial edge case where there are no fresh oranges at all (should immediately return 0).
- Confusing 'minutes' with 'number of oranges processed', rather than 'number of BFS levels'.

If I Get Stuck — Recovery

- If stuck: 'Simulating minute by minute with a full grid rescan each time seems wasteful. Since rot spreads outward like a wave, this feels like a multi-source BFS problem, where distance corresponds directly to minutes.'
- If minutes come out wrong: check whether I'm using the queue-snapshot trick (for `_ in range(len(queue))`) to correctly separate one minute's processing from the next.
- If the interviewer hints 'think about all the rotten oranges spreading at once': confirm and describe seeding the BFS queue with all of them simultaneously.
- If `fresh_count` logic seems off: trace through a small example by hand to confirm it decrements exactly once per orange that turns rotten.

Pattern Recognition

Trigger words: 'spreads over time', 'minutes until all infected/reached', 'simultaneous spreading from multiple sources'

Pattern: Multi-Source BFS (level-by-level = time steps)

First thought: Time steps in the problem map directly to BFS levels/distance — seed the queue with all starting 'infected' cells and expand level by level.

Common trap: Not using the level-snapshot trick, causing minutes to be counted incorrectly.

Memory Hook

KEY INSIGHT

Rotting Oranges → "BFS levels ARE the minutes — seed with every rotten orange at once."

30-Second Recap

Pattern	Multi-Source BFS
Key idea	Multi-source BFS from all initially rotten oranges; each BFS level equals one minute; track fresh count to
Data structure	A queue (deque) for level-by-level BFS.
Main algorithm	Multi-Source BFS with level counting.
Time	$O(\text{rows} * \text{cols})$
Space	$O(\text{rows} * \text{cols})$
Biggest edge case	A fresh orange that can never be reached (return -1).

Full Amazon Talk Track

Interviewer: Find the minimum minutes until no fresh orange remains, or -1 if impossible.

Me: Got it — rot spreads in four directions each minute, and I return -1 if some fresh orange can never be reached, correct?

Interviewer: Yes.

Me: A direct simulation would rescan the whole grid every minute to find newly infected oranges, which is $O((\text{rows times cols})^2)$ in the worst case.

Me: Since every rotten orange spreads simultaneously, I can treat this as multi-source BFS: seed the queue with all initially rotten oranges, and process it level by level, where each level equals one minute.

Me: Let me code that, using the queue-snapshot trick to make sure I count minutes correctly, and tracking a fresh-orange counter to know when everything's infected.

Me: Tracing a 3x3 grid with one rotten orange in the corner: the rot spreads outward, and it takes exactly 4 minutes for everything to turn rotten.

Me: Edge case: if a fresh orange is completely isolated and can never be reached, the fresh count never reaches zero, and we correctly return -1.

Me: Final complexity: $O(\text{rows times cols})$ time and space, since every cell is processed at most once.

9. Heaps / Priority Queue

Maintaining a bounded top-K set efficiently.

Kth Largest Element in a Stream

LeetCode #703 | **EASY** | Pattern: Heap (Min-Heap of size k) | Design-heavy problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We need to design a class that, given an integer k and an initial stream of numbers, supports adding new numbers to the stream one at a time and always returning the k th largest element in the stream so far.

What I should clarify

- Does 'kth largest' mean the k th largest counting duplicates (so $[5,5]$ with $k=2$ gives 5, not the 2nd distinct value)?
- Is k guaranteed to be less than or equal to the number of elements added so far after initialization?
- How many times will 'add' be called — does that affect whether we optimize for frequent calls?
- Can the numbers include duplicates or negative values?

SAY THIS —

Just to confirm, kth largest counts duplicates, k stays fixed for the object's lifetime, and 'add' will likely be called many times, so I should optimize for repeated calls, not just one query, correct?

STEP 2 — Brute Force First

Idea

Keep a plain list of all numbers seen so far, and every time 'add' is called, insert the new number, sort the whole list, and return the k th element from the end.

SAY THIS —

The simplest approach is to store every number in a list, and each time a new number is added, re-sort the entire list and pick out the kth largest.

STEP 3 — Brute Force Complexity

Time Complexity

Each call to 'add' costs $O(n \log n)$ for the sort, where n is the number of elements seen so far.

Space Complexity

$O(n)$ to store all the numbers.

SAY THIS —

This is $O(n \log n)$ time per add call, which adds up badly if 'add' is called many times, since we're re-sorting everything from scratch every single time.

STEP 4 — Identify the Bottleneck

I'm re-sorting the entire history of numbers on every single call, when I really only ever need to know the k largest values — everything smaller than that is irrelevant and doesn't need to be tracked precisely at all.

SAY THIS —

The bottleneck is treating every number as equally important and re-sorting everything, when I only actually care about the k largest values at any given time.

STEP 5 — Optimization

Key insight / pattern: Heap (Min-Heap of size k)

I maintain a Min-Heap that never holds more than k elements — specifically, the heap always holds exactly the k largest numbers seen so far, with the SMALLEST of those k sitting at the top of the heap. When a new number arrives: if the heap has fewer than k elements, I just add it. Otherwise, I compare the new number to the heap's smallest (top) element; if the new number is bigger, it deserves to be in the top k , so I pop the current smallest and push the new number in its place. In either case, the k th largest element is always exactly the top of this size- k min-heap.

KEY INSIGHT

If I only ever keep the k largest elements seen so far, then the smallest one AMONG those k is, by definition, the k th largest element overall — and a min-heap gives me $O(1)$ access to that smallest-of-the- k value at all times.

SAY THIS —

I'll maintain a min-heap capped at size k , holding exactly the k largest numbers seen so far.

When a new number comes in, if the heap isn't full yet, I just add it; if it's full, I only replace the heap's smallest element if the new number is bigger.

The top of this heap is always the smallest of the current top- k , which is exactly the k th largest value overall.

STEP 6 — Code While Narrating

```
import heapq

class KthLargest:
    def __init__(self, k, nums):
        self.k = k
        self.heap = []
        for num in nums:
            self.add(num)
```

SAY THIS —

In the constructor, I store k and start with an empty heap, then add every number from the initial stream one at a time using the same 'add' method I'll use later.

WHY: Reusing 'add' for the initial numbers avoids duplicating logic — the constructor is really just a sequence of the same operation that happens later for each new number.

```
def add(self, val):
    if len(self.heap) < self.k:
        heapq.heappush(self.heap, val)
    elif val > self.heap[0]:
        heapq.heapreplace(self.heap, val)

    return self.heap[0]
```

SAY THIS —

If the heap hasn't reached size k yet, I just push the new value in. Once it's at size k , I only push a new value in (replacing the current smallest) if it's bigger than the heap's current smallest element. Either way, I return the heap's top, which is the current k th largest value.

WHY: `heapq.heapreplace` does an efficient pop-then-push in one operation, and only doing the replace when `val` is bigger than the current minimum avoids unnecessary heap operations when the new value wouldn't even make it into the top k anyway.

STEP 7 — Dry Run

$k=3$, initial `nums=[4,5,8,2]`. `add(4)`: heap size $0 < 3$, push $\rightarrow [4]$. `add(5)`: size $1 < 3$, push $\rightarrow [4,5]$ (heap keeps 4 on top). `add(8)`: size $2 < 3$, push $\rightarrow [4,5,8]$ (heap keeps 4 on top). `add(2)`: size 3, not < 3 . $2 > \text{heap}[0]=4$? No. So heap unchanged, still `[4,5,8]`, top is 4. Return 4 (the 3rd largest among 4,5,8,2 is indeed 4). Now call `add(3)`: size 3, $3 > 4$? No, heap unchanged, return 4. Call `add(10)`: size 3, $10 > 4$? Yes, `heapreplace` pops 4 and pushes 10 $\rightarrow$ heap becomes `[5,8,10]` (top now 5). Return 5, which is the 3rd largest among `{4,5,8,2,3,10}`: sorted desc is 10,8,5,4,3,2, and the 3rd largest is indeed 5.

SAY THIS —

With $k=3$ and numbers 4,5,8,2 added initially, the heap settles on holding `{4,5,8}` with 4 on top, meaning 4 is the 3rd largest so far. Adding 3 doesn't change anything since it's smaller than the current minimum of the top 3. Adding 10 replaces 4 with 10, and the new 3rd largest becomes 5.

STEP 8 — Edge Case Test

Adding a value smaller than everything currently in the heap once the heap is already at size k , e.g. continuing the example above and adding 1. Since 1 is not greater than `heap[0]=5`, the heap remains unchanged, and the k th largest stays 5, correctly ignoring numbers that don't even make it into the current top k .

SAY THIS —

Let me check adding a very small number once the heap is already full, like adding 1 to a heap holding `{5,8,10}` — since 1 isn't bigger than the current minimum of 5, the heap correctly stays unchanged, and the answer remains 5.

Other edge cases to remember

- k equals 1 (just track the single largest value seen)
- Duplicate values being added multiple times
- The initial `nums` array being empty
- k equal to the total count of numbers added so far, right at the boundary

STEP 9 — Final Complexity

	Time	Space
<code>add()</code>	$O(\log k)$	$O(k)$

SAY THIS —

Each call to 'add' runs in $O(\log k)$ time, since heap push/replace operations on a heap of size k cost $O(\log k)$, and the heap uses $O(k)$ space total, regardless of how many numbers have been added overall, which is a big improvement over the brute force's $O(n \log n)$ per call.

If the Interviewer Pushes Back

Why use a min-heap and not a max-heap?

Because I specifically need fast access to the SMALLEST of the current top-k elements (to decide whether a new number should replace it, and to answer the 'kth largest' query directly), and a min-heap gives $O(1)$ access to its minimum at the top, which is exactly the value I need.

What if k could change after the object is created?

The problem specifies k is fixed for the object's lifetime; if it needed to change dynamically, I'd need additional logic to resize the heap, potentially rebuilding it from a larger stored history, which would add complexity.

Can you avoid `heapq.heapreplace` and just use separate pop and push?

Yes, that would work too, but `heapreplace` does it as a single optimized operation (pop then push) rather than two separate calls, which is slightly more efficient in practice.

Can you optimize this further?

$O(\log k)$ per add is already quite efficient; the only way to do better would be $O(1)$ amortized, which isn't generally achievable for this kind of 'maintain the kth order statistic under insertions' problem with a comparison-based structure.

Common Mistakes

- Re-sorting the entire history on every add call instead of maintaining a bounded-size heap.
- Using a max-heap of ALL elements instead of a min-heap capped at size k, which wastes both time and space.
- Forgetting the '`val > heap[0]`' check before replacing, and always replacing regardless of whether the new value actually belongs in the top k.
- Not reusing the 'add' method for processing the initial nums list in the constructor, leading to duplicated logic.

If I Get Stuck — Recovery

- If stuck: 'Re-sorting on every call is wasteful since I only ever care about the top k values. Let me think about a structure that only remembers the k largest values efficiently.'
- If I use the wrong heap type: reason aloud, 'I need quick access to the smallest of my current best k, so that's a min-heap, not a max-heap.'
- If the interviewer hints 'you don't need the full history': confirm and describe capping the heap at size k.
- If unsure about `heapreplace`: it's equivalent to a pop followed by a push, just combined into one call for efficiency — explain it that way if the exact function name is forgotten.

Pattern Recognition

Trigger words: 'kth largest in a stream', 'design a class that supports adding and querying kth largest'

Pattern: Heap (Min-Heap bounded to size k)

First thought: I only need to track the top k values at any time — a size-k min-heap gives me instant access to the smallest of those top k, which is the kth largest overall.

Common trap: Rebuilding/re-sorting the whole history on every call instead of maintaining a small bounded heap incrementally.

Memory Hook

KEY INSIGHT

Kth Largest in a Stream → "Min-heap of size k; the top of the heap IS the kth largest."

30-Second Recap

Pattern	Heap (Min-Heap of size k)
Key idea	Maintain a min-heap capped at size k; a new value only gets added if it beats the current minimum of the heap.
Data structure	A min-heap (Python's <code>heapq</code>) of size at most k.
Main algorithm	Bounded heap maintenance across repeated 'add' calls.
Time	$O(\log k)$ per add
Space	$O(k)$
Biggest edge case	Adding a value smaller than everything already in a full heap (should be ignored).

Full Amazon Talk Track

Interviewer: Design a class that returns the kth largest element in a stream after each new number is added.

Me: Got it — kth largest counts duplicates, k stays fixed, and I should expect 'add' to be called many times, so I should optimize for that, correct?

Interviewer: Yes.

Me: The simplest approach would re-sort the entire history on every add call, which is $O(n \log n)$ per call — that gets expensive fast.

Me: Since I only ever care about the k largest values, I'll maintain a min-heap capped at size k. If it's not full yet, I just add the new number. Once it's full, I only replace the heap's smallest element if the new number beats it.

Me: The heap's top is always the smallest of the current top-k, which is exactly the kth largest value, so I can return it directly after every add.

Me: Let me code that, reusing the add method itself to process the initial numbers in the constructor.

Me: Tracing $k=3$ with initial numbers 4,5,8,2: the heap settles on {4,5,8}, so the 3rd largest is 4. Adding 10 replaces 4, giving a new 3rd largest of 5.

Me: Edge case: adding a small number once the heap is full and it doesn't beat the current minimum — the heap correctly stays unchanged.

Me: Final complexity: $O(\log k)$ per add call, $O(k)$ space, much better than re-sorting the whole stream every time.

10. Greedy

Provably-optimal local choices: reachability, scheduling, capacity sweeps.

Jump Game

LeetCode #55 | **MEDIUM** | Pattern: Greedy | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array where each element represents the maximum jump length from that position, starting at index 0, and we need to determine if we can reach the last index.

What I should clarify

- Can jump lengths be 0, potentially trapping us at a position?
- Are all values guaranteed non-negative?
- Is the array guaranteed non-empty, and does a single-element array trivially return true?
- Do we need the actual path, or just whether it's reachable?

SAY THIS —

Just to confirm, from any index I can jump up to (but not exceeding) that index's value forward, and I just need a yes/no on whether index 0 can reach the last index, correct?

STEP 2 — Brute Force First

Idea

Use recursion or DFS: from the current index, try every possible jump length, and recursively check if any of them can eventually reach the end.

SAY THIS —

A direct approach is recursive: from any position, try every possible jump size and recursively check if any of those choices eventually reaches the last index.

STEP 3 — Brute Force Complexity

Time Complexity

In the worst case, this branches into many choices at every step, giving exponential $O(2^n)$ time without memoization.

Space Complexity

$O(n)$ for the recursion stack.

SAY THIS —

This is exponential time in the worst case, since at every index we might branch into many different jump choices, and those branches multiply quickly.

STEP 4 — Identify the Bottleneck

I'm exploring many different specific jump choices, when actually all I really need to know is the single furthest index reachable so far — the exact path taken doesn't matter, only how far it lets me go.

SAY THIS —

The bottleneck is treating this like a full path-search problem, when actually I only need to track the single furthest reachable index as I scan through, not every possible route to get there.

STEP 5 — Optimization

Key insight / pattern: Greedy

I use a Greedy approach, scanning left to right while tracking the furthest index reachable so far. At each index i , if i is beyond the furthest reachable point, I can never even get here, so it's impossible to continue; otherwise, I update $\text{furthest} = \max(\text{furthest}, i + \text{nums}[i])$. If at any point furthest reaches or exceeds the last index, I know I can get there.

KEY INSIGHT

I don't need to know exactly which sequence of jumps gets me somewhere — I only need to know the single furthest index that's reachable using any combination of jumps up to the current point, and that value can be tracked and updated in one simple linear scan.

SAY THIS —

I'll scan left to right, keeping track of the furthest index reachable so far using any jumps up to this point.

At each index, if it's beyond what I could ever reach, I'm stuck, so I return False immediately.

Otherwise, I update the furthest reachable index using the current position's jump length, and if that ever covers the last index, I return True.

STEP 6 — Code While Narrating

```
def can_jump(nums):  
    furthest = 0
```

SAY THIS —

I initialize furthest, the farthest index reachable so far, to 0, since we start at index 0.

WHY: This single variable is all the state I need to track — it summarizes every possible combination of jumps taken so far into one number.

```
    for i in range(len(nums)):  
        if i > furthest:  
            return False  
        furthest = max(furthest, i + nums[i])
```

SAY THIS —

For each index, if it's beyond the furthest point I could have reached, I could never have gotten here in the first place, so I return False. Otherwise, I update furthest using how far I could jump from here.

WHY: Checking ' $i > \text{furthest}$ ' before updating catches the case where we've hit a dead index that was never actually reachable, which would otherwise silently produce a wrong (too optimistic) answer.

```
    return True
```

SAY THIS —

If the loop finishes without ever returning False, that means every index along the way was reachable, so I return True.

WHY: Reaching the end of the loop means the furthest reachable point never fell short of any index we needed to pass through, which is exactly the condition for successfully reaching the last index.

STEP 7 — Dry Run

nums = [2,3,1,1,4]. i=0: $0 \leq \text{furthest}(0)$. $\text{furthest} = \max(0, 0+2) = 2$. i=1: $1 \leq 2$. $\text{furthest} = \max(2, 1+3) = 4$. i=2: $2 \leq 4$. $\text{furthest} = \max(4, 2+1) = 4$. i=3: $3 \leq 4$. $\text{furthest} = \max(4, 3+1) = 4$. i=4: $4 \leq 4$. $\text{furthest} = \max(4, 4+4) = 8$. Loop ends (i only goes to $\text{len}-1=4$). Return True — index 4 (the last index) was reachable.

SAY THIS —

Tracing [2,3,1,1,4]: the furthest reachable index quickly grows to 4, covering the last index, so we correctly return true.

STEP 8 — Edge Case Test

A 0 that traps us, e.g. nums=[3,2,1,0,4]. i=0: $\text{furthest} = \max(0, 3) = 3$. i=1: $1 \leq 3$, $\text{furthest} = \max(3, 1+2) = 3$. i=2: $2 \leq 3$, $\text{furthest} = \max(3, 2+1) = 3$. i=3: $3 \leq 3$, $\text{furthest} = \max(3, 3+0) = 3$. i=4: $4 > \text{furthest}(3)$, return False immediately — correctly detecting we can never reach index 4 because the 0 at index 3 traps us there.

SAY THIS —

Let me check a case with a 0 that traps us, like [3,2,1,0,4] — the furthest reachable index gets stuck at 3, and when we reach index 4, it's beyond what's reachable, so we correctly return false.

Other edge cases to remember

- Single-element array (trivially true, we're already at the last index)
- Array starting with a 0 (only true if the array has length 1)
- Array where every element is the same large jump value
- Array with a very long stretch of zeros

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time since it's a single pass through the array, and $O(1)$ space since only one running variable, furthest, is tracked.

If the Interviewer Pushes Back

Why does the greedy approach work here?

Because reachability is monotonic — if I can reach some index, tracking the single furthest point reachable using ALL jumps available so far always dominates any other bookkeeping; there's no advantage to remembering a specific path, since only the furthest extent matters for future reachability.

Can you solve this with Dynamic Programming instead?

Yes, a DP approach would define $\text{dp}[i]$ as whether index i is reachable, checking all previous reachable indices that could jump to i , but that's $O(n^2)$ in the naive form, strictly worse than the $O(n)$ greedy approach.

What if you needed to find the actual sequence of jumps, not just whether it's possible?

I'd need to track more information, like a 'came from' pointer at each step, and greedy would need to be adjusted to a proper shortest-path or backtracking approach to reconstruct one valid path.

Can you optimize further?

No, $O(n)$ is optimal since we must examine every index at least once to determine overall reachability.

Common Mistakes

- Forgetting the check 'if $i > \text{furthest}$ ' and blindly updating furthest even at an unreachable index, giving an incorrect true result.
- Confusing this with the related 'Jump Game II' (minimum number of jumps), which requires additional bookkeeping.
- Assuming jump values are always positive, missing that a 0 can create a trap.
- Trying to solve this recursively without memoization, leading to exponential blowup.

If I Get Stuck — Recovery

- If stuck: 'Instead of tracking every possible jump path, let me just track the single furthest index reachable overall — that should be enough information.'
- If I forget to check ' $i > \text{furthest}$ ': notice the bug during a dry run involving a trapping zero, where the algorithm would otherwise wrongly continue past an unreachable index.
- If the interviewer hints 'you don't need the actual path': confirm and describe the greedy furthest-reachable-index tracking.
- If confused with Jump Game II: clarify out loud that this version only asks yes/no reachability, not minimum jumps.

Pattern Recognition

Trigger words: 'can you reach the last index', 'maximum jump length from each position'

Pattern: Greedy (track the furthest reachable index)

First thought: I don't need the actual jump sequence, just the single furthest point reachable so far, updated as I scan.

Common trap: Forgetting to check if the current index itself is still reachable before updating the furthest-reach variable.

Memory Hook

KEY INSIGHT

Jump Game → "Track the furthest you could ever reach — forget the actual path."

30-Second Recap

Pattern	Greedy
Key idea	Scan left to right, tracking the furthest reachable index; fail immediately if the current index is beyond that
Data structure	One scalar variable, furthest.
Main algorithm	Greedy single pass.
Time	$O(n)$

Space	$O(1)$
Biggest edge case	A 0 value that traps progress before reaching the end.

Full Amazon Talk Track

Interviewer: Can you reach the last index of this array, jumping at most `nums[i]` steps from index `i`?

Me: Got it — just a yes/no answer, and jump values could include 0, correct?

Interviewer: Yes.

Me: A recursive approach trying every jump choice would be exponential in the worst case.

Me: Instead, I only need to track the furthest index reachable so far as I scan left to right. If I ever reach an index beyond that furthest point, I'm stuck and return false; otherwise, I keep updating the furthest reach.

Me: Let me code that — a single pass with one running variable.

Me: Tracing `[2,3,1,1,4]`: the furthest reachable index quickly covers the last index, so it's true.

Me: Edge case: a trapping zero, like `[3,2,1,0,4]` — the furthest reach gets stuck at index 3, and index 4 becomes unreachable, correctly returning false.

Me: Final complexity: $O(n)$ time, one pass, $O(1)$ space.

Task Scheduler

LeetCode #621 | **MEDIUM** | Pattern: Greedy + Math (or Heap simulation) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a list of tasks (each represented by a letter) and a cooldown period n , meaning the same task must be separated by at least n other tasks or idle slots, and we need to find the minimum total time (including idle time) needed to complete all tasks.

What I should clarify

- Can different task types be scheduled back-to-back with no restriction, only the SAME task type needs the cooldown?
- Is n guaranteed to be non-negative?
- If there's idle time needed, does it count toward the total time returned?
- Can the order of different task types be chosen freely to minimize idle time?

SAY THIS —

Just to confirm, only repeating the SAME task requires waiting n slots (other tasks or idle time) in between, and I want to minimize total time including any necessary idle slots, correct?

STEP 2 — Brute Force First

Idea

Simulate scheduling minute by minute: at each time slot, greedily pick any available task (not on cooldown) with the highest remaining count, or idle if none are available, and track the total time elapsed using a priority queue.

SAY THIS —

A direct simulation approach processes one time unit at a time, using a priority queue (or similar structure) to always pick the most frequent available task, adding idle time when nothing's available.

STEP 3 — Brute Force Complexity

Time Complexity

This simulation runs in roughly $O(\text{total_time} * \log(\text{unique_tasks}))$ since each time step involves a heap operation.

Space Complexity

$O(\text{unique_tasks})$ for the counts and heap.

SAY THIS —

This simulation approach is efficient, roughly $O(\text{total time times log of the number of distinct tasks})$, and is actually a fully valid solution — but there's also a neat closed-form math formula that avoids simulating minute by minute at all.

STEP 4 — Identify the Bottleneck

Simulating every single time unit works but is more computation than necessary; since the structure of the optimal schedule is very predictable (it's dominated by however many times the MOST frequent task appears), there's a direct formula that skips the simulation entirely.

SAY THIS —

Simulating minute by minute isn't wrong, but it's more work than needed, since the answer's shape is actually determined by simple counting: how many times does the most frequent task appear, and how many DIFFERENT tasks tie for that maximum frequency.

STEP 5 — Optimization

Key insight / pattern: Greedy + Math (or Heap simulation)

I use a Greedy + Math formula approach. I count each task's frequency. Let `max_freq` be the highest frequency among all tasks, and `max_count` be how many distinct tasks share that highest frequency. Picture the schedule as $(\text{max_freq} - 1)$ complete 'chunks', each chunk being $(n+1)$ slots wide (the most frequent task, plus n cooldown slots to fill with other tasks or idle time), plus one final chunk containing just the `max_count` tasks that tied for the highest frequency. This gives a formula: $(\text{max_freq} - 1) * (n + 1) + \text{max_count}$. However, if there are enough OTHER tasks to fill in all the idle gaps, idle time is never actually needed, so the true answer is the maximum of that formula and simply the total number of tasks.

KEY INSIGHT

The tightest constraint on the schedule always comes from the single most frequent task (or tasks, if there's a tie), since scheduling it optimally with the mandatory cooldown gaps determines a minimum schedule length; if there are enough other distinct tasks to fill every gap, the schedule can be fully packed with no idle time, in which case the answer is just simply the total number of tasks.

SAY THIS —

I'll count the frequency of every task, then find the maximum frequency and how many tasks share that maximum.

I'll use those two numbers directly in a formula: $(\text{max_freq} \text{ minus } 1) \text{ times } (n \text{ plus } 1)$, plus however many tasks tie for the max frequency.

Since idle time is only actually needed if there aren't enough other tasks to fill the gaps, I'll take the maximum of that formula and the total task count, which naturally handles both cases.

STEP 6 — Code While Narrating

```
from collections import Counter

def least_interval(tasks, n):
    counts = Counter(tasks)
    max_freq = max(counts.values())
```

SAY THIS —

I count how many times each task appears, then find the highest frequency among all of them.

WHY: The most frequent task is what dictates the minimum spacing structure of the entire schedule, so it's the key number I need first.

```
max_count = sum(1 for count in counts.values() if count == max_freq)
```

SAY THIS —

I count how many distinct tasks share that maximum frequency.

WHY: If multiple tasks tie for the most frequent, they all need to occupy the final 'leftover' chunk together, which adds directly to the schedule length rather than needing extra cooldown-driven idle time.

```
formula = (max_freq - 1) * (n + 1) + max_count
return max(formula, len(tasks))
```

SAY THIS —

I compute the theoretical minimum length using the chunk-based formula, then return the larger of that and the total number of tasks.

WHY: Taking the max with len(tasks) correctly handles the case where there are so many different tasks that every cooldown gap gets filled naturally, meaning no idle time is ever needed and the answer is simply the total task count.

STEP 7 — Dry Run

tasks = ['A','A','A','B','B','B'], n=2. counts={'A':3,'B':3}. max_freq=3. max_count=2 (both A and B have frequency 3). formula = (3-1)*(2+1)+2 = 2*3+2 = 8. len(tasks)=6. max(8,6)=8. A valid schedule: A B idle A B idle A B, which is 8 slots — matches!

SAY THIS —

With tasks A,A,A,B,B and n=2: both A and B tie for the max frequency of 3, so the formula gives (3-1)(2+1)+2 = 8, and indeed a valid schedule like A-B-idle-A-B-idle-A-B takes exactly 8 time units.*

STEP 8 — Edge Case Test

n=0, meaning no cooldown is needed at all. formula = (max_freq-1)*(0+1)+max_count = max_freq - 1 + max_count. Taking the max with len(tasks) will typically just resolve to len(tasks) directly, since with no cooldown restriction, tasks can just be run back-to-back in any order, needing exactly as many slots as there are tasks.

SAY THIS —

Let me check n=0, meaning there's no cooldown restriction at all — the formula still computes correctly, but the max with len(tasks) naturally resolves to just the total number of tasks, since we can run everything back-to-back with no idle time needed.

Other edge cases to remember

- Only one type of task with a large n (lots of forced idle time)
- Many different task types, each appearing only once (no idle time ever needed)
- All tasks identical (worst-case spacing scenario)
- n larger than the number of distinct other tasks available to fill gaps

STEP 9 — Final Complexity

	Time	Space
Optimized (formula)	O(t)	O(1)

SAY THIS —

This is $O(t)$ time, where t is the number of tasks, just to count frequencies, and $O(1)$ extra space (a fixed 26-letter alphabet at most for typical constraints), which is much faster than simulating minute by minute.

If the Interviewer Pushes Back

Why does taking the max with `len(tasks)` work?

Because the chunk-based formula assumes the worst case where idle time is genuinely necessary to satisfy cooldowns; but if there are enough distinct OTHER tasks to naturally fill every gap without ever needing to idle, the true minimum schedule length is simply the total number of tasks, which is what the `max()` correctly captures.

Can you solve this with a heap-based simulation instead?

Yes, that's a fully valid alternative: repeatedly pick the most frequent available task each round (using a max-heap) and track a cooldown period per task type; it directly simulates the schedule and naturally handles all edge cases, at the cost of being a bit more code and slightly higher complexity than the closed-form formula.

Why does the most frequent task determine the chunk structure?

Because that task needs the most total spacing overall; scheduling it as evenly as possible (its instances forming the 'skeleton' of the schedule, each followed by n cooldown slots) creates the minimum possible structure, and other tasks simply get slotted into those gaps.

Can you optimize this further?

$O(t)$ is already optimal since we must at least look at every task once to count frequencies.

Common Mistakes

- Forgetting to take the max with `len(tasks)`, which would undercount when there's no idle time actually needed.
- Miscounting `max_count` (forgetting to count ALL tasks tied for the maximum frequency, not just one).
- Off-by-one in the formula (forgetting the '-1' on `max_freq`, since the last occurrence of the max-frequency task doesn't need a trailing cooldown).
- Trying to simulate this manually without recognizing the direct formula, leading to unnecessarily complex code.

If I Get Stuck — Recovery

- If stuck: 'Let me think about which single task type is the hardest to schedule — probably whichever one appears most often, since it dictates the spacing needed throughout.'
- If the formula gives a wrong answer: double check I'm taking the max with `len(tasks)`, and that `max_count` correctly counts ALL tied tasks, not just one.
- If the interviewer asks for a simulation instead: pivot to a heap-based approach that greedily always schedules the currently most frequent available task.
- If unsure why '-1' appears in the formula: reason it through with a tiny example, like a single task type with frequency 3 and $n=2$, which needs exactly 2 gaps (not 3) after its first, second, but not third occurrence.

Pattern Recognition

Trigger words: 'cooldown between repeated tasks', 'minimum time to complete all tasks with spacing constraint'

Pattern: Greedy + Math (chunk-based formula), or Heap-based simulation

First thought: The most frequent task(s) determine the schedule's minimum structure; a formula based on max frequency and its count often beats simulating minute by minute.

Common trap: Forgetting to compare the formula's result against the plain total task count, missing the 'no idle time needed' case.

Memory Hook

KEY INSIGHT

Task Scheduler → "The busiest task sets the pace — formula beats simulating minute by minute."

30-Second Recap

Pattern	Greedy + Math (or Heap simulation)
Key idea	Compute $(\text{max_freq} - 1) * (n + 1) + \text{max_count}$, then take the max with the total task count.
Data structure	A frequency counter (hash map).
Main algorithm	Greedy + closed-form counting formula.
Time	$O(t)$
Space	$O(1)$
Biggest edge case	Enough distinct tasks to eliminate all idle time (formula < total task count).

Full Amazon Talk Track

Interviewer: Find the minimum time to schedule all these tasks, with a cooldown of n between repeats of the same task.

Me: Got it — only the SAME task needs the cooldown, other tasks can fill those gaps freely, and I return the total time including any idle slots, correct?

Interviewer: Yes.

Me: One valid approach is simulating minute by minute with a max-heap, always picking the most frequent available task — that works, but there's actually a direct formula that avoids simulation.

Me: I'll count each task's frequency, find the maximum frequency, and how many tasks tie for it. The schedule structure is dominated by the most frequent task, which needs $(\text{max_freq} - 1)$ complete cycles of $(n+1)$ slots each, plus a final chunk holding however many tasks tied for that max frequency.

Me: But if there are enough other distinct tasks to fill every gap, no idle time is actually needed, so I take the max of that formula and just the total number of tasks.

Me: Let me code that — counting frequencies, then applying the formula.

Me: Tracing tasks A,A,A,B,B,B with $n=2$: both A and B tie for max frequency 3, giving a formula result of 8, matching a valid schedule like A-B-idle-A-B-idle-A-B.

Me: Edge case: $n=0$ means no cooldown at all, and the max with the total task count correctly resolves to just running everything back-to-back.

Me: Final complexity: $O(t)$ time to count frequencies, $O(1)$ extra space.

Car Pooling

LeetCode #1094 | **MEDIUM** | Pattern: Difference Array / Line Sweep (Greedy) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a list of trips, each with a number of passengers and a start/end location, and a car capacity, and we need to determine if it's possible to pick up and drop off all passengers for all trips without ever exceeding the car's capacity.

What I should clarify

- Does a passenger getting dropped off at location x free up capacity immediately at that same location, before new passengers boarding at x are counted?
- Are start and end locations guaranteed to be non-negative, with start always less than end?
- Can multiple trips overlap at the same locations simultaneously?
- Is capacity a fixed constant for the whole scenario?

SAY THIS —

Just to confirm, at any given location, passengers dropping off free up capacity before new passengers at that same location board, and I need to check that total passengers never exceed capacity at any point along the route, correct?

STEP 2 — Brute Force First

Idea

For every unit of distance along the route, sum up all passengers from trips that are currently 'active' (started but not yet ended) at that point, and check if it ever exceeds capacity.

SAY THIS —

A direct approach is to check, at every possible location along the route, how many passengers are currently in the car from all overlapping trips, and see if that ever exceeds capacity.

STEP 3 — Brute Force Complexity

Time Complexity

If checking each location requires scanning all trips, and there are many possible locations, this could be $O(\text{trips} * \text{max_location})$, which can be quite slow.

Space Complexity

$O(1)$ beyond tracking the trips themselves.

SAY THIS —

This naive approach can be $O(\text{trips times max_location})$ if I check every location against every trip, which is inefficient, especially over a large range of locations.

STEP 4 — Identify the Bottleneck

I'm recomputing the total passenger count from scratch at every location, when passenger count only actually CHANGES at the specific start and end points of trips — I should only need to look at those change points, not every possible location in between.

SAY THIS —

The bottleneck is checking every single location individually, when the passenger count only changes at a handful of specific points (trip starts and ends), so I should focus only on those.

STEP 5 — Optimization

Key insight / pattern: Difference Array / Line Sweep (Greedy)

I use a Difference Array (also called a Line Sweep) technique. I create an array representing the change in passenger count at each location: at a trip's start location, I ADD its passenger count (people getting on); at its end location, I SUBTRACT that same count (people getting off). Then I do a single pass across all locations in order, maintaining a running total (a prefix sum of these changes), and check at every point whether that running total ever exceeds capacity.

KEY INSIGHT

Instead of tracking 'how many passengers are on the car at each location' directly, I only need to track WHERE passenger count changes (a trip's start and end), and a running sum across those change points reconstructs the exact passenger count at every location efficiently.

SAY THIS —

I'll create a difference array where each trip adds its passenger count at its start location and subtracts it at its end location.

I'll then do a single pass across all locations, keeping a running sum of these changes.

If that running sum ever exceeds capacity at any point, I know it's not possible to fulfill all the trips.

STEP 6 — Code While Narrating

```
def car_pooling(trips, capacity):
    changes = [0] * 1001
    for passengers, start, end in trips:
        changes[start] += passengers
        changes[end] -= passengers
```

SAY THIS —

I set up a changes array large enough to cover the maximum possible location (the constraints typically cap locations at 1000). For every trip, I add its passenger count at the start location, and subtract it at the end location.

WHY: Subtracting at the END location (not end-1) correctly reflects that those passengers are no longer in the car starting exactly at that point, which matches the convention that drop-offs free capacity immediately at that same location.

```

current_passengers = 0
for change in changes:
    current_passengers += change
    if current_passengers > capacity:
        return False

return True

```

SAY THIS —

I sweep across the changes array left to right, maintaining a running total of passengers, and checking after each step whether it exceeds capacity.

WHY: This running sum (essentially a prefix sum over the difference array) reconstructs the exact number of passengers in the car at every single location, letting me catch a capacity violation the moment it would occur.

STEP 7 — Dry Run

trips = [[2,1,5],[3,3,7]], capacity=4. changes array (showing relevant indices): changes[1]+=2 -> changes[1]=2. changes[5]=-2 -> changes[5]=-2. changes[3]+=3 -> changes[3]=3. changes[7]=-3 -> changes[7]=-3. Sweep: index0:0,current=0. index1: current=0+2=2, <=4 ok. index2: current=2 (no change), ok. index3: current=2+3=5, 5>4! Return False — correctly identifying that at location 3, both trips overlap (2+3=5 passengers), exceeding capacity 4.

SAY THIS —

Tracing trips [[2,1,5],[3,3,7]] with capacity 4: at location 3, both trips are active simultaneously, putting 5 passengers in the car, which exceeds the capacity of 4, so we correctly return false.

STEP 8 — Edge Case Test

A drop-off and a pick-up happening at the exact same location, e.g. one trip ending at location 5 and another starting at location 5. Since the end location's subtraction and the start location's addition both apply exactly AT index 5, the running sum correctly reflects that the departing passengers' capacity is freed up before or simultaneously with the new passengers boarding, matching the standard convention for this problem.

SAY THIS —

Let me check a case where one trip ends exactly where another begins — since both the subtraction (for the ending trip) and the addition (for the new trip) apply at that same location in the difference array, the running sum correctly reflects that departing passengers free up room right at that point.

Other edge cases to remember

- A single trip that exactly fills the capacity (should be fine, not exceeding it)
- Trips that don't overlap at all (should always be fine regardless of capacity, as long as each individual trip fits)
- Many small trips that individually fit but collectively overlap and exceed capacity
- A trip with 0 passengers (a no-op, shouldn't affect anything)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\text{trips} + \text{max_location})$	$O(\text{max_location})$

SAY THIS —

This is $O(\text{trips} + \text{max_location})$ time, since building the difference array is proportional to the number of trips, and sweeping across it is proportional to the location range, and $O(\text{max_location})$ space for the difference array itself.

If the Interviewer Pushes Back

Why subtract exactly at the end location, not end - 1?

Because the problem's convention is that a passenger occupies the car from their start location up to but not including their end location, so subtracting at the end index correctly removes them from the running count starting exactly there, matching real-world 'drop off, then possibly pick someone else up at the same stop' semantics.

What if location values could be very large or sparse?

Instead of a fixed-size array, I could use a hash map or sort all the distinct start/end points and use a compressed range, doing coordinate compression to keep the sweep proportional to the number of DISTINCT points rather than the absolute value range.

Can you solve this without an array, using sorting instead?

Yes, an alternative is to create a list of (location, change) events, sort them by location (with drop-offs processed before pick-ups at the same location), and do the same running-sum sweep over that sorted list, which is $O(\text{trips} \log \text{trips})$ time instead of $O(\text{max_location})$ space.

Can you optimize this further?

The array-based sweep is already quite efficient given the bounded location range; if that range were much larger, the sorting/event-based approach would be preferable to avoid excessive space usage.

Common Mistakes

- Subtracting at end - 1 instead of end, misrepresenting when passengers actually leave the car.
- Forgetting to check capacity DURING the sweep, only checking the final total (which would miss violations that resolve themselves later).
- Assuming trips are given in any particular order; the difference array technique doesn't require sorting since additions/subtractions are order-independent as long as they're applied at the correct absolute location.
- Using an array sized incorrectly, either too small (missing high locations) or unnecessarily large (wasting space).

If I Get Stuck — Recovery

- If stuck: 'Checking every single location directly against all trips is repetitive. Passenger count only really changes at specific start/end points — let me track just those changes and sum across them.'
- If capacity checks come out wrong: verify whether I'm checking after each running sum update, not just at the very end.
- If the interviewer hints 'think about where the count actually changes': confirm and describe the difference array / line sweep approach.
- If the location range seems too large for a fixed array: pivot to a sorted-events approach with coordinate compression.

Pattern Recognition

Trigger words: 'overlapping intervals with capacity limit', 'passengers picked up and dropped off along a route'

Pattern: Difference Array / Line Sweep

First thought: The total at any point only changes where an interval starts or ends — track just those change points instead of every possible position.

Common trap: Subtracting at the wrong index (end-1 instead of end), or only checking the final total instead of checking during the sweep.

Memory Hook

KEY INSIGHT

Car Pooling → "Add at pickup, subtract at drop-off, sweep and watch the running total."

30-Second Recap

Pattern	Difference Array / Line Sweep (Greedy)
Key idea	Build a difference array marking +passengers at start and -passengers at end; sweep with a running sum
Data structure	A difference array indexed by location.
Main algorithm	Difference Array / Line Sweep.
Time	$O(\text{trips} + \text{max_location})$
Space	$O(\text{max_location})$
Biggest edge case	A drop-off and pick-up occurring at the exact same location.

Full Amazon Talk Track

Interviewer: Determine if this car can complete all the given trips without exceeding capacity.

Me: Got it — drop-offs free up capacity right at that same location, before new pick-ups at that spot count, correct?

Interviewer: Yes.

Me: Checking every single location against every trip directly would be inefficient if the location range is large.

Me: Instead, passenger count only actually changes at trip start and end points, so I'll use a difference array: add the passenger count at the start location, subtract it at the end location.

Me: Then I sweep across all locations once, keeping a running sum, and check if it ever exceeds capacity.

Me: Let me code that — building the difference array from the trips, then the sweep with the capacity check.

Me: Tracing trips $[[2,1,5],[3,3,7]]$ with capacity 4: at location 3, both trips overlap, putting 5 passengers in the car, which exceeds capacity, so the answer is false.

Me: Edge case: a drop-off and pick-up at the exact same location — the difference array correctly applies both changes right at that index, reflecting that capacity frees up immediately.

Me: Final complexity: $O(\text{trips plus max location})$ time and space for the sweep.

11. Backtracking

Constrained exploration with pruning, plus one constraint-checking problem (Valid Sudoku).

Generate Parentheses

LeetCode #22 | **MEDIUM** | Pattern: Backtracking | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an integer n , and we need to generate all combinations of n pairs of parentheses that are well-formed (validly balanced).

What I should clarify

- Does 'well-formed' mean every open parenthesis has a matching close, and they're never mismatched in order?
- Is n guaranteed to be a small positive integer, given the output size grows quickly (Catalan numbers)?
- Does the order of the returned combinations matter?
- Should duplicates ever occur, or is each valid combination distinct by construction?

SAY THIS —

Just to confirm, I need every distinct valid combination of n open and n close parentheses, and the order of the output doesn't matter, correct?

STEP 2 — Brute Force First

Idea

Generate all possible strings of length $2n$ using '(' and ')' in every combination, then filter out the ones that aren't validly balanced.

SAY THIS —

A brute-force way is to generate every possible string of length $2n$ made of parentheses, then check each one for validity and keep only the valid ones.

STEP 3 — Brute Force Complexity

Time Complexity

There are $2^{(2n)}$ total strings of length $2n$, and validating each takes $O(n)$, giving $O(2^{(2n)} * n)$.

Space Complexity

$O(n)$ per validity check, plus storing all generated strings.

SAY THIS —

This is $O(2$ to the $2n$ times n) time, since we're generating every possible arrangement, most of which aren't even valid, and then filtering afterward.

STEP 4 — Identify the Bottleneck

I'm generating a huge number of strings that are invalid from very early on (e.g., starting with a close parenthesis), when I could avoid ever building those invalid prefixes in the first place.

SAY THIS —

The bottleneck is generating strings that are already broken from an early point and only checking validity at the very end, instead of pruning invalid choices as soon as they become impossible to fix.

STEP 5 — Optimization

Key insight / pattern: Backtracking

I use Backtracking. I build the string one character at a time, and at each step, I only add a character if it keeps the string 'on track' to possibly become valid: I can add an open parenthesis as long as I haven't used all n of them yet, and I can add a close parenthesis only if doing so wouldn't exceed the number of opens used so far (i.e., there are more opens than closes currently placed). When the string reaches length $2n$, it's guaranteed to be a valid combination, so I add it to the results.

KEY INSIGHT

A partial string can only ever become valid if, at every point, the number of close parentheses placed never exceeds the number of opens placed so far, and we never use more than n opens total — checking these two conditions at each step prunes away every invalid path immediately, rather than discovering the mistake only at the very end.

SAY THIS —

I'll build the string character by character using backtracking, tracking how many opens and closes I've used so far.

I can add an open parenthesis if I haven't used all n yet, and a close parenthesis only if it wouldn't create more closes than opens at that point.

Whenever the string reaches length $2n$, it's guaranteed valid, so I record it as one of the answers.

STEP 6 — Code While Narrating

```
def generate_parenthesis(n):
    result = []

    def backtrack(current, open_count, close_count):
        if len(current) == 2 * n:
            result.append(current)
            return
```

SAY THIS —

I set up a results list and a recursive helper tracking the string built so far, plus how many opens and closes have been used. The base case: once the string reaches the full length $2n$, it's a complete valid combination, so I record it.

WHY: Because I only ever add characters that keep the string 'on track' to be valid, reaching length $2n$ guarantees the result is automatically well-formed, with no separate validation step needed.

```
        if open_count < n:
            backtrack(current + '(', open_count + 1, close_count)
```

SAY THIS —

If I haven't used all n open parentheses yet, I try adding one, recursing with the updated counts.

WHY: Limiting opens to at most n directly enforces that we never use more open parentheses than allowed.

```
if close_count < open_count:
    backtrack(current + ')', open_count, close_count + 1)
```

SAY THIS —

I only try adding a close parenthesis if doing so wouldn't create more closes than opens so far.

WHY: This is exactly the condition that guarantees validity at every prefix: a close parenthesis can only ever be placed if there's a corresponding unmatched open parenthesis already placed before it.

```
backtrack('', 0, 0)
return result
```

SAY THIS —

I kick off the recursion with an empty string and zero opens/closes used, then return the accumulated results.

WHY: Starting from the empty string and exploring both valid choices at each step (open, close, or both when applicable) systematically builds every valid combination without ever constructing an invalid one.

STEP 7 — Dry Run

$n=2$. `backtrack("",0,0)`: `open_count(0)<2`, try '(' -> `backtrack('(',1,0)`. Inside: `open_count(1)<2`, try '(' -> `backtrack('(',2,0)`. Inside: `open_count(2) not <2`, skip open. `close_count(0)` `backtrack('()',2,1)`. Inside: `open not <2` skip. `close(1)<2`, try '()' -> `backtrack('()',2,2)`. `length=4=2n`, add '()' to result. Back up: `backtrack('(',1,0)` also tries close: `close_count(0)` `backtrack('()',1,1)`. Inside: `open(1)<2`, try '()' -> `backtrack('()',2,1)`. `length not yet 4`, `close(1)<2`, try '()' -> `backtrack('()',2,2)`, `length=4`, add '()'. Also from '()',1,1: `close(1) not < open(1)`, skip close there. Final result after full exploration: ['()', '()'] — matches the expected 2 valid combinations for $n=2$.

SAY THIS —

Tracing $n=2$: the backtracking explores adding opens and closes while respecting the validity rules at every step, correctly producing exactly the two valid combinations, '()' and '()'.

STEP 8 — Edge Case Test

$n=0$. `backtrack("",0,0)` immediately checks `len(current)==0==2*0`, true, so it appends the empty string "" to result right away. Final result: [""], which is the (arguably trivial but technically correct) single valid combination when zero pairs are requested.

SAY THIS —

Let me check $n=0$ — the base case triggers immediately since the target length is also 0, so we get a single result: the empty string, which is technically valid for zero pairs.

Other edge cases to remember

- $n=1$ (should produce just '()')
- Larger n verifying the count matches the n th Catalan number
- Ensuring no duplicate strings are ever generated
- Ensuring no invalid combination like '()' is ever produced

STEP 9 — Final Complexity

	Time	Space
--	------	-------

Optimized	$O(4^n / \sqrt{n})$	$O(4^n / \sqrt{n})$
-----------	---------------------	---------------------

SAY THIS —

This is bounded by the n th Catalan number, which grows roughly as $O(4^n / n^{1.5})$, since that's exactly how many valid combinations exist and each takes $O(n)$ to build, and the space is similarly proportional to the total output size (the Catalan number) times the string length.

If the Interviewer Pushes Back

Why is backtracking better than generate-then-filter here?

Because backtracking prunes invalid branches immediately — the moment a choice would make the string unfixable (too many closes, or too many opens), we simply don't explore that branch at all, whereas generate-then-filter wastes time constructing and then discarding a huge number of invalid full-length strings.

What is the exact count of valid combinations for a given n ?

It's the n th Catalan number, $C(n) = (2n \text{ choose } n) / (n+1)$, which is a well-known combinatorial quantity that also shows up in many other problems involving balanced structures.

Can you solve this iteratively instead of recursively?

Yes, using an explicit stack to simulate the recursive backtracking, storing (current_string, open_count, close_count) tuples, achieving the same result without relying on the call stack.

Can you optimize this further?

Not meaningfully — since we must actually output every valid combination, the time complexity is inherently tied to how many valid combinations exist (the Catalan number), so this approach is already asymptotically optimal for the output size.

Common Mistakes

- Forgetting the constraint on when a close parenthesis can be added ($\text{close_count} < \text{open_count}$), which would allow invalid combinations like ')('.
- Forgetting the constraint limiting opens to at most n , potentially generating overly long strings.
- String concatenation inefficiency in some languages (using a list/StringBuilder and backtracking by removing the last character might be preferred over creating new strings each call, though in Python this is a minor concern).
- Not handling $n=0$ correctly if the base case check isn't exactly right.

If I Get Stuck — Recovery

- If stuck: 'Generating everything and filtering afterward wastes a lot of work on strings that are already broken early on. Let me think about what makes a PARTIAL string still fixable, and only build those.'
- If I generate an invalid combination: check the two conditions — $\text{open_count} < n$ for adding '(' and $\text{close_count} < \text{open_count}$ for adding ')' — and verify both are being enforced.
- If the interviewer hints 'can you avoid wasted work': confirm and describe pruning invalid branches early via backtracking.
- If asked about the exact output count: mention the Catalan number formula, even if just to acknowledge the connection without deriving it from scratch.

Pattern Recognition

Trigger words: 'generate all valid combinations', 'well-formed parentheses'

Pattern: Backtracking (constrained choice at each step)

First thought: Only build partial strings that could still possibly become valid — prune invalid choices immediately instead of filtering complete strings afterward.

Common trap: Forgetting the close-count-less-than-open-count constraint, which is what actually enforces validity at every prefix.

Memory Hook

KEY INSIGHT

Generate Parentheses → "Only add ')' if there's an unmatched '(' waiting for it."

30-Second Recap

Pattern	Backtracking
Key idea	Backtrack, only adding '(' if under the limit n, and ')' only if fewer closes than opens so far.
Data structure	A results list; implicit recursion call stack.
Main algorithm	Backtracking with two pruning constraints.
Time	$O(4^n / \sqrt{n})$
Space	$O(4^n / \sqrt{n})$
Biggest edge case	$n = 0$ (single empty-string result).

Full Amazon Talk Track

Interviewer: Generate all valid combinations of n pairs of parentheses.

Me: Got it — I need every distinct well-formed combination, and order doesn't matter, right?

Interviewer: Correct.

Me: Brute force would generate every string of length $2n$ and filter for validity afterward, which wastes a lot of time on strings that are broken from the very first few characters.

Me: Instead, I'll use backtracking: build the string one character at a time, only adding '(' if I haven't used all n yet, and only adding ')' if doing so wouldn't create more closes than opens so far.

Me: Once the string reaches length $2n$, it's guaranteed to be valid, so I add it directly to my results, no separate validation needed.

Me: Let me code that recursive structure.

Me: Tracing $n=2$: the backtracking explores valid choices at each step and correctly produces exactly '()' and '()()', the two valid combinations.

Me: Edge case: $n=0$ immediately produces just the empty string as the single trivial result.

Me: Final complexity is tied to the n th Catalan number, since that's exactly how many valid combinations exist, and we must output all of them.

Combination Sum

LeetCode #39 | **MEDIUM** | Pattern: Backtracking | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a list of distinct positive integers and a target, and we need to find all unique combinations where the chosen numbers sum to the target, allowing the same number to be reused an unlimited number of times.

What I should clarify

- Can the same number be used multiple times within one combination?
- Are all the candidate numbers guaranteed distinct and positive?
- Should combinations that use the same numbers in a different order be considered duplicates?
- Is the target guaranteed to be a positive integer?

SAY THIS —

Just to confirm, the same number can be reused multiple times within a combination, and combinations that use the same multiset of numbers in different orders count as the same (shouldn't be duplicated), correct?

STEP 2 — Brute Force First

Idea

Try every possible combination of numbers (with repetition) using nested loops or recursion without pruning, checking whether each complete selection sums to the target.

SAY THIS —

A very naive approach would try building every possible sequence of chosen numbers and checking at the end whether it sums to the target, without any early pruning.

STEP 3 — Brute Force Complexity

Time Complexity

Without pruning, this can be enormously wasteful, potentially unbounded, since numbers can repeat, so a bound must be set some other way; even with a reasonable bound, it can be exponential.

Space Complexity

$O(\text{target}/\text{min_candidate})$ for the depth of recursion in the worst case.

SAY THIS —

Without pruning, this approach can spiral into massive redundant exploration, since we might keep adding numbers well past the target before ever checking if we've overshot.

STEP 4 — Identify the Bottleneck

I'm continuing to explore branches even after the running sum has already exceeded the target, and exploring numbers in an order that lets the same combination be discovered multiple times in different arrangements, leading

to duplicate work and duplicate answers.

SAY THIS —

The bottleneck is not stopping early once the running sum exceeds the target, and not enforcing an order on which candidates can be picked next, which causes both wasted exploration and duplicate combinations.

STEP 5 — Optimization

Key insight / pattern: Backtracking

I use Backtracking with two key prunes. First, I stop exploring a branch immediately if the running sum exceeds the target (no point continuing). Second, to avoid generating the same combination in different orders, I only allow picking candidates from the current index ONWARD (never going back to an earlier candidate), which means each combination is only ever discovered once, in a single canonical (sorted) order.

KEY INSIGHT

By only ever choosing the current candidate again or moving forward to later candidates (never backward), each valid combination is generated in exactly one canonical order, completely avoiding the need to de-duplicate results afterward; combined with stopping as soon as the running sum exceeds the target, this prunes the search dramatically.

SAY THIS —

I'll use backtracking, tracking a running sum and the current path of chosen numbers.

At each step, I only consider candidates from the current index onward, which naturally prevents generating the same combination in a different order.

I stop exploring a branch immediately if the running sum would exceed the target, and record a valid combination whenever the running sum exactly equals it.

STEP 6 — Code While Narrating

```
def combination_sum(candidates, target):
    result = []

    def backtrack(start, path, remaining):
        if remaining == 0:
            result.append(list(path))
            return
        if remaining < 0:
            return
```

SAY THIS —

I define a recursive helper tracking the starting index for candidates, the current path of chosen numbers, and how much of the target is still remaining. If remaining hits exactly 0, I've found a valid combination, so I record a COPY of the current path. If remaining goes negative, this branch has overshoot the target, so I stop exploring it.

WHY: Copying the path (`list(path)`) is important because the same path list object gets mutated throughout the backtracking, so without copying, all recorded results would end up referencing the same (eventually emptied) list.

```

    for i in range(start, len(candidates)):
        path.append(candidates[i])
        backtrack(i, path, remaining - candidates[i])
        path.pop()

    backtrack(0, [], target)
    return result

```

SAY THIS —

I loop through candidates starting from 'start' (never going backward), trying each one: add it to the path, recurse with the SAME index i (allowing it to be reused), subtracting its value from the remaining target, and then remove it from the path afterward to try the next candidate.

WHY: Passing 'i' (not 'i+1') as the next starting index is what allows the same number to be reused multiple times, while still preventing combinations from being generated in a different order, since we never go back to try an earlier candidate after moving forward.

STEP 7 — Dry Run

candidates=[2,3,6,7], target=7. backtrack(0,[],7): i=0 (2): path=[2], backtrack(0,[2],5). i=0(2): path=[2,2], backtrack(0,[2,2],3). i=0(2): path=[2,2,2], backtrack(0,...,1). i=0(2): remaining becomes -1, return (overshoot). i=1(3): remaining -1? Let's recompute: at path=[2,2,2], remaining=1; try i=0(2): 1-2=-1, overshoot, return. i=1(3): 1-3=-2, overshoot, return. i=2(6): overshoot. i=3(7): overshoot. Pop back to [2,2], remaining=3. i=1(3): path=[2,2,3], remaining=0! Record [2,2,3]. Pop. i=2(6): 3-6 overshoot. i=3(7): overshoot. Pop back further... eventually also finds [7] directly: backtrack(0,[],7) after exhausting the '2' branch tries i=3 at the top level (7): path=[7], remaining=0, record [7]. Final result: [[2,2,3],[7]].

SAY THIS —

Tracing candidates=[2,3,6,7] with target=7: the backtracking explores reusing 2 repeatedly, eventually finding [2,2,3], and separately finds [7] directly, giving the correct final answer of [[2,2,3],[7]].

STEP 8 — Edge Case Test

No combination sums to the target at all, e.g. candidates=[5], target=3. backtrack(0,[],3): i=0(5): remaining=3-5=-2, overshoot, return immediately. No valid combination is ever recorded. Final result: [] (empty list), correctly indicating no solution exists.

SAY THIS —

Let me check a case where no combination can reach the target, like candidates=[5] with target=3 — every attempt immediately overshoots, so we correctly end up with an empty result list.

Other edge cases to remember

- Target exactly equal to one of the candidates (a single-element combination)
- Candidates that individually already exceed the target (immediately pruned)
- A target of 0 (only the empty combination would sum to it, though typically targets are positive)
- Candidates list containing just one very small number requiring many repetitions

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(N^{(T/\min)})$	$O(T/\min)$

SAY THIS —

The time complexity is difficult to express simply since it depends on the branching factor and depth, roughly bounded by the number of candidates raised to the power of (target divided by the smallest candidate), and space is $O(\text{target}/\text{min_candidate})$ for the maximum recursion depth (the path length in the worst case).

If the Interviewer Pushes Back

Why pass 'i' instead of 'i+1' as the next start index?

Passing 'i' allows the same candidate to be chosen again in the next recursive call, which is required since numbers can be reused an unlimited number of times; if repetition weren't allowed, I'd pass $i+1$ instead, as in the related 'Combination Sum II' variant.

How do you avoid duplicate combinations in a different order?

By always choosing candidates in non-decreasing index order (never going back to an earlier index), each valid multiset of numbers is only ever discovered in one canonical order, so there's no need for any post-processing deduplication.

Can you optimize by sorting the candidates first?

Yes — sorting lets me break out of the loop early as soon as a candidate alone (added to the current running sum) would exceed the target, since all later candidates would be even larger, which is a nice additional pruning on top of the core backtracking.

What if candidates could include duplicates in the input list?

I'd want to deduplicate the candidates list first (or handle it explicitly), since this specific algorithm assumes distinct candidate values as given in the problem constraints.

Common Mistakes

- Passing $i+1$ instead of i as the next starting index, which would incorrectly disallow reusing the same number.
- Forgetting to copy the path list when appending to results, causing all recorded combinations to reference the same mutated list.
- Not pruning when remaining goes negative, wasting time exploring clearly-overshot branches.
- Allowing candidates to be chosen from any earlier index, which would generate duplicate combinations in different orders.

If I Get Stuck — Recovery

- If stuck: 'I need a way to explore combinations without generating the same set of numbers in multiple different orders. If I only ever move forward through the candidate list, that naturally avoids duplicates.'
- If duplicates appear in my results: check whether I'm passing the correct next-start index (should be non-decreasing, never allowing a backward jump).
- If the interviewer hints 'can you prune more aggressively': mention sorting the candidates first to break out of the loop early once a candidate alone would overshoot.
- If `path.pop()` is missing: recognize the bug when a dry run shows stale values persisting from a previous branch that should have been un-chosen.

Pattern Recognition

Trigger words: 'find all combinations that sum to a target', 'numbers can be reused'

Pattern: Backtracking with a forward-only index constraint

First thought: To avoid duplicate combinations in different orders, only ever pick candidates at the current index or later, never going backward.

Common trap: Forgetting to pass 'i' (allowing reuse) vs. 'i+1' (disallowing reuse), or forgetting to copy the path before appending to results.

Memory Hook

KEY INSIGHT

Combination Sum → "Only move forward through candidates; reuse allowed by staying at the same index."

30-Second Recap

Pattern	Backtracking
Key idea	Backtrack forward-only through candidates, allowing reuse of the current candidate, pruning when remain
Data structure	A results list and a path list; implicit recursion call stack.
Main algorithm	Backtracking with a forward-only index and early pruning.
Time	Exponential (bounded by branching and depth)
Space	$O(\text{target}/\text{min_candidate})$
Biggest edge case	No combination reaches the target at all.

Full Amazon Talk Track

Interviewer: Find all unique combinations of these numbers that sum to the target, reusing numbers freely.

Me: Got it — the same number can be used multiple times, and combinations using the same numbers in a different order should only count once, right?

Interviewer: Yes.

Me: A naive approach would explore every possible sequence without pruning, which risks huge redundant work and duplicate results in different orders.

Me: I'll use backtracking: track a running remaining target, and only ever consider candidates from the current index forward, which naturally prevents generating the same combination in different orders. I'll also stop immediately if the remaining target goes negative.

Me: When the remaining target hits exactly 0, I record a copy of the current path as a valid combination.

Me: Let me code that, being careful to pass the SAME index forward (not index+1) so numbers can be reused.

Me: Tracing candidates=[2,3,6,7] with target=7: the search finds [2,2,3] and [7] as the two valid combinations.

Me: Edge case: if no combination can reach the target, like candidates=[5] with target=3, every attempt immediately overshoots, giving an empty result.

Me: The complexity is exponential in the worst case, but the pruning on overshooting the target and the forward-only ordering keep the actual search space manageable.

Letter Combinations of a Phone Number

LeetCode #17 | **MEDIUM** | Pattern: Backtracking | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a string of digits from 2 to 9, each mapping to a set of letters like on a phone keypad, and we need to return all possible letter combinations that the digit string could represent.

What I should clarify

- Should an empty input string return an empty list, or a list containing just the empty string?
- Is the digit-to-letters mapping the standard phone keypad mapping (2=abc, 3=def, etc.)?
- Does the order of the returned combinations matter?
- Can digits 0 or 1 appear, which typically have no letters mapped to them?

SAY THIS —

Just to confirm, I use the standard phone keypad mapping, digits are 2 through 9 only, and an empty input should return an empty list, correct?

STEP 2 — Brute Force First

Idea

Use nested loops, one for each digit's possible letters, to build every combination directly — but this requires knowing the input length in advance, which doesn't generalize to a variable-length string.

SAY THIS —

For a fixed number of digits, I could write nested loops, one per digit, to enumerate every combination directly, but that doesn't generalize since the input string can be any length.

STEP 3 — Brute Force Complexity

Time Complexity

If it could be done, this would still be $O(4^n)$ in the worst case (some digits map to 4 letters), since that's how many total combinations exist.

Space Complexity

$O(n)$ for tracking a single combination during construction.

SAY THIS —

Even a hypothetical fixed-nested-loop approach would still be $O(4 \text{ to the } n)$ time in the worst case, since that's fundamentally how many combinations exist; the real issue is that nested loops don't generalize to arbitrary input lengths.

STEP 4 — Identify the Bottleneck

Nested loops require knowing the exact number of digits in advance, which doesn't work for a general variable-length input; I need a technique that naturally handles any input length by building up combinations one digit at a time.

SAY THIS —

The bottleneck is that a fixed set of nested loops can't handle inputs of varying length; I need an approach that generalizes by processing one digit at a time regardless of how many digits there are.

STEP 5 — Optimization

Key insight / pattern: Backtracking

I use Backtracking, which naturally generalizes to any input length. I maintain a current partial combination as I recursively process each digit's position. For each digit, I try every letter it maps to: add that letter to the current combination, recurse to handle the next digit, and then remove the letter (backtrack) to try the next possible letter for that same digit. When the current combination's length equals the number of digits, it's complete, and I add it to the results.

KEY INSIGHT

Backtracking naturally generalizes to any number of digits because it processes one digit at a time regardless of how many total digits there are, whereas nested loops would need to be manually written out for a specific fixed count, which doesn't scale.

SAY THIS —

I'll map each digit to its corresponding letters using a fixed lookup table.

I'll recursively build up a combination one digit at a time: try each possible letter for the current digit, recurse into the next digit, then backtrack to try the next letter.

Once the combination's length matches the number of input digits, it's complete, and I record it.

STEP 6 — Code While Narrating

```
def letter_combinations(digits):
    if not digits:
        return []

    phone = {
        '2': 'abc', '3': 'def', '4': 'ghi', '5': 'jkl',
        '6': 'mno', '7': 'pqrs', '8': 'tuv', '9': 'wxyz'
    }
    result = []
```

SAY THIS —

I handle the empty input case right away, returning an empty list. Then I set up the standard digit-to-letters phone keypad mapping and a results list.

WHY: Explicitly checking for an empty string avoids accidentally producing a list containing just the empty string, matching the expected convention for this problem.

```
def backtrack(index, current):
    if index == len(digits):
        result.append(current)
        return

    possible_letters = phone[digits[index]]
    for letter in possible_letters:
        backtrack(index + 1, current + letter)

backtrack(0, '')
return result
```

SAY THIS —

My recursive helper tracks which digit index I'm currently processing and the combination built so far. Once index reaches the end of the digits, the combination is complete, so I record it. Otherwise, I look up the current digit's possible letters and try each one, recursing to handle the next digit.

WHY: Because index moves forward by exactly one with each recursive call, and the base case triggers precisely when we've assigned a letter for every digit, this naturally produces every possible combination of letters, one full assignment per recursive path.

STEP 7 — Dry Run

digits = '23'. backtrack(0, ""): digit[0]='2', letters='abc'. Try 'a': backtrack(1,'a'). digit[1]='3', letters='def'. Try 'd': backtrack(2,'ad'), index==2==len, record 'ad'. Try 'e': backtrack(2,'ae'), record 'ae'. Try 'f': backtrack(2,'af'), record 'af'. Back to backtrack(0,"), try 'b': backtrack(1,'b') similarly produces 'bd','be','bf'. Try 'c': backtrack(1,'c') produces 'cd','ce','cf'. Final result: ['ad','ae','af','bd','be','bf','cd','ce','cf'] — all 9 combinations for digits '23'.

SAY THIS —

Tracing digits='23': the backtracking pairs every letter for '2' (a,b,c) with every letter for '3' (d,e,f), producing all 9 combinations: ad, ae, af, bd, be, bf, cd, ce, cf.

STEP 8 — Edge Case Test

Empty input string. The very first check 'if not digits: return []' triggers immediately, correctly returning an empty list rather than a list containing an empty string.

SAY THIS —

Let me check the empty string input — the initial guard clause catches it and correctly returns an empty list right away, rather than accidentally including an empty combination.

Other edge cases to remember

- A single digit (returns just that digit's individual letters as single-character combinations)
- A digit mapping to 4 letters, like '7' or '9' (verifies all 4 branches are explored)
- The maximum expected input length for the given constraints
- Multiple identical digits in a row, like '22' (still produces distinct letter combinations correctly)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(4^n * n)$	$O(n)$

SAY THIS —

This is $O(4^n \text{ times } n)$ time in the worst case, since there are up to 4^n total combinations (for digits like 7 and 9 that map to 4 letters), and building each combination string takes $O(n)$, and $O(n)$ space for the recursion depth (not counting the output storage itself).

If the Interviewer Pushes Back

Why can't nested loops solve this directly?

Because the number of digits (and therefore the number of nested loops needed) varies per input, and you can't write a variable number of nested loops in most languages without some other mechanism, which is exactly what recursion/backtracking provides naturally.

Can you solve this iteratively instead of recursively?

Yes, an iterative approach can build up the result list progressively: start with [''], and for each digit, expand every existing partial combination by appending each of that digit's possible letters, replacing the result list with these expanded combinations after each digit is processed.

What's the true size of the output?

It's the product of the number of letters mapped to each digit (3 or 4 depending on the digit), which in the worst case (all digits being 7 or 9) is 4^n , matching the stated time complexity.

Can you optimize this further?

No, since we must actually produce every valid combination as output, the time complexity is inherently tied to the total number of combinations that exist.

Common Mistakes

- Forgetting the empty-string edge case, which could otherwise produce a list containing a spurious empty combination.
- Using a wrong or incomplete digit-to-letters mapping (some digits like 7 and 9 have 4 letters instead of 3).
- Not advancing the index correctly in the recursive call, causing infinite recursion or missing digits.
- Trying to hardcode a fixed number of nested loops instead of a general recursive approach that handles any input length.

If I Get Stuck — Recovery

- If stuck: 'Nested loops would work for a fixed number of digits, but I need something that generalizes to any length — that's exactly what recursion is good for, processing one digit at a time.'
- If I get the wrong number of combinations: double-check the phone mapping, especially for digits 7 and 9 which have 4 letters instead of the usual 3.
- If the interviewer hints 'think about a general approach, not hardcoded loops': confirm and describe the recursive backtracking solution.
- If asked for an iterative version: describe progressively expanding a list of partial combinations, one digit at a time.

Pattern Recognition

Trigger words: 'phone keypad letter combinations', 'all combinations across multiple independent choices'

Pattern: Backtracking (one choice dimension per recursive level)

First thought: Each digit contributes an independent set of choices; recursion naturally handles a variable number of these choice dimensions where nested loops couldn't.

Common trap: Forgetting the empty-input edge case, or using an incorrect digit-to-letter mapping.

Memory Hook

KEY INSIGHT

Letter Combinations → "One recursive level per digit — try every letter, then move to the next digit."

30-Second Recap

Pattern	Backtracking
Key idea	Recursively try every possible letter for the current digit, advancing to the next digit index, until all digits are processed.
Data structure	A results list; implicit recursion call stack.
Main algorithm	Backtracking, one recursive level per input digit.
Time	$O(4^n * n)$
Space	$O(n)$
Biggest edge case	Empty input string (should return an empty list).

Full Amazon Talk Track

Interviewer: Given a string of digits, return all possible letter combinations using the phone keypad mapping.

Me: Got it — standard keypad mapping, and an empty input should return an empty list rather than a list with an empty string, correct?

Interviewer: Yes.

Me: For a fixed number of digits I could write nested loops, one per digit, but that doesn't generalize to arbitrary input lengths, so I'll use recursion instead.

Me: I'll build up a combination one digit at a time: for the current digit, try every possible letter, recurse into the next digit, and once the combination's length matches the number of digits, record it.

Me: Let me code that with the standard digit-to-letters mapping.

Me: Tracing digits='23': it pairs every letter of 2 (a,b,c) with every letter of 3 (d,e,f), producing all 9 combinations, ad through cf.

Me: Edge case: an empty input string correctly returns an empty list immediately via a guard clause.

Me: Final complexity: $O(4^n \text{ times } n)$ time in the worst case, since some digits map to 4 letters, and $O(n)$ space for the recursion depth.

Valid Sudoku

LeetCode #36 | **MEDIUM** | Pattern: Hash Set (Constraint Checking) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a 9x9 Sudoku board partially filled with digits 1-9 and empty cells marked '.', and we need to determine if the current board configuration is valid, meaning no row, column, or 3x3 sub-box contains a repeated digit.

What I should clarify

- Do I need to check that the board is SOLVABLE, or just that the CURRENT configuration doesn't violate any rules?
- Are empty cells always represented by the character '.'?
- Is the board always exactly 9x9?
- Should I validate that only digits 1-9 (and '.') appear, or can I assume the input is already well-formed in that sense?

SAY THIS —

Just to confirm, I only need to check that the CURRENT filled-in numbers don't violate any row/column/box rule, not whether the puzzle is ultimately solvable, correct?

STEP 2 — Brute Force First

Idea

For each row, column, and 3x3 box separately, scan through and check for duplicate non-empty digits using a temporary set for each one.

SAY THIS —

A straightforward approach checks each row, then each column, then each 3x3 box separately, using a temporary set each time to detect duplicates.

STEP 3 — Brute Force Complexity

Time Complexity

Checking all rows is $O(81)$, all columns is another $O(81)$, and all boxes another $O(81)$, so overall it's $O(81)$ times a constant factor of 3, which simplifies to $O(1)$ since the board size is fixed at 9x9, but conceptually $O(n^2)$ for a general $n \times n$ board.

Space Complexity

$O(9)$ per temporary set, reused across checks.

SAY THIS —

Since the board size is fixed at 9x9, this is technically $O(1)$ time and space in an absolute sense, but conceptually it's $O(n^2)$ for an $n \times n$ board, since we make three separate full passes over the board.

STEP 4 — Identify the Bottleneck

Making three completely separate passes (one for rows, one for columns, one for boxes) means visiting every cell three times total, when I could check all three constraints for each cell in a SINGLE pass instead.

SAY THIS —

The bottleneck, such as it is, is visiting the board three separate times instead of combining all three checks into one single pass over every cell.

STEP 5 — Optimization

Key insight / pattern: Hash Set (Constraint Checking)

I use three sets of Hash Sets (one per row, one per column, one per 3x3 box), and do a single pass over every cell in the board. For each filled cell, I compute which row-set, column-set, and box-set it belongs to (the box index is computed as $(row // 3) * 3 + (col // 3)$), and check if the digit is already in any of those three corresponding sets. If it is, the board is invalid; otherwise, I add the digit to all three sets and continue.

KEY INSIGHT

Every cell belongs to exactly one row, one column, and one box, and a hash set gives $O(1)$ average time to check 'have I seen this digit in this row/column/box before' — doing all three checks together for each cell as I scan once is strictly more efficient than three separate full scans.

SAY THIS —

I'll create 9 sets for rows, 9 for columns, and 9 for boxes.

I'll do a single pass through every cell; for each filled cell, I check whether its digit is already in the corresponding row, column, or box set.

If it is, the board is invalid immediately; otherwise, I add the digit to all three relevant sets and move on.

STEP 6 — Code While Narrating

```
def is_valid_sudoku(board):
    rows = [set() for _ in range(9)]
    cols = [set() for _ in range(9)]
    boxes = [set() for _ in range(9)]
```

SAY THIS —

I create a list of 9 empty sets each for rows, columns, and boxes.

WHY: Using a list of sets, one per row/column/box index, lets me check and update the right set in $O(1)$ using simple index lookups as I scan the board.

```
    for r in range(9):
        for c in range(9):
            val = board[r][c]
            if val == '.':
                continue
            box_index = (r // 3) * 3 + (c // 3)
```

SAY THIS —

I loop through every cell. If it's empty, I skip it. Otherwise, I compute which of the 9 boxes this cell belongs to using integer division on the row and column.

WHY: Dividing row and column by 3 (integer division) groups cells into their correct 3x3 box, and the formula $(r//3)*3 + (c//3)$ maps that 3x3 grid position into a single flat index from 0 to 8.

```

        if val in rows[r] or val in cols[c] or val in boxes[box_index]:
            return False

        rows[r].add(val)
        cols[c].add(val)
        boxes[box_index].add(val)

    return True

```

SAY THIS —

I check if this digit already exists in its row's set, column's set, or box's set; if so, the board is invalid. Otherwise, I add the digit to all three relevant sets and continue to the next cell.

WHY: Checking and updating all three constraints together, for every cell, in a single pass, is what lets this whole validation run in just one scan over the board instead of three separate scans.

STEP 7 — Dry Run

A tiny conceptual trace: suppose row 0 has '5' at column 0 and later another '5' at column 4 in the same row. Processing (0,0): val='5', rows[0] empty, cols[0] empty, boxes[0] empty, no conflict, add '5' to rows[0], cols[0], boxes[0]. Later processing (0,4): val='5', check rows[0]: '5' is already in rows[0]! Return False immediately — correctly catching the duplicate in the same row.

SAY THIS —

Tracing a small case where the digit 5 appears twice in row 0: the first occurrence adds 5 to that row's set, and the second occurrence immediately finds it already there, correctly flagging the board as invalid.

STEP 8 — Edge Case Test

A duplicate digit within the same 3x3 box but in different rows and columns, e.g. '3' at (0,0) and '3' at (2,2), both within the top-left box. Both map to box_index = (0//3)*3+(0//3)=0 and (2//3)*3+(2//3)=0 respectively — same box! The second occurrence finds '3' already in boxes[0], correctly returning False even though they're in different rows and columns.

SAY THIS —

Let me check a duplicate that's only a box violation, not a row or column violation — like '3' appearing at both (0,0) and (2,2), which are in different rows and columns but the same 3x3 box. The box index calculation correctly maps both to the same box, catching the violation.

Other edge cases to remember

- A completely empty board (should be trivially valid)
- A completely and correctly filled valid Sudoku solution
- Only a single digit filled in anywhere (trivially valid)
- A duplicate that spans exactly a row-column-box intersection in a tricky way

STEP 9 — Final Complexity

	Time	Space
Optimized	O(1) (fixed 81 cells)	O(1) (fixed 243 slots)

SAY THIS —

Since the board is a fixed 9x9 size, this is technically $O(1)$ time and space in absolute terms (81 cells, at most 243 total set entries), but thinking of it generally as an $n \times n$ board, it's $O(n^2)$ time and space, achieved in a single pass rather than three separate passes.

If the Interviewer Pushes Back

Why compute the box index as $(r//3)*3 + (c//3)$?

Because dividing the row and column by 3 groups them into one of 3 'super-rows' and 3 'super-columns' of boxes; multiplying the super-row by 3 and adding the super-column flattens that 3x3 arrangement of boxes into a single index from 0 to 8, uniquely identifying each 3x3 box.

Could you use a single combined set with tagged entries instead of three separate structures?

Yes, a common trick is to insert strings like `f'row{r}-{val}'`, `f'col{c}-{val}'`, `f'box{box_index}-{val}'` all into one big set, checking membership before adding; this achieves the same result with a single set instead of three separate lists of sets, just with slightly more string overhead per entry.

What if the board could contain invalid characters or wrong dimensions?

I'd assume, based on problem constraints, that the board is always a well-formed 9x9 grid with only digits 1-9 or '.', but if that weren't guaranteed, I'd add validation for those cases first.

Can you optimize this further?

Since we must examine every filled cell at least once to detect any violation, and the board size is fixed, this single-pass approach is already essentially optimal.

Common Mistakes

- Doing three separate full passes over the board (once each for rows, columns, boxes) instead of combining all three checks into one pass.
- Getting the box index formula wrong, such as forgetting to multiply by 3 or dividing incorrectly.
- Forgetting to skip empty cells ('.'), which would otherwise be treated as a value to check/add.
- Checking validity as if a full board solution were required, when only the current partial configuration needs to be conflict-free.

If I Get Stuck — Recovery

- If stuck: 'Checking rows, columns, and boxes separately means visiting every cell three times. Let me see if I can check all three constraints for each cell in one single pass instead.'
- If the box index seems wrong: work out a small example by hand, mapping cell (row, col) pairs to expected box numbers 0-8, and verify the formula against that.
- If the interviewer hints 'can you do this in one pass': confirm and describe combining all three hash-set checks together per cell.
- If confused about what's being validated: clarify explicitly that this checks CURRENT validity, not whether the puzzle is solvable to completion.

Pattern Recognition

Trigger words: 'validate a Sudoku board', 'no duplicates in row/column/box'

Pattern: Hash Set (multiple constraint tracking in a single pass)

First thought: Each cell belongs to exactly one row, column, and box — check all three constraints together per cell using hash sets, rather than three separate full passes.

Common trap: Getting the box-index formula wrong, or doing three separate scans instead of one combined scan.

Memory Hook

KEY INSIGHT

Valid Sudoku → "One pass, three sets per cell: row, column, and box."

30-Second Recap

Pattern	Hash Set (Constraint Checking)
Key idea	For each filled cell, check and update its row, column, and box hash sets together in a single pass.
Data structure	Three lists of 9 hash sets each (rows, columns, boxes).
Main algorithm	Single-pass constraint checking with hash sets.
Time	$O(1)$ [$O(n^2)$ general]
Space	$O(1)$ [$O(n^2)$ general]
Biggest edge case	A duplicate that violates only the box constraint, not row or column.

Full Amazon Talk Track

Interviewer: Determine if this partially filled Sudoku board is currently valid.

Me: Just to confirm, I only need to check the current configuration for conflicts, not whether the puzzle is solvable to completion, right?

Interviewer: Correct.

Me: A straightforward approach checks all rows, then all columns, then all boxes separately, each with a temporary set — that's three full passes over the board.

Me: I can combine that into a single pass: I'll keep 9 sets each for rows, columns, and boxes, and for every filled cell, check and update all three relevant sets together.

Me: The box index for a cell comes from dividing its row and column by 3 and combining them into a single 0-8 index.

Me: Let me code that — looping through every cell once, skipping empties, and checking membership in the row, column, and box sets before adding.

Me: Tracing a small case with a duplicate 5 in the same row: the second occurrence is immediately caught since 5 is already in that row's set.

Me: Edge case: a duplicate that only violates the box constraint, like the same digit at two different cells within the same 3x3 box but different rows and columns — the box index correctly identifies and catches that.

Me: Since the board is a fixed 9x9, this is effectively $O(1)$ time and space, achieved in one single pass instead of three.

12. Dynamic Programming

Building answers from smaller subproblems, bottom-up.

Climbing Stairs

LeetCode #70 | **EASY** | Pattern: Dynamic Programming (Fibonacci-style) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a staircase with n steps, and at each move we can climb either 1 or 2 steps, and we need to find how many distinct ways there are to reach the top.

What I should clarify

- Can we only move 1 or 2 steps at a time, never more?
- Is n guaranteed to be a positive integer?
- Do different orderings of the same steps count as distinct ways (e.g., $1+2$ vs $2+1$)?
- Is there a size constraint suggesting recursion without memoization would be too slow?

SAY THIS —

Just to confirm, each move is exactly 1 or 2 steps, and different orderings of the same combination of moves count as distinct ways, correct?

STEP 2 — Brute Force First

Idea

Use plain recursion: the number of ways to reach step n is the number of ways to reach step $n-1$ plus the number of ways to reach step $n-2$, without storing any intermediate results.

SAY THIS —

The direct recursive definition is that $\text{ways}(n)$ equals $\text{ways}(n-1)$ plus $\text{ways}(n-2)$, but without storing results, we'd recompute the same subproblems over and over.

STEP 3 — Brute Force Complexity

Time Complexity

Without memoization, this recursion branches into two calls at every level, giving $O(2^n)$ time in the worst case.

Space Complexity

$O(n)$ for the recursion call stack.

SAY THIS —

This is $O(2^n)$ time because of the massive redundant recomputation — the same subproblems, like $\text{ways}(5)$, get computed many times across different recursive branches.

STEP 4 — Identify the Bottleneck

I'm recomputing the same smaller subproblems repeatedly across different branches of the recursion tree, when each distinct subproblem (ways to reach a given step) only ever needs to be computed once.

SAY THIS —

The bottleneck is redundant recomputation — ways(k) for any given k is always the same value no matter which branch of the recursion tree asks for it, so I should compute it once and reuse it.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (Fibonacci-style)

I use Dynamic Programming, recognizing this is structurally identical to the Fibonacci sequence. Instead of recomputing, I build up the answer from the bottom: ways to reach step 1 is 1, ways to reach step 2 is 2, and every step after that is the sum of the previous two. I only need to keep the last two values at any point, not a full array, to compute the next one.

KEY INSIGHT

ways(n) = ways(n-1) + ways(n-2) is exactly the Fibonacci recurrence, and just like Fibonacci, I only ever need the two most recent values to compute the next one, avoiding both the exponential blowup of naive recursion and the need for a full array of size n.

SAY THIS —

I recognize this as the Fibonacci recurrence: ways to reach step n is the sum of ways to reach step n-1 and step n-2.

I'll build this up from the bottom, starting with the known base cases for 1 and 2 steps.

I only need to keep track of the two most recent values at any time, updating them as I move toward n.

STEP 6 — Code While Narrating

```
def climb_stairs(n):  
    if n <= 2:  
        return n
```

SAY THIS —

I handle the base cases directly: reaching step 1 has exactly 1 way, and reaching step 2 has exactly 2 ways (1+1 or 2).

WHY: These base cases match the actual number of ways for very small staircases, and serve as the starting point for building up to larger n.

```
    previous_two = 1  
    previous_one = 2  
    for step in range(3, n + 1):  
        current = previous_one + previous_two  
        previous_two = previous_one  
        previous_one = current  
  
    return previous_one
```

SAY THIS —

I set up two variables representing the ways to reach the previous two steps, then iterate from step 3 up to n, each time computing the current step's ways as the sum of the previous two, and shifting my tracking variables forward.

WHY: Because each new value only depends on the two immediately preceding values, I never need to store the entire history — just these two rolling variables, which keeps space usage constant regardless of how large n is.

STEP 7 — Dry Run

$n=5$. Base case skipped since $n > 2$. $\text{previous_two}=1$ (ways(1)), $\text{previous_one}=2$ (ways(2)). $\text{step}=3$: $\text{current}=2+1=3$ (ways(3)=3). Update: $\text{previous_two}=2$, $\text{previous_one}=3$. $\text{step}=4$: $\text{current}=3+2=5$ (ways(4)=5). Update: $\text{previous_two}=3$, $\text{previous_one}=5$. $\text{step}=5$: $\text{current}=5+3=8$ (ways(5)=8). Update: $\text{previous_two}=5$, $\text{previous_one}=8$. Loop ends (range stops at $n+1=6$). Return $\text{previous_one}=8$. Final answer: 8 ways to climb 5 steps.

SAY THIS —

Tracing $n=5$: the ways build up as 1, 2, 3, 5, 8, which is exactly the Fibonacci sequence, and the final answer is 8 distinct ways to climb 5 steps.

STEP 8 — Edge Case Test

$n=1$, the smallest meaningful staircase. The base case check ' $n \leq 2$ ' immediately returns n , which is 1, correctly representing the single way to climb a 1-step staircase.

SAY THIS —

Let me check $n=1$ — the base case handles it directly and correctly returns 1, since there's exactly one way to climb a single step.

Other edge cases to remember

- $n=2$ (exactly 2 ways: 1+1 or 2)
- $n=0$ if it could occur (typically considered out of the problem's valid range, or trivially 1 way — doing nothing)
- A moderately large n , verifying the iterative approach doesn't hit recursion limits the way naive recursion would
- Verifying the sequence matches known Fibonacci numbers for sanity checking

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n)$	$O(1)$

SAY THIS —

This is $O(n)$ time since we compute each value from 3 to n exactly once in a single loop, and $O(1)$ space since we only keep two rolling variables instead of a full array.

If the Interviewer Pushes Back

Can you solve this with full memoization instead of the rolling-variable trick?

Yes, top-down recursion with a memo dictionary (or array) caching each $\text{ways}(k)$ the first time it's computed would also achieve $O(n)$ time, though it uses $O(n)$ space for the memo table (plus recursion stack), compared to $O(1)$ space for the iterative rolling-variable version.

Why is this the same as Fibonacci?

Because the recurrence relation is identical: each value is the sum of the two preceding values, with similar base cases, so the sequence of answers for $n=1,2,3,4,5,\dots$ exactly matches the Fibonacci sequence shifted slightly.

What if you could also climb 3 steps at a time?

The recurrence would extend to $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2) + \text{ways}(n-3)$, and I'd need to track the three most recent values instead of two, but the same bottom-up rolling approach would still apply.

Can you optimize this further, e.g., using matrix exponentiation?

Yes, since this is structurally Fibonacci, matrix exponentiation (or a closed-form formula using the golden ratio) can compute the n th value in $O(\log n)$ time, though that's typically overkill unless n is extremely large and the interviewer specifically asks for it.

Common Mistakes

- Using naive recursion without memoization, leading to exponential time.
- Getting the base cases wrong (e.g., starting from 0 and 1 like standard Fibonacci, without adjusting for this problem's specific indexing).
- Off-by-one in the loop range, either starting or stopping at the wrong step.
- Using a full array to store all values when only the last two are ever needed.

If I Get Stuck — Recovery

- If stuck: 'The direct recursive definition, $ways(n) = ways(n-1) + ways(n-2)$, has a lot of repeated subproblems. This looks exactly like Fibonacci, so let me build it up from the bottom instead.'
- If my base cases seem off: verify them concretely — climbing 1 step has 1 way, climbing 2 steps has 2 ways (1+1 or 2), and build up from there.
- If the interviewer hints 'have you seen this recurrence before': confirm it's Fibonacci and describe the $O(1)$ -space rolling variable technique.
- If unsure about the loop bounds: trace through a small n by hand to verify the loop correctly reaches and includes step n .

Pattern Recognition

Trigger words: 'number of distinct ways to climb/reach', 'each step is 1 or 2 (or a small fixed set of choices)'

Pattern: Dynamic Programming (Fibonacci-style recurrence)

First thought: Does the answer at n depend only on the answers at a small number of previous positions? If so, build up from the bottom with rolling variables instead of full recursion.

Common trap: Using naive unmemoized recursion, or storing a full array when only the last couple of values are ever needed.

Memory Hook

KEY INSIGHT

Climbing Stairs → "It's just Fibonacci — today's ways is yesterday's plus the day before."

30-Second Recap

Pattern	Dynamic Programming (Fibonacci-style)
Key idea	$ways(n) = ways(n-1) + ways(n-2)$; build up iteratively with two rolling variables.
Data structure	Two scalar variables — no array needed.
Main algorithm	Bottom-up Dynamic Programming (Fibonacci recurrence).
Time	$O(n)$
Space	$O(1)$

Biggest edge case	$n = 1$ (smallest valid staircase).
--------------------------	-------------------------------------

Full Amazon Talk Track

Interviewer: How many distinct ways can you climb n stairs, taking 1 or 2 steps at a time?

Me: Got it — different orderings of the same steps count as distinct ways, correct?

Interviewer: Yes.

Me: The direct recursive definition is $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$, but without memoization that recomputes the same subproblems repeatedly, giving exponential time.

Me: This is exactly the Fibonacci recurrence, so I'll build it up from the bottom instead, using just two rolling variables for the last two values, rather than a full recursion tree or even a full array.

Me: Let me code that — base cases for $n=1$ and $n=2$, then a loop from 3 up to n updating the rolling variables.

Me: Tracing $n=5$: the sequence builds 1, 2, 3, 5, 8, giving a final answer of 8 ways.

Me: Edge case: $n=1$ is handled directly by the base case, correctly returning 1.

Me: Final complexity: $O(n)$ time for the single loop, $O(1)$ space since I only keep two rolling variables.

Perfect Squares

LeetCode #279 | **MEDIUM** | Pattern: Dynamic Programming | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an integer n , and we need to find the minimum number of perfect square numbers (like 1, 4, 9, 16, ...) that sum up to exactly n .

What I should clarify

- Is n guaranteed to be a positive integer?
- Can the same perfect square be used multiple times in the sum?
- Is there always a valid answer (since 1 is always a perfect square, any n can be expressed as a sum of 1's)?
- Is there a size constraint on n that would rule out certain approaches?

SAY THIS —

Just to confirm, I can reuse the same perfect square multiple times, and since 1 is always available, there's always at least one valid decomposition, correct?

STEP 2 — Brute Force First

Idea

Try every possible combination of perfect squares summing to n using recursion, exploring all ways to subtract a perfect square and recursing on the remainder.

SAY THIS —

A direct recursive approach tries subtracting every possible perfect square less than or equal to n , and recursively finds the minimum count for the remainder, without caching any results.

STEP 3 — Brute Force Complexity

Time Complexity

Without memoization, this branches into multiple choices at every level, leading to exponential time in the worst case.

Space Complexity

$O(n)$ for the recursion depth in the worst case.

SAY THIS —

This is exponential time without memoization, since the same remainder values get recomputed repeatedly across different branches of the recursion tree.

STEP 4 — Identify the Bottleneck

I'm recomputing the minimum count for the same remainder values over and over across different recursive paths, when each distinct remainder only needs to be solved once.

SAY THIS —

The bottleneck is redundant recomputation of the same subproblems — the minimum number of squares needed to sum to a given remainder is always the same value, regardless of which path in the recursion got us there.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming

I use Dynamic Programming. I build an array `dp` where `dp[i]` represents the minimum number of perfect squares that sum to `i`. I initialize `dp[0] = 0` (zero squares needed for a sum of zero). For every `i` from 1 to `n`, I try every perfect square `j*j` that's less than or equal to `i`, and set `dp[i]` to the minimum of its current value and `dp[i - j*j] + 1` (using that square, plus however many squares were needed for the remainder).

KEY INSIGHT

The minimum number of squares to reach `i` is 1 (for using some perfect square `j*j`) plus the minimum number of squares needed to reach the remainder (`i - j*j`), and since the remainder is always smaller than `i`, computing `dp` in increasing order of `i` means every smaller subproblem is already solved when I need it.

SAY THIS —

I'll build a `dp` array where `dp[i]` holds the minimum number of perfect squares summing to `i`, with `dp[0] = 0` as the base case.

For each `i` from 1 to `n`, I'll try every perfect square that fits, and take the best (minimum) option of using that square plus the precomputed answer for the remainder.

By the time I reach `dp[n]`, I've correctly built up every smaller answer it could possibly depend on.

STEP 6 — Code While Narrating

```
def num_squares(n):
    dp = [float('inf')] * (n + 1)
    dp[0] = 0
```

SAY THIS —

I create a `dp` array of size `n+1`, initialized to infinity everywhere except `dp[0]`, which is 0, since zero squares are needed to sum to zero.

WHY: Using infinity as the initial value ensures that any real found combination will always be smaller and correctly overwrite it during the `min()` comparisons.

```
    for i in range(1, n + 1):
        j = 1
        while j * j <= i:
            dp[i] = min(dp[i], dp[i - j * j] + 1)
            j += 1

    return dp[n]
```

SAY THIS —

For each value `i` from 1 to `n`, I try every perfect square `j*j` that's at most `i`, and update `dp[i]` to be the smaller of its current value or using that square plus the answer for the remaining amount.

WHY: Trying every possible perfect square as the 'last square used' and taking the best result across all of them guarantees `dp[i]` ends up holding the true minimum, since we've considered every valid way to decompose `i`.

STEP 7 — Dry Run

$n=12$. $dp[0]=0$. $i=1$: $j=1(1 \leq 1)$: $dp[1]=\min(\text{inf}, dp[0]+1)=1$. $i=2$: $j=1$: $dp[2]=\min(\text{inf}, dp[1]+1)=2$. $i=3$: $j=1$: $dp[3]=\min(\text{inf}, dp[2]+1)=3$. $i=4$: $j=1$: $dp[4]=\min(\text{inf}, dp[3]+1)=4$. $j=2(4 \leq 4)$: $dp[4]=\min(4, dp[0]+1)=1$. $i=5..8$ similarly build up using combinations of 1 and 4. $i=9$: $j=1$: $dp[9]=dp[8]+1$. $j=2(4 \leq 9)$: $dp[9]=\min(\dots, dp[5]+1)$. $j=3(9 \leq 9)$: $dp[9]=\min(\dots, dp[0]+1)=1$. $i=12$: $j=1$: $dp[12]=dp[11]+1$. $j=2$: $dp[12]=\min(\dots, dp[8]+1)$. $j=3(9 \leq 12)$: $dp[12]=\min(\dots, dp[3]+1)=3+1=4$? Let's verify known answer: $12 = 4+4+4$, three squares. $dp[8]$ should be 2 ($4+4$), so $dp[12]$ via $j=2$ (using 4): $dp[8]+1 = 2+1=3$. Final $dp[12]=3$, matching the known correct answer of 3 ($4+4+4$).

SAY THIS —

Tracing $n=12$: after building up the dp array from 0, the algorithm correctly finds that 12 needs a minimum of 3 perfect squares ($4+4+4$), matching the known answer.

STEP 8 — Edge Case Test

n itself is already a perfect square, e.g. $n=16$. When $i=16$, $j=4$ gives $4*4=16 \leq 16$, so $dp[16]$ considers $dp[0]+1=1$, which becomes the minimum, correctly recognizing that a perfect square number needs just 1 square (itself) to represent it.

SAY THIS —

Let me check when n is itself a perfect square, like 16 — the algorithm correctly finds $dp[16]=1$, since 16 can be represented by the single perfect square 16 itself.

Other edge cases to remember

- $n=1$ (trivially needs exactly 1 square, itself)
- A number requiring exactly 4 squares (the theoretical maximum needed for any positive integer, per Lagrange's four-square theorem)
- A prime number that's not close to any perfect square
- Small values like $n=2$ or $n=3$ that must be built from multiple 1's

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n * \sqrt{n})$	$O(n)$

SAY THIS —

This is $O(n \text{ times } \sqrt{n})$ time, since for each of the n values we try up to $\sqrt{i}$ possible perfect squares, and $O(n)$ space for the dp array.

If the Interviewer Pushes Back

Can you solve this with BFS instead?

Yes — treating each number from 0 to n as a graph node, with an edge between i and $i - j^2$ for every perfect square j^2 , the answer is the shortest path (in terms of number of edges) from n to 0, which BFS naturally finds level by level, giving a similar $O(n \sqrt{n})$ time complexity through a different lens.

Is there a mathematical shortcut, like Lagrange's Four Square Theorem?

Yes — Lagrange's theorem guarantees every positive integer can be expressed as the sum of at most 4 perfect squares, and there are number-theoretic conditions (related to a number's prime factorization, specifically forms like $4^a(8b+7)$) that determine whether 1, 2, 3, or 4 squares are needed, which can answer this in $O(\sqrt{n})$ time without

a full DP table, though it's more specialized number theory.

Why do you iterate i from 1 to n in increasing order?

Because $dp[i]$ depends on $dp[i - j^2]$, which is always a strictly smaller index; computing in increasing order guarantees every value $dp[i]$ needs has already been correctly computed before it's used.

Can you reduce the space usage?

Not meaningfully below $O(n)$, since we generally need the full history of smaller subproblem answers to compute each new $dp[i]$ (unlike some DP problems where only a fixed small window of previous values is needed).

Common Mistakes

- Forgetting to initialize $dp[0] = 0$, or initializing all entries to 0 instead of infinity, breaking the $\min()$ comparisons.
- Using an off-by-one boundary for j (should try while $j^2 \leq i$, not $j^2 < i$).
- Recomputing without memoization (falling back to the exponential recursive approach) despite recognizing the DP structure.
- Forgetting to consider ALL possible squares at each i , only checking one or a few instead of every valid j .

If I Get Stuck — Recovery

- If stuck: 'Trying every combination recursively recomputes the same remainders repeatedly. Let me build up a table of answers for every value from 0 up to n instead.'
- If $dp[i]$ comes out wrong: verify the transition, $dp[i] = \min$ over all valid j of $dp[i - j^2] + 1$, and check that $j^2 \leq i$ is enforced correctly.
- If the interviewer hints 'have you heard of Lagrange's theorem': acknowledge the mathematical fact but note the DP or BFS approach is more general and easier to derive under interview pressure.
- If runtime seems too slow for very large n : mention the number-theoretic approach as a theoretical alternative, while defaulting to the standard, reliable DP solution.

Pattern Recognition

Trigger words: 'minimum number of perfect squares/coins summing to a target', 'fewest number of X to reach Y '

Pattern: Dynamic Programming (unbounded knapsack / coin-change style)

First thought: This is structurally identical to the coin change problem, where 'coins' are perfect squares — build a dp array where $dp[i]$ is the best answer for target i , using smaller already-solved subproblems.

Common trap: Forgetting to try ALL possible perfect squares at each step, or getting the $\min()$ initialization/comparison wrong.

Memory Hook

KEY INSIGHT

Perfect Squares $\rightarrow$ "Coin change, but the coins are perfect squares."

30-Second Recap

Pattern	Dynamic Programming
Key idea	$dp[i] = \min$ over every valid perfect square $j^2 \leq i$ of $(dp[i - j^2] + 1)$, built bottom-up.
Data structure	A dp array of size $n+1$.
Main algorithm	Bottom-up Dynamic Programming (unbounded knapsack style).

Time	$O(n * \sqrt{n})$
Space	$O(n)$
Biggest edge case	n itself being a perfect square (answer should be 1).

Full Amazon Talk Track

Interviewer: Find the minimum number of perfect squares that sum to n.

Me: Got it — the same perfect square can be reused multiple times, and there's always a valid answer since 1 is always available, correct?

Interviewer: Yes.

Me: A direct recursive approach would try every possible square to subtract and recurse on the remainder, but without caching, that recomputes the same remainders repeatedly, giving exponential time.

Me: This is structurally the same as coin change, just with perfect squares as the 'coins'. I'll build a dp array bottom-up, where $dp[i]$ is the minimum squares summing to i.

Me: For each i, I try every perfect square that fits, and take the best option of using that square plus the precomputed answer for the remainder.

Me: Let me code that — $dp[0]=0$ as the base case, then building up to $dp[n]$.

Me: Tracing $n=12$: the dp array correctly builds up to show that 12 needs a minimum of 3 squares, since $4+4+4=12$.

Me: Edge case: if n is itself a perfect square, like 16, the algorithm correctly finds the answer is just 1.

Me: Final complexity: $O(n \text{ times square root of } n)$ time, since for each value we try up to root-n candidate squares, and $O(n)$ space for the dp array.

Is Subsequence

LeetCode #392 | **EASY** | Pattern: Two Pointers (Greedy Matching) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two strings, s and t , and we need to determine if s is a subsequence of t , meaning s can be formed by deleting some (possibly zero) characters from t without changing the relative order of the remaining characters.

What I should clarify

- Do characters need to remain in the same relative order, though not necessarily contiguous?
- Can s be an empty string (trivially a subsequence of anything)?
- Is there a scenario where this check happens many times against the same t , which might favor a different approach?
- Are we comparing case-sensitively?

SAY THIS —

Just to confirm, s 's characters need to appear in t in the same relative order, though not necessarily consecutively, and an empty s should trivially count as a subsequence, correct? Will this be checked against the same t repeatedly?

STEP 2 — Brute Force First

Idea

Try every way to select characters from t that could form s , essentially checking all possible subsequences of t for a match, which is extremely wasteful.

SAY THIS —

An overly broad approach would try generating or checking many possible subsequences of t , but that's needlessly complex given a much simpler direct method exists.

STEP 3 — Brute Force Complexity

Time Complexity

Generating all subsequences of t would be $O(2^{\text{len}(t)})$, astronomically inefficient.

Space Complexity

$O(2^{\text{len}(t)})$ if actually generating them.

SAY THIS —

Trying to explicitly generate or consider all subsequences of t would be exponential time, which is wildly impractical and not the intended approach here.

STEP 4 — Identify the Bottleneck

There's no need to consider all subsequences at all; I can check greedily and directly whether s's characters appear in order within t using a straightforward linear scan.

SAY THIS —

The real insight is recognizing that checking a subsequence relationship doesn't require generating anything — a simple greedy linear scan through both strings suffices.

STEP 5 — Optimization

Key insight / pattern: Two Pointers (Greedy Matching)

I use Two Pointers, one for s and one for t, both starting at the beginning. I advance the t pointer through every character; whenever it matches the current character of s, I also advance the s pointer. If the s pointer reaches the end of s at any point, that means every character of s was found in order within t, so s is indeed a subsequence.

KEY INSIGHT

Greedily matching each character of s to the EARLIEST possible position in t (rather than trying to be clever about which occurrence to pick) is always optimal, since matching earlier never closes off options and can only leave more of t available for matching the rest of s.

SAY THIS —

I'll use two pointers, one for s and one for t, both starting at index 0.

As I scan through t, whenever the current character matches what s's pointer is looking for, I advance the s pointer too.

If the s pointer reaches the end of s by the time I've scanned all of t, then s is a subsequence.

STEP 6 — Code While Narrating

```
def is_subsequence(s, t):  
    s_pointer = 0
```

SAY THIS —

I set up a pointer into s, starting at 0, representing 'the next character of s I need to find'.

WHY: I don't need a separate pointer into t as a variable, since I can just iterate through t directly with a for loop, only tracking progress through s explicitly.

```
    for char in t:  
        if s_pointer < len(s) and char == s[s_pointer]:  
            s_pointer += 1  
  
    return s_pointer == len(s)
```

SAY THIS —

I scan through every character of t. If I still need more characters from s (s_pointer hasn't reached the end) and the current character of t matches what I'm looking for, I advance s_pointer. At the end, s is a subsequence exactly if I've matched every character, meaning s_pointer reached the full length of s.

WHY: Checking 's_pointer < len(s)' before comparing prevents an index-out-of-range error once all of s has already been matched, letting the loop harmlessly continue through the rest of t without further action.

STEP 7 — Dry Run

s='abc', t='ahbgdc'. s_pointer=0. char 'a': matches s[0]='a', s_pointer=1. char 'h': s[1]='b', no match. char 'b': matches s[1]='b', s_pointer=2. char 'g': s[2]='c', no match. char 'd': no match. char 'c': matches s[2]='c', s_pointer=3. Loop ends. s_pointer(3)==len(s)(3), return True.

SAY THIS —

Tracing s='abc' against t='ahbgdc': the pointer into s advances each time a match is found in t, ('a', then 'b', then 'c'), and by the end all of s has been matched, so it's correctly identified as a subsequence.

STEP 8 — Edge Case Test

s is longer than t, e.g. s='abc', t='ab'. s_pointer advances to 2 after matching 'a' and 'b', but t is exhausted with s_pointer still at 2, not 3 (len(s)). Return s_pointer==len(s) -> 2==3 is False, correctly identifying that s can't be a subsequence since t doesn't even have enough characters.

SAY THIS —

Let me check when s is longer than t, like s='abc' and t='ab' — t runs out before s can be fully matched, so s_pointer never reaches len(s), correctly returning false.

Other edge cases to remember

- s is an empty string (trivially a subsequence of anything, including an empty t)
- s equals t exactly
- t contains all of s's characters but in the wrong order
- t is empty and s is non-empty (should be false)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\text{len}(t))$	$O(1)$

SAY THIS —

This is $O(\text{len}(t))$ time since we make a single pass through t, and $O(1)$ extra space since only one pointer variable is used beyond the input strings themselves.

If the Interviewer Pushes Back

Why is greedily matching the earliest occurrence always correct?

Because matching as early as possible in t leaves the maximum remaining portion of t available for matching the rest of s; delaying a match to a later occurrence could only reduce the remaining options, never improve them, so greedy-earliest is provably at least as good as any other strategy.

What if this check needs to run many times against the same t with different s values?

That's a classic follow-up (LeetCode 792, 'Number of Matching Subsequences'), where precomputing, for each character, a sorted list of its positions in t, and using binary search to find the next valid position for each character of s, makes repeated queries much faster than a fresh $O(\text{len}(t))$ scan each time.

Can you solve this recursively instead of iteratively?

Yes, a two-pointer recursive version works the same way: compare the current characters, advance both pointers on a match (advance only t's pointer otherwise), with base cases for reaching the end of either string — functionally identical to the iterative version, just expressed recursively.

Can you optimize this further for a single query?

No, $O(\text{len}(t))$ is optimal for a single query since we must potentially examine every character of t to confirm or deny the subsequence relationship.

Common Mistakes

- Forgetting the bounds check ' $s_pointer < \text{len}(s)$ ' before indexing into s , causing an index error once s has already been fully matched.
- Advancing both pointers on every character of t regardless of whether there's a match, instead of only advancing $s_pointer$ conditionally.
- Assuming subsequence means contiguous (that's 'substring', a different and stricter concept).
- Not handling the empty-string s case correctly (though the given logic actually handles it correctly without special-casing, since $s_pointer$ starts and ends at $0 == \text{len}(s) = 0$).

If I Get Stuck — Recovery

- If stuck: 'This doesn't need any complex algorithm — let me just try matching characters of s to t greedily, one at a time, in order.'
- If I get an index error: check that I'm guarding array access into s with a bounds check before comparing.
- If the interviewer hints 'what if this were called many times': mention precomputing character positions in t and using binary search for repeated queries.
- If confused about subsequence vs substring: clarify explicitly that subsequence allows gaps, unlike a substring which must be contiguous.

Pattern Recognition

Trigger words: 'is X a subsequence of Y ', 'can be formed by deleting some characters'

Pattern: Two Pointers (Greedy Matching)

First thought: Greedily match each needed character to the earliest possible occurrence, advancing through the larger string once.

Common trap: Forgetting the bounds check on the shorter string's pointer, or confusing subsequence with substring.

Memory Hook

KEY INSIGHT

Is Subsequence → "Greedily match each letter of s to the next occurrence in t ."

30-Second Recap

Pattern	Two Pointers (Greedy Matching)
Key idea	Two pointers: scan through t , advancing the s -pointer only when characters match; s is a subsequence if
Data structure	A single integer pointer into s .
Main algorithm	Two Pointers (Greedy Matching).
Time	$O(\text{len}(t))$
Space	$O(1)$
Biggest edge case	s longer than t (impossible to be a subsequence).

Full Amazon Talk Track

Interviewer: Determine if s is a subsequence of t .

Me: Got it — characters of s need to appear in the same relative order in t , though not necessarily consecutively, correct?

Interviewer: Yes.

Me: This doesn't need anything complex — I can just greedily scan through t , and whenever a character matches what I'm currently looking for in s , advance my pointer into s .

Me: If my pointer into s reaches the end of s by the time I've scanned all of t , then s is a subsequence.

Me: Let me code that — a single pointer into s , iterating through every character of t .

Me: Tracing $s='abc'$ against $t='ahbgdc'$: the pointer advances on 'a', 'b', and 'c' in order, successfully matching all of s .

Me: Edge case: if s is longer than t , like $s='abc'$ and $t='ab'$, t runs out before s can be fully matched, correctly returning false.

Me: Final complexity: $O(\text{len}(t))$ time for one pass, $O(1)$ space.

Interviewer: What if this check happened many times with different s values against the same t ?

Me: I'd precompute, for each letter, a sorted list of its positions in t , then use binary search to quickly find the next valid position for each character of s , which speeds up repeated queries significantly.

Distinct Subsequences (Number of Subsequences)

LeetCode #115 | **HARD** | Pattern: Dynamic Programming (2D) | Advanced pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two strings, s and t , and we need to count how many distinct ways t can be formed as a subsequence of s (choosing different sets of index positions in s counts as different ways, even if the resulting characters are identical).

What I should clarify

- Does choosing different positions in s that happen to spell the same characters count as separate distinct ways?
- Can t be longer than s (in which case the answer should be 0)?
- Can either string be empty?
- Are we matching case-sensitively?

SAY THIS —

Just to confirm, I'm counting distinct ways based on WHICH positions in s are chosen, not just distinct resulting substrings, and if t is longer than s the answer should be 0, correct?

STEP 2 — Brute Force First

Idea

Recursively try, for each character of t , every possible matching position in the remaining portion of s , counting all the ways the rest of t can then be matched in what's left of s .

SAY THIS —

A direct recursive approach considers, for each character we need from t , every possible place it could be matched in s , and recursively counts the ways to match the rest of t in what remains.

STEP 3 — Brute Force Complexity

Time Complexity

Without memoization, this branches heavily and can be exponential, since overlapping subproblems (matching a given suffix of t within a given suffix of s) get recomputed repeatedly.

Space Complexity

$O(\text{len}(s) + \text{len}(t))$ for the recursion depth.

SAY THIS —

This is exponential time without memoization, since the same (remaining s , remaining t) subproblems recur across many different recursive branches.

STEP 4 — Identify the Bottleneck

The recursion keeps recomputing 'how many ways can I match the remaining part of t within the remaining part of s' for the same pairs of remaining substrings, when I could cache each pair's answer and reuse it.

SAY THIS —

The bottleneck is redundant recomputation of the same (s-suffix, t-suffix) subproblems, which is a strong signal that Dynamic Programming with a 2D table indexed by position in each string will help.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (2D)

I use 2D Dynamic Programming. I define $dp[i][j]$ as the number of distinct ways to form $t[0:j]$ (the first j characters of t) using $s[0:i]$ (the first i characters of s). The base case is $dp[i][0] = 1$ for all i (there's always exactly one way to form an empty t : choose nothing). For each cell, if $s[i-1] == t[j-1]$ (the current characters match), I can either use this matching character (contributing $dp[i-1][j-1]$ ways) or skip it entirely (contributing $dp[i-1][j]$ ways, ignoring this character of s); the total is their sum. If the characters don't match, I can only skip this character of s , so $dp[i][j] = dp[i-1][j]$.

KEY INSIGHT

At each position, the character in s is either used to match the corresponding position in t , or it's skipped over entirely — when a match is possible, BOTH options contribute valid ways and must be added together, which is what makes this fundamentally a counting (not just yes/no) DP problem.

SAY THIS —

I'll build a 2D dp table where $dp[i][j]$ means 'ways to form the first j characters of t using the first i characters of s '.

The base case is that there's always exactly 1 way to form an empty target string, regardless of how much of s we have available.

When characters match, I add together the ways from 'using this character' and 'skipping this character'; when they don't match, I can only skip.

STEP 6 — Code While Narrating

```
def num_distinct(s, t):
    m, n = len(s), len(t)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = 1
```

SAY THIS —

I create a 2D table of size $(m+1)$ by $(n+1)$, and initialize the first column entirely to 1, since there's always exactly one way to form an empty target string (choose nothing) regardless of how much of s we have.

WHY: This base case anchors the entire DP: every other value ultimately builds up from these initial 1's representing the trivial 'match nothing' case.

```

for i in range(1, m + 1):
    for j in range(1, n + 1):
        dp[i][j] = dp[i - 1][j]
        if s[i - 1] == t[j - 1]:
            dp[i][j] += dp[i - 1][j - 1]

return dp[m][n]

```

SAY THIS —

For every cell, I start by assuming we skip the current character of s (carrying over $dp[i-1][j]$). Then, if the current characters of s and t actually match, I also add in the ways from using this character as a match, $dp[i-1][j-1]$.

WHY: Starting with the 'skip' option unconditionally, then conditionally adding the 'use as match' option only when characters actually match, correctly captures every distinct way of forming t as a subsequence of s, counting position choices as distinct even when they spell identical substrings.

STEP 7 — Dry Run

$s='rabbbit'$, $t='rabbit'$. This is a classic example where the answer is 3 (there are 3 ways to choose which 'b' to skip among the three b's in 'rabbbit' to spell 'rabbit'). Building the dp table bottom-up: $dp[i][0]=1$ for all i. As we process matching characters ('r','a','b','b','i','t'), the table accumulates counts, particularly around the three 'b' characters in s matching the two 'b's needed in t, where multiple valid position choices contribute to the count. The final value $dp[7][6]$ correctly comes out to 3, matching the well-known answer for this classic example.

SAY THIS —

Tracing the classic example $s='rabbbit'$, $t='rabbit'$: building up the 2D table row by row, the multiple 'b' characters in s create multiple valid ways to match the two 'b's needed in t, and the final answer correctly comes out to 3.

STEP 8 — Edge Case Test

t is longer than s, e.g. $s='ab'$, $t='abc'$. Since it's structurally impossible for a longer string to be a subsequence of a shorter one, every relevant $dp[i][j]$ where j exceeds what's achievable stays 0, and the final $dp[\text{len}(s)][\text{len}(t)]$ correctly evaluates to 0.

SAY THIS —

Let me check when t is longer than s, like $s='ab'$ and $t='abc'$ — there's no way to fit a longer subsequence into a shorter string, and the dp table correctly ends up with 0 at the final cell.

Other edge cases to remember

- t is an empty string (should be exactly 1 way — choosing nothing)
- s is an empty string but t is not (should be 0 ways)
- s and t are identical (exactly 1 way — use every character)
- s contains many repeated characters relevant to matching t (verifies the counting logic handles multiplicities correctly)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(m * n)$	$O(m * n)$

SAY THIS —

This is $O(m \text{ times } n)$ time and space, where m and n are the lengths of s and t respectively, since we fill in a table with that many cells, each computed in $O(1)$ time from previously computed cells.

If the Interviewer Pushes Back

Can you reduce the space complexity?

Yes — since $dp[i][j]$ only depends on the previous row ($dp[i-1][...]$), I can use a 1D rolling array instead of a full 2D table, updating it in place (being careful about iteration direction) to reduce space from $O(m*n)$ down to $O(n)$.

Why do we add both options when characters match, rather than choosing the better one?

Because this problem asks for a COUNT of distinct ways, not just whether a match is possible, so both the 'use this character' and 'skip this character' options represent genuinely different valid subsequence choices that must both be counted, not compared and picked between.

What's the intuition behind counting position choices as distinct even for identical resulting substrings?

The problem defines distinctness based on which specific indices in s were chosen, not just the resulting character sequence, so even if two different sets of chosen positions spell the same substring, they still count as separate valid ways.

Can you optimize this further?

The $O(m*n)$ time is generally necessary since in the worst case, meaningful interactions between characters of s and t can occur across the entire table; the 1D space optimization is the main practical improvement typically expected.

Common Mistakes

- Forgetting to initialize $dp[i][0] = 1$ for all i , which would break the entire base case for matching an empty target string.
- Only adding $dp[i-1][j-1]$ when characters match, forgetting to also always carry forward $dp[i-1][j]$ (the 'skip' option).
- Confusing this counting problem with a boolean subsequence-check problem, and not correctly summing both options.
- Off-by-one errors in translating between 0-indexed strings and the 1-indexed dp table.

If I Get Stuck — Recovery

- If stuck: 'A direct recursive count-the-ways approach recomputes overlapping (s-suffix, t-suffix) subproblems repeatedly. Let me set up a 2D table indexed by position in each string to avoid that.'
- If my counts come out too low: check whether I'm always carrying forward the 'skip' option ($dp[i-1][j]$) in addition to conditionally adding the 'match' option.
- If the interviewer hints 'this is about counting, not just checking': confirm this means summing options rather than picking one when characters match.
- If space is a concern for large inputs: mention the 1D rolling-array optimization as a natural follow-up.

Pattern Recognition

Trigger words: 'count distinct ways to form one string as a subsequence of another'

Pattern: 2D Dynamic Programming (counting variant, sum options rather than choosing one)

First thought: This is a counting problem over two strings simultaneously — a 2D dp table indexed by position in both strings, where matches contribute additively (not by picking the better option).

Common trap: Forgetting to always carry forward the 'skip this character of s' option in addition to the 'use it as a match' option.

Memory Hook

KEY INSIGHT

Distinct Subsequences → "On a match, add BOTH 'use it' and 'skip it' — it's counting, not choosing."

30-Second Recap

Pattern	Dynamic Programming (2D)
Key idea	$dp[i][j] = dp[i-1][j]$ (skip), plus $dp[i-1][j-1]$ if $s[i-1] == t[j-1]$ (use as match).
Data structure	A 2D dp table of size $(m+1) \times (n+1)$.
Main algorithm	Bottom-up 2D Dynamic Programming.
Time	$O(m * n)$
Space	$O(m * n)$ [$O(n)$ with rolling array]
Biggest edge case	t longer than s (answer must be 0).

Full Amazon Talk Track

Interviewer: Count the number of distinct ways t appears as a subsequence of s.

Me: Just to confirm, different position choices in s count as distinct even if they spell the same characters, and if t is longer than s the answer should be 0, correct?

Interviewer: Yes.

Me: A direct recursive approach would try every possible matching position for each character of t, but that recomputes the same (remaining s, remaining t) subproblems repeatedly, giving exponential time.

Me: I'll use a 2D dp table instead, where $dp[i][j]$ is the number of ways to form the first j characters of t using the first i characters of s.

Me: The base case is $dp[i][0] = 1$ for all i, since there's always exactly one way to form an empty target. For each cell, I always carry forward the option of skipping the current character of s, and if the characters match, I also add in the option of using it as a match.

Me: Let me code that with the 2D table and the two nested loops.

Me: Tracing the classic example $s = \text{'rabbbit'}$, $t = \text{'rabbit'}$: the multiple 'b' characters in s create multiple valid ways to match, and the final answer correctly comes out to 3.

Me: Edge case: if t is longer than s, like $s = \text{'ab'}$ and $t = \text{'abc'}$, the final answer correctly evaluates to 0, since it's structurally impossible.

Me: Final complexity: $O(m \text{ times } n)$ time and space, where m and n are the string lengths.

Interviewer: Can you reduce the space?

Me: Yes, since each row only depends on the previous row, I can use a 1D rolling array instead of the full 2D table, bringing space down to $O(n)$.

Unique Binary Search Trees

LeetCode #96 | **MEDIUM** | Pattern: Dynamic Programming (Catalan Numbers) | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an integer n , and we need to find how many structurally unique Binary Search Trees can be formed using exactly the values 1 through n .

What I should clarify

- Are we counting only the number of distinct STRUCTURES, not enumerating the actual trees?
- Is n guaranteed to be a positive integer?
- Are the values always exactly 1 through n , with no gaps or repeats?
- Is there a known relationship to Catalan numbers that I should be aware of?

SAY THIS —

Just to confirm, I need to count distinct tree STRUCTURES using values 1 through n , not actually build or list every tree, correct?

STEP 2 — Brute Force First

Idea

Recursively try every possible value as the root, then recursively count the unique BSTs possible for the resulting left subtree (values less than the root) and right subtree (values greater), multiplying the counts together, without caching any repeated subproblems.

SAY THIS —

A direct recursive approach tries every value as the potential root, splitting the remaining values into left and right subtree ranges, and recursively multiplies the counts for each side, but without caching results, it recomputes shared subproblems repeatedly.

STEP 3 — Brute Force Complexity

Time Complexity

Without memoization, this recursion has significant overlapping subproblems, potentially leading to exponential-like behavior in practice, though bounded by the Catalan number growth itself in terms of distinct answers.

Space Complexity

$O(n)$ for the recursion depth.

SAY THIS —

Without memoization, we recompute the same 'number of unique BSTs for a range of a given SIZE' repeatedly across different specific ranges that happen to be the same length, wasting significant work.

STEP 4 — Identify the Bottleneck

The number of unique BSTs for any contiguous range of values only depends on how MANY values are in that range, not which specific values they are, so I'm needlessly recomputing the same answer for different ranges of the same size.

SAY THIS —

The bottleneck is recomputing the same answer multiple times for different ranges that happen to have the same size, when the count only actually depends on the SIZE of the range, not its specific values.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (Catalan Numbers)

I use Dynamic Programming. I define $dp[i]$ as the number of unique BSTs that can be formed using i distinct, ordered values (regardless of which specific values they are, since only relative order matters for tree SHAPE). The base cases are $dp[0] = 1$ (an empty tree is one valid 'shape') and $dp[1] = 1$ (a single node is the only shape). For larger i , I consider every possible root position: if the root is the k th smallest value (for k from 1 to i), then there are $(k-1)$ values in the left subtree and $(i-k)$ values in the right subtree, contributing $dp[k-1] * dp[i-k]$ unique combinations for that choice of root; I sum this over every possible k .

KEY INSIGHT

Because BST structure only depends on the relative ORDER of values, not their specific identities, the number of unique BSTs for a contiguous range of size i is always the same regardless of which actual values are in that range — this collapses what looks like many different subproblems into just $n+1$ distinct ones ($dp[0]$ through $dp[n]$).

SAY THIS —

I'll define $dp[i]$ as the number of unique BST shapes possible using any i values in a row, with base cases $dp[0]=1$ and $dp[1]=1$.

For a given size i , I'll consider every possible root position k , where the left subtree gets $k-1$ values and the right subtree gets $i-k$ values.

I'll sum $dp[k-1]$ times $dp[i-k]$ over every possible k from 1 to i , since each choice of root and each combination of left/right shapes is a distinct overall tree.

STEP 6 — Code While Narrating

```
def num_trees(n):
    dp = [0] * (n + 1)
    dp[0] = 1
    dp[1] = 1
```

SAY THIS —

I create a dp array of size $n+1$, and set the base cases: $dp[0]=1$ (an empty range still counts as one valid, empty tree shape) and $dp[1]=1$ (a single value has exactly one shape).

WHY: These base cases anchor the recursive formula, since every larger $dp[i]$ value is built from combinations of dp values at smaller indices, ultimately bottoming out at these two.

```
    for i in range(2, n + 1):
        for k in range(1, i + 1):
            dp[i] += dp[k - 1] * dp[i - k]

    return dp[n]
```

SAY THIS —

For each size i from 2 up to n , I consider every possible root position k , and add $dp[k-1]$ times $dp[i-k]$ to $dp[i]$, accumulating the count across all root choices.

WHY: Each choice of k represents a genuinely distinct family of trees (differing in which value is the root), and multiplying $dp[k-1]$ by $dp[i-k]$ correctly counts every combination of a valid left-subtree shape with a valid right-subtree shape for that specific root choice.

STEP 7 — Dry Run

$n=3$. $dp[0]=1$, $dp[1]=1$. $i=2$: $k=1$: $dp[2] += dp[0]*dp[1]=1*1=1$. $k=2$: $dp[2] += dp[1]*dp[0]=1*1=1$. $dp[2]=2$. $i=3$: $k=1$: $dp[3] += dp[0]*dp[2]=1*2=2$. $k=2$: $dp[3] += dp[1]*dp[1]=1*1=1$. $k=3$: $dp[3] += dp[2]*dp[0]=2*1=2$. $dp[3]=2+1+2=5$. Final answer: $dp[3]=5$, matching the known result that there are exactly 5 unique BSTs possible with values 1,2,3.

SAY THIS —

Tracing $n=3$: building up $dp[0]=1$, $dp[1]=1$, $dp[2]=2$, and finally $dp[3]=5$, which matches the well-known result that there are exactly 5 distinct BST shapes using values 1 through 3.

STEP 8 — Edge Case Test

$n=0$, representing an empty set of values. $dp[0]$ is set directly to 1 as a base case, and if the function were called with $n=0$, it would correctly return 1, representing that there's exactly one 'shape' (the empty tree) for zero values.

SAY THIS —

Let me check $n=0$ — the base case $dp[0]=1$ handles it directly, representing that there's exactly one valid shape (the empty tree) when there are no values at all.

Other edge cases to remember

- $n=1$ (trivially just 1 shape, a single node)
- $n=2$ (exactly 2 shapes, since the smaller value can be root with the larger as its right child, or vice versa)
- Verifying $dp[n]$ matches the known Catalan number formula for sanity checking
- Larger n , checking that growth matches the expected rapid (though not exponential in n^2 sense) increase of Catalan numbers

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n^2)$	$O(n)$

SAY THIS —

This is $O(n^2)$ time since for each of the n sizes, we sum over up to n possible root positions, and $O(n)$ space for the dp array.

If the Interviewer Pushes Back

Is there a direct formula instead of building the dp table?

Yes — $dp[n]$ is exactly the n th Catalan number, given by the closed-form formula $C(n) = \frac{(2n \text{ choose } n)}{(n+1)}$, which can be computed directly using factorials or a running product formula, potentially avoiding the $O(n^2)$ table entirely in favor of an $O(n)$ or even $O(1)$ -with-precomputed-factorials approach.

Why does $dp[i]$ only depend on the SIZE of a range, not its specific values?

Because BST structure (the shape of the tree) is entirely determined by the relative ORDER of values, not their specific magnitudes — any set of i values with a fixed relative order produces exactly the same set of possible tree shapes as any other set of i values in the same relative order.

Can you actually construct all the unique trees, not just count them?

Yes, that's a related problem (Unique Binary Search Trees II) that uses a very similar recursive splitting idea, but instead of just counting, it recursively builds and returns actual list combinations of left and right subtrees for every root choice, which is more expensive since it must construct the actual tree objects.

Can you optimize the counting further?

Using the direct Catalan number formula would reduce this from $O(n^2)$ to $O(n)$ (or better, depending on how efficiently large factorials or binomial coefficients are computed), which is worth mentioning as a mathematical shortcut on top of the DP solution.

Common Mistakes

- Forgetting the base case $dp[0] = 1$, which is needed for the 'root is the smallest or largest value' cases where one side of the split is empty.
- Using an incorrect summation range for k (should be 1 through i inclusive, representing every possible root position).
- Confusing 'counting unique structures' with 'counting unique VALUES arrangements', which would give a different (and generally much larger) answer.
- Not recognizing the Catalan number connection, missing an opportunity to mention a more elegant closed-form alternative.

If I Get Stuck — Recovery

- If stuck: 'For any range of values, if I fix which one is the root, everything smaller goes left and everything bigger goes right — let me count combinations for every possible root choice.'
- If $dp[i]$ comes out wrong: manually verify a small case, like $n=2$ or $n=3$, against known correct values (1, 2, 5 for $n=0,1,2,3$ respectively, i.e., $dp[0]=1, dp[1]=1, dp[2]=2, dp[3]=5$) to catch summation errors.
- If the interviewer hints 'do you recognize this sequence': mention the Catalan number connection if it comes to mind, or continue confidently with the DP approach if not.
- If confused about why size alone matters: reason through it concretely — relabeling values 4,5,6 as 1,2,3 doesn't change how many tree SHAPES are possible, since shape only depends on relative order.

Pattern Recognition

Trigger words: 'count unique/structurally distinct binary search trees', 'Catalan number problems in general'

Pattern: Dynamic Programming (Catalan Number recurrence)

First thought: BST shape depends only on relative order, not specific values, so the count for a range only depends on its SIZE — build a dp table indexed by size, summing over every possible root choice.

Common trap: Forgetting the $dp[0]=1$ base case, which is essential for handling the smallest/largest root edge cases in the summation.

Memory Hook

KEY INSIGHT

Unique BSTs $\rightarrow$ "Fix a root; multiply left-shape count by right-shape count; sum over every root choice."

30-Second Recap

Pattern	Dynamic Programming (Catalan Numbers)
Key idea	$dp[i] = \text{sum over } k=1..i \text{ of } dp[k-1] * dp[i-k]$, with $dp[0]=dp[1]=1$; this is the Catalan number recurrence.
Data structure	A 1D dp array of size $n+1$.
Main algorithm	Bottom-up Dynamic Programming (Catalan recurrence).
Time	$O(n^2)$
Space	$O(n)$
Biggest edge case	$n = 0$ (empty tree, base case $dp[0]=1$).

Full Amazon Talk Track

Interviewer: How many structurally unique BSTs can be formed with values 1 through n ?

Me: Got it — I just need to count distinct STRUCTURES, not actually build or list every tree, correct?

Interviewer: Yes.

Me: A direct recursive approach would try every value as root, splitting into left and right ranges recursively, but without caching, the same range-size subproblems get recomputed repeatedly.

Me: I realized the count for a range only depends on its SIZE, not which specific values are in it, since BST shape only cares about relative order. So I'll build a dp table where $dp[i]$ is the count for any i values in a row.

Me: For each size i , I sum over every possible root position k : $dp[k-1]$ times $dp[i-k]$, since the left and right subtree shapes combine independently for each root choice.

Me: Let me code that with the base cases $dp[0]=1$ and $dp[1]=1$.

Me: Tracing $n=3$: dp builds up to 1, 1, 2, 5, matching the known result that there are exactly 5 unique BSTs for values 1 through 3.

Me: Edge case: $n=0$ is handled directly by the $dp[0]=1$ base case, representing the single empty-tree shape.

Me: Final complexity: $O(n^2)$ time, since each of the n sizes sums over up to n root choices, and $O(n)$ space for the dp array.

Interviewer: Is there a faster way?

Me: Yes, this sequence is exactly the Catalan numbers, which have a closed-form formula using binomial coefficients, potentially avoiding the full table.

13. Design / Data Structure Design

Combining structures to hit multiple operation complexity targets at once.

Min Stack

LeetCode #155 | **MEDIUM** | Pattern: Stack (Auxiliary Min-Tracking) | Design-heavy problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We need to design a stack that supports the usual push, pop, and top operations, plus a getMin operation that retrieves the minimum element currently in the stack, all in $O(1)$ time.

What I should clarify

- Do push, pop, top, AND getMin all need to run in $O(1)$ time, not just amortized?
- Can the stack contain duplicate values, including duplicates of the current minimum?
- Is it guaranteed that pop and top are never called on an empty stack?
- Is there a constraint on how much extra space I can use?

SAY THIS —

Just to confirm, ALL four operations — push, pop, top, and getMin — need to run in $O(1)$ time, and duplicate values (including duplicate minimums) are possible, correct?

STEP 2 — Brute Force First

Idea

Use a plain stack, and whenever getMin is called, scan through the entire stack to find the minimum value.

SAY THIS —

The simplest approach is a regular stack, and whenever getMin is called, scan through all the current elements to find the smallest one.

STEP 3 — Brute Force Complexity

Time Complexity

Push, pop, and top are all $O(1)$, but getMin costs $O(n)$ since it scans the whole stack.

Space Complexity

$O(n)$ for the stack itself.

SAY THIS —

This gives $O(1)$ for push, pop, and top, but $O(n)$ for getMin, which violates the requirement that ALL operations run in $O(1)$ time.

STEP 4 — Identify the Bottleneck

I'm recomputing the minimum from scratch every time getMin is called, when I could instead maintain the minimum incrementally as elements are pushed and popped, so it's always instantly available.

SAY THIS —

The bottleneck is treating getMin as a fresh computation every time, instead of maintaining a running 'current minimum' that updates automatically as the stack changes.

STEP 5 — Optimization

Key insight / pattern: Stack (Auxiliary Min-Tracking)

I maintain a second, Auxiliary Stack that tracks the minimum value at each point in the main stack's history. Whenever I push a value onto the main stack, I also push onto the min-stack: either the new value (if it's less than or equal to the min-stack's current top) or the min-stack's current top again (if the new value isn't a new minimum). This way, the min-stack's top always reflects the minimum of everything currently in the main stack. When I pop from the main stack, I also pop from the min-stack, keeping both stacks perfectly synchronized.

KEY INSIGHT

By pushing a 'running minimum as of this point' onto a parallel stack every time I push a real value, popping from both stacks together automatically 'un-does' the minimum tracking too, since whatever minimum was true before that push is exactly what's revealed again on the min-stack after popping.

SAY THIS —

I'll maintain a second stack alongside the main one, specifically for tracking minimums.

Every time I push a value, I also push the smaller of that value and the current minimum onto the min-stack, so its top is always the overall minimum so far.

Every time I pop, I pop from both stacks together, which automatically restores the correct previous minimum.

STEP 6 — Code While Narrating

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
```

SAY THIS —

I initialize two empty lists: the main stack for regular values, and the min-stack for tracking running minimums.

WHY: Keeping these as two separate, perfectly synchronized stacks (always the same length) is what lets every operation, including getMin, run in O(1) time.

```
def push(self, val):
    self.stack.append(val)
    if not self.min_stack or val <= self.min_stack[-1]:
        self.min_stack.append(val)
    else:
        self.min_stack.append(self.min_stack[-1])
```

SAY THIS —

I push the value onto the main stack as usual. For the min-stack, if it's empty or the new value is less than or equal to its current top, I push the new value; otherwise, I push a duplicate of the current top.

WHY: Using <= (not just <) correctly handles duplicate minimum values, ensuring that popping one copy of the minimum still leaves the min-stack correctly reporting that value as the minimum if another copy remains below it.

```

def pop(self):
    self.stack.pop()
    self.min_stack.pop()

def top(self):
    return self.stack[-1]

def get_min(self):
    return self.min_stack[-1]

```

SAY THIS —

Popping removes from both stacks together, keeping them synchronized. top() just returns the main stack's top as usual, and get_min() simply returns the min-stack's top, which is always correct by construction.

WHY: Because both stacks are always pushed to and popped from together, the min-stack's top always accurately reflects the minimum of exactly what's currently in the main stack, with no scanning ever required.

STEP 7 — Dry Run

Operations: push(-2), push(0), push(-3), getMin(), pop(), top(), getMin(). push(-2): stack=[-2], min_stack empty so push -2 -> min_stack=[-2]. push(0): stack=[-2,0]. 0<=-2? No. push min_stack[-1]=-2 again -> min_stack=[-2,-2]. push(-3): stack=[-2,0,-3]. -3<=-2? Yes. push -3 -> min_stack=[-2,-2,-3]. getMin(): return min_stack[-1]=-3. Correct, min of [-2,0,-3] is -3. pop(): stack=[-2,0], min_stack=[-2,-2]. top(): return stack[-1]=0. getMin(): return min_stack[-1]=-2. Correct, min of [-2,0] is -2.

SAY THIS —

Tracing push(-2), push(0), push(-3), getMin() (gives -3), pop(), top() (gives 0), getMin() (gives -2): every operation returns the correct value, and the min-stack automatically 'remembers' the right minimum at each point as we pop back through history.

STEP 8 — Edge Case Test

Pushing duplicate minimum values, e.g. push(1), push(1), pop(), getMin(). push(1): stack=[1], min_stack=[1] (empty check, push 1). push(1): 1<=1? Yes, push 1 again -> min_stack=[1,1]. pop(): stack=[1], min_stack=[1]. getMin(): return 1, still correctly the minimum even though we just removed one of the two 1's.

SAY THIS —

Let me check pushing the same minimum value twice, then popping once — using <= (not strict <) when deciding what to push onto the min-stack ensures that after popping one copy, getMin still correctly returns the remaining 1.

Other edge cases to remember

- Popping until the stack is completely empty
- Pushing values in strictly increasing order (min-stack always just duplicates the very first value)
- Pushing values in strictly decreasing order (min-stack mirrors the main stack exactly)
- A stack containing only a single element

STEP 9 — Final Complexity

	Time	Space
All operations	O(1)	O(n)

SAY THIS —

Every operation — push, pop, top, and getMin — runs in $O(1)$ time, since each just accesses or modifies the end of one or both stacks, and the overall space is $O(n)$ for maintaining the two parallel stacks.

If the Interviewer Pushes Back

Why use $\leq$ instead of $<$ when deciding what to push onto the min-stack?

Because using strict $<$ would fail to correctly track a duplicate minimum: if I pushed a value equal to the current minimum but didn't also duplicate it onto the min-stack, popping the main stack later could leave the min-stack out of sync, no longer correctly reflecting that the same minimum value still exists deeper in the stack.

Can you do this with less extra space, like $O(1)$ instead of $O(n)$ for the min-stack?

Yes, an alternative encodes the minimum differently: instead of a parallel stack, store just a single running minimum variable, and when pushing a new minimum, push an ENCODED value onto the main stack itself (like $2 \cdot \text{val} - \text{old_min}$) that lets you recover the previous minimum during a pop; this trades some arithmetic complexity for avoiding a full second stack, though it requires careful handling of potential overflow in some languages.

Does this handle negative numbers correctly?

Yes, the min-stack approach works identically regardless of whether values are positive, negative, or zero, since it's just comparing values directly with $\leq$, with no assumption about sign.

Can you optimize this further?

No, $O(1)$ per operation is already optimal, since these are meant to be constant-time stack primitives by design; the only real tradeoff available is the space optimization mentioned above.

Common Mistakes

- Using strict $<$ instead of $\leq$ when pushing onto the min-stack, breaking correctness with duplicate minimum values.
- Forgetting to pop from BOTH stacks together, causing them to fall out of sync.
- Trying to maintain just a single 'current minimum' variable without a full auxiliary stack, which can't correctly restore a PREVIOUS minimum after a pop.
- Not initializing the min-stack correctly on the very first push (missing the 'is min_stack empty' check).

If I Get Stuck — Recovery

- If stuck: 'Scanning for the minimum every time is $O(n)$, but I need $O(1)$. Let me think about maintaining the minimum incrementally instead, updated alongside every push and pop.'
- If `getMin()` gives a wrong answer after a pop: verify that both stacks are always pushed and popped together, keeping them the same length at all times.
- If the interviewer hints 'can you use less space': mention the single running-minimum-with-encoding trick as a space-optimized alternative.
- If confused about $\leq$ vs $<$: trace through a duplicate-minimum example by hand, which will clearly reveal why $\leq$ is required for correctness.

Pattern Recognition

Trigger words: 'design a stack that also supports `getMin/getMax` in $O(1)$ '

Pattern: Stack with an Auxiliary Tracking Stack

First thought: Maintain a second, perfectly synchronized stack that tracks the running minimum, so it's always immediately available without scanning.

Common trap: Using strict $<$ instead of $\leq$ when pushing onto the min-stack, which breaks correctness for duplicate minimum values.

Memory Hook

KEY INSIGHT

Min Stack $\rightarrow$ "A second stack, always tracking the minimum-so-far, pushed and popped in lockstep."

30-Second Recap

Pattern	Stack (Auxiliary Min-Tracking)
Key idea	Maintain a parallel min-stack; each push also pushes the running minimum; pop from both together.
Data structure	Two stacks (lists): the main stack and the min-stack.
Main algorithm	Auxiliary stack synchronized push/pop.
Time	$O(1)$ all operations
Space	$O(n)$
Biggest edge case	Duplicate minimum values pushed and later popped.

Full Amazon Talk Track

Interviewer: Design a stack that also supports retrieving the minimum element in $O(1)$ time.

Me: Got it — push, pop, top, AND getMin all need to be $O(1)$, and duplicate values including duplicate minimums are possible, correct?

Interviewer: Yes.

Me: A regular stack handles push, pop, and top in $O(1)$ easily, but getMin would need to scan the whole stack, $O(n)$, unless I track it incrementally.

Me: I'll maintain a second, auxiliary min-stack alongside the main one. Every push also pushes the current running minimum onto this second stack, and every pop removes from both stacks together, so they stay perfectly synchronized.

Me: Let me code that — using $\leq$ rather than strict $<$ when deciding what to push onto the min-stack, which matters for correctly handling duplicate minimums.

Me: Tracing push(-2), push(0), push(-3), getMin(), pop(), top(), getMin(): every operation returns the correct value, and the min-stack automatically restores the right minimum as we pop.

Me: Edge case: pushing the same minimum value twice, then popping once — the $\leq$ comparison ensures getMin still correctly returns that value afterward.

Me: Final complexity: $O(1)$ for every single operation, $O(n)$ space for the two parallel stacks.

Interviewer: Can you reduce the space usage?

Me: Yes, there's a trick using a single running-minimum variable and encoding values pushed onto the main stack itself, avoiding a full second stack, though it adds some arithmetic complexity.

Insert Delete GetRandom O(1)

LeetCode #380 | **MEDIUM** | Pattern: Hash Map + Array | Design-heavy problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We need to design a data structure that supports inserting a value, removing a value, and getting a truly random element from the current set of values, all in average $O(1)$ time.

What I should clarify

- Should insert and remove return whether the operation actually changed anything (e.g., false if inserting a duplicate or removing something not present)?
- Does every element (including duplicates, if allowed) need equal probability of being chosen by getRandom, or are all values guaranteed unique?
- Is there a constraint on extra space usage?
- Should getRandom use a uniform distribution across all current elements?

SAY THIS —

Just to confirm, all values are unique (it's a set, not a multiset), insert/remove should indicate whether something actually changed, and getRandom needs uniform probability across current elements, correct?

STEP 2 — Brute Force First

Idea

Use just a Python list: insert appends (checking for duplicates first), remove searches for and removes the value by scanning, and getRandom picks a random index directly.

SAY THIS —

Using just a plain list, insert can check for duplicates and append, getRandom can pick a random index directly in $O(1)$, but remove has to search through the list to find the value's position first.

STEP 3 — Brute Force Complexity

Time Complexity

Insert is $O(n)$ to check for duplicates, remove is $O(n)$ to find the value, and getRandom is $O(1)$.

Space Complexity

$O(n)$ for the list.

SAY THIS —

Insert and remove both risk being $O(n)$ with a plain list, since checking for duplicates or finding a value's exact position requires scanning, which violates the $O(1)$ requirement for those operations.

STEP 4 — Identify the Bottleneck

A list alone doesn't give $O(1)$ lookup for 'does this value exist' or 'where is this value', but an array alone is what gives $O(1)$ random access needed for `getRandom` — I need to combine both structures to get the best of each.

SAY THIS —

The bottleneck is that a hash map alone can't give $O(1)$ random access (needed for `getRandom`), and an array alone can't give $O(1)$ lookup/removal by value — I need both structures working together.

STEP 5 — Optimization

Key insight / pattern: Hash Map + Array

I combine a Hash Map and an Array (list). The array stores all the actual values, giving $O(1)$ random access for `getRandom` (just pick a random valid index). The hash map stores value-to-index-in-the-array, giving $O(1)$ lookup to check existence or find a value's position. The tricky part is removal: to remove a value in $O(1)$, instead of shifting all elements after it (which would be $O(n)$), I swap it with the LAST element in the array, update the hash map for that swapped element's new position, then simply pop the last element off (which is now the value I wanted to remove) — all $O(1)$ operations.

KEY INSIGHT

Removing from the middle of an array is normally $O(n)$ because of the need to shift everything after it, but if you don't care about preserving order, swapping the target with the very last element first turns 'remove from the middle' into 'remove from the end', which is $O(1)$ — the hash map is what lets you instantly find where the target is without searching.

SAY THIS —

I'll keep both an array of values (for $O(1)$ random access) and a hash map from value to its index in that array (for $O(1)$ lookup).

Insert appends to the array and records its index in the map, if it's not already present.

Remove is the clever part: I swap the target value with the array's last element, update the map for the swapped element, then pop the last element off, turning an $O(n)$ middle-removal into an $O(1)$ operation.

STEP 6 — Code While Narrating

```
import random

class RandomizedSet:
    def __init__(self):
        self.values = []
        self.value_to_index = {}
```

SAY THIS —

I initialize an empty list to store values, and an empty hash map from value to its current index in that list.

WHY: These two structures together are what let every operation run in $O(1)$: the array for random access, the map for instant lookups.

```
def insert(self, val):
    if val in self.value_to_index:
        return False
    self.value_to_index[val] = len(self.values)
    self.values.append(val)
    return True
```

SAY THIS —

If the value already exists, insertion fails (returns False) since we maintain a set of unique values. Otherwise, I record its new index (the current length of the list, before appending) and add it to the end of the list.

WHY: Recording the index BEFORE appending correctly captures 'where this new value will land', since appending to a list of current length L places the new item at index L.

```
def remove(self, val):
    if val not in self.value_to_index:
        return False
    index_to_remove = self.value_to_index[val]
    last_value = self.values[-1]
    self.values[index_to_remove] = last_value
    self.value_to_index[last_value] = index_to_remove
    self.values.pop()
    del self.value_to_index[val]
    return True
```

SAY THIS —

If the value doesn't exist, removal fails. Otherwise, I find its index using the map, then move the array's LAST value into that position (overwriting the target), update the map so the moved value points to its new index, and finally pop the true last slot off the array (which now correctly represents the removed target's old duplicate) and delete the removed value's own map entry.

WHY: This swap-with-last trick is exactly what avoids an O(n) shift: instead of removing from the middle (expensive), we move the problem to the end (cheap), since popping the last element of a list is O(1).

```
def get_random(self):
    return random.choice(self.values)
```

SAY THIS —

I return a uniformly random element directly from the array.

WHY: Since the array holds every current value exactly once with no gaps, random.choice gives a perfectly uniform O(1) random selection.

STEP 7 — Dry Run

insert(1): values=[1], map={1:0}. insert(2): values=[1,2], map={1:0,2:1}. insert(3): values=[1,2,3], map={1:0,2:1,3:2}.
remove(2): index_to_remove=map[2]=1. last_value=values[-1]=3. values[1]=3 -> values=[1,3,3] temporarily.
map[3]=1 (updated). values.pop() -> values=[1,3]. del map[2]. Final: values=[1,3], map={1:0,3:1}. Correctly removed 2, and 3 (which was swapped into 2's old slot) now correctly maps to index 1.

SAY THIS —

Tracing insert(1), insert(2), insert(3), then remove(2): the value 3 gets swapped into 2's old position, the map is updated to reflect that, and the last slot is popped off, leaving values=[1,3] with a correctly updated map — all in O(1) steps, no shifting required.

STEP 8 — Edge Case Test

Removing the very last element in the array, e.g. after the above state (values=[1,3]), remove(3).
index_to_remove=map[3]=1. last_value=values[-1]=3 (same as the value being removed, since it IS the last element). values[1]=3 (no actual change, since it's swapping the last element with itself). map[3]=1 (unchanged).

values.pop() -> values=[1]. del map[3]. Correctly ends with values=[1], map={1:0}, with no corruption despite the target and the 'last value' being the same element.

SAY THIS —

Let me check removing an element that's ALREADY the last one in the array — the swap-with-last logic ends up swapping it with itself, which is harmless, and the pop correctly removes it, leaving the structure in a valid state.

Other edge cases to remember

- Removing a value that doesn't exist (should return False, no change)
- Inserting a duplicate value (should return False, no change)
- getRandom called when only one element exists
- Repeated insert/remove cycles verifying the map and array stay correctly synchronized

STEP 9 — Final Complexity

	Time	Space
All operations	$O(1)$ average	$O(n)$

SAY THIS —

Insert, remove, and getRandom all run in average $O(1)$ time, since each involves only a constant number of hash map and array operations, and the overall space is $O(n)$ for storing all the current values in both structures.

If the Interviewer Pushes Back

Why swap with the LAST element specifically, rather than any other element?

Because popping from the END of a Python list is $O(1)$, while removing from the middle would require shifting all subsequent elements, which is $O(n)$; by moving the 'gap' created by removal to the very end first, we can close it with a cheap pop instead of an expensive shift.

What if the value being removed is already the last element?

As shown in the edge case, the algorithm still works correctly, though the swap becomes a harmless no-op (swapping an element with itself) before the final pop removes it.

Does getRandom truly give a uniform distribution?

Yes, since the array contains every current value exactly once with no gaps or duplicates, and random.choice selects uniformly among all current array positions, every element has an equal $1/n$ chance of being selected.

Can you extend this to a multiset that allows duplicate values?

Yes, that's a related follow-up (Insert Delete GetRandom $O(1)$ - Duplicates allowed), which requires the hash map to store a SET of indices per value instead of a single index, adding some complexity to correctly handle removal when duplicates exist.

Common Mistakes

- Trying to remove from the array by searching for the value's position instead of using the hash map, reintroducing $O(n)$ removal.
- Shifting elements after removal instead of using the swap-with-last-then-pop trick, which is $O(n)$ instead of $O(1)$.
- Forgetting to update the hash map entry for the swapped (formerly-last) value after moving it.

- Forgetting to delete the removed value's own entry from the hash map, leaving stale data.

If I Get Stuck — Recovery

- If stuck: 'An array gives me $O(1)$ random access but $O(n)$ removal by value; a hash map gives me $O(1)$ lookup but no random access. Let me combine both, and think about how to make removal $O(1)$ too.'
- If removal seems inherently $O(n)$: reason aloud, 'popping from the END of an array is $O(1)$ — can I turn removing from the middle into removing from the end somehow?' which leads to the swap trick.
- If the interviewer hints 'think about swapping': confirm and describe swapping the target with the last element before popping.
- If confused about updating the map after a swap: trace through a small example by hand to see exactly which map entries need updating.

Pattern Recognition

Trigger words: 'O(1) insert, delete, AND random access all together'

Pattern: Hash Map + Array combined (Hash Map for lookup, Array for random access)

First thought: No single structure gives $O(1)$ for both lookup/removal AND random access — combine a hash map (value to index) with an array (values), and use a swap-with-last trick to make removal $O(1)$ too.

Common trap: Forgetting to update the hash map for the swapped element's new index after a removal, corrupting future lookups.

Memory Hook

KEY INSIGHT

Insert Delete GetRandom → "Array for random access, hash map for lookup, swap-with-last to remove."

30-Second Recap

Pattern	Hash Map + Array
Key idea	Combine an array (random access) with a hash map (value-to-index lookup); remove by swapping with the last element.
Data structure	An array (list) plus a hash map (value → index).
Main algorithm	Swap-with-last removal technique.
Time	$O(1)$ average, all operations
Space	$O(n)$
Biggest edge case	Removing a value that is already the array's last element.

Full Amazon Talk Track

Interviewer: Design a structure supporting insert, remove, and getRandom, all in average $O(1)$ time.

Me: Got it — all values are unique, insert/remove should indicate whether something actually changed, and getRandom needs to be uniformly random, correct?

Interviewer: Yes.

Me: An array alone gives $O(1)$ getRandom, but removal by value would require an $O(n)$ search. A hash map alone gives $O(1)$ lookup, but no $O(1)$ random access. I'll combine both.

Me: The array stores the values; the hash map stores value-to-index. Insert appends to the array and records its index. GetRandom just picks a random array index directly.

Me: Removal is the interesting part: instead of shifting elements after removing from the middle, which would be $O(n)$, I swap the target with the array's LAST element, update the map for that swapped element, and pop the last slot off — all $O(1)$.

Me: Let me code that up.

Me: Tracing insert(1), insert(2), insert(3), then remove(2): 3 gets swapped into 2's old slot, the map updates accordingly, and the last slot pops off cleanly, leaving [1,3] correctly.

Me: Edge case: removing a value that's already the last element in the array — the swap becomes a harmless no-op with itself, and the pop still correctly removes it.

Me: Final complexity: $O(1)$ average time for every operation, $O(n)$ space for the array and hash map together.

14. Other Important Amazon-Style Problems

Bit manipulation, simulation, and greedy-formatting problems that round out the playlist.

Reverse Bits

LeetCode #190 | **EASY** | Pattern: Bit Manipulation | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given a 32-bit unsigned integer, and we need to reverse the order of its bits and return the resulting integer.

What I should clarify

- Is the input always exactly 32 bits, zero-padded on the left if needed?
- Should the output also be treated as a 32-bit unsigned integer?
- Is there a preference for a bit-by-bit approach versus a built-in string reversal trick?
- Will this function be called repeatedly, which might favor a precomputation-based approach?

SAY THIS —

Just to confirm, the input is always a 32-bit unsigned integer, and I need to reverse the actual bit pattern (not just the numeric value) and return it as an integer, correct?

STEP 2 — Brute Force First

Idea

Convert the integer to its 32-character binary string representation, reverse that string, and convert it back to an integer.

SAY THIS —

A simple approach is to convert the number to its 32-bit binary string, reverse that string directly, and parse it back into an integer.

STEP 3 — Brute Force Complexity

Time Complexity

Converting to binary and back are both $O(32)$, which is effectively $O(1)$ for a fixed-width integer.

Space Complexity

$O(32)$ for the string representation.

SAY THIS —

Since we're always dealing with exactly 32 bits, this is $O(1)$ time and space in an absolute sense, though conceptually it's $O(k)$ for a k -bit integer.

STEP 4 — Identify the Bottleneck

There isn't a deeper algorithmic bottleneck to eliminate for a fixed 32-bit width; the interesting part is understanding both the straightforward string-based method and the more 'bit manipulation native' iterative method, since interviewers often want to see manipulation with shifts and masks directly.

SAY THIS —

This one doesn't really have an asymptotic bottleneck to fix, given the fixed 32-bit size; the goal is showing comfort with direct bit manipulation (shifts and masks) as an alternative to string conversion.

STEP 5 — Optimization

Key insight / pattern: Bit Manipulation

I use direct Bit Manipulation with shifts. I initialize a result to 0, and for each of the 32 bit positions, I extract the least significant bit of the input (using $n \& 1$), shift the result left by 1 to make room, and OR in that extracted bit. Then I shift the input right by 1 to expose the next bit. Repeating this 32 times naturally builds up the reversed bit pattern, since each bit extracted from the low end of the input gets placed into a progressively higher position of the result.

KEY INSIGHT

Building the result by repeatedly shifting it left and OR-ing in the next bit from the input's low end naturally reverses bit order, since the FIRST bit extracted (originally the least significant) ends up shifted all the way to the most significant position by the end of the 32 iterations.

SAY THIS —

I'll process the number 32 bits at a time, since it's a fixed-width integer.

At each step, I extract the current least significant bit of the input, shift my result left to make room, and OR in that extracted bit.

I then shift the input right to expose the next bit, and repeat for all 32 positions.

STEP 6 — Code While Narrating

```
def reverse_bits(n):
    result = 0
    for i in range(32):
        bit = n & 1
        result = (result << 1) | bit
        n = n >> 1

    return result
```

SAY THIS —

I start result at 0. For each of the 32 iterations, I extract the current lowest bit of n using a bitwise AND with 1, shift result left by one position to make room for this new bit, OR it into that new open slot, and then shift n right by one to expose the next bit for the following iteration.

WHY: Because each extracted bit gets shifted progressively further left in the result as more iterations happen, the very first bit extracted (originally n's least significant bit) ends up in the most significant position of the final result, and vice versa — exactly reversing the bit order.

STEP 7 — Dry Run

$n = 0b00000010100101000001111010011100$ (a specific 32-bit example, simplified conceptually). Let's use a smaller conceptual trace with 4 bits for clarity: $n = 0b1011$ (conceptually, ignoring the full 32-bit requirement for illustration). $i=0$: $bit=1011\&1=1$. $result=(0\<<1)|1=1$. $n=1011\>>1=101$. $i=1$: $bit=101\&1=1$. $result=(1\<<1)|1=3$ ($0b11$). $n=101\>>1=10$. $i=2$: $bit=10\&1=0$. $result=(3\<<1)|0=6$ ($0b110$). $n=10\>>1=1$. $i=3$: $bit=1\&1=1$. $result=(6\<<1)|1=13$ ($0b1101$). Final result= $0b1101$, which is indeed $0b1011$ reversed. The same exact logic applies across all 32 iterations for the real problem.

SAY THIS —

Using a simplified 4-bit example, $n=0b1011$: processing bit by bit, the result builds up to $0b1101$, which is exactly 1011 reversed — the same mechanism scales directly to the full 32-bit case.

STEP 8 — Edge Case Test

An input of all zeros, $n=0$. Every iteration extracts $\text{bit}=0$, and result stays 0 throughout all 32 iterations, correctly returning 0, since the reverse of all zeros is still all zeros.

SAY THIS —

Let me check an input of all zeros — every extracted bit is 0, so the result correctly stays 0 throughout, since reversing an all-zero bit pattern is still all zeros.

Other edge cases to remember

- An input of all ones (should also return all ones, unchanged)
- An input with a single set bit at the very front (becomes a single set bit at the very back)
- An input with a single set bit at the very back (becomes a single set bit at the very front)
- A palindromic bit pattern (should return the exact same value)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(1)$	$O(1)$

SAY THIS —

Since we always process exactly 32 bits regardless of the input's value, this is $O(1)$ time and space in absolute terms (or $O(k)$ for a general k -bit integer if generalized beyond a fixed width).

If the Interviewer Pushes Back

Why not just use the string-reversal approach?

The string approach works and is simple to write, but this bit-shifting approach demonstrates direct comfort with bitwise operations, which many interviewers specifically want to see for a 'bit manipulation' tagged problem, and it also avoids the overhead of string conversion in performance-sensitive contexts.

What if this function were called many times on different inputs?

The direct bit-shifting or string-reversal approaches are already $O(1)$ per call given the fixed width, so there's limited room for further amortized optimization unless we specifically cache results for repeated identical inputs, which is unlikely to be useful in practice.

Can you do this with fewer than 32 iterations?

Yes, there's a well-known 'divide and conquer' bit-reversal trick that swaps bits in groups of decreasing size (first swap 16-bit halves, then 8-bit chunks within those, then 4, 2, and finally 1), completing the reversal in only 5 steps using precomputed bit masks, which is a classic low-level optimization technique.

How would you handle a different bit width, like 16 or 64 bits?

I'd simply change the loop bound from 32 to the appropriate bit width; the core logic (extract, shift result left, OR in, shift input right) generalizes directly to any fixed width.

Common Mistakes

- Off-by-one in the loop count, using something other than exactly 32 iterations for a 32-bit integer.
- Forgetting to shift the input right after extracting each bit, causing the same bit to be read repeatedly.
- Confusing the order of shifting result left versus OR-ing in the new bit (should shift first, then OR).
- Not handling potential sign issues in languages with signed integer types (less of a concern in Python, but worth mentioning for languages like Java).

If I Get Stuck — Recovery

- If stuck: 'I know I need to process all 32 bits somehow. Let me think about extracting one bit at a time from one end and building up the result by placing it at the OTHER end.'
- If my dry run doesn't reverse correctly: check the order of operations — shift result left, THEN OR in the new bit, not the other way around.
- If the interviewer hints 'can you do it without converting to a string': pivot directly to the shift-and-mask bitwise approach.
- If asked about a faster method: mention the divide-and-conquer bit-swapping technique as a more advanced optimization.

Pattern Recognition

Trigger words: 'reverse the bits of an integer', 'bit manipulation with fixed width'

Pattern: Bit Manipulation (Shift and mask, bit by bit)

First thought: Extract bits from one end of the input one at a time, and build the result by shifting them into the opposite end.

Common trap: Getting the order of 'shift result left' vs 'OR in new bit' backwards, or forgetting to shift the input to expose the next bit.

Memory Hook

KEY INSIGHT

Reverse Bits → "Peel one bit off the front, tuck it onto the back of the result."

30-Second Recap

Pattern	Bit Manipulation
Key idea	For 32 iterations, extract the lowest bit of n, shift result left and OR it in, then shift n right.
Data structure	No extra data structure — just integer shifts and masks.
Main algorithm	Bit Manipulation (iterative shift-and-mask).
Time	$O(1)$ [$O(k)$ general]
Space	$O(1)$
Biggest edge case	All-zero or all-one bit patterns.

Full Amazon Talk Track

Interviewer: Reverse the bits of this 32-bit unsigned integer.

Me: Got it — always exactly 32 bits, and I return the reversed bit pattern as an integer, correct?

Interviewer: Yes.

Me: One simple approach is converting to a 32-character binary string, reversing that string, and parsing it back — that works and is $O(1)$ given the fixed width.

Me: I can also do this with direct bit manipulation, which is often what's specifically being tested here: extract the lowest bit of the input, shift my result left to make room, OR that bit in, then shift the input right to expose the next bit, repeating 32 times.

Me: Let me code that with a simple loop.

Me: Tracing a small 4-bit example, 1011: after processing each bit, the result builds up to 1101, which is exactly the reverse — the same logic scales directly to the full 32-bit case.

Me: Edge case: an all-zero input stays all zeros throughout, which is correctly unchanged after reversal.

Me: Final complexity: $O(1)$ since we always process exactly 32 bits regardless of the input value.

Interviewer: Can you do it faster?

Me: There's a divide-and-conquer bit-swapping trick that reverses in just 5 steps by swapping progressively smaller chunks, using precomputed masks, if constant-factor speed really mattered.

Number of 1 Bits

LeetCode #191 | **EASY** | Pattern: Bit Manipulation | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an unsigned integer, and we need to count how many bits in its binary representation are set to 1 (this is often called the 'Hamming weight').

What I should clarify

- Is the input always treated as a 32-bit unsigned integer, or could it be an arbitrary-size integer in Python?
- Is there a preference for using built-in functions versus manual bit manipulation?
- Will this be called many times, favoring a precomputed lookup approach?
- Should negative numbers be handled, or is the input guaranteed non-negative (unsigned)?

SAY THIS —

Just to confirm, the input is a non-negative (unsigned) integer, and I need to count how many of its bits are 1, correct? Should I assume a specific bit width like 32 bits?

STEP 2 — Brute Force First

Idea

Check every one of the 32 bit positions individually: for each position, shift the number right by that many bits and check if the lowest bit is 1, incrementing a counter each time.

SAY THIS —

The direct approach checks all 32 possible bit positions one at a time, testing whether each one is set, and counts how many are.

STEP 3 — Brute Force Complexity

Time Complexity

This always takes exactly 32 steps, so $O(32)$, effectively $O(1)$ for a fixed-width integer, or $O(k)$ for a general k -bit integer.

Space Complexity

$O(1)$ for the counter.

SAY THIS —

This is $O(1)$ time in an absolute sense since we always check exactly 32 positions, but there's a neater trick that can skip over runs of zero bits entirely.

STEP 4 — Identify the Bottleneck

I'm checking every single bit position even when many of them might be 0, when there's a bit trick that can jump directly from one set bit to the next, skipping any runs of zeros in between entirely.

SAY THIS —

The straightforward version checks all 32 positions regardless of how many are actually set; there's a classic trick (Brian Kernighan's algorithm) that only does as many steps as there are actual 1 bits, which can be meaningfully faster for numbers with few set bits.

STEP 5 — Optimization

Key insight / pattern: Bit Manipulation

I use Brian Kernighan's Algorithm. The key trick is that $n \& (n - 1)$ always clears the LOWEST set bit of n (turns the rightmost 1 into a 0), without affecting any other bits. So I repeatedly apply $n = n \& (n - 1)$ and count how many times I do this until n becomes 0 — that count is exactly the number of 1 bits originally in n .

KEY INSIGHT

Subtracting 1 from a number flips all the bits from the lowest set bit down to the end (turning that lowest 1 into a 0 and all the 0's below it into 1's), so ANDing the original number with $(n - 1)$ exactly cancels out and clears just that lowest set bit, leaving every other bit unchanged — letting us count set bits in exactly as many steps as there are 1's, not always 32 steps.

SAY THIS —

I'll use the trick that $n \text{ AND } (n \text{ minus } 1)$ always clears the lowest set bit of n , leaving everything else unchanged.

I'll repeatedly apply that operation and count how many times it takes until n becomes 0.

That count is exactly the number of 1 bits, and this only takes as many steps as there are actual set bits, not a fixed 32 steps.

STEP 6 — Code While Narrating

```
def hamming_weight(n):
    count = 0
    while n != 0:
        n = n & (n - 1)
        count += 1

    return count
```

SAY THIS —

I start a counter at 0. As long as n isn't already 0, I apply the $n \& (n-1)$ trick to clear its lowest set bit, and increment my counter once per clear. Once n becomes 0, every set bit has been cleared and counted.

WHY: Each iteration removes exactly one set bit (the lowest one), so the number of iterations needed to reduce n all the way to 0 is precisely the count of set bits it originally had — no wasted iterations on positions that are already 0.

STEP 7 — Dry Run

$n = 11$ (binary 1011). $\text{count} = 0$. $n = 1011$, $n-1 = 1010$. $n \& (n-1) = 1011 \& 1010 = 1010$. $\text{count} = 1$. $n = 1010$ (binary, =10 decimal), $n-1 = 1001$. $n \& (n-1) = 1010 \& 1001 = 1000$. $\text{count} = 2$. $n = 1000$ (=8), $n-1 = 0111$. $n \& (n-1) = 1000 \& 0111 = 0000$. $\text{count} = 3$. $n = 0$, loop ends. Final $\text{count} = 3$, matching that binary 1011 has three 1 bits.

SAY THIS —

Tracing $n=11$ (binary 1011): each application of the $n \& (n-1)$ trick clears exactly one set bit, taking exactly 3 iterations to reach 0, correctly matching that 1011 has three 1 bits.

STEP 8 — Edge Case Test

$n = 0$. The while loop condition ' $n \neq 0$ ' is immediately false, so the loop body never executes, and count remains 0, correctly reflecting that 0 has no set bits at all.

SAY THIS —

Let me check $n=0$ — the loop condition is false right away, so we correctly return a count of 0, since there are no set bits in an all-zero number.

Other edge cases to remember

- n with only a single set bit, like a power of 2 (should return 1)
- n being all 1 bits for a given width (e.g., 0xFFFFFFFF for 32 bits, should return 32)
- $n = 1$ (the smallest number with exactly one set bit)
- A number with alternating bit pattern, like 0b10101010

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(k)$	$O(1)$

SAY THIS —

This is $O(k)$ time, where k is the number of set bits (not the total bit width), since each iteration clears exactly one set bit, and $O(1)$ space since only a counter is used; this can be meaningfully faster than a fixed 32-iteration approach when there are few set bits.

If the Interviewer Pushes Back

Why does $n \& (n-1)$ clear exactly the lowest set bit?

Subtracting 1 from n flips every bit starting from the lowest set bit (turning it into 0) down through all the trailing 0's below it (turning them into 1's); ANDing this modified value with the original n keeps every higher bit unchanged (since those didn't change), but the lowest set bit becomes 0 AND 1 = 0, and everything below it becomes 1 AND 0 = 0, so only that lowest set bit gets cleared.

Can you also just check all 32 bit positions directly?

Yes, that's the straightforward alternative: for each of 32 positions, check $(n \gg i) \& 1$, which is always $O(32)$ regardless of how many bits are actually set; Brian Kernighan's trick is an improvement specifically when the number of set bits is small relative to the total width.

Does Python have a built-in for this?

Yes, in modern Python, `bin(n).count('1')` or the integer method `n.bit_count()` (available in Python 3.10+) directly gives this count; mentioning the built-in shows practical awareness, though implementing the bit trick demonstrates understanding of the underlying technique, which is often what's being tested.

Can you optimize this further?

For very performance-critical code, a lookup-table approach (precomputing the bit count for every possible byte value, then combining 4 byte-lookups for a 32-bit integer) can get this down to $O(1)$ with a small constant, trading

memory for speed.

Common Mistakes

- Using the straightforward 32-iteration check-every-bit approach when the interviewer specifically wants to see the more clever Kernighan's trick.
- Forgetting the loop condition should be 'while n != 0', not a fixed range of 32 iterations, when using the bit-clearing trick.
- Misremembering the trick as $n \& (n+1)$ or some other variant instead of $n \& (n-1)$.
- Assuming this handles negative numbers directly without considering how negative numbers are represented in two's complement in a given language.

If I Get Stuck — Recovery

- If stuck: 'Checking all 32 positions always works, but I recall there's a trick where $n \text{ AND } (n \text{ minus } 1)$ clears the lowest set bit — let me think through why that's true and use it.'
- If unsure why the trick works: reason through a small binary example by hand, like $n=1100$, and verify that $n-1=1011$, and $n \& (n-1)=1000$, clearing exactly the lowest set bit.
- If the interviewer hints 'can you do better than checking every bit': confirm and describe Brian Kernighan's algorithm.
- If asked about built-ins: mention `bin(n).count('1')` or `n.bit_count()` as a practical shortcut, while still being able to implement the manual approach.

Pattern Recognition

Trigger words: 'count set bits', 'Hamming weight', 'number of 1 bits'

Pattern: Bit Manipulation (Brian Kernighan's algorithm)

First thought: $n \& (n-1)$ clears the lowest set bit — repeat until n is 0, counting iterations, rather than checking every fixed bit position.

Common trap: Defaulting to a fixed 32-iteration check-every-position approach when a more efficient bit trick is expected.

Memory Hook

KEY INSIGHT

Number of 1 Bits → " $n \text{ AND } (n-1)$ erases the lowest 1 — count how many erasures until zero."

30-Second Recap

Pattern	Bit Manipulation
Key idea	Repeatedly apply $n = n \& (n-1)$, counting iterations until n becomes 0.
Data structure	No extra data structure — just a counter.
Main algorithm	Brian Kernighan's Algorithm.
Time	$O(k)$ [k = number of set bits]
Space	$O(1)$
Biggest edge case	$n = 0$ (should return 0 immediately).

Full Amazon Talk Track

Interviewer: Count the number of 1 bits in this integer's binary representation.

Me: Got it — the input is non-negative, and I should count set bits regardless of a specific fixed width, correct?

Interviewer: Yes.

Me: A direct approach checks all 32 bit positions individually, which is $O(1)$ given a fixed width, but there's a neater trick that only takes as many steps as there are actual set bits.

Me: The trick is that $n \text{ AND } (n \text{ minus } 1)$ always clears the lowest set bit of n , leaving everything else unchanged. So I repeatedly apply that and count how many times it takes until n becomes 0.

Me: Let me code that — a simple while loop.

Me: Tracing $n=11$, binary 1011: each application clears one set bit, taking exactly 3 iterations to reach 0, matching that 1011 has three 1 bits.

Me: Edge case: $n=0$ immediately satisfies the loop's exit condition, correctly returning a count of 0.

Me: Final complexity: $O(k)$ time, where k is the number of set bits, which can be meaningfully faster than checking all 32 positions when few bits are set, and $O(1)$ space.

Pascal's Triangle

LeetCode #118 | **EASY** | Pattern: Dynamic Programming (Simple Row Building) | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an integer `numRows`, and we need to generate the first `numRows` rows of Pascal's Triangle, where each number is the sum of the two numbers directly above it in the previous row.

What I should clarify

- Are the first and last elements of every row always 1?
- Is `numRows` guaranteed to be a positive integer?
- Should the output be a list of lists, one per row?
- Is there a size constraint that would make this impractical to build directly?

SAY THIS —

Just to confirm, every row starts and ends with 1, and each middle element is the sum of the two elements above it in the previous row, and I return a list of lists, one per row, correct?

STEP 2 — Brute Force First

Idea

There isn't really a distinct 'brute force' here beyond the direct row-by-row construction, but one could imagine computing each entry using the binomial coefficient formula ($n \text{ choose } k$) independently for every position, without building rows incrementally.

SAY THIS —

An alternative to building rows incrementally would be computing each entry directly using the binomial coefficient formula, $n \text{ choose } k$, for every position independently.

STEP 3 — Brute Force Complexity

Time Complexity

Computing each binomial coefficient directly (using factorials) costs $O(\text{row_index})$ per entry, and there are $O(\text{numRows}^2)$ entries total, giving $O(\text{numRows}^3)$ if done very naively with repeated factorial computations.

Space Complexity

$O(\text{numRows}^2)$ for storing the full triangle.

SAY THIS —

Computing each entry independently via factorials is more expensive than necessary, potentially $O(\text{numRows}^3)$ if done naively, since it recomputes overlapping factorial calculations repeatedly.

STEP 4 — Identify the Bottleneck

I don't need to compute each entry from scratch using a formula; each row can be built directly and cheaply from the previous row, since Pascal's Triangle's defining property IS that relationship.

SAY THIS —

The bottleneck in the factorial-based approach is recomputing large factorials repeatedly, when each row can instead be built directly and cheaply from the row just before it.

STEP 5 — Optimization

Key insight / pattern: Dynamic Programming (Simple Row Building)

I build the triangle row by row using simple Dynamic Programming. Each row starts and ends with 1. Every middle element in a new row is the sum of the two elements directly above it (at the same index and the previous index) in the row before it. I build up one row at a time, using the previously completed row to construct each new one.

KEY INSIGHT

Pascal's Triangle is defined recursively (each entry is the sum of the two above it), so building it row by row, reusing the previous row directly, is both the most natural and most efficient approach — no need for factorials or binomial coefficient formulas at all.

SAY THIS —

I'll build the triangle one row at a time, starting with the trivial first row [1].

For each new row, I'll start and end with 1, and for every middle position, sum the two adjacent elements from the previous row.

I'll repeat this until I've built all of the requested numRows rows.

STEP 6 — Code While Narrating

```
def generate(num_rows):  
    triangle = [[1]]
```

SAY THIS —

I initialize the triangle with just its very first row, which is always simply [1].

WHY: Starting with this known, trivial base case gives me a previous row to build from when constructing the second row.

```
    for row_num in range(1, num_rows):  
        previous_row = triangle[-1]  
        new_row = [1]
```

SAY THIS —

For each additional row I need to build, I grab the most recently completed row, and start the new row with a leading 1.

WHY: Every row in Pascal's Triangle always starts with 1, since there's nothing to its upper-left to sum with.

```
        for i in range(1, len(previous_row)):  
            new_row.append(previous_row[i - 1] + previous_row[i])  
        new_row.append(1)  
        triangle.append(new_row)  
  
    return triangle
```

SAY THIS —

For each middle position, I sum the two adjacent values from the previous row. After filling in all the middle values, I append a trailing 1 to complete the row, then add this new row to the triangle.

WHY: Summing `previous_row[i-1]` and `previous_row[i]` directly implements the defining rule of Pascal's Triangle: each interior value is the sum of the two values diagonally above it.

STEP 7 — Dry Run

`numRows=5`. `triangle=[[1]]`. `row_num=1`: `previous_row=[1]`. `new_row=[1]`. Loop `range(1,1)` is empty (no middle elements). append 1 -> `new_row=[1,1]`. `triangle=[[1],[1,1]]`. `row_num=2`: `previous_row=[1,1]`. `new_row=[1]`. `i=1`: `previous_row[0]+previous_row[1]=1+1=2`, append -> `[1,2]`. append 1 -> `[1,2,1]`. `triangle=[[1],[1,1],[1,2,1]]`. `row_num=3`: `previous_row=[1,2,1]`. `new_row=[1]`. `i=1`: `1+2=3`, append. `i=2`: `2+1=3`, append. `new_row=[1,3,3]`, append 1 -> `[1,3,3,1]`. `row_num=4`: `previous_row=[1,3,3,1]`. `new_row=[1]`. `i=1`: `1+3=4`. `i=2`: `3+3=6`. `i=3`: `3+1=4`. `new_row=[1,4,6,4]`, append 1 -> `[1,4,6,4,1]`. Final triangle: `[[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]` — exactly the first 5 rows of Pascal's Triangle.

SAY THIS —

Tracing `numRows=5`: building row by row from `[1]`, each new row correctly sums adjacent pairs from the previous row, producing the exact expected first 5 rows of Pascal's Triangle.

STEP 8 — Edge Case Test

`numRows = 1`. The loop '`range(1, 1)`' never executes, so the triangle stays as its initial value, `[[1]]`, which is correctly just the single first row.

SAY THIS —

Let me check `numRows=1` — the loop to build additional rows never runs, so the triangle correctly stays as just the single row `[[1]]`.

Other edge cases to remember

- `numRows = 2` (should produce `[[1],[1,1]]`)
- A larger `numRows` verifying the symmetric structure of each row
- Confirming the outermost values in every row are always exactly 1
- `numRows = 0` (edge case depending on constraints, likely returns an empty list if allowed)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\text{numRows}^2)$	$O(\text{numRows}^2)$

SAY THIS —

This is $O(\text{numRows squared})$ time and space, since the total number of entries across all rows of Pascal's Triangle up to `numRows` is proportional to `numRows squared`, and each entry is computed in $O(1)$ from the previous row.

If the Interviewer Pushes Back

Why not compute each entry directly with the binomial coefficient formula?

That would work too (entry at row r , position k is ' r choose k '), but computing factorials directly is more work per entry and doesn't reuse previously computed values the way building row by row does, making the incremental approach simpler and just as efficient overall.

Can you reduce space by not storing the whole triangle?

If only the LAST row were needed (not all rows), I could maintain just a single rolling row and update it in place, working from right to left to avoid overwriting values I still need; but since the problem asks for ALL rows as output, storing the full triangle is necessary regardless.

Why does every row start and end with 1?

Because the elements at the very edges of each row have nothing above and to one side of them in the triangle (conceptually, they're bordered by 'invisible zeros'), so summing an existing value with 0 just gives back that value, which is always 1 at the edges by the triangle's construction.

Can you optimize this further?

No, $O(\text{numRows}^2)$ is optimal since that's exactly the total number of values that must be output across all the rows.

Common Mistakes

- Off-by-one in the inner loop range, either missing the first or last middle element.
- Forgetting the leading or trailing 1 on a new row.
- Not properly referencing the PREVIOUS row when building sums, accidentally using values from the current, still-incomplete row.
- Mishandling the `numRows=1` case if the initial triangle isn't seeded correctly with `[[1]]`.

If I Get Stuck — Recovery

- If stuck: 'Each row can be built directly from the one before it, following the sum-of-two-above rule — let me build the triangle row by row rather than computing entries independently.'
- If a row comes out with a wrong length or values: verify with a small dry run, comparing against the known correct triangle for small `numRows`.
- If the interviewer hints 'do you need factorials for this': confirm that building incrementally avoids that complexity entirely.
- If confused about indices: carefully trace the inner loop range with a concrete `previous_row` example to make sure boundaries are correct.

Pattern Recognition

Trigger words: 'Pascal's Triangle', 'each value is the sum of the two above it'

Pattern: Simple Dynamic Programming (build each row from the previous row)

First thought: This is inherently defined recursively, row by row — build directly from the previous row rather than using a closed-form formula.

Common trap: Off-by-one errors in the inner loop bounds, or forgetting the leading/trailing 1 on each new row.

Memory Hook

KEY INSIGHT

Pascal's Triangle → "Each new row: start with 1, sum adjacent pairs above, end with 1."

30-Second Recap

Pattern	Dynamic Programming (Simple Row Building)
Key idea	Build each row from the previous one: start/end with 1, middle values are sums of adjacent pairs above.
Data structure	A list of lists, growing one row at a time.
Main algorithm	Simple Dynamic Programming (row-by-row construction).
Time	$O(\text{numRows}^2)$
Space	$O(\text{numRows}^2)$
Biggest edge case	$\text{numRows} = 1$ (just the single row $[1]$).

Full Amazon Talk Track

Interviewer: Generate the first numRows rows of Pascal's Triangle.

Me: Got it — every row starts and ends with 1, and middle values are sums of the two values above them in the previous row, correct?

Interviewer: Yes.

Me: I could compute each entry directly with the binomial coefficient formula, but that recomputes factorials unnecessarily; building each row directly from the previous one is simpler and just as efficient.

Me: I'll start with the trivial first row $[1]$, and for each additional row, start with 1, sum adjacent pairs from the previous row for the middle values, and end with 1.

Me: Let me code that with a loop building one row at a time.

Me: Tracing $\text{numRows}=5$: the rows build up correctly to $[1]$, $[1,1]$, $[1,2,1]$, $[1,3,3,1]$, $[1,4,6,4,1]$, exactly matching Pascal's Triangle.

Me: Edge case: $\text{numRows}=1$ just returns the single row $[[1]]$, since the row-building loop never runs.

Me: Final complexity: $O(\text{numRows}^2)$ time and space, since that's exactly how many total values need to be output.

Longest Common Prefix

LeetCode #14 | **EASY** | Pattern: Horizontal/Vertical Scanning | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an array of strings, and we need to find the longest common prefix shared by all of them, returning an empty string if there's no common prefix at all.

What I should clarify

- Can the array of strings be empty?
- Can any individual string in the array be empty?
- Should comparison be case-sensitive?
- Is there a size constraint that would favor one particular scanning strategy?

SAY THIS —

Just to confirm, I need the longest prefix shared by EVERY string in the array, returning an empty string if there's no common prefix, or if any string itself is empty, correct?

STEP 2 — Brute Force First

Idea

Compare every pair of strings character by character to find their common prefix length, then take the minimum common prefix length across all pairs.

SAY THIS —

One approach is to compare every pair of strings to find their pairwise common prefix, then take the shortest one across all pairs as the overall answer.

STEP 3 — Brute Force Complexity

Time Complexity

Comparing all pairs is $O(n^2)$ pairs, each taking $O(m)$ to compare (m being average string length), giving $O(n^2 * m)$.

Space Complexity

$O(1)$ extra space beyond the input.

SAY THIS —

This pairwise comparison approach is $O(n \text{ squared times } m)$, which is more comparisons than necessary, since we don't actually need every pairwise relationship, just a single shared answer across ALL strings at once.

STEP 4 — Identify the Bottleneck

I'm comparing pairs of strings redundantly, when I really just need to check, character position by character position, whether ALL strings agree at that position simultaneously.

SAY THIS —

The bottleneck is doing pairwise comparisons, when I can instead check all strings together at once, position by position, stopping the moment any disagreement is found.

STEP 5 — Optimization

Key insight / pattern: Horizontal/Vertical Scanning

I use Vertical Scanning (character position by character position, across all strings simultaneously). I take the first string as a reference. For each character position (starting from 0), I check if that position exists in every other string and whether the character there matches the reference string's character at that position. The moment either check fails, the common prefix ends exactly there, and I return everything up to (but not including) that position.

KEY INSIGHT

Instead of comparing whole strings to each other one pair at a time, I can compare a single character POSITION across every string at once, and stop immediately at the very first position where any string disagrees or runs out of characters — that position is exactly the length of the longest common prefix.

SAY THIS —

I'll use the first string in the array as a reference for what the prefix might look like.

For each character position in that reference string, I'll check if every OTHER string has a matching character there.

The moment any string disagrees or is too short, I stop and return everything matched so far.

STEP 6 — Code While Narrating

```
def longest_common_prefix(strs):
    if not strs:
        return ''

    for i in range(len(strs[0])):
        char = strs[0][i]
        for other in strs[1:]:
            if i >= len(other) or other[i] != char:
                return strs[0][:i]

    return strs[0]
```

SAY THIS —

I handle the empty-array case first. Then, for each character position in the FIRST string, I check every OTHER string: if that string is too short to have a character at this position, or its character doesn't match, I immediately return everything matched so far (a slice of the first string up to position i). If I get through every position of the first string without any mismatch, the entire first string itself is the common prefix (meaning it's a prefix of, or equal to, every other string).

WHY: Checking position by position across ALL strings at once, and returning the instant any mismatch is found, avoids ever doing more comparisons than strictly necessary to find where agreement breaks down.

STEP 7 — Dry Run

strs = ['flower', 'flow', 'flight']. i=0: char='f'. Check 'flow'[0]='f' match. Check 'flight'[0]='f' match. i=1: char='l'. 'flow'[1]='l' match. 'flight'[1]='l' match. i=2: char='o'. 'flow'[2]='o' match. 'flight'[2]='i' — mismatch! Return strs[0][:2] = 'fl'. Final

answer: 'fl', the correct longest common prefix.

SAY THIS —

Tracing ['flower', 'flow', 'flight']: positions 0 and 1 ('f' and 'l') match across all three strings, but position 2 mismatches between 'flow' ('o') and 'flight' ('i'), so we correctly stop and return 'fl'.

STEP 8 — Edge Case Test

No common prefix at all, e.g. `strs=['dog','racecar','car']`. `i=0: char='d'`. Check `'racecar'[0]='r'` — immediate mismatch. Return `strs[0]:0 = ""` (empty string), correctly indicating no common prefix exists.

SAY THIS —

Let me check a case with no common prefix at all, like ['dog','racecar','car'] — the very first character position already mismatches, so we correctly return an empty string right away.

Other edge cases to remember

- An array containing just a single string (the whole string is trivially the common prefix)
- An array where one of the strings is empty (result should be empty, since nothing can be common with an empty string except an empty prefix)
- All strings being identical (the whole string is the common prefix)
- An empty array of strings (should return an empty string, per the initial guard clause)

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(n * m)$	$O(1)$

SAY THIS —

This is $O(n \text{ times } m)$ time in the worst case, where n is the number of strings and m is the length of the shortest string (or the common prefix length), since we check every string at every relevant position, and $O(1)$ extra space beyond the output.

If the Interviewer Pushes Back

Why use the first string as the reference instead of the shortest one?

Using the first string works correctly regardless, since the moment we hit a string shorter than the current position, that comparison itself fails and we return early; using the shortest string as reference is a common further optimization to avoid unnecessary iterations, but isn't strictly required for correctness.

Can you solve this with a different scanning order, like Horizontal Scanning?

Yes — horizontal scanning compares two strings' common prefix at a time (start with the first string as the running prefix, then repeatedly shrink it based on comparing against each subsequent string), which is also $O(n*m)$ but processes strings sequentially rather than position by position across all simultaneously.

Can you solve this with a Divide and Conquer approach?

Yes, split the array into two halves, recursively find the common prefix of each half, then find the common prefix of those two results; this is also $O(n*m)$ in the worst case but can be more cache-friendly and parallelizable in some contexts.

Can you optimize this using a Trie?

Yes, inserting all strings into a Trie and finding the longest path with only a single child at each node down to the first branching point (or end of a string) gives the common prefix; this is more useful if this operation needs to be repeated multiple times against an evolving set of strings.

Common Mistakes

- Forgetting the empty-array guard clause, causing an index error when accessing `strs[0]`.
- Forgetting to check `'i >= len(other)'` before indexing into `other[i]`, causing an index error for shorter strings.
- Comparing only pairs of strings pairwise instead of checking all strings together at each position.
- Not handling the case where the first string itself IS the common prefix (i.e., the loop completes without ever returning early).

If I Get Stuck — Recovery

- If stuck: 'Instead of comparing whole strings pairwise, let me check one character POSITION at a time, across all the strings simultaneously — stopping the moment any string disagrees.'
- If I get an index error: make sure I check the current string's length before indexing into it at the current position.
- If the interviewer hints 'can you avoid pairwise comparisons': confirm and describe vertical (position-by-position) scanning across all strings.
- If asked about Tries: describe the general idea (shared prefixes correspond to a single path with no branching) even if not implementing it directly.

Pattern Recognition

Trigger words: 'longest common prefix among multiple strings'

Pattern: Vertical Scanning (character position by character position across all strings)

First thought: Compare a single character position across ALL strings simultaneously, stopping at the first disagreement, rather than pairwise string comparisons.

Common trap: Forgetting to bounds-check a string's length before indexing into it at the current position.

Memory Hook

KEY INSIGHT

Longest Common Prefix → "Check one position at a time across every string — stop at first disagreement."

30-Second Recap

Pattern	Horizontal/Vertical Scanning
Key idea	For each position in the first string, verify every other string agrees there; stop and return at the first mismatch.
Data structure	No extra data structure — direct index comparisons.
Main algorithm	Vertical Scanning.
Time	$O(n * m)$
Space	$O(1)$
Biggest edge case	No common prefix at all (return empty string immediately).

Full Amazon Talk Track

Interviewer: Find the longest common prefix among these strings.

Me: Got it — return an empty string if there's no common prefix, or if the array is empty, or if any string itself is empty, correct?

Interviewer: Yes.

Me: A pairwise comparison approach would compare every pair of strings, which is more work than necessary, $O(n^2 \times m)$.

Me: Instead, I'll check one character position at a time, but across ALL strings simultaneously, using the first string as a reference. The moment any string disagrees at a position, or is too short, I stop and return everything matched so far.

Me: Let me code that with a loop over positions in the first string, and an inner check against every other string.

Me: Tracing ['flower', 'flow', 'flight']: positions 0 and 1 match everywhere, but position 2 mismatches between 'flow' and 'flight', so I correctly return 'fl'.

Me: Edge case: no common prefix at all, like ['dog', 'racecar', 'car'] — the very first position already mismatches, correctly returning an empty string.

Me: Final complexity: $O(n \times m)$ time, where n is the number of strings and m is the common prefix length or shortest string length, and $O(1)$ extra space.

Integer to Roman

LeetCode #12 | **MEDIUM** | Pattern: Greedy | Common interview pattern

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given an integer, and we need to convert it into its Roman numeral representation, following the standard subtractive notation rules (like IV for 4, IX for 9).

What I should clarify

- Is the input guaranteed to be within the standard Roman numeral range, typically 1 to 3999?
- Should I handle the subtractive cases (IV, IX, XL, XC, CD, CM) explicitly, or derive them some other way?
- Is the input always a positive integer, with no need to handle 0 or negative numbers?
- Should the output use uppercase Roman numeral characters?

SAY THIS —

Just to confirm, the input is a positive integer within the standard range (typically up to 3999), and I need to include the standard subtractive notation like IV and IX, correct?

STEP 2 — Brute Force First

Idea

Handle each digit place (thousands, hundreds, tens, ones) separately with a big set of if/elif conditions for every possible digit value (0-9) at each place, mapping directly to the right combination of Roman numeral symbols.

SAY THIS —

A very direct but repetitive approach would write out explicit if/elif branches for every possible digit (0 through 9) at each place value (ones, tens, hundreds, thousands), each producing the correct Roman numeral symbols.

STEP 3 — Brute Force Complexity

Time Complexity

$O(1)$ since there's a fixed, bounded number of digit places and values to check for any valid input.

Space Complexity

$O(1)$ for the fixed set of hardcoded mappings.

SAY THIS —

This hardcoded, exhaustive-branching approach is technically $O(1)$ since the input range is bounded, but it's extremely repetitive and error-prone to write out every single case by hand.

STEP 4 — Identify the Bottleneck

Writing out every single digit-value combination by hand is repetitive and easy to get wrong; there's a more elegant pattern-based approach that captures the SAME logic using an ordered list of value-symbol pairs instead of dozens

of hardcoded branches.

SAY THIS —

The real issue with the hardcoded branching approach isn't speed, it's that it's needlessly verbose and error-prone; a greedy approach with an ordered list of values captures the exact same logic far more concisely.

STEP 5 — Optimization

Key insight / pattern: Greedy

I use a Greedy approach with a precomputed, ordered list of (value, symbol) pairs, listed from largest to smallest, including the special subtractive combinations (like (900, 'CM'), (400, 'CD'), (90, 'XC'), (40, 'XL'), (9, 'IX'), (4, 'IV')) alongside the standard ones. I repeatedly find the largest value in this list that's less than or equal to the remaining number, append its corresponding symbol to the result, and subtract that value from the remaining number, continuing until the remaining number reaches 0.

KEY INSIGHT

By explicitly including the subtractive combinations (like 900 for 'CM') as their own entries in the ordered value list, the same simple greedy 'always take the biggest bite possible' logic that works for straightforward Roman numerals ALSO correctly handles all the special subtractive cases, without needing separate special-case logic.

SAY THIS —

I'll use an ordered list of value-symbol pairs, from largest to smallest, that explicitly includes the special subtractive combinations like 900 for 'CM' and 4 for 'IV'.

I'll repeatedly find the largest value in that list that still fits within what's remaining, append its symbol, and subtract it from the remaining amount.

I keep doing this until the remaining amount reaches 0, which builds up the correct Roman numeral string one greedy 'largest possible bite' at a time.

STEP 6 — Code While Narrating

```
def int_to_roman(num):
    value_symbols = [
        (1000, 'M'), (900, 'CM'), (500, 'D'), (400, 'CD'),
        (100, 'C'), (90, 'XC'), (50, 'L'), (40, 'XL'),
        (10, 'X'), (9, 'IX'), (5, 'V'), (4, 'IV'), (1, 'I')
    ]
    result = []
```

SAY THIS —

I define the ordered list of value-symbol pairs, from largest to smallest, explicitly including all the subtractive combinations, and set up a list to build the result string.

WHY: Including entries like (900, 'CM') directly in this ordered list, rather than handling them as a special case afterward, is what lets the simple greedy loop below correctly produce proper Roman numerals without any extra logic.

```

for value, symbol in value_symbols:
    while num >= value:
        result.append(symbol)
        num -= value

return ''.join(result)

```

SAY THIS —

For each value-symbol pair, in order from largest to smallest, I keep appending that symbol and subtracting its value from num as many times as it still fits. Once I've gone through the whole list, I join everything into the final string.

WHY: Because the list is sorted largest to smallest, greedily taking the biggest value that fits at each step always produces the correct, most compact standard Roman numeral representation.

STEP 7 — Dry Run

num = 1994. Check (1000,'M'): 1994>=1000, append 'M', num=994. 994<1000, move on. Check (900,'CM'): 994>=900, append 'CM', num=94. 94<900, move on. Check (500,'D') through (100,'C'): 94<100, skip. Check (90,'XC'): 94>=90, append 'XC', num=4. 4<90, move on. Check (50,'L') through (10,'X'): 4<10, skip. Check (9,'IX'): 4<9, skip. Check (5,'V'): 4<5, skip. Check (4,'IV'): 4>=4, append 'IV', num=0. Check (1,'I'): 0<1, skip. Final result: 'M'+'CM'+'XC'+'IV' = 'MCMXCIV', the correct Roman numeral for 1994.

SAY THIS —

Tracing num=1994: the greedy approach correctly builds up 'M' (1000), then 'CM' (900), then 'XC' (90), then 'IV' (4), giving 'MCMXCIV', which is indeed the correct Roman numeral representation of 1994.

STEP 8 — Edge Case Test

A number entirely made of subtractive combinations, e.g. num=444. Check (400,'CD'): 444>=400, append 'CD', num=44. Check (90,'XC') down to (50,'L'): 44<90,<50, skip those. Check (40,'XL'): 44>=40, append 'XL', num=4. Check (10,'X'),(9,'IX'),(5,'V'): 4< all of those, skip. Check (4,'IV'): 4>=4, append 'IV', num=0. Final: 'CD'+'XL'+'IV' = 'CDXLIV', correctly using three separate subtractive combinations in a row.

SAY THIS —

Let me check a number that uses several subtractive combinations in a row, like 444 — the greedy approach correctly produces 'CDXLIV', combining CD (400), XL (40), and IV (4).

Other edge cases to remember

- The smallest valid input, num=1 (just 'I')
- The largest typical input, num=3999 ('MMMCMXCIX')
- A round number like 1000 ('M') or 500 ('D') with no subtractive parts needed
- A number requiring several repeats of the same symbol, like 3 ('III')

STEP 9 — Final Complexity

	Time	Space
Optimized	O(1)	O(1)

SAY THIS —

This is $O(1)$ time and space since the number of value-symbol pairs is fixed (13 entries) and the input is bounded within the standard Roman numeral range, so the total work done is bounded by a small constant regardless of the specific input.

If the Interviewer Pushes Back

Why include the subtractive combinations as explicit entries rather than detecting them separately?

Because doing so lets the same simple greedy 'take the biggest value that fits' loop handle every case uniformly, without needing separate conditional logic to detect and construct subtractive patterns — it's a clean unification of what would otherwise be two different sets of rules.

Why does the greedy approach work here specifically?

Because Roman numeral values were specifically designed such that greedily taking the largest applicable value at each step always produces the standard, correct, and most compact representation; this isn't true for arbitrary sets of denominations in general (like some coin systems), but it happens to hold for the specific set of Roman numeral values.

Can you solve this without the explicit subtractive list, deriving it programmatically instead?

Yes, you could detect subtractive patterns by examining digit-by-digit values (checking if a digit's place value is 4 or 9 specifically) and handling those with separate logic, but this is more complex and error-prone than simply encoding the subtractive combinations directly into the ordered list.

Can you optimize this further?

Since the input range and the number of value-symbol pairs are both small and fixed, this is already effectively $O(1)$; there's no meaningful further optimization needed for this specific problem's constraints.

Common Mistakes

- Forgetting to include the subtractive combinations (900, 400, 90, 40, 9, 4) in the value-symbol list, producing incorrect (non-standard) representations like 'VIII' instead of 'IX'.
- Ordering the value-symbol list incorrectly (not strictly descending), which would break the greedy logic.
- Using a while loop instead of correctly checking with $\geq$ at each step, potentially skipping valid repeated symbols.
- Assuming Roman numerals allow more than 3 repeated symbols in standard notation, missing the subtractive rule replacing 4+ repeats.

If I Get Stuck — Recovery

- If stuck: 'Handling every digit place with hardcoded branches would work but is very repetitive. Let me think about a general list of value-symbol pairs and a consistent rule for using them.'
- If my output doesn't match standard Roman numerals: check whether I've included the subtractive combinations (900, 400, 90, 40, 9, 4) explicitly in my value list.
- If the interviewer hints 'can you avoid hardcoding every digit case': confirm and describe the greedy value-list approach.
- If unsure about ordering: emphasize that the list must be strictly descending by value for the greedy approach to work correctly.

Pattern Recognition

Trigger words: 'convert integer to Roman numeral', 'convert to a different fixed symbolic representation with special combined cases'

Pattern: Greedy (always take the largest applicable value)

First thought: Can I list all possible symbol values, including special combined ones, in descending order, and greedily take the biggest one that fits repeatedly?

Common trap: Forgetting to explicitly include the subtractive combinations in the value list, producing non-standard output.

Memory Hook

KEY INSIGHT

Integer to Roman → "Ordered list of values (including CM, CD, XC...) — always take the biggest bite that fits."

30-Second Recap

Pattern	Greedy
Key idea	Use an ordered (descending) list of value-symbol pairs, including subtractive combinations, and greedily
Data structure	A fixed ordered list of (value, symbol) pairs.
Main algorithm	Greedy largest-value-first construction.
Time	$O(1)$
Space	$O(1)$
Biggest edge case	A number requiring several subtractive combinations in a row (e.g., 444 → CDXLIV).

Full Amazon Talk Track

Interviewer: Convert this integer to its Roman numeral representation.

Me: Got it — the input's within the standard range, and I need to correctly handle subtractive notation like IV and IX, correct?

Interviewer: Yes.

Me: A very direct approach would hardcode every digit-place combination with if/elif branches, but that's repetitive and easy to get wrong.

Me: Instead, I'll use a greedy approach: an ordered list of value-symbol pairs from largest to smallest, explicitly including the subtractive combinations like 900 for CM and 4 for IV. I repeatedly take the biggest value that still fits, append its symbol, and subtract it, until nothing remains.

Me: Let me code that with the value list and a simple greedy loop.

Me: Tracing num=1994: the greedy approach correctly builds 'M', then 'CM', then 'XC', then 'IV', giving MCMXCIV.

Me: Edge case: a number using several subtractive combinations in a row, like 444, correctly produces CDXLIV, combining CD, XL, and IV.

Me: Final complexity: $O(1)$ time and space, since the value list and input range are both small and fixed.

Add Binary

LeetCode #67 | **EASY** | Pattern: Two Pointers + Carry Propagation | Useful foundational problem

Video source: Not available from the provided source — no playlist link or video URL was included with this request, so no transcript could be retrieved. The explanation below is written from first principles / general knowledge of this problem (in the style of standard references such as NeetCode and LeetCode), not transcribed from any video.

STEP 1 — Restate the Problem

We're given two binary strings, and we need to add them together and return the sum, also as a binary string.

What I should clarify

- Can the two input strings be of different lengths?
- Are the strings guaranteed to contain only '0' and '1' characters, with no sign or other characters?
- Should leading zeros in the result be avoided (except for the result '0' itself)?
- Is there a size constraint that would make converting to integers and back impractical (very long strings)?

SAY THIS —

Just to confirm, the two binary strings can be different lengths, contain only 0s and 1s, and I return their sum as a binary string with no unnecessary leading zeros, correct?

STEP 2 — Brute Force First

Idea

Convert both binary strings to integers using Python's built-in `int(s, 2)`, add them as regular integers, then convert the sum back to a binary string using `bin()` and stripping the '0b' prefix.

SAY THIS —

In Python, I could convert both binary strings to integers directly, add them normally, and convert the sum back to a binary string.

STEP 3 — Brute Force Complexity

Time Complexity

Converting each string is $O(n)$, addition is effectively $O(n)$ for big integers, and converting back is $O(n)$, so $O(n)$ overall.

Space Complexity

$O(n)$ for the intermediate integer and string representations.

SAY THIS —

This is $O(n)$ time and space in Python, and it's actually a perfectly valid solution here — but the more 'manual' digit-by-digit approach is often what's expected to demonstrate understanding of binary addition directly, and would be necessary in a language without arbitrary-precision integers or built-in base conversion.

STEP 4 — Identify the Bottleneck

Relying on built-in conversions sidesteps demonstrating the actual mechanics of binary addition, which is likely what the interviewer wants to see, especially since real-world equivalents of this problem (like adding numbers represented as arrays/lists) don't always have such convenient built-ins available.

SAY THIS —

Using built-in integer conversion works in Python, but it doesn't demonstrate the underlying mechanics of binary addition with carries, which is probably the specific skill being tested here.

STEP 5 — Optimization

Key insight / pattern: Two Pointers + Carry Propagation

I simulate binary addition directly, digit by digit, from the rightmost (least significant) end of both strings using Two Pointers, just like manual addition on paper. I maintain a carry value starting at 0. At each step, I sum the current digits from both strings (defaulting to 0 if a string has been fully consumed) plus the carry, determine the result bit (sum modulo 2) and the new carry (sum divided by 2, integer division), and prepend that result bit to my growing answer. I continue until both pointers have gone past the start of their strings and there's no carry left.

KEY INSIGHT

Binary addition works exactly like decimal addition on paper, just with a base of 2 instead of 10: I process one digit position at a time from the right, carry over anything that overflows past 1 (since binary only has two, digits 0 and 1), and stop once both strings and any leftover carry are fully accounted for.

SAY THIS —

I'll process both strings from right to left simultaneously, using two pointers.

At each step, I sum the current digits from both strings (treating a missing digit as 0) plus any carry, and compute the resulting bit (sum mod 2) and new carry (sum divided by 2).

I keep going until both pointers have exhausted their strings and there's no carry left over, building the result from right to left.

STEP 6 — Code While Narrating

```
def add_binary(a, b):  
    result = []  
    i, j = len(a) - 1, len(b) - 1  
    carry = 0
```

SAY THIS —

I set up a list to build the result (since prepending to a string repeatedly is inefficient, I'll append and reverse at the end instead), two pointers starting at the last character of each string, and a carry starting at 0.

WHY: Using a list and appending, then reversing once at the end, is more efficient than repeatedly prepending to a string, which would be $O(n)$ per prepend in many implementations.

```
    while i >= 0 or j >= 0 or carry:  
        digit_a = int(a[i]) if i >= 0 else 0  
        digit_b = int(b[j]) if j >= 0 else 0  
        total = digit_a + digit_b + carry
```

SAY THIS —

I loop as long as either string still has digits left, or there's a carry to apply. I treat a missing digit (when a pointer has gone past its string's start) as 0, and sum both current digits plus the carry.

WHY: Continuing while 'carry' is truthy correctly handles cases like '1'+ '1', which needs one more digit than either input string alone provides, similar to the leftover-carry case in Add Two Numbers.

```
        carry = total // 2
        result.append(str(total % 2))
        i -= 1
        j -= 1

    return ''.join(reversed(result))
```

SAY THIS —

I compute the new carry (total divided by 2) and the current result bit (total mod 2), append that bit to my result list, and move both pointers one position to the left. At the end, I reverse the accumulated bits (since I built them from least to most significant) and join them into the final string.

WHY: Because I processed digits from right to left but need to return the result left to right (most significant digit first), reversing the accumulated list at the very end correctly restores proper reading order.

STEP 7 — Dry Run

a='11', b='1'. i=1,j=0,carry=0. Step1: digit_a=a[1]='1'->1, digit_b=b[0]='1'->1, total=1+1+0=2. carry=1, append str(0)='0'. i=0,j=-1. Step2: digit_a=a[0]='1'->1, digit_b=0 (j<0), total=1+0+1=2. carry=1, append '0'. i=-1,j=-2. Step3: digit_a=0,digit_b=0,carry=1, total=0+0+1=1. carry=0, append '1'. i=-2,j=-3. Loop condition: i<0,j<0,carry=0, stop. result=['0','0','1'], reversed='100'. Final answer: '100', which is indeed 3+1=4 in binary (11 is 3, 1 is 1, sum is 4 = 100 in binary).

SAY THIS —

Tracing a='11' (which is 3) and b='1' (which is 1): processing right to left with carries, the result builds up to '100', which is 4 in binary — correctly matching 3 plus 1.

STEP 8 — Edge Case Test

Both inputs are '1', producing a result requiring an extra digit due to carry, e.g. a='1', b='1'. i=0,j=0,carry=0. Step1: digit_a=1,digit_b=1,total=2. carry=1,append '0'. i=-1,j=-1. Step2: both digits 0, carry=1, total=1. carry=0, append '1'. Loop ends. result=['0','1'], reversed='10'. Final answer '10', which is 2 in binary, correctly matching 1+1=2.

SAY THIS —

Let me check a='1' plus b='1', which should need an extra digit due to the carry — the trace correctly produces '10', which is 2 in binary, matching 1 plus 1.

Other edge cases to remember

- Strings of very different lengths
- One of the strings being just '0'
- Both strings being all 1's (maximal carry propagation)
- Strings representing a sum that's an exact power of 2

STEP 9 — Final Complexity

	Time	Space
Optimized	$O(\max(n, m))$	$O(\max(n, m))$

SAY THIS —

This is $O(\max(n, m))$ time, where n and m are the lengths of the two input strings, since we process at most that many positions (plus possibly one more for a final carry), and $O(\max(n, m))$ space for the resulting string.

If the Interviewer Pushes Back

Why not just use Python's built-in `int(s, 2)` and `bin()`?

That's a perfectly valid and much shorter solution in Python specifically, since Python has arbitrary-precision integers and built-in base conversion; however, demonstrating the manual digit-by-digit simulation shows understanding of the underlying mechanics, which is often exactly what's being tested, and is necessary in languages or contexts without such convenient built-ins (or for extremely large inputs where such conversions might be impractical).

Why build the result with a list and reverse at the end, instead of prepending to a string?

Because prepending to a string is generally an $O(n)$ operation each time in many language implementations (creating a new string every time), while appending to a list is $O(1)$ amortized; reversing once at the very end is much more efficient than reversing implicitly on every single prepend.

What if the strings could contain characters other than 0 and 1?

I'd assume, per problem constraints, that inputs are always valid binary strings; if that weren't guaranteed, I'd add validation logic before processing.

Can you optimize this further?

No, $O(\max(n, m))$ is optimal since we must examine every digit of the longer string at least once to correctly compute the sum.

Common Mistakes

- Forgetting to continue the loop when both strings are exhausted but a carry remains (missing the final carry digit).
- Building the result by prepending to a string repeatedly instead of appending to a list and reversing at the end.
- Off-by-one in the pointer initialization (should start at `len-1`, the LAST character, not `len`).
- Not handling strings of different lengths correctly, causing an index error when one pointer goes out of bounds.

If I Get Stuck — Recovery

- If stuck: 'This is just binary addition, structurally identical to how I'd add two numbers by hand in decimal, just working in base 2 with carries.'
- If a final carry gets dropped: check the while loop condition includes 'or carry', not just the two pointer conditions.
- If the interviewer is fine with using built-ins: mention `int(s, 2)` and `bin()` as a quick Python-specific solution, but be ready to also explain the manual approach.
- If confused about append-then-reverse: reason aloud that since I'm processing right to left but need to output left to right, reversing once at the end restores correct order more efficiently than repeated prepending.

Pattern Recognition

Trigger words: 'add two binary strings', 'simulate addition digit by digit in a given base'

Pattern: Two Pointers with Carry Propagation (manual digit-by-digit addition)

First thought: Simulate manual addition directly, processing from the least significant digit (rightmost) with a carry, just like paper arithmetic but in base 2.

Common trap: Forgetting to continue the loop for one final step if a carry remains after both strings are exhausted.

Memory Hook

KEY INSIGHT

Add Binary → "Paper addition in base 2 — sum digits plus carry, right to left."

30-Second Recap

Pattern	Two Pointers + Carry Propagation
Key idea	Process both strings right to left with two pointers, summing digits plus carry, and finish with a check for a
Data structure	A list for building the result, reversed at the end.
Main algorithm	Two Pointers with Carry Propagation.
Time	$O(\max(n,m))$
Space	$O(\max(n,m))$
Biggest edge case	A leftover carry after both strings are exhausted (e.g., '1' + '1').

Full Amazon Talk Track

Interviewer: Add these two binary strings and return the sum as a binary string.

Me: Got it — the strings can be different lengths, contain only 0s and 1s, and I return the sum with no unnecessary leading zeros, correct?

Interviewer: Yes.

Me: In Python, I could convert both to integers, add them, and convert back to binary — that works, but I think demonstrating the manual digit-by-digit simulation is probably what's being tested here.

Me: I'll process both strings from right to left with two pointers, summing corresponding digits plus a carry, just like paper addition but in base 2, treating a missing digit as 0 for the shorter string.

Me: Let me code that, building the result in a list and reversing it at the end, since prepending to a string repeatedly would be less efficient.

Me: Tracing $a='11'$ (3) and $b='1'$ (1): processing with carries gives '100', which is 4 — correct.

Me: Edge case: $a='1'$ plus $b='1'$, which needs an extra digit due to carry propagation — correctly gives '10', which is 2.

Me: Final complexity: $O(\max \text{ of the two lengths})$ time and space, since we process at most that many digit positions, plus possibly one more for a final carry.

Amazon SDE1 Pattern Cheat Sheet

Use this as a lookup table during practice: hear the clue on the left, think the pattern on the right.

Interview clue	Pattern	Example problem(s)
Need a pair/complement quickly	Hash Map	Group Anagrams, Product Except Self
Sorted array + search for a value/boundary	Binary Search	Find K Closest Elements, Pow(x,n)
Sorted array, converge from both ends	Two Pointers	Two Sum II, Boats to Save People
Contiguous subarray/substring with a condition	Sliding Window	Min Size Subarray Sum, Find All Anagrams
Matching brackets / most-recent-first processing	Stack	Evaluate RPN, Decode String, Simplify Path
Next greater/smaller element to the side	Monotonic Stack	Next Greater Element I
Shortest path in an unweighted grid/graph	BFS	Rotting Oranges, Walls and Gates
Multiple starting points spreading simultaneously	Multi-Source BFS	Rotting Oranges, Walls and Gates
Explore all possibilities / generate combinations	Backtracking	Generate Parentheses, Combination Sum
Top-K / Kth largest or smallest, ongoing stream	Heap	Kth Largest in a Stream, Top K Frequent
Best result built from repeated subproblems	Dynamic Programming	Climbing Stairs, Perfect Squares
Cycle detection / duplicate via pointers	Fast & Slow Pointers	Linked List Cycle, Find the Duplicate Number
Locally best choice is provably globally best	Greedy	Jump Game, Task Scheduler, Car Pooling
Node value bounded by ALL ancestors	DFS with Valid Range	Validate Binary Search Tree
Tree values in sorted order	In-order Traversal	Kth Smallest Element in a BST
Rebuild a structure from a serialized form	Preorder DFS + markers	Serialize/Deserialize Binary Tree
Need O(1) insert, remove, AND random access	Hash Map + Array	Insert Delete GetRandom O(1)
Overlapping intervals with a running capacity	Difference Array / Line Sweep	Car Pooling
Minimize the maximum / maximize the minimum	Binary Search on the Answer	Split Array Largest Sum
Pairs cancel out, one value doesn't	Bit Manipulation (XOR)	Single Number

How to Talk in an Amazon Coding Interview

This is the universal flow. Print it, read it right before you walk in.

- **Listen** — let the interviewer finish the full problem statement before saying anything.
- **Restate** — "So just to confirm, we're given..." in your own words.
- **Clarify** — ask 2-4 real questions: constraints, edge cases, input properties, expected output format.
- **Example** — walk through a small concrete example together if one isn't given.
- **Brute force** — state the simplest correct approach, even if obviously slow.
- **Complexity** — say the brute force's time and space complexity out loud, and WHY.
- **Bottleneck** — name specifically what's slow or wasteful about the brute force.
- **Optimize** — introduce the better approach and name the pattern if you recognize one.
- **Explain the data structure** — say why THIS structure fixes THIS bottleneck.
- **Code while talking** — narrate what each block does and why, in small pieces.
- **Test** — dry run your code on the given example, out loud, tracking variables.
- **Edge cases** — test at least one meaningful edge case explicitly.
- **Complexity again** — restate the FINAL time and space complexity, unprompted.
- **Ask if they want anything else** — "Would you like me to handle any other cases, or discuss alternatives?"

Short Example Script

Me: So just to make sure I understand the problem correctly, we're given...

Interviewer: Yes, that's right.

Me: Okay. A couple of clarifying questions first: can the input be empty, and are duplicates possible?

Interviewer: Assume non-empty, duplicates possible.

Me: Got it. The straightforward approach would be to check every pair, which is $O(n^2)$ time and $O(1)$ space.

Interviewer: Can you do better?

Me: The bottleneck is that I'm re-scanning for information I've already seen. I can fix that with a hash map, which gets us to $O(n)$ time.

Me: Let me code that... here I'm building a map of value to index as I scan, so I can check for a complement in $O(1)$.

Me: Let me test this on the example... and here's an edge case with duplicates to make sure it still holds.

Me: So the final solution is $O(n)$ time and $O(n)$ space.

24-Hour Amazon Coding Crash Review

If you only have one day, prioritize by PATTERN, not by individual question — patterns transfer to problems you haven't seen before; memorized answers to specific questions don't. These groupings reflect how foundational and how often each pattern recurs across this guide's problems, not any claim about which exact questions Amazon will ask.

Must Understand (do these first)

- **Hash Map** — Group Anagrams, Product of Array Except Self, Top K Frequent Elements. Foundation for a huge fraction of interview problems.
- **Two Pointers** — Two Sum II, Boats to Save People. Simple, very common, easy points if solid.
- **Sliding Window** — Minimum Size Subarray Sum, Find All Anagrams. Natural extension of two pointers.
- **BFS/DFS on trees and graphs** — Max Depth, Validate BST, Rotting Oranges, Walls and Gates. Amazon asks tree/graph traversal constantly.
- **Dynamic Programming basics** — Climbing Stairs, Maximum Subarray (Kadane's). The gateway to all other DP.

Should Understand

- **Stacks & Monotonic Stack** — Evaluate RPN, Next Greater Element, Decode String.
- **Heaps** — Kth Largest Element in a Stream. Recognize the 'top-K' signal.
- **Backtracking** — Generate Parentheses, Combination Sum. Understand the prune-as-you-go mindset.
- **Binary Search variants** — Find K Closest Elements, and Binary Search on the Answer (Split Array Largest Sum).
- **Linked List manipulation** — Add Two Numbers, Linked List Cycle (Fast & Slow Pointers).

If Time Remains

- **Design problems** — Min Stack, Insert Delete GetRandom O(1). Common but lower frequency than core patterns.
- **Advanced DP** — Distinct Subsequences, Unique Binary Search Trees.
- **Bit manipulation** — Single Number, Number of 1 Bits, Reverse Bits.
- **Niche/simulation problems** — Car Pooling, Task Scheduler, Integer to Roman, Add Binary.

KEY INSIGHT

Under time pressure, review the 30-Second Recap and Memory Hook for every problem in "Must Understand" first, then "Should Understand." Skim the Pattern Cheat Sheet last, right before you walk in.

Sources

Primary source

None of the requested YouTube playlist videos could be retrieved or transcribed for this guide. The original request described a playlist of problems by title but did not include an actual playlist URL or individual video links, so this guide's explanations, code, dry runs, and talk tracks were **not** transcribed from, or based on, any specific video's transcript or narration. No quotes are attributed to any video anywhere in this guide.

Supporting sources

- LeetCode problem statements and constraints (used to confirm exact problem definitions, e.g. return formats, index conventions).
- General, widely-known algorithmic techniques and terminology consistent with standard references such as NeetCode's pattern catalog (Two Pointers, Sliding Window, Backtracking, etc.) — no specific NeetCode video content was transcribed or quoted.
- Standard computer science background on data structures and algorithms (hash maps, heaps, trees, graphs, dynamic programming, bit manipulation).

Per-problem video source note

Every problem page in this guide includes an italicized note under its title stating explicitly that video transcript access was not available for that problem, consistent with the note above. This is repeated per-problem, as requested, rather than only stated once here.

Problems covered in this guide

Maximum Subarray, Maximum Product Subarray, Group Anagrams, Product of Array Except Self, Top K Frequent Elements, Find the Duplicate Number, Single Number, Move Zeroes, Two Sum II - Input Array Is Sorted, Boats to Save People, Minimum Size Subarray Sum, Find All Anagrams in a String, Evaluate Reverse Polish Notation, Next Greater Element I, Decode String, Simplify Path, Find K Closest Elements, Split Array Largest Sum, Pow(x, n), Add Two Numbers, Linked List Cycle, Intersection of Two Linked Lists, Copy List with Random Pointer, Maximum Depth of Binary Tree, Same Tree, Validate Binary Search Tree, Kth Smallest Element in a BST, Construct Binary Tree from Preorder and Inorder Traversal, Serialize and Deserialize Binary Tree, Balanced Binary Tree, Subtree of Another Tree, Walls and Gates, Rotting Oranges, Kth Largest Element in a Stream, Jump Game, Task Scheduler, Car Pooling, Generate Parentheses, Combination Sum, Letter Combinations of a Phone Number, Valid Sudoku, Climbing Stairs, Perfect Squares, Is Subsequence, Distinct Subsequences (Number of Subsequences), Unique Binary Search Trees, Min Stack, Insert Delete GetRandom O(1), Reverse Bits, Number of 1 Bits, Pascal's Triangle, Longest Common Prefix, Integer to Roman, Add Binary